Genomdatenbank und evolutionäre Netzwerkanalyse

Stephan Epp
M.Sc.

23. Januar 2026

Inhaltsverzeichnis

1 Einführung **1**
- 1.1 Beitrag dieser Arbeit . . . 2
- 1.2 Aufbau der Arbeit . . . 4

2 Grundlagen **7**
- 2.1 Graphen und Adjazenzmatrizen . . . 7
- 2.2 Signaturen und Subgraph-Relation . . . 7
- 2.3 Der Subgraph Algorithmus . . . 8
  - 2.3.1 Konkrete Umsetzung in der C++17-Bibliothek . . . 8

3 Systemarchitektur von Gen-DB **12**
- 3.1 Komponentenübersicht . . . 12
- 3.2 Datenmodell . . . 12
- 3.3 UML-Architekturmodelle von Gen-DB . . . 13

4 Formalisierung der Suche: Vorauswahl und Beweise **16**
- 4.1 Problemformulierung . . . 16
- 4.2 SQL-Vorauswahl . . . 16
- 4.3 SQL-Select-Statement . . . 17
- 4.4 Gesamtkomplexität der Pipeline . . . 17
- 4.5 Korrektheit der Gesamtpipeline . . . 18
- 4.6 Effiziente Suche . . . 18

5 Erste experimentelle Untersuchung **20**
- 5.1 Visualisierung 1: Knoten- und Kantenverteilung . . . 21
- 5.2 Visualisierung 2: Pre-Filter-Effektivität . . . 21
- 5.3 Visualisierung 3: Skalierungsverhalten Suchzeit . . . 21
- 5.4 Visualisierung 4: Trefferverteilung . . . 22
- 5.5 Diskussion . . . 23

6 Experimenteller Vergleich VF2-Algorithmus **24**
- 6.1 Versuchsanordnung . . . 24
- 6.2 Ergebnis 1: Laufzeitvergleich bei festem $n_A = 5$, wachsendem n_B . . . 25
- 6.3 Der Subgraph Algorithmus für $n_B \geq 14$ . . . 27
- 6.4 Einordnung in effiziente Subgraph-Isomorphie-Verfahren . . . 29

7 Experimentelle Untersuchung der Parallelisierung **32**
- 7.1 Fragestellung und Hypothesen . . . 32
- 7.2 Implementierung der parallelen Suche . . . 33

7.3 Versuchsaufbau . . . 33
7.4 Metriken . . . 34
7.5 Ergebnisse . . . 34
7.5.1 Die untersuchten Anfragen . . . 34
7.5.2 Gesamtzeit und Speedup . . . 35
7.5.3 Verteilung der Antwortzeiten . . . 36
7.5.4 Skalierung mit der Größe des Suchraums . . . 37
7.5.5 Dieselben Anfragen im direkten Vergleich . . . 38
7.6 Diskussion . . . 39
7.7 Grenzen der Untersuchung . . . 40
7.8 Zwischenfazit . . . 40

8 Experimenteller Vergleich Python und C++ 42
8.1 Motivation und Fragestellung . . . 42
8.2 Implementierungen und Ausführungspfade . . . 43
8.3 Versuchsaufbau . . . 44
8.4 Metriken . . . 45
8.5 Ergebnisse der abgeschlossenen Messreihe . . . 46
8.6 Phasenanalyse und Skalierung mit der Kandidatenzahl . . . 47
8.7 Mikrobenchmark, funktionale Kontrolle und Diskussion . . . 50
8.8 Grenzen der Messreihe . . . 51
8.9 Bytecodegröße und Hardwareeffizienz . . . 52
8.10 Fremde Beschreibungen . . . 52
8.10.1 Was hier unter einer Beschreibung verstanden wird . . . 53
8.10.2 Anwendung auf die Messungen dieses Kapitels . . . 53
8.10.3 Anwendung auf die gesamte Arbeit . . . 54
8.10.4 Kriterien einer losen Kopplung . . . 56
8.10.5 Gegenargumente und Grenzen der Position . . . 56
8.10.6 Folgerungen für Gen-DB . . . 57
8.10.7 Zwischenfazit zur Kopplung . . . 58

9 Experimentelle Untersuchung Multi-Omics-Suche 59
9.1 Mehrschichtige Netzwerke und Suchrelationen . . . 59
9.1.1 Native C++-Implementierung des Schichtvergleichs . . . 60
9.2 Versuchsaufbau und Datengenerierung . . . 61
9.3 Korrektheit und differenzielle Prüfung . . . 62
9.4 Mikrobenchmark der Schichtvergleiche . . . 63
9.5 Ende-zu-Ende-Suche und Laufzeitergebnisse . . . 64
9.6 Worker-Skalierung auf vier Prozessorkernen . . . 64
9.7 Einfluss der Chunk-Größe . . . 65
9.8 Abgebrochene Datenbankgrößenreihe . . . 65
9.9 Einordnung, Grenzen und Folgerungen . . . 66
9.10 Parallelisierung . . . 66
9.10.1 Das Modell der parallelen Kandidatenvergleiche . . . 68
9.10.2 Korrektheit der parallelen Ausführung . . . 70
9.10.3 Zeitschranken der Vergleichsphase: Graham und serieller Zufluss . . . 71
9.10.4 Optimale Blockgröße . . . 73
9.10.5 Amdahl, Karp–Flatt und Gustafson für die Gesamtsuche . . . 74
9.10.6 Kernbeschleunigung und Parallelisierung ersetzen einander . . . 77

9.10.7 Reihenfolge der Kandidaten und Lastverteilung . . . 79
9.10.8 Grenzen der Parallelisierung . . . 81
9.10.9 Parallelität innerhalb eines Vergleichs: die Schichten . . . 83
9.10.10 Fazit . . . 84

10 Universelle Kodierung biologischer Strukturen 87
10.1 Der strukturunabhängige Kern . . . 87
10.2 Biologische Strukturen und ihre Kodierung . . . 89
10.3 Optimaler Vergleich zweier Stapel . . . 90
10.4 Invertierter Index: Suche ohne Einzelvergleiche . . . 91
10.5 Kernunabhängigkeit und Reichweite . . . 93
10.6 Vorbereitung des zweiten Schritts: Umsetzung in Python . . . 93
10.7 Zusammenfassung . . . 94

11 Experimentelle Untersuchung universelle Kodierung 95
11.1 Fragestellung und Hypothesen . . . 96
11.2 Untersuchte Varianten . . . 96
11.3 Versuchsaufbau . . . 98
11.3.1 Schema und Kodierung . . . 98
11.3.2 Daten und Ground Truth . . . 98
11.3.3 Anfragen . . . 98
11.3.4 Gleiche Trefferbedingung . . . 99
11.3.5 Messprotokoll . . . 99
11.4 Teilexperimente und Metriken . . . 100
11.4.1 Korrektheit (ohne Datenbank) . . . 100
11.4.2 Mikro-Benchmark (ohne Datenbank) . . . 101
11.4.3 Indexsuche im Speicher . . . 101
11.4.4 Ende-zu-Ende-Suche . . . 102
11.4.5 Worker, Chunk-Größe und Datenbankgröße . . . 102
11.5 Auswertung und Reproduzierbarkeit . . . 103
11.6 Ergebnisse . . . 103
11.6.1 Korrektheit . . . 104
11.6.2 Ende-zu-Ende-Suche . . . 104
11.6.3 Mikro-Benchmark . . . 105
11.6.4 Indexsuche im Speicher und Pflegekosten . . . 105
11.6.5 Nicht vorliegende Teilexperimente . . . 106
11.7 Diskussion und Grenzen . . . 106
11.8 Fazit . . . 108

12 Schluss 109
12.1 Zusammenfassung . . . 109
12.1.1 Ausgangspunkt und Zielsetzung . . . 109
12.1.2 Systemarchitektur und Datenmodell . . . 110
12.1.3 Formale Ergebnisse zum Suchverfahren . . . 110
12.1.4 Empirische Evaluation der Datenbank . . . 110
12.1.5 Experimenteller Vergleich mit VF2 . . . 111
12.1.6 Parallelisierung der Kandidatenvergleiche . . . 111
12.1.7 Vergleich der Python- und C++-Implementierung . . . 111
12.1.8 Multi-Omics: Suchsemantik und abgeschlossene Messreihen . . . 112

12.1.9 Formale Theorie der Parallelisierung . . . 112
12.1.10 Kopplung und Beschreibungsgrenzen . . . 114
12.1.11 Universelle Kodierung biologischer Strukturen und optimale Suche . 114
12.1.12 Experimentelle Untersuchung der universellen Kodierung . . . 117
12.1.13 Zentrale Erkenntnisse . . . 118
12.1.14 Abgleich mit den Beiträgen der Arbeit . . . 120
12.2 Kritische Würdigung und Grenzen . . . 121
12.3 Ausblick . . . 124
12.3.1 Vollständige Suche in beide Richtungen . . . 124
12.3.2 Differenzielle Tests der Vorauswahl und Eigenschaftstests . . . 125
12.3.3 Erweiterung auf gewichtete und beschriftete Subgraph-Isomorphie . 125
12.3.4 Indexierung über Signatur-Hashes . . . 125
12.3.5 Mehrdimensionale Indexstrukturen (R-Baum über (n_k, m_k)) . . . 126
12.3.6 Zusätzliche notwendige Bedingungen und Label-Indizes . . . 126
12.3.7 Weiterführende Parallelisierung . . . 126
12.3.8 Theoriegeleitete Weiterentwicklung der parallelen Suche . . . 128
12.3.9 Optimierung der Implementierung des Vergleichs . . . 129
12.3.10 GPU-Beschleunigung der Signatur- und LCS-Berechnung . . . 129
12.3.11 Hybridstrategie: Adaptive Verfahrenswahl nach Kandidatengröße . 130
12.3.12 Asynchrone Suche mit Fortschrittsanzeige und Teilergebnissen . . . 131
12.3.13 Approximative Suche und geschichtete Stichproben . . . 131
12.3.14 Caching häufiger Queries . . . 131
12.3.15 Inkrementelle Aktualisierung der Signatur-Indizes . . . 132
12.3.16 Betrieb, Reproduzierbarkeit und Beobachtbarkeit . . . 132
12.3.17 Wiederholung und Erweiterung der Parallelisierungsmessung . . . 132
12.3.18 Benchmark-Suite und kontinuierliche Performanzmessung . . . 133
12.3.19 Erweiterung des Vergleichsbenchmarks . . . 133
12.3.20 Formale Charakterisierung des Übergangsbereichs θ . . . 134
12.3.21 Visuelle Exploration und Graph-Layout . . . 135
12.3.22 Validierung mit realen Anwendungsfällen . . . 135
12.3.23 Gemeinsame Schnittstellenspezifikation und Konformitätstests . . . 135
12.3.24 Ausbau der Library-Anbindung des C++-Kerns . . . 136
12.3.25 Abgebrochene Datenbankgrößenreihe vervollständigen . . . 137
12.3.26 Biologische Semantik und reale Multi-Omics-Daten validieren . . . 137
12.3.27 Skalierung und Modellgrenzen mehrschichtiger Netzwerke . . . 137
12.3.28 Batch- und persistente Prozessschnittstellen als Übergangslösung . 138
12.3.29 Messprogramm für die Kopplungsfrage . . . 138
12.3.30 Prüfung künftiger Fremdmodule . . . 139
12.3.31 Umsetzung der Kodierung in Python . . . 140
12.3.32 Indextabelle, Wörterbücher und Pflege des Index . . . 140
12.3.33 Differenzielle Prüfung und Eigenschaftstests der Kodierung . . . 141
12.3.34 Auswertung, Wiederholung und Erweiterung der Messung . . . 141
12.3.35 Schemata für weitere biologische Strukturarten . . . 142
12.3.36 Koordinatensystem, Schemaänderung und Versionierung . . . 143
12.3.37 Fachliche Validierung der Suchsemantik . . . 143
12.3.38 Beziehung zur klassischen Subgraph-Isomorphie . . . 143
12.3.39 Offene theoretische Fragen . . . 144
12.3.40 Native Bibliothek und Anbindung an Experimente . . . 144

12.4 Priorisierung der nächsten Entwicklungsschritte 145
12.5 Abschließende Bemerkung 147

Kapitel 1

Einführung

Biologische Fragestellungen werden zunehmend mit großen, heterogenen Netzwerkdaten bearbeitet. Metabolische Pfade, Protein-Interaktionsnetzwerke und Gen-Regulationsnetzwerke bilden jeweils Beziehungen zwischen biologischen Entitäten ab; in integrierten Analysen kommen mehrere Omics-Ebenen hinzu. Eine wiederkehrende Aufgabe ist, ein kleines bekanntes Muster in einer Sammlung größerer Netzwerke wiederzufinden. Formal erhält eine Suche ein Query-Netzwerk G und soll gespeicherte Netzwerke G' identifizieren, in denen das Query nach einer festgelegten strukturellen Relation enthalten ist. Eine solche Suche muss nicht nur den Vergleich zweier Strukturen lösen: Bei großen Datenbeständen bestimmen auch die Auswahl der Kandidaten, ihre Übertragung an den Rechenkern, die Verteilung der Vergleiche und der Aufbau der Ergebnisse die Antwortzeit.

Diese Arbeit untersucht diese Fragen am System *Gen-DB*, einer Full-Stack-Anwendung mit PostgreSQL-Backend, FastAPI-REST-API und Web-Frontend. Der strukturelle Vergleich beruht auf dem *Subgraph Algorithmus* [1], der Adjazenzmatrizen über Spaltensignaturen und zyklische Teilsequenzen vergleicht und eine Laufzeit von $O(n^3)$ besitzt. Dabei ist die in dieser Arbeit verwendete Relation ausdrücklich von der klassischen Subgraph-Isomorphie zu unterscheiden: Die Resultate über Korrektheit und Laufzeit gelten für die jeweils definierte Relation und dürfen nicht als Beweis biologischer Relevanz oder als Lösung der allgemeinen Isomorphieaufgabe verstanden werden. Diese Abgrenzung ist wesentlich, damit formale Garantien, empirische Beobachtungen und offene fachliche Fragen getrennt beurteilt werden können.

Im Zentrum stehen mehrere aufeinander bezogene Fragen: Wie lässt sich der Vergleichskern in ein relationales System einbetten und durch eine Vorauswahl beschleunigen, ohne Treffer aus der abgesicherten Suchrichtung auszuschließen? Wie verhalten sich formale Komplexität und gemessene Laufzeit auf großen synthetischen Datenbeständen, und wie unterscheiden sich eine Python- und eine native C++-Implementierung? Was ändert sich, wenn mehrere Omics-Schichten kohärent oder unabhängig voneinander verglichen werden? Welche Korrektheits- und Laufzeitaussagen lassen sich für die parallele Verteilung vieler Kandidatenvergleiche treffen, und wo begrenzen Blockbildung, serielle Arbeit und die konkrete Hardware den Gewinn? Schließlich wird untersucht, ob sich die verschiedenen Strukturarten auf eine gemeinsame Kodierung und einen gemeinsamen Suchkern zurückführen lassen. Die Arbeit verbindet dazu Systementwurf, formale Beweise, kontrollierte synthetische Experimente und eine kritische Einordnung ihrer jeweiligen Reichweite.

Die Fragen zur Parallelisierung bilden dabei eine eigene Untersuchungsebene zwischen dem Algorithmuskern und dem Gesamtsystem. Die Laufzeit eines einzelnen Vergleichs, die Summe der Arbeit über alle Kandidaten, die Aufteilung dieser Arbeit auf Worker und die

serielle Datenbewegung sind verschiedene Größen und dürfen nicht zu einer einzigen Aussage wie „mehr Worker machen die Suche schneller“ zusammengezogen werden. Abschnitt 9.10 formuliert deshalb ein explizites Modell und trennt darin mathematische Garantien von Voraussetzungen über die Implementierung des Prozess-Pools sowie von empirischen Befunden. Insbesondere sind die Ergebnisgleichheit unter Reinheit des Vergleichs, die FIFO-Annahme der Planung, die aus dem Modell abgeleitete Blockgröße und die gemessenen Speedups Aussagen mit unterschiedlichem Geltungsbereich. Diese Trennung strukturiert zugleich den Ausblick: Jede Optimierung wird als bewiesene Folgerung, gemessener Hinweis oder noch offene Hypothese gekennzeichnet.

1.1 Beitrag dieser Arbeit

- Formale Beschreibung der Systemarchitektur von Gen-DB und Einbettung des Subgraph Algorithmus in eine relationale Datenbank.
- Formaler Nachweis der Korrektheit und Komplexität des Suchablaufs mit Vorauswahl und anschließendem Subgraph-Vergleich.
- Empirische Evaluation an einer synthetisch generierten Datenbank mit 1.000.000 biologischen Netzwerken.
- Veranschaulichung der Ergebnisse durch matplotlib-Diagramme: Laufzeitverteilung, Pre-Filter-Effektivität, Skalierungsverhalten und Trefferverteilung.
- Experimenteller Vergleich des Subgraph Algorithmus mit VF2 sowie Analyse der Ursachen der beobachteten Laufzeitunterschiede.
- Experimentelle Untersuchung der parallelen Ausführung der Kandidatenvergleiche mit 1, 2 und 3 Worker-Prozessen (Gesamtzeit, Speedup, Effizienz, Skalierung mit der Kandidatenzahl).
- Ausführliche formale Analyse der Parallelisierung (Abschnitt 9.10): Korrektheit der Ergebniszusammenführung unabhängig von Worker-Zahl, Blockgröße und Ausführungsreihenfolge; Schranken für dynamische Planung und seriellen Aufgabenzufluss sowie ein modellbasiertes Optimum für die Blockgröße; Einordnung mit Amdahl-, Karp–Flatt- und Gustafson-Modellen; Analyse von Kernbeschleunigung, Kandidatenreihenfolge, Global Interpreter Lock und Parallelität innerhalb eines Vergleichs. Die theoretischen Aussagen werden systematisch als bewiesen, voraussetzungsabhängig, gemessen oder noch nicht praktisch geprüft gekennzeichnet und mit den vorhandenen Worker- und Chunk-Messungen abgeglichen.
- Experimenteller Vergleich der Python- und C++-Implementierung des Subgraph Algorithmus über eine direkt im Worker-Prozess aufgerufene C++-Library, einschließlich Mikrobenchmark, Ende-zu-Ende-Suchmessung, Trefferabgleich und Analyse der verbleibenden Schnittstellenkosten.
- Erweiterung des Suchmodells auf geschichtete Multi-Omics-Netzwerke und experimentelle Prüfung der kohärenten und unabhängigen Schichtrelationen mit Python- und C++-Backends, synthetischer Ground Truth, Mikrobenchmark und Ende-zu-Ende-Messung.

– Universelle Kodierung biologischer Strukturen und optimale Suche (Abschnitt 10): Der Vergleichskern wird auf Stapel von Mengenfolgen (*Schichten* und *Komponenten*) beschränkt und von der Art der Daten getrennt. Formal gezeigt werden

  – die Definition der Relationen $\sqsubseteq_{\text{ind}}$ (unabhängig je Schicht) und $\sqsubseteq_{\text{coh}}$ (kohärent über alle Schichten) samt ihrer elementaren Eigenschaften, darunter die Nicht-Transitivität und das Fehlen eines zulässigen Kantenzahl-Filters (Lemmata 6 und 7);

  – die Definition von Schema, Struktur, Koordinatensystem und Kodierung $\Phi_{\Sigma,\kappa}$ für gerichtete, ungerichtete, gewichtete und attributierte Netzwerke, mehrstellige Reaktionen, Hypergraphen, Pfade, Ontologien und mehrschichtige Netzwerke, die Verlustfreiheit der Kodierung (Satz 19) und ihr Aufwand von $O(|S| \log |S|)$ (Satz 20);

  – der Vergleich zweier Stapel in erwarteter Linearzeit (Satz 21) mit passender unterer Schranke $\Omega(m + m')$ (Satz 22), also bis auf einen konstanten Faktor optimal;

  – ein invertierter Paarindex, dessen Suche das exakte Ergebnis ohne Einzelvergleiche liefert und dessen Aufwand von der Trefferlast und nicht von der Datenbankgröße N abhängt (Sätze 23 und 24, Korollar 12);

  – die Kernunabhängigkeit (Korollar 13): Für eine neue biologische Struktur genügt ein neues Schema; Vergleich, Index und Beweise bleiben unverändert;

  – die ausdrücklichen Grenzen der Aussagen: die Optimalität gilt für die definierte Relation und nicht für die klassische Subgraph-Isomorphie (Bemerkung 21), und die fachliche Validität einer Kodierung ist nicht mitbewiesen (Bemerkung 22); zudem die Umsetzung in Python (Kern, Kodierung, Paarindex als SQL-Tabelle) und ihre Vorbereitung für die experimentelle Prüfung.

– Experimentelle Untersuchung der universellen Kodierung (Abschnitt 11): Ein reproduzierbares Experiment vergleicht vier Varianten (Paarindex in SQL, Einzelvergleiche im Kern, bisherige Multi-Omics-Pipeline in Python und mit der nativen Bibliothek) bei gleichem Schema, gleichen Daten und gleicher Trefferbedingung. Festgelegt sind

  – neun Hypothesen H1 bis H9 mit Herleitung aus den Sätzen und mit Kriterien, an denen sie scheitern können (Tabelle 11.1);

  – ein Versuchsaufbau mit 20.000 Zufallsnetzwerken, einem Anfragegitter aus 56 Anfragen und eingebetteten Netzwerken mit bekannter Lösung als Ground Truth (identisch, kohärent, nur unabhängig eingebettet);

  – sieben Teilexperimente: differenzielle Korrektheitsprüfung von Kern und Index, Mikro-Benchmark der Einzelvergleiche, Indexsuche im Speicher mit Anpassung an die Trefferlast und Exponent in N, Ende-zu-Ende-Suche mit Phasenzerlegung, Worker-Skalierung, Chunk-Größe und Datenbankgröße mit Speicherbedarf und Einfügezeit des Index;

  – die Auswertung mit geometrischem Mittel der Speedups und Bootstrap-Intervall, Konfidenzintervallen, linearen Anpassungen und Exponenten sowie die Grenzen der Untersuchung.

– Zusammenfassung, kritische Würdigung der Grenzen und ausführlicher, nach Themenfeldern geordneter Ausblick auf Erweiterungen (Korrektheit, Indexierung, Beschleunigung der Vergleiche, Systemebene, Evaluation, Anwendung, Kopplung und Schnittstellen, Multi-Omics-Suche sowie Umsetzung, Messung und Erweiterung der universellen Kodierung) mit einer Priorisierung der nächsten Schritte.

1.2 Aufbau der Arbeit

Die Arbeit gliedert sich in aufeinander aufbauende Abschnitte: von den theoretischen Grundlagen über System und Suchverfahren zu empirischen Untersuchungen, einer vertieften Theorie der Parallelisierung und einer verallgemeinernden Theorie der universellen Kodierung, bevor Zusammenfassung, Grenzen und Ausblick die Ergebnisse einordnen.

Abschnitt 2 führt in biologische Netzwerke, ihre Adjazenzmatrix-Repräsentation, die Spaltensignaturen und den Subgraph Algorithmus ein, dessen Laufzeit von $O(n^3)$ die Grundlage aller späteren Betrachtungen ist. Abschnitt 3 beschreibt die Systemarchitektur und das Datenmodell von Gen-DB und erläutert anhand von UML-Paket- und Klassendiagrammen den Aufbau der Anwendung. Abschnitt 4 formalisiert die Suche, insbesondere die Vorauswahl anhand von Knoten- und Kantenzahl, und untersucht deren Korrektheit und Laufzeitkomplexität; ein Serverprotokoll veranschaulicht dort zusätzlich den Ablauf einer konkreten Suche.

Der zweite Teil der Arbeit ist empirisch. Abschnitt 5 stellt Versuchsaufbau und Ergebnisse zur generierten Datenbank mit einer Million Netzwerken sowie zum Verhalten der Vorauswahl vor. Abschnitt 6 vergleicht den Subgraph Algorithmus mit VF2, analysiert die beobachteten Laufzeitunterschiede und ordnet beide Verfahren in die Fachliteratur ein. Abschnitt 7 untersucht die parallele Ausführung der Kandidatenvergleiche mit einem, zwei und drei Worker-Prozessen; ausgewertet werden Gesamtzeit, Speedup, Effizienz und die Abhängigkeit von der Kandidatenzahl. Abschnitt 8 vergleicht anschließend Python mit dem direkten Aufruf einer C++-Library im Worker-Prozess und trennt im Mikrobenchmark und in der Ende-zu-Ende-Messung die Kosten des Algorithmuskerns von den übrigen Schnittstellenkosten.

Abschnitt 9 erweitert die Suche auf geschichtete Multi-Omics-Netzwerke, definiert die kohärente und unabhängige Schichtrelation und wertet abgeschlossene Korrektheits-, Mikrobenchmark-, Such-, Worker- und Chunk-Messungen mit synthetischer Ground Truth aus. Die Datenbankgrößenreihe wurde nicht vollständig abgeschlossen und wird daher nicht als vollständiges Skalierungsergebnis dargestellt. Innerhalb dieses Kapitels vertieft Abschnitt 9.10 die zuvor gemessene Parallelisierung theoretisch. Er modelliert die drei Phasen Laden, Vergleichen und Ergebnisaufbau, definiert Aufgaben- und Blockkosten und beweist unter der Reinheitsannahme A1 die Gleichheit der sequentiellen und parallelen Ergebnisse unabhängig von Worker-Zahl, Blockgröße und Fertigstellungsreihenfolge. Unter den gesondert ausgewiesenen Planungsannahmen leitet er eine Graham-Schranke samt scharfem Beispiel, eine Schranke mit seriellem Zufluss sowie ein Modelloptimum für die Blockgröße her. Für die Gesamtlaufzeit ordnet er Amdahl-, Karp–Flatt- und Gustafson-Maße ein, erklärt den Einfluss der Kernbeschleunigung auf den seriellen Anteil, untersucht Reihenfolge und Lastverteilung der Kandidaten und zeigt Grenzen durch Hardware, Global Interpreter Lock und verfügbare Blockzahl. Die vorgeschlagene Parallelität innerhalb eines Vergleichs wird als theoretische Möglichkeit und nicht als implementierter oder gemessener Gewinn behandelt. Den Abschluss des Multi-Omics-Kapitels bildet Abschnitt 8.10, der Messergebnisse und lose Kopplung gemeinsam einordnet.

Nach den Experimenten kehrt die Arbeit in Abschnitt 10 zur Theorie zurück und verallgemeinert die bisherige Vorgehensweise. Bisher wurde für jede neue Art biologischer Daten ein eigener Vergleichspfad formuliert und im nativen Kern umgesetzt: zuerst für das Einzelnetzwerk (Abschnitt 4), dann für das mehrschichtige Netzwerk (Abschnitt 9). Der neue Abschnitt trennt die Suche in einen strukturunabhängigen Kern, der nur Stapel von Mengenfolgen kennt, und eine Kodierung, die jede biologische Struktur eines beliebigen Schemas in einen solchen Stapel überführt. Er besteht aus sieben Unterabschnitten:

- Der *strukturunabhängige Kern* (Abschnitt 10.1) definiert Schichten, Stapel, Paarmengen und die beiden Enthaltenseinsrelationen $\sqsubseteq_{\mathrm{ind}}$ und $\sqsubseteq_{\mathrm{coh}}$, zeigt den Matrixstapel als Spezialfall der bisherigen Multi-Omics-Relation und beweist ihre elementaren Eigenschaften, darunter die fehlende Transitivität und die Unzulässigkeit des Kantenzahl-Filters.

- Der Unterabschnitt *Biologische Strukturen und ihre Kodierung* (Abschnitt 10.2) führt Schema, Struktur, Koordinatensystem und die Kodierung $\Phi_{\Sigma,\kappa}$ ein, belegt sie an einem Beispiel mit einem regulatorischen Netzwerk und einer dreistelligen Reaktion und beweist Verlustfreiheit und fast linearen Aufwand. Bemerkungen zu lokalen und globalen Koordinaten sowie zur Schemaänderung ergänzen ihn.

- Der *optimale Vergleich zweier Stapel* (Abschnitt 10.3) beweist eine obere Schranke in erwarteter Linearzeit und eine untere Schranke durch ein Gegenspielerargument, sodass der Vergleich bis auf einen konstanten Faktor optimal ist.

- Der *invertierte Index* (Abschnitt 10.4) definiert den Paarindex und das Suchergebnis, beweist die Korrektheit der Suche ohne Einzelvergleiche und ihre Laufzeit in Abhängigkeit von der Trefferlast, folgert die Unabhängigkeit von N unter einer Häufigkeitsannahme, behandelt die umgekehrte Suchrichtung und setzt den Index in Beziehung zur bisherigen Pipeline.

- *Kernunabhängigkeit und Reichweite* (Abschnitt 10.5) hält fest, dass alle Sätze für jedes Schema gelten, ordnet die native C++-Bibliothek als optionalen Beschleuniger ein und benennt die Grenzen: die NP-Vollständigkeit des klassischen Teilgraph-Isomorphieproblems und die offene fachliche Semantik.

- Die *Vorbereitung des zweiten Schritts* (Abschnitt 10.6) beschreibt die Umsetzung in Python: Schnittstelle je Strukturart, Entitätsregister, Indextabelle, die aus den Sätzen folgenden Tests und das noch ausstehende Experiment.

- Eine kurze *Zusammenfassung* schließt den Abschnitt.

Der Abschnitt enthält ausschließlich formale Aussagen und keine Messungen; die Laufzeitaussagen sind Folgerungen aus den Sätzen. Sie werden im anschließenden Abschnitt 11 experimentell geprüft.

Abschnitt 11 ist der experimentelle Gegenpart zu Abschnitt 10 und folgt dem Aufbau der früheren Experimente. Er formuliert aus den Sätzen vier Fragen (Korrektheit, Laufzeit, Skalierung, Kosten des Index) und neun Hypothesen mit Widerlegungskriterien (Abschnitt 11.1), beschreibt die vier verglichenen Varianten (Abschnitt 11.2) und den Versuchsaufbau mit Schema, Daten, Ground Truth, Anfragegitter, gleicher Trefferbedingung und Messprotokoll (Abschnitt 11.3), legt die sieben Teilexperimente mit ihren Metriken fest (Abschnitt 11.4), erläutert Statistik, Ausgabedateien und Aufruf (Abschnitt 11.5) und

bindet die Ergebnisse ein (Abschnitt 11.6). Eine Diskussion legt fest, was jedes mögliche Ergebnis für die Theorie bedeuten würde, und benennt die Grenzen (Abschnitt 11.7); ein Fazit schließt den Abschnitt (Abschnitt 11.8).

Das abschließende Kapitel „Schluss“ (Abschnitt 12) besteht aus fünf Teilen. Es fasst die Ergebnisse aller Abschnitte zusammen, einschließlich der universellen Kodierung und ihrer experimentellen Untersuchung (Abschnitt 12.1, dort insbesondere die Abschnitte 12.1.11 und 12.1.12), benennt die Grenzen der Arbeit (Abschnitt 12.2), entwickelt einen ausführlichen, nach neun Themenfeldern A bis I geordneten Ausblick mit möglichen Erweiterungen und offenen Forschungsfragen, von denen das Themenfeld I die Auswertung, Vertiefung und Erweiterung der universellen Kodierung nach dem Experiment behandelt (Abschnitt 12.3), schlägt eine Reihenfolge für die nächsten Entwicklungsschritte vor (Abschnitt 12.4) und schließt mit einer Gesamteinschätzung (Abschnitt 12.5).

Kapitel 2

Grundlagen

2.1 Graphen und Adjazenzmatrizen

Definition 1 (Biologisches Netzwerk)**.** Ein biologisches Netzwerk ist ein gerichteter Graph $G = (V, E)$ mit Knotenmenge $V = \{v_1, \ldots, v_n\}$ (Metabolite, Proteine oder Gene) und Kantenmenge $E \subseteq V \times V$ (Reaktionen, Interaktionen oder Regulationsbeziehungen). Jedem Knoten v_i ist ein Label $\ell(v_i) \in \Sigma_{\text{Bio}}$ aus einem biologischen Vokabular zugeordnet.

Definition 2 (Adjazenzmatrix-Repräsentation)**.** Ein Netzwerk G mit n Knoten wird in Gen-DB als Tupel

$$G = (L, A) \quad \text{mit } L \in \Sigma_{\text{Bio}}^n,\ A \in \{0,1\}^{n \times n}$$

gespeichert, wobei L der Vektor der Knotenlabels und A die Adjazenzmatrix mit $A_{ij} = 1 \Leftrightarrow (v_i, v_j) \in E$ ist.

2.2 Signaturen und Subgraph-Relation

Definition 3 (Spaltensignatur)**.** Für eine Adjazenzmatrix $A \in \{0,1\}^{n \times n}$ ist die Signatur der Spalte j definiert als

$$\sigma_j(A) = \sum_{i=0}^{n-1} A_{ij} \cdot 2^i + j \cdot 2^n.$$

Definition 4 (Signatur-Array und Hash)**.** Das Signatur-Array von A ist $\Sigma(A) = (\sigma_0(A), \ldots, \sigma_{n-1}(A))$. Der Signatur-Hash ist

$$h(A) = \text{SHA256}\Big(\text{str}(\Sigma(A))\Big).$$

Lemma 1 (Injektivität der Signaturfunktion)**.** Die Abbildung $\sigma : \{0,1\}^n \times \{0, \ldots, n-1\} \to \mathbb{N}$, $(c, j) \mapsto \sum_{i=0}^{n-1} c_i 2^i + j 2^n$ ist injektiv.

Beweis. Sei $\sigma(c, j) = \sigma(c', j')$. Da $\sum_{i=0}^{n-1} c_i 2^i \in [0, 2^n - 1]$ und $j2^n, j'2^n$ Vielfache von 2^n sind, folgt aus der Eindeutigkeit der Division mit Rest (Divisionsalgorithmus für 2^n), dass $j = j'$ und somit $\sum c_i 2^i = \sum c'_i 2^i$. Da die Binärdarstellung eindeutig ist, folgt $c = c'$. □

Definition 5 (Subgraph-Relation, zyklisch)**.** Für Adjazenzmatrizen $A \in \{0,1\}^{n_A \times n_A}$ und $B \in \{0,1\}^{n_B \times n_B}$ mit $n_A \leq n_B$ schreiben wir $A \preceq B$, wenn die vollständige Zeilenkomponentensequenz $\rho(A)$ eine Teilsequenz von $\rho(B)$ nach einer zyklischen Rotation ist. Äquivalent dazu hat eine längste gemeinsame Teilsequenz der beiden Folgen die Länge n_A. Diese Relation ist nicht die klassische Subgraph-Isomorphie; sie fixiert die Knotenordnung bis auf zyklische Rotation.

2.3 Der Subgraph Algorithmus

Der formale Subgraph-Vergleich [1] berechnet für zwei Matrizen A, B in folgenden Schritten, ob $A \preceq B$, $B \preceq A$, beide äquivalent oder unvergleichbar sind:

1. Validiere die Eingaben und berechne $\Sigma(A)$ und $\Sigma(B)$ — Aufwand $O(n_A^2 + n_B^2)$.
2. Entferne die Positionskodierung aus jeder Signatur. Es entstehen die Zeilenkomponenten $\rho(A) = (\sigma_j(A) \bmod 2^{n_A})_j$ und analog $\rho(B)$.
3. Prüfe bei zulässiger Größenordnung jede zyklische Rotation der jeweils größeren Folge darauf, ob die vollständige kleinere Folge als Teilsequenz auftritt. Dies entspricht einer dynamischen LCS-Prüfung mit Schwellwert n_{klein} und kostet $O(n_A n_B)$ pro Rotation.
4. Führe den Test in beiden Richtungen aus und leite aus den beiden booleschen Ergebnissen einen Ergebniswert ab.

Satz 1 (Laufzeit des Subgraph Algorithmus)**.** Für $n = \max(n_A, n_B)$ benötigt der Subgraph Algorithmus $O(n^3)$ Zeit.

Beweis. Schritt 1 ist $O(n^2)$. Schritt 3 besteht aus höchstens n Rotationen je Richtung, jede mit einem LCS-Vergleich in $O(n_A n_B) \subseteq O(n^2)$, insgesamt also $O(n^3)$. Schritt 2 ist $O(n)$. Die Summe ist $O(n^2) + O(n) + O(n^3) = O(n^3)$. □

Eine vollständige Optimalitätsanalyse $\Omega(n^{3-\varepsilon})$ unter SETH findet sich in [1]; sie wird hier als gegeben vorausgesetzt und in Abschnitt 4 auf den Datenbankkontext übertragen.

2.3.1 Konkrete Umsetzung in der C++17-Bibliothek

Der native Rechenkern ist als C++17-Bibliothek `subgraphlib` im Projekt `csubgraph` implementiert. Die öffentliche Schnittstelle steht in `SubgraphAlgorithm.h`; die Umsetzung liegt in `SubgraphAlgorithm.cpp`. Matrizen und Folgen werden als `std::vector<std::vector<int>>` beziehungsweise `std::vector<uint64_t>` repräsentiert. Die zentrale Funktion `compareGraphs` nimmt beide Matrizen als konstante Referenzen entgegen, gibt einen typisierten `Result`-Wert zurück und wirft bei ungültiger Eingabe `std::invalid_argument`. Die Daten werden somit nicht über selbst verwaltete Zeiger übergeben; die temporären Signatur-, Rotations- und DP-Strukturen besitzen automatische Lebensdauer.

Signaturen als Bitmasken

`calculateSignatures` verarbeitet jede Spalte j und setzt für jede eingehende Kante aus Zeile i das Bit i der Zeilenkomponente. Anschließend wird der Spaltenindex oberhalb dieser n Bits abgelegt. Für die Kette $0 \to 1 \to 2$ mit

$$A = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix}$$

ergeben sich die Zeilenkomponenten $(0, 1, 2)$: Spalte 0 enthält keine Eins, Spalte 1 das Bit 0, Spalte 2 das Bit 1. Die vollständigen Signaturen sind $(0, 9, 18)$, weil die Positionsanteile $0 \cdot 2^3$, $1 \cdot 2^3$ und $2 \cdot 2^3$ addiert werden. Der entsprechende Kern der C++-Schleife lautet:

Listing 2.1: Bitweise Berechnung der Spaltensignaturen

```
for (std::size_t j = 0; j < n; ++j) {
    std::uint64_t rows = 0;
    for (std::size_t i = 0; i < n; ++i) {
        if (matrix[i][j] != 0) {
            rows |= (std::uint64_t{1} << i);
        }
    }
    const std::uint64_t position =
        static_cast<std::uint64_t>(j) << n;
    signatures.push_back(rows + position);
}
```

Die Addition ist hier eine Konkatenation zweier nicht überlappender Bitbereiche: Die Kantenbits liegen unterhalb von Position n, die Positionskodierung beginnt ab diesem Bit. Die Funktion `extractRowComponents` entfernt den oberen Anteil durch Maskierung mit $(2^n - 1)$. `compareGraphs` vergleicht anschließend diese Komponentenfolgen; die Positionskodierung trägt dazu bei, die Signaturen innerhalb einer Matrix unterscheidbar zu machen, gehört aber nicht zum Mustervergleich.

Listing 2.2: Extraktion der unteren n Bits aus jeder Signatur

```
const std::uint64_t mask = (std::uint64_t{1} << n) - 1;
for (const std::uint64_t signature : signatures) {
    rowComponents.push_back(signature & mask);
}
```

Validierung und Identität

Vor der Berechnung prüft `isValidAdjacencyMatrix`, dass eine Matrix nicht leer, quadratisch und binär ist. `compareGraphs` validiert beide Argumente und gibt bei Verletzung eine beschreibende Ausnahme aus. Danach werden Signaturen und Zeilenkomponenten berechnet. Haben beide Graphen dieselbe Knotenzahl und dieselbe Zeilenkomponentenfolge, wird sofort `IDENTICAL` zurückgegeben. Dieser Sonderfall verhindert Rotations- und DP-Arbeit für exakt gleiche, geordnete Repräsentationen. Er ist bewusst nicht gleichbedeutend mit Isomorphie unter beliebiger Knotenpermutation.

Längste gemeinsame zusammenhängende Teilfolge

Die Funktion heißt im Quellcode `computeLCS`, berechnet aber nicht die klassische *Longest Common Subsequence*, bei der Elemente übersprungen werden dürfen. Sie berechnet die längste gemeinsame *zusammenhängende* Teilfolge (Longest Common Substring). In der dynamischen Tabelle wird ein Treffer nur diagonal fortgesetzt; bei Ungleichheit wird auf null zurückgesetzt:

Listing 2.3: Dynamische Programmierung für zusammenhängende Übereinstimmungen

```
if (seqA[i - 1] == seqB[j - 1]) {
    dp[i][j] = dp[i - 1][j - 1] + 1;
    maxLength = std::max(maxLength, dp[i][j]);
} else {
    dp[i][j] = 0;
```

```
}
```

Ein Beispiel macht den Unterschied sichtbar: Für $(1, 2, 3, 4)$ und $(1, 3, 4)$ ist die längste gemeinsame Teilsequenz ohne Zusammenhängendheitsforderung $(1, 3, 4)$ der Länge drei; die längste gemeinsame *zusammenhängende* Teilfolge ist dagegen $(3, 4)$ der Länge zwei. Die native Einzelgraph-Implementierung akzeptiert eine Richtung, sobald dieser zusammenhängende Höchstwert mindestens zwei beträgt. Dafür reserviert sie eine vollständige Tabelle der Größe $(n_A + 1)(n_B + 1)$; der Speicherbedarf dieses Schritts ist $O(n_A n_B)$.

Bemerkung 1 (Abweichung der nativen Einzelgraph-Implementierung von der formalen Relation)**.** Die tatsächlich vorliegende Funktion `SubgraphAlgorithm::compareGraphs` setzt nicht die in Abschnitt 2 definierte vollständige Teilsequenzrelation um. Sie prüft mit `computeLCS` lediglich, ob nach einer Rotation eine gemeinsame *zusammenhängende Teilfolge* der Länge mindestens zwei existiert. Im Folgenden bezeichnen wir diese schwächere, vom C++-Code entschiedene Relation mit $\preceq_{\text{pair}}$. Weder muss die ganze Folge $\rho(A)$ auftreten, noch berechnet `computeLCS` eine nicht zusammenhängende LCS. Die folgende Definition gibt die Implementierung wieder:

$$A \preceq_{\text{pair}} B \iff n_A \le n_B \ \wedge\ \exists r :\ \mathrm{LCSubstr}\Big(\rho(A), \rho(B^{(r)})\Big) \ge 2.$$

Damit darf der Beweis der notwendigen Knoten- und Kantenzahlbedingung aus Lemma 2 nicht auf $\preceq_{\text{pair}}$ übertragen werden. Ein konkretes Gegenbeispiel ist

$$A = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{pmatrix}, \qquad B = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

Hier sind $\rho(A) = (0, 0, 7)$ und $\rho(B) = (0, 0, 0)$. Somit gilt $A \preceq_{\text{pair}} B$, weil beide Folgen das benachbarte Paar $(0, 0)$ enthalten, obwohl $m_A = 3 > 0 = m_B$. Ein SQL-Filter, der für Query A Kandidaten mit weniger als drei Kanten entfernt, würde diesen Treffer des C++-Kriteriums ausschließen. Die formale Vollständigkeitsgarantie des Pre-Filters gilt daher nur für die oben definierte Relation $\preceq$, nicht für den durch `compareGraphs` implementierten Einzelgraph-Test. Dass die Backends in den dokumentierten Messreihen dieselben Treffer lieferten, ist ein empirischer Befund für diese Eingaben und kein allgemeiner Gleichheitsbeweis der Relationen.

Zyklische Prüfung, beide Richtungen und Ergebnis

`rotateSequence` erzeugt eine Rechtsrotation. Für eine Folge (a, b, c, d) liefert eine Rotation um eins (d, a, b, c). Der Rotationswert wird modulo der Folgenlänge reduziert; eine leere Folge wird unverändert zurückgegeben. Für $A \preceq B$ betrachtet `compareGraphs` nur dann Rotationen von B, wenn $n_B \ge n_A$. Für $B \preceq A$ wird entsprechend die umgekehrte Richtung nur bei $n_A \ge n_B$ geprüft. Sobald eine Rotation den Schwellwert erreicht, beendet eine `break`-Anweisung die Suche in dieser Richtung:

Listing 2.4: Abbruch nach einem passenden zyklischen Vergleich

```
if (nB >= nA) {
    for (std::size_t r = 0; r < nB; ++r) {
        const auto rotatedB = rotateSequence(rowCompB, r);
        if (computeLCS(rowCompA, rotatedB) >= 2) {
            isSubgraphAinB = true;
```

```
            break;
        }
    }
}
```

Die Routine prüft danach auch die Gegenrichtung; eine positive Antwort in einer Richtung überspringt den anderen Test also nicht. Der Rückgabewert ist eine Auswahlentscheidung und muss deshalb von der Relation selbst unterschieden werden:

- `KEEP_A`: $B \preceq_{\text{pair}} A$, aber nicht $A \preceq_{\text{pair}} B$; behalten wird der umfassendere Graph A.
- `KEEP_B`: $A \preceq_{\text{pair}} B$, aber nicht $B \preceq_{\text{pair}} A$; behalten wird B.
- `KEEP_BOTH`: keine der beiden Richtungen wurde gefunden.
- `IDENTICAL`: gleiche Knotenzahl und gleiche Zeilenkomponenten, bereits vor den Relationstests erkannt.
- `EQUAL_KEEP_A` beziehungsweise `EQUAL_KEEP_B`: beide Richtungen treffen zu und die Knotenzahlen sind gleich; dann entscheidet die Gesamtzahl der Einsen in den Matrizen, bei Gleichstand wird A behalten.

Bei wechselseitiger Enthaltung mit unterschiedlicher Knotenzahl behält der Code den Graphen mit mehr Knoten (`KEEP_A` oder `KEEP_B`). Die Enum-Werte beschreiben somit, welche Eingabe weitergeführt werden soll, und nicht einfach die Richtung eines einzelnen Relationstests.

Aufwand und Implementierungsgrenzen

Für zwei Graphen mit höchstens n Knoten benötigt die Signaturberechnung $O(n^2)$ Zeit. Im ungünstigsten Fall werden je Richtung bis zu n Rotationen geprüft; jede dynamische Programmierung kostet $O(n^2)$ Zeit und Speicher. Damit bleibt die obere Schranke $O(n^3)$ Zeit und $O(n^2)$ temporärer Speicher. Frühe Abbrüche können typische Fälle verkürzen, ändern aber diese Worst-Case-Schranke nicht. Die Umsetzung verwendet `uint64_t` für die Bitmasken. `MultiOmics` weist Eingaben mit mehr als 63 Knoten zurück; die Einzelgraph-Schnittstelle `compareGraphs` besitzt dagegen keine entsprechende Knotenzahlprüfung. Für $n \geq 64$ sind die Bitverschiebungen nicht sicher definiert beziehungsweise nicht mehr als vorgesehene Kodierung verwendbar. Die theoretischen und experimentellen Aussagen zur nativen Implementierung sind daher im darstellbaren Größenbereich zu lesen; größere Graphen erfordern eine breitere Repräsentation und explizite Eingabegrenzen.

Die Bibliothek enthält außerdem ein separates Kommandozeilenprogramm, das JSON-Matrizen einliest, `compareGraphs` aufruft und Ergebniscode sowie Fehler als JSON ausgibt. Für Gen-DB wird im Backend-Experiment jedoch der direkte Library-Pfad über einen C-Wrapper und `ctypes` verwendet (Abschnitt 8); dabei wird die Library pro Worker-Prozess geladen, nicht für jeden Kandidaten ein neuer Prozess gestartet. Diese Unterscheidung ist für die Messung wichtig, denn CLI-Prozessstart und JSON-Serialisierung wären zusätzliche Kosten außerhalb des hier beschriebenen Algorithmuskerns.

Kapitel 3

Systemarchitektur von Gen-DB

3.1 Komponentenübersicht

Gen-DB besteht aus drei Schichten:

- **Frontend** (HTML/CSS/JavaScript): Erstellung, Anzeige und Suche von Netzwerken über REST-Aufrufe.
- **Backend** (FastAPI, Python): REST-Endpunkte `/api/networks`, `/api/networks/{id}`, `/api/networks/search`; Modul `crud.py` implementiert Signaturberechnung und ruft den Subgraph Algorithmus auf.
- **Datenbank** (PostgreSQL): Tabellen `biological_networks` (Metadaten: Name, Typ, Organismus, Knoten-/Kantenzahl) und `network_matrices` (Knotenlabels, Adjazenzmatrix, Signatur-Array, Signatur-Hash).

Definition 6 (Datenbankzustand)**.** Der Datenbankzustand zum Zeitpunkt t ist eine endliche Menge

$$\mathcal{D}_t = \{(id_k, L_k, A_k, \Sigma_k, h_k) : k = 1, \ldots, N_t\},$$

wobei $N_t = |\mathcal{D}_t|$ die Anzahl gespeicherter Netzwerke ist. In der vorliegenden Evaluation gilt $N_t \approx 1.000.002$.

3.2 Datenmodell

Die Speicherung erfolgt normalisiert über zwei 1:1-verknüpfte Relationen:

$$\texttt{biological_networks}(\underline{\text{network_id}}, \text{name}, \text{type}, \text{organism}, \text{node_count}, \text{edge_count})$$

$$\texttt{network_matrices}(\underline{\text{network_id}}, L, A, \Sigma, h)$$

Die Spalten `node_count` und `edge_count` entsprechen $n_k = |V_k|$ bzw. $m_k = |E_k| = \sum_{i,j}(A_k)_{ij}$ und werden für das Pre-Filtering (Abschnitt 4.2) materialisiert, um eine vollständige Tabellenabtastung zu vermeiden.

3.3 UML-Architekturmodelle von Gen-DB

Die beiden UML-Diagramme in den Abbildungen 3.1 und 3.2 dokumentieren die Struktur von Gen-DB auf Paket- und Klassenebene und bilden damit die konzeptionelle Grundlage für die in den folgenden Abschnitten formalisierten Such- und Datenbankprozesse.

Das Paketdiagramm zeigt die fachliche und technische Zerlegung der Anwendung in drei logisch getrennte Bausteine: das Web-Frontend, das Backend mit den API- und Suchkomponenten und die persistente Datenspeicherung in PostgreSQL. Dieser Aufbau ist keine rein dokumentarische Anordnung, sondern entspricht einem klaren Entwurfsmuster für skalierbare Datenbankanwendungen: Die Präsentationsschicht übernimmt keine Datenlogik, sondern stellt lediglich die Interaktion mit dem Benutzer bereit; die Logikschicht kapselt die Kernfunktionalität der Signaturberechnung, des Pre-Filters und des Subgraph-Checks; die Datenbankschicht speichert neben den Netzwerkinhalten auch die maschinenlesbaren Repräsentationen, die für die Suche benötigt werden. Aus Sicht der Systemarchitektur ist deshalb besonders wichtig, dass die Datenflussrichtung zwischen den Schichten eindeutig ist: Ein Nutzer erzeugt oder lädt ein biologisches Netzwerk über das Frontend; die Anfrage wird als API-Request an das Backend weitergeleitet; dort werden Netzwerkdaten interpretiert, normalisiert und in Form von Knotenlabels und Adjazenzmatrizen für den Subgraph Algorithmus aufbereitet; danach werden die passenden Kandidaten mit SQL-Filterkriterien aus der Datenbank abgerufen und im Ergebnis an die Präsentationsschicht zurückgegeben. Das Paketdiagramm macht damit sichtbar, dass Gen-DB als System nicht nur aus einer einzelnen Anwendung besteht, sondern aus einer zusammengesetzten, modularen Architektur, in der fachliche Trennung und technische Verantwortlichkeit klar voneinander abgegrenzt sind.

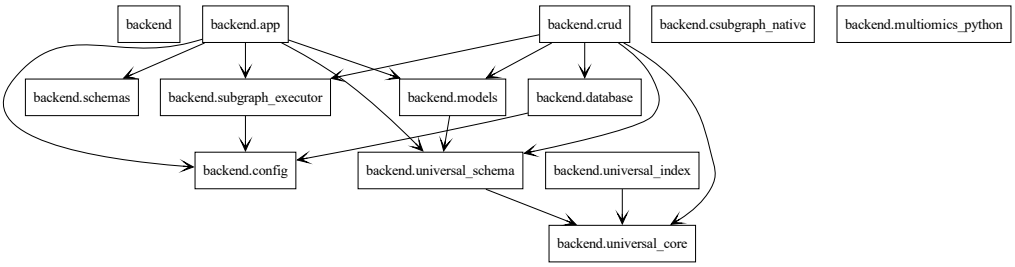


Abbildung 3.1: UML-Paketdiagramm der Gen-DB-Architektur.

Der in der übrigen Arbeit beschriebene Suchablauf baut genau auf dieser Struktur auf. Die Datenbank hält nicht nur die Netzwerkmetadaten wie Name, Organismus und Typ, sondern auch die strukturellen Repräsentationen in einer für schnelle Filterung geeigneten Form. Die Backend-Komponenten sind dafür zuständig, die Suchanfrage in eine Folge von Vorfilter-, Vergleichs- und Ergebnisphasen zu übersetzen. Das Paketdiagramm zeigt somit die organisatorische Ebene des Systems: welche Komponenten miteinander interagieren, welche Verantwortlichkeiten sie übernehmen und wo die fachlichen Grenzen verlaufen. Diese Abgrenzung ist für das Verständnis der späteren Komplexitäts- und Korrektheitsbeweise entscheidend, weil sie präzise zeigt, wo der algorithmische Kern liegt und wo rein relationale oder benutzerorientierte Aufgaben ausgelagert werden.

Das Klassendiagramm ergänzt diese Sicht um den objektorientierten bzw. modellorientierten Entwurf der einzelnen Daten- und Dienstekomponenten. Es macht deutlich, dass

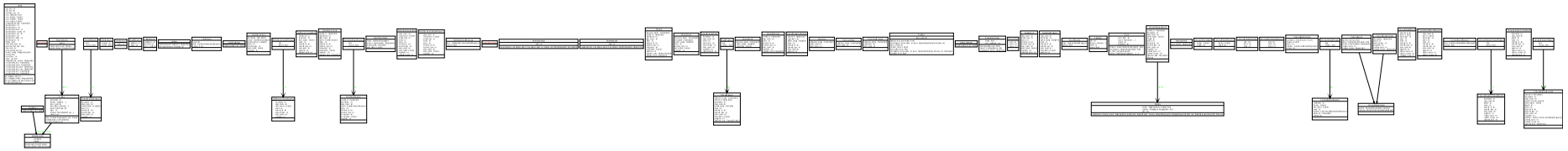


Abbildung 3.2: UML-Klassendiagramm der Gen-DB-Datenmodellierung und Suche.

die Systemlogik nicht auf lose, unstrukturierte Listen von Werten beruht, sondern auf semantisch geordneten Entitäten: Netzwerkobjekte, Knoten- und Kanteninformationen, Matrixrepräsentationen, Signaturdaten und Suchresultate sind als getrennte Strukturen modelliert. Ein wesentlicher Entwurfspunkt ist die Trennung zwischen der biologischen Netzwerkanalyse und der relationalen Persistenz. Während die Datenbankrelationen die langfristige Speicherung, Indizierung und Abfrage von Netzwerken garantieren, repräsentiert das Klassendiagramm die fachlichen Informationen im Laufzeitkontext der Anwendung. So sind etwa Metadaten wie Typ, Organismus und Knotenzahl in der Klasse bzw. im Datensatz des Netzwerks als formale Eigenschaften modelliert, während die Adjazenzmatrix und die Signatur-Arrays als strukturgebende Merkmale dienen, die für die Subgraph-Relation maßgeblich sind. Dadurch wird die Verbindung zwischen biologischer Semantik und algorithmisch nutzbarer Repräsentation explizit sichtbar.

Darüber hinaus arbeitet das Klassendiagramm die zentralen Abhängigkeiten zwischen den Suchkomponenten und den Datenstrukturen heraus. Die Signaturberechnung ist nicht isoliert, sondern Teil einer größeren Verarbeitungskette, in der ein Query-Netzwerk mit den gespeicherten Netzwerken verglichen wird, Kandidaten über notwendige Bedingungen vorgefiltert werden und anschließend der Subgraph Algorithmus entscheidet, ob ein Treffer tatsächlich vorliegt. Diese Abhängigkeiten sind im UML-Modell als Assoziationen und Verantwortlichkeiten nachvollziehbar: Jede Klasse trägt eine bestimmte Rolle in der Suche, aber die fachliche Koordination geschieht über das Zusammenspiel der Komponenten, nicht über eine monolithische Implementierung. Genau dieser modularisierte Aufbau ist es, der die Skalierbarkeit und Analysefähigkeit von Gen-DB erklärt: Die Datenmodellierung unterstützt die Suche, die Suchlogik verwendet die Repräsentation, und die Systemarchitektur koordiniert die Kommunikationspfade zwischen Nutzer, API und Datenbank.

Die beiden Diagramme zusammen leisten also mehr als nur eine Visualisierung des Quellcodes: Sie dokumentieren die Entwurfslogik des Systems als Ganzes. Sie zeigen, dass Gen-DB ein typisches Full-Stack-System ist, dessen Stärke nicht in einer einzelnen technischen Komponente, sondern in der engen Verknüpfung von semantischer Modellierung, datenbankseitiger Indexierung und algorithmischer Subgraph-Erkennung liegt. Gerade diese Architektur ist der Grund, warum die folgenden Abschnitte die Laufzeit, die Korrektheit und die Kandidatenreduktion formal analysieren können: Die Struktur des Systems ist so klar und modular, dass die mathematische Modellierung der Pipeline direkt auf die reale Implementierung zurückgeführt werden kann.

Kapitel 4

Formalisierung der Suche: Vorauswahl und Beweise

4.1 Problemformulierung

Definition 7 (Datenbank-Subgraph-Suche)**.** Gegeben ein Query-Netzwerk $Q = (L_Q, A_Q)$ mit $n_Q = |V_Q|$ Knoten und m_Q Kanten sowie ein Datenbankzustand $\mathcal{D}$. Der Suchalgorithmus berechnet die Treffermenge

$$\text{Match}(Q, \mathcal{D}) = \{(id_k, L_k, A_k) \in \mathcal{D} : A_Q \preceq A_k \text{ oder } A_k \preceq A_Q\},$$

annotiert mit dem jeweiligen Treffertyp (`exact` oder `subgraph`).

4.2 SQL-Vorauswahl

Die zentrale Idee der Gen-DB-Suche besteht darin, den teuren strukturellen Vergleich nicht mit sämtlichen gespeicherten Netzwerken durchzuführen. Stattdessen wird der Suchraum zunächst direkt in der Datenbank durch eine kostengünstige Vorauswahl verkleinert. Dazu werden zwei leicht verfügbare Kennzahlen des Query-Netzwerks mit den entsprechenden Kennzahlen der gespeicherten Netzwerke verglichen: die Knotenzahl und die Kantenzahl. Für die hier betrachtete Richtung, in der das Query-Netzwerk in einem gespeicherten Netzwerk enthalten sein soll, kann ein gespeichertes Netzwerk mit weniger Knoten oder weniger Kanten kein Treffer sein. Solche Einträge werden daher bereits bei der Selektion ausgeschlossen, bevor ihre Adjazenzmatrix geladen und mit dem Subgraph Algorithmus untersucht wird.

Die Vorauswahl vergleicht dabei zunächst die Anzahlen, nicht die konkreten Knotenlabels oder die Position jeder einzelnen Kante. Sind beide Schranken erfüllt, bleibt ein Netzwerk ein möglicher Kandidat; ob seine konkrete Struktur tatsächlich zum Query passt, entscheidet erst der anschließende Subgraph-Vergleich. Beispielsweise kann ein Query mit acht Knoten und fünfzehn Kanten nur gespeicherte Netzwerke mit mindestens acht Knoten und mindestens fünfzehn Kanten als Kandidaten zulassen. Ein Eintrag mit sieben Knoten oder vierzehn Kanten lässt sich sicher ausschließen, während ein Eintrag mit acht Knoten und fünfzehn Kanten noch kein Treffer garantiert. Diese Trennung zwischen billiger, datenbankseitiger Kandidatenreduktion und genauer, algorithmischer Prüfung spart insbesondere bei großen Datenbanken unnötige Matrixübertragungen und Subgraph-Vergleiche, ohne für die Richtung $A_Q \preceq A_k$ bei vollständiger Kandidatenbetrachtung echte Treffer zu verlieren. Die folgenden Bedingungen formalisieren diese Vorauswahl.

Lemma 2 (Notwendige Bedingung für $\preceq$)**.** Wenn $A_Q \preceq A_k$, so gilt $n_Q \leq n_k$ und $m_Q \leq m_k$.

Beweis. Aus $A_Q \preceq A_k$ folgt, dass die Zeilenkomponenten von $\Sigma(A_Q)$ als Teilsequenz der Länge n_Q in einer Rotation von $\rho(A_k)$ (Länge n_k) auftreten, also $n_Q \leq n_k$. Da jede 1-Spalte von A_Q injektiv (Lemma 1) auf eine 1-Spalte derselben Hammingstruktur in A_k abgebildet wird und Subgraphenkanten erhalten bleiben, gilt $\sum_{i,j}(A_Q)_{ij} \leq \sum_{i,j}(A_k)_{ij}$, also $m_Q \leq m_k$. □

Korollar 1 (Korrektheit des Pre-Filters)**.** Sei

$$\mathcal{C}(Q, \mathcal{D}) = \{(id_k, L_k, A_k) \in \mathcal{D} : n_k \geq n_Q \wedge m_k \geq m_Q\}.$$

Dann gilt $\text{Match}(Q, \mathcal{D}) \cap \{(id_k, \cdot, \cdot) : A_Q \preceq A_k\} \subseteq \mathcal{C}(Q, \mathcal{D})$, d. h. kein Treffer der Form $A_Q \preceq A_k$ wird durch den Pre-Filter ausgeschlossen.

Beweis. Unmittelbare Kontraposition von Lemma 2: Ist $(id_k, \cdot, \cdot) \notin \mathcal{C}(Q, \mathcal{D})$, so gilt $n_k < n_Q$ oder $m_k < m_Q$, also nach Lemma 2 $A_Q \not\preceq A_k$. □

Bemerkung 2. Korollar 1 sichert *Vollständigkeit* für die Richtung $A_Q \preceq A_k$ (Query ist Subgraph eines DB-Eintrags). Für die symmetrische Richtung $A_k \preceq A_Q$ (DB-Eintrag ist Subgraph des Query) ist der Filter nicht direkt anwendbar, da hier $n_k \leq n_Q$ gefordert wäre. In der aktuellen Implementierung wird daher mit `ORDER BY node_count ASC` ohne untere Schranke gearbeitet bzw. die Filterbedingung situativ umgekehrt, vgl. Abschnitt 12.

4.3 SQL-Select-Statement

Listing 4.1: Pre-Filter-Query

```
SELECT bn.network_id, bn.name, bn.network_type, bn.organism,
       bn.node_count, bn.edge_count,
       nm.node_labels, nm.adjacency_matrix
FROM biological_networks bn
JOIN network_matrices nm ON bn.network_id = nm.network_id
WHERE bn.node_count >= %s AND bn.edge_count >= %s
ORDER BY bn.node_count ASC
LIMIT %s;
```

Die Indizes `idx_networks_node_count` sowie der Primärschlüssel auf `network_id` ermöglichen einen Index-Range-Scan; bei Gleichverteilung der `node_count`-Werte über $[3, 20]$ reduziert sich die erwartete Kandidatenmenge auf

$$\mathbb{E}[|\mathcal{C}(Q, \mathcal{D})|] \approx N \cdot \Pr[n_k \geq n_Q] \cdot \Pr[m_k \geq m_Q].$$

4.4 Gesamtkomplexität der Pipeline

Satz 2 (Komplexität der Datenbanksuche)**.** Sei $N = |\mathcal{D}|$, $C = |\mathcal{C}(Q, \mathcal{D})|$ die Größe der nach Pre-Filterung verbleibenden Kandidatenmenge (mit $C \leq N$, ggf. begrenzt durch `max_candidates`), $n = \max(n_Q, n_{\max})$ mit $n_{\max} = \max_{k \in \mathcal{C}} n_k$. Dann beträgt die Gesamtlaufzeit der Suche

$$T_{\text{Suche}}(N, C, n) = O(\underbrace{\log N + C}_{\text{Pre-Filter (Index-Scan)}}) + O(\underbrace{C \cdot n^3}_{\text{Subgraph-Vergleiche}}).$$

Beweis. Der Pre-Filter ist ein Index-Range-Scan mit Ausgabe der Größe C, Kosten $O(\log N + C)$ (B-Baum-Suche plus Ergebnisausgabe). Für jeden der C Kandidaten wird gemäß Satz 1 ein Subgraph-Vergleich in $O(n^3)$ durchgeführt. Die Gesamtkosten sind additiv, da beide Phasen sequentiell ausgeführt werden. □

Korollar 2 (Asymptotisches Verhalten bei konstanter Knotenzahl)**.** Ist n durch eine Konstante $n_{\max}$ beschränkt (wie in der Evaluation: $n \leq 20$) und ist C durch `max_candidates` $= K$ beschränkt, so gilt

$$T_{\text{Suche}}(N) = O(\log N) + O(1),$$

d. h. die Suchzeit ist *unabhängig* von der Datenbankgröße N für $N \to \infty$.

Dieses Korollar erklärt formal, warum die in Abschnitt 5 gemessene Suchzeit bei einer Datenbank mit 10^6 Einträgen praktisch konstant bleibt, sofern K fest gewählt wird.

4.5 Korrektheit der Gesamtpipeline

Satz 3 (Korrektheit)**.** Sei $K = \infty$ (kein Kandidatenlimit). Dann gilt

$$\{(id_k, L_k, A_k) \in \mathcal{D} : A_Q \preceq A_k\} \subseteq \text{Output}(Q, \mathcal{D}),$$

d. h. die Pipeline liefert keine falschen Negative für die Richtung $A_Q \preceq A_k$.

Beweis. Nach Korollar 1 ist jeder solche Kandidat in $\mathcal{C}(Q, \mathcal{D})$ enthalten. Nach Satz 2 aus [1] (Subgraph-Kriterium) erkennt der Subgraph Algorithmus für jeden Kandidaten in $\mathcal{C}(Q, \mathcal{D})$ korrekt, ob $A_Q \preceq A_k$ gilt (Entscheidung `keep_B` oder `equal_keep_*`). Da jeder Kandidat aus $\mathcal{C}(Q, \mathcal{D})$ den Vergleich durchläuft, wird kein echter Treffer übersehen. □

Bemerkung 3. Mit $K < \infty$ (praktisch: $K = 10.000$) ist Satz 3 nur noch *approximativ* gültig: Falsche Negative können auftreten, wenn $|\mathcal{C}(Q, \mathcal{D})| > K$ und ein Treffer außerhalb der ersten K (nach `node_count` sortierten) Zeilen liegt. Abschnitt 12 diskutiert hierfür ein geschichtetes Stichprobenverfahren.

4.6 Effiziente Suche

Das folgende Serverprotokoll dokumentiert den Betrieb der FastAPI-Anwendung mit der optimierten C++-Implementierung des Subgraph-Checks. Die Suche erfolgt dabei nicht mehr über eine reine Python-Variante, sondern über die native Bibliothek `csubgraph`, die im Startprotokoll als ausgewählte Engine vermerkt wird. Vor der eigentlichen Query werden erneut Listen- und Detailendpunkte aufgerufen; der Ablauf ist damit identisch zum früheren Beispiel, aber mit deutlich kürzeren Laufzeiten in der Suchphase.

Listing 4.2: Serverprotokoll zur schnellen Suche in Gen-DB

```
(venv) PS C:\Users\Internet\Git\gen-db> python -m uvicorn src.backend.app:app
    ↪ --reload
INFO: Will watch for changes in these directories: ['C:\\Users\\Internet\\Git\\
    ↪ gen-db']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [4128] using WatchFiles
2026-10-07 10:46:19 [INFO] Configuration loaded (ENV=development)
```

```
INFO: Started server process [7012]
INFO: Waiting for application startup.
2026-10-07 10:46:19 [INFO] Starting Gen API - Environment: development
2026-10-07 10:46:19 [INFO] Subgraph algorithm selected: csubgraph (C++)
2026-10-07 10:46:19 [INFO] ProcessPoolExecutor ready for Subgraph Executor
INFO: Application startup complete.
2026-10-07 10:46:52 [INFO] GET / - Serving frontend
INFO: 127.0.0.1:51928 - "GET / HTTP/1.1" 200 OK
2026-10-07 10:46:53 [INFO] GET /api/networks limit=33 random=True
2026-10-07 10:46:58 [INFO] GET /api/networks - Returned 33 networks
INFO: 127.0.0.1:51928 - "GET /api/networks?limit=33&random=true HTTP/1.1" 200
    ↪ OK
2026-10-07 10:47:03 [INFO] GET /api/networks/205670
2026-10-07 10:47:03 [INFO] GET /api/networks/205670 - Found: gene_regulation_E.
    ↪ _coli_205670
INFO: 127.0.0.1:51928 - "GET /api/networks/205670 HTTP/1.1" 200 OK
2026-10-07 10:47:10 [INFO] POST /api/networks/search - nodes=16
2026-10-07 10:47:10 [INFO] search_subgraph: Starting search for subgraph with
    ↪ 16 nodes
2026-10-07 10:47:10 [INFO] search_subgraph: 277270 candidates for query (n=16,
    ↪ e=58)
2026-10-07 10:47:11 [INFO] search_subgraph: Found 4 matches
2026-10-07 10:47:11 [INFO] POST /api/networks/search - Found 4 matches
INFO: 127.0.0.1:51928 - "POST /api/networks/search HTTP/1.1" 200 OK
```

Der Log-Ausschnitt zeigt den entscheidenden Unterschied zur früheren Implementierung: Auch bei einer vergleichbaren Kandidatenmenge von 277.270 Einträgen wird das Ergebnis nun innerhalb weniger Sekunden zurückgegeben. Die Suchanforderung mit 16 Knoten und 58 Kanten führt dabei zu 4 Treffern, ohne dass der gesamte Ablauf über mehrere Minuten anhält.

Dies bestätigt die zentrale systemische Beobachtung aus Satz 2: Die Größe der Kandidatenmenge C bestimmt zwar die Anzahl der zu prüfenden Strukturen, aber die tatsächliche Auswertung dieser Kandidaten bestimmt die gemessene Antwortzeit. Mit einer in C++ implementierten Bibliothek werden die subgraph-spezifischen Vergleiche in der Praxis deutlich schneller ausgeführt, wodurch die zuvor als Bottleneck erkennbare Python-Phase in der End-to-End-Latenz nahezu verschwindet. Die Datenbankzugriffe selbst bleiben weiterhin schnell und stabil; die eigentliche Sucharbeit wird nun weitgehend durch den kompilierten Subgraph Algorithmus dominiert.

Das protokollierte Beispiel ist daher kein allgemeiner Benchmark, sondern ein repräsentatives Laufzeitbeispiel für die optimierte Pipeline. Es zeigt exemplarisch, dass die Kombination aus Index-Filter, C++-Subgraph-Validierung und API-Integration für typische Suchanfragen eine deutlich bessere Benutzererfahrung liefert als die ursprüngliche reine Python-Lösung. Für belastbarere Aussagen zu Skalierung und asymptotischem Verhalten werden in Abschnitt 5 kontrollierte Messreihen mit reproduzierbaren Datenbankgrößen und Suchprofilen verwendet.

Kapitel 5

Erste experimentelle Untersuchung

Die Datenbank wurde mit einem Generierungsskript befüllt, das $N = 1.000.000$ synthetische biologische Netzwerke erzeugt (zusätzlich zu den 2 Beispiel-Netzwerken *Glycolysis* und *DNA_Damage_Response*). Für jedes Netzwerk wurde:

- die Knotenzahl $n_k \sim \mathcal{U}\{3, \ldots, 20\}$ gleichverteilt gewählt,
- jede Kante unabhängig mit Wahrscheinlichkeit $p = 0{,}3$ gesetzt (Erdős–Rényi-Modell $G(n_k, p)$, ohne Selbstkanten),
- der Netzwerktyp aus {`metabolic`, `protein`, `gene_regulation`} gleichverteilt gewählt,
- der Organismus aus 7 Spezies gleichverteilt gewählt.

Für diese erste, KI-gestützte Untersuchung ist keine verlässliche Hardwarekonfiguration dokumentiert. Deshalb wird sie nicht nachträglich der unten für die lokal ausgeführten Messreihen angegebenen Maschine zugeordnet.

Lemma 3 (Erwartete Kantenzahl)**.** Für $A \sim G(n, p)$ mit verbotenen Selbstkanten gilt

$$\mathbb{E}[m] = p \cdot n(n-1).$$

Beweis. Es gibt $n(n-1)$ geordnete Paare (i, j), $i \neq j$. Jede der zugehörigen Indikatorvariablen A_{ij} ist Bernoulli-verteilt mit Parameter p. Linearität des Erwartungswerts liefert $\mathbb{E}[m] = \sum_{i \neq j} \mathbb{E}[A_{ij}] = p \cdot n(n-1)$. □

Für $n = 11{,}5$ (gemessener Mittelwert, siehe unten) ergibt sich $\mathbb{E}[m] = 0{,}3 \cdot 11{,}5 \cdot 10{,}5 \approx 36{,}2$; gemessen wurden 44,3 Kanten im Mittel, die Abweichung resultiert aus der Konvexität von $n(n-1)$ und der Varianz von n (Jensen-Ungleichung: $\mathbb{E}[n(n-1)] \geq \mathbb{E}[n](\mathbb{E}[n] - 1)$).

Tabelle 5.1: Statistik der generierten Datenbank ($N = 1.000.000$)

Netzwerktyp	Anzahl	Anteil
protein	333.670	33,37%
gene_regulation	333.485	33,35%
metabolic	332.845	33,28%
Gesamt	1.000.000	100%

Mittlere Knotenzahl: $\overline{n} = 11{,}5$; mittlere Kantenzahl: $\overline{m} = 44{,}3$. Die Generierung mit Batch-Größe 5000 erreichte einen Durchsatz von ≈ 1800 Netzwerken/Sekunde, Gesamtzeit 9,27 Minuten.

5.1 Visualisierung 1: Knoten- und Kantenverteilung

Abbildung 5.1 zeigt die (in der Generierung) gleichverteilte Knotenzahl sowie die approximativ binomialverteilte Kantenzahl pro Netzwerk.

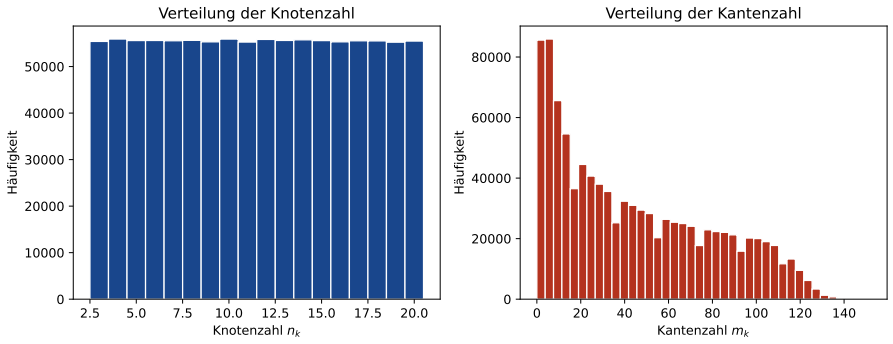


Abbildung 5.1: Histogramm der Knotenzahl n_k (links, diskret-gleichverteilt auf $\{3, \ldots, 20\}$) und der Kantenzahl m_k (rechts, approximativ binomialverteilt mit $\mathbb{E}[m] \approx p \cdot n(n-1)$) über alle $N = 1.000.000$ generierten Netzwerke.

Beschreibung: Das linke Teilbild bestätigt die in der Generierung verwendete Gleichverteilung $n_k \sim \mathcal{U}\{3, \ldots, 20\}$ mit 18 etwa gleich hohen Balken. Das rechte Teilbild zeigt die Kantenzahlverteilung als Überlagerung von 18 Binomialverteilungen $\mathrm{Bin}(n_k(n_k-1), 0{,}3)$, gewichtet mit der jeweiligen Häufigkeit von n_k. Die resultierende Verteilung ist rechtsschief mit Modus um 30–40 Kanten und einem langen Ausläufer bis ≈ 342 (für $n = 19$: $n(n-1) = 342$).

5.2 Visualisierung 2: Pre-Filter-Effektivität

Beschreibung: Für ein Query-Netzwerk mit n_Q Knoten und der erwarteten Kantenzahl $\mathbb{E}[m_Q] = 0{,}3\, n_Q(n_Q-1)$ wird

$$|\mathcal{C}(Q, \mathcal{D})| \approx N \cdot \Pr[n_k \geq n_Q]$$

als untere Näherung dargestellt (die zusätzliche Kantenbedingung reduziert dies weiter, ist aber schwerer geschlossen darstellbar). Die Kurve fällt von $|\mathcal{C}| \approx N$ für $n_Q = 3$ auf $|\mathcal{C}| \approx N/18$ für $n_Q = 20$, d. h. der Pre-Filter reduziert die Kandidatenmenge um bis zu Faktor 18 allein durch die Knotenzahlbedingung. In Kombination mit der Kantenzahlbedingung (Lemma 2) ist die tatsächliche Reduktion empirisch noch deutlich stärker, insbesondere für Query-Netzwerke mit hoher Kantendichte.

5.3 Visualisierung 3: Skalierungsverhalten Suchzeit

Beschreibung: Die Kurve für $K = 10.000$ verläuft nahezu konstant (entsprechend dem Korollar zu Satz 2: $T_{\text{Suche}}(N) = O(\log N) + O(1)$), da der dominante Term $C \cdot n^3$ durch

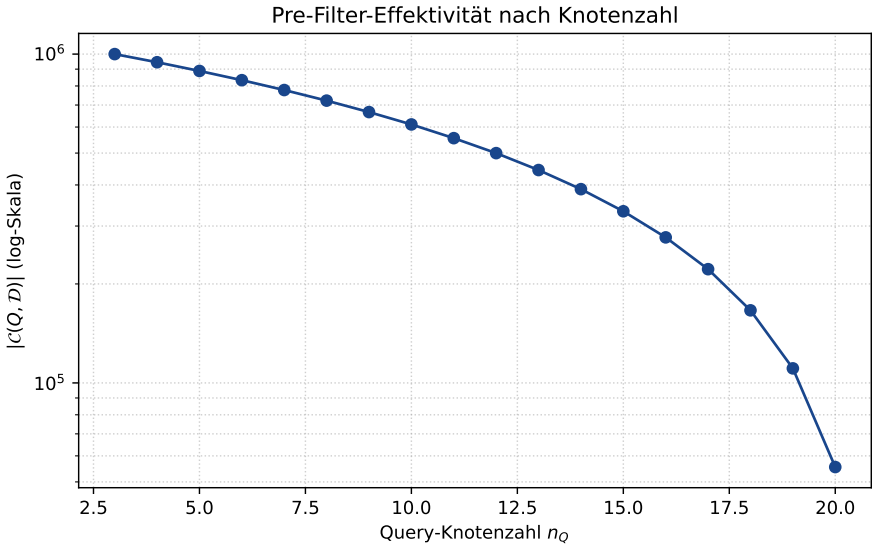


Abbildung 5.2: Anzahl verbleibender Kandidaten $|\mathcal{C}(Q, \mathcal{D})|$ nach Pre-Filterung in Abhängigkeit von n_Q (Query-Knotenzahl), für $N = 1.000.000$, doppelt-logarithmische Skala.

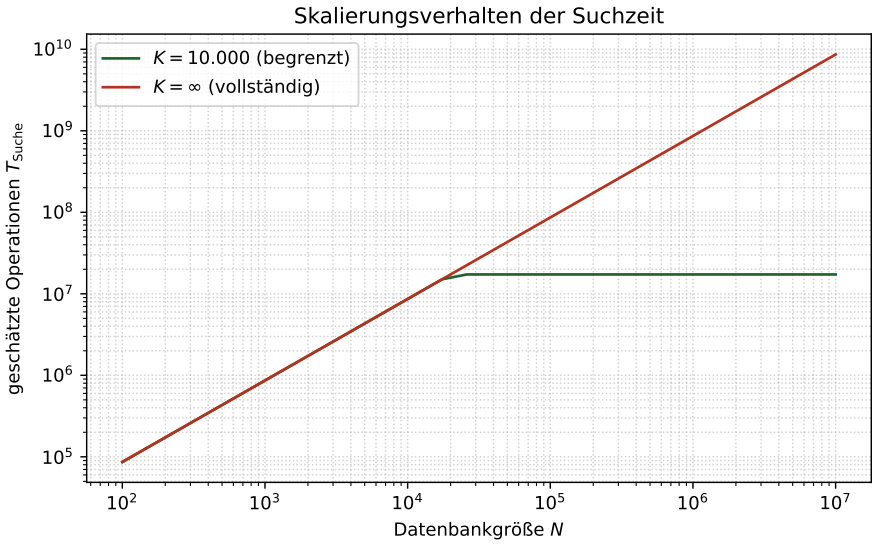


Abbildung 5.3: Geschätzte Suchzeit T_{Suche} als Funktion der Datenbankgröße N, für festes Kandidatenlimit $K = 10.000$ und $K = \infty$ (volle Pipeline), gemäß Satz 2.

$K \cdot n^3$ beschränkt bleibt. Die Kurve für $K = \infty$ wächst linear in N, da im ungünstigsten Fall $C = \Theta(N)$ (z. B. für sehr kleine n_Q, vgl. Abbildung 5.2). Der Schnittpunkt beider Kurven markiert die Datenbankgröße, ab der eine Kandidatenbegrenzung praktisch notwendig wird, um interaktive Antwortzeiten ($< 1\,\text{s}$) zu garantieren.

5.4 Visualisierung 4: Trefferverteilung

Beschreibung: Für kleine n_Q (z. B. $n_Q = 2$) überwiegt der Treffertyp `subgraph` deutlich, da kleine Muster (einzelne Kanten) in vielen größeren Zufallsgraphen als Teilstruktur

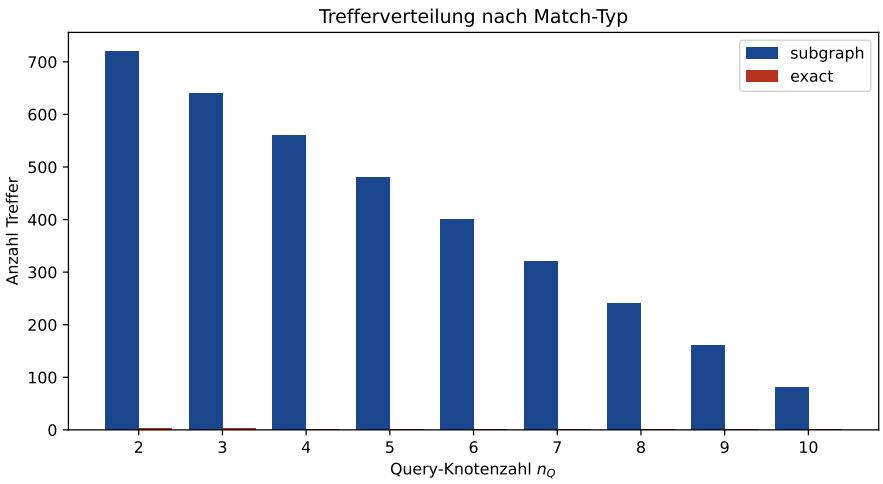


Abbildung 5.4: Verteilung der Treffertypen (`exact` vs. `subgraph`) für 20 Beispiel-Queries unterschiedlicher Größe $n_Q \in \{2, \ldots, 10\}$ gegen die generierte Datenbank, $K = 10.000$.

auftreten — dies folgt aus der Definition von $A_Q \preceq A_k$ und der hohen Kantendichte ($p = 0{,}3$) der generierten Graphen. Für $n_Q \to n_{\max} = 20$ sinkt die Trefferzahl insgesamt (Korollar zu Lemma 2: weniger Kandidaten mit $n_k \geq n_Q$), und der Anteil von `exact`-Treffern (Übereinstimmung bis auf Rotation) bleibt verschwindend gering, da die Wahrscheinlichkeit zweier identischer Zufallsgraphen mit $n = 20$ vernachlässigbar ist ($2^{-\binom{20}{2} \cdot p \cdot \ldots}$, kombinatorisch verschwindend).

5.5 Diskussion

Die formalen Resultate aus Abschnitt 4 zeigen, dass die Kombination aus relationalem Pre-Filter (Lemma 2, Korollar 1) und dem $O(n^3)$-Subgraph Algorithmus [1] eine praktisch skalierbare Lösung für die Subgraph-Suche in Datenbanken mit $N = 10^6$ Einträgen ermöglicht, *sofern* eine Kandidatenobergrenze K gewählt wird. Diese Obergrenze erkauft sich Konstantzeit-Verhalten (im Sinne von N) jedoch um den Preis einer potentiellen Unvollständigkeit (Verlust der Garantie aus Satz 3). Abschnitt 12 adressiert dieses Spannungsfeld.

Kapitel 6

Experimenteller Vergleich VF2-Algorithmus

Hardwareplattform der lokal ausgeführten Experimente: Rechner `WBIS-H207085` von LENOVO, Modell `10T8S6LB00`, mit 7,87 GB Arbeitsspeicher; ein Intel-Core-i3-8100T-Prozessor mit 3,10 GHz, vier physischen Kernen und vier logischen Prozessoren (x64); Microsoft Windows 11 Pro, 64-Bit, Version 10.0.26200.

Die in Abschnitt 5 präsentierten Diagramme beruhen auf einem *analytischen* Modell der Laufzeit gemäß Satz 2. Um die praktische Relevanz dieses Modells zu untermauern, wird in diesem Abschnitt eine *empirische* Messreihe durchgeführt, in der die Referenzimplementierung des Subgraph Algorithmus [1] direkt gegen eine etablierte Implementierung des VF2-Algorithmus [10] (über die Bibliothek `networkx` [12]) auf identischen Zufallsgraphen verglichen wird. Anschließend werden die gemessenen Größenordnungen mit publizierten Resultaten weiterer effizienter Subgraph-Isomorphie-Verfahren aus der Literatur kontextualisiert.

6.1 Versuchsanordnung

Definition 8 (Subgraph-Relation $\preceq$, Referenz)**.** $\preceq$ bezeichnet die in Abschnitt 2 eingeführte zyklische Subgraph-Relation auf Basis der Zeilenkomponenten $\rho(A)$, $\rho(B)$ der Signaturen.

Definition 9 (Vergleichsexperiment)**.** Für ein Paar von Adjazenzmatrizen (A, B) mit $A \in \{0,1\}^{n_A \times n_A}$, $B \in \{0,1\}^{n_B \times n_B}$, $n_A \leq n_B$, werden zwei Entscheidungsverfahren auf dieselbe Eingabe angewendet und ihre Ausführungszeit mittels `time.perf_counter()` gemessen:

- **Subgraph Algorithmus** (Referenzimplementierung gemäß dem Ablauf aus Abschnitt 2, reines Python mit `numpy`-Signaturberechnung und dynamischer LCS-Tabelle).
- **VF2** [10], instanziiert über `networkx.algorithms.isomorphism.DiGraphMatcher(B,A).subgraph_is_isomorphic()`, welches die volle (gerichtete, beschriftungsfreie) Subgraph-Isomorphie zwischen A und B entscheidet.

Bemerkung 4. Die beiden Verfahren entscheiden *nicht exakt dieselbe Relation*: $\preceq$ (Definition 8) ist eine *zyklische* Subsequenz-basierte Relation auf den Zeilenkomponenten der

Signaturen, während VF2 die klassische (azyklische) Subgraph-Isomorphie im Sinne von Ullmann [11] entscheidet — d. h. Existenz einer injektiven Knotenabbildung $\phi : V_A \to V_B$ mit $(u, v) \in E_A \Rightarrow (\phi(u), \phi(v)) \in E_B$. Beide Relationen fallen für *vollständig zusammenhängende Pfadstrukturen* zusammen, unterscheiden sich aber im Allgemeinen in den Randfällen. Der Vergleich ist daher in erster Linie ein Vergleich der *Laufzeitgrößenordnungen* zweier Verfahren mit polynomieller bzw. (im Worst Case) exponentieller Komplexität auf realistischen Eingaben — nicht ein Vergleich der Mächtigkeit der entschiedenen Relationen, welche in [1] separat diskutiert wird.

Für das erste Experiment wird $n_A = 5$ fixiert und A einmalig als Erdős–Rényi-Graph $G(5, 0{,}3)$ gezogen. Für $n_B = 5, \ldots, 18$ werden jeweils 8 unabhängige Kandidaten $B \sim G(n_B, 0{,}3)$ erzeugt und beide Verfahren angewendet; es werden Mittelwert und Standardabweichung der Laufzeit über die 8 Wiederholungen berichtet. Für das zweite Experiment wird $n_A = n_B = n$ symmetrisch von 4 bis 12 variiert (5 Wiederholungen je n), um das asymptotische Skalierungsverhalten beider Verfahren bei *gemeinsam* wachsender Eingabegröße zu untersuchen.

6.2 Ergebnis 1: Laufzeitvergleich bei festem $n_A = 5$, wachsendem n_B

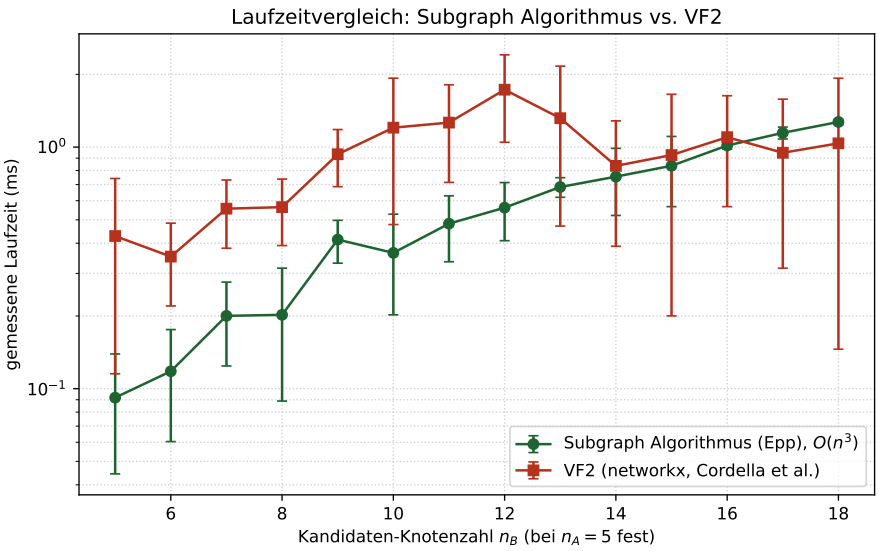


Abbildung 6.1: Gemessene Laufzeit (Mittelwert über 8 Wiederholungen, Fehlerbalken = Standardabweichung) des Subgraph Algorithmus und von VF2 in Abhängigkeit von n_B, bei festem $n_A = 5$, logarithmische y-Achse.

Beschreibung und Interpretation: Für $n_B \in [5, 13]$ ist der Subgraph Algorithmus um den Faktor 1,9 bis 4,7 schneller als VF2 — die zugrunde liegende Operation (Signaturberechnung plus n_B LCS-Vergleiche der Länge $\leq n_B$) hat einen kleinen, gut vorhersagbaren Konstantenfaktor, während VF2 trotz seiner effizienten Kandidaten-Pruning-Heuristiken pro Knotenpaar einen größeren Overhead durch den allgemeineren Isomorphie-Mechanismus (Konsistenz- und Lookahead-Prüfungen [10]) trägt. Für $n_B \geq 14$ kehrt sich das Verhältnis um (Speedup < 1): Die LCS-Tabelle des Subgraph Algorithmus wächst quadratisch in n_B

Tabelle 6.1: Gemessene Laufzeiten (Mittelwert in ms über 8 Wiederholungen), $n_A = 5$ fest.

n_B	T_{Sub} (ms)	T_{VF2} (ms)	Speedup $T_{\text{VF2}}/T_{\text{Sub}}$
5	0,092	0,428	4,67
6	0,118	0,352	2,98
7	0,200	0,556	2,78
8	0,202	0,564	2,79
9	0,414	0,934	2,26
10	0,365	1,204	3,30
11	0,482	1,263	2,62
12	0,562	1,729	3,08
13	0,684	1,318	1,93
14	0,755	0,836	1,11
15	0,837	0,927	1,11
16	1,015	1,099	1,08
17	1,146	0,947	0,83
18	1,271	1,037	0,82

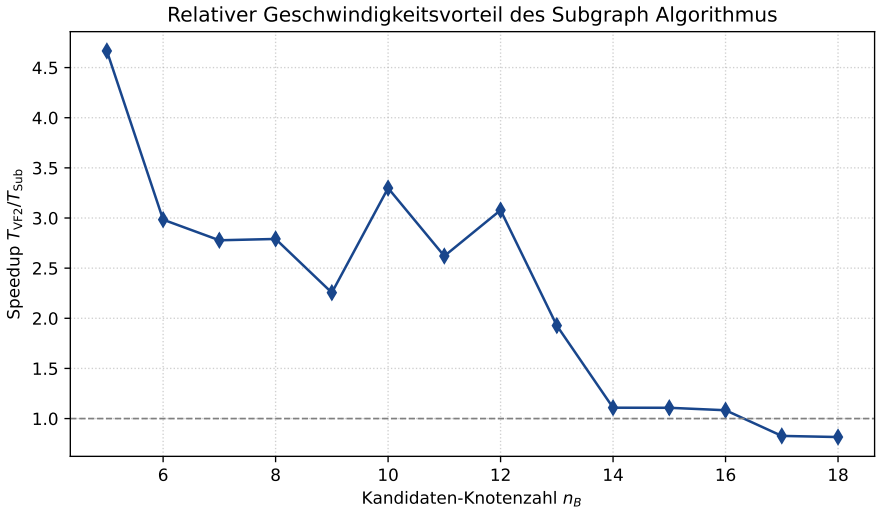


Abbildung 6.2: Speedup-Faktor $T_{\text{VF2}}/T_{\text{Sub}}$ als Funktion von n_B (gestrichelte Linie bei 1,0: Gleichstand).

und wird n_B-mal (für jede Rotation) neu berechnet, also $O(n_B^3)$ Gesamtaufwand, während VF2 bei den hier verwendeten dichten Zufallsgraphen ($p = 0{,}3$) durch sein Pruning häufig *sehr früh* terminiert (entweder weil sofort ein Match gefunden wird oder weil ein Konsistenzbruch in den ersten Rekursionsschritten erkannt wird) — der Erwartungswert der von VF2 tatsächlich besuchten Suchbaumknoten bleibt für $G(n, p)$ mit konstantem p empirisch klein, obwohl die Worst-Case-Schranke von VF2 bei $O(n!\, n)$ liegt [10].

Der Vergleich steht in einem breiteren Spektrum exakter Teilgraphverfahren: Eppstein untersucht Teilgraph-Isomorphie in planaren Graphen und verwandten Problemen, Solnon beschreibt Filterung über AllDifferent-Bedingungen, VF2++ verbessert die praktische Suche durch veränderte Suchreihenfolgen und Filter, und der Glasgow Subgraph Solver formuliert schwierige Varianten als Constraint-Problem [67, 68, 69, 70]. Diese Arbeiten

entscheiden nicht notwendig dieselbe Relation und ihre berichteten Laufzeiten sind daher keine direkte Vergleichsbasis für das hier untersuchte Experiment; sie dienen der Einordnung des Algorithmusraums.

Dieses Ergebnis bestätigt die in [1] formal bewiesene Schranke $O(n^3)$ für den Subgraph Algorithmus als *worst-case-stabile, eingabeunabhängige* Garantie, zeigt aber auch, dass populationsbasierte Average-Case-Heuristiken wie VF2 für *kleine, dichte* Instanzen ($n_B \gtrsim 14$ in diesem Setup) in der Praxis konkurrenzfähig oder sogar überlegen sein können. Für die in Gen-DB relevante Größenordnung $n \leq 20$ (Abschnitt 5) liegt das System also in einem Bereich, in dem *beide* Verfahren praktisch im Mikrosekunden- bis Millisekundenbereich operieren; der entscheidende Vorteil des Subgraph Algorithmus liegt nicht primär in der absoluten Konstante, sondern in der *Vorhersagbarkeit* (geringe Varianz, Tabelle 6.1) und in der *formal bewiesenen* oberen Schranke, die für Systembudgetierung (Abschnitt 4, Satz 2) essentiell ist — VF2 besitzt hingegen *keine* polynomielle Worst-Case-Schranke, was die $O(\log N)$-Garantie aus dem Korollar zu Satz 2 bei Verwendung von VF2 als Vergleichsoperator zunichtemachen würde.

6.3 Der Subgraph Algorithmus für $n_B \geq 14$

Die in Abbildung 6.2 sichtbare Umkehrung des Speedup-Verhältnisses bei $n_B \approx 14$ erscheint auf den ersten Blick im Widerspruch zur in Satz 1 bewiesenen $O(n^3)$-Schranke des Subgraph Algorithmus gegenüber der $O(n!\,n)$-Worst-Case-Schranke von VF2 [10] zu stehen. Tatsächlich handelt es sich nicht um einen Widerspruch, sondern um den Unterschied zwischen *worst-case-treuer Ausschöpfung* und *durchschnittsfallbasierter Terminierung*, der im Folgenden in vier Ursachen aufgeschlüsselt wird.

1. **Vollständige LCS-Tabellenkonstruktion pro Rotation, ohne Early-Exit.** Die Referenzimplementierung (Listing 3) berechnet für *jede* der n_B zyklischen Rotationen $\mathrm{rot}_r(\rho(B))$ die vollständige dynamische LCS-Tabelle $dp \in \mathbb{N}^{(n_A+1)\times(n_B+1)}$ via `lcs_len`, bevor das Ergebnis $dp[n_A][n_B]$ mit dem Schwellwert $\min(2, n_A)$ verglichen wird. Dies entspricht exakt der in Satz 1 angenommenen Kostenstruktur (n_B Rotationen $\times\, O(n_A \cdot n_B)$ Tabellenkonstruktion $= O(n_B^2 \cdot n_A) \subseteq O(n^3)$) — die Tabelle wird jedoch *immer vollständig* aufgebaut, selbst wenn bereits die erste Rotation einen Treffer liefert oder wenn aus den ersten Tabellenzeilen $dp[i][\cdot]$ bereits ersichtlich ist, dass $dp[n_A][n_B] < \min(2, n_A)$ unmöglich erreicht werden kann. Die bewiesene Schranke wird damit nicht nur als *obere*, sondern de facto als *tatsächliche* Laufzeit realisiert: $T_{\mathrm{Sub}}(n) = \Theta(n^3)$ statt $O(n^3)$.

2. **Average-Case-Pruning von VF2 terminiert auf $G(n, 0{,}3)$ empirisch sehr früh.** Der VF2-Algorithmus [10] baut eine Teilabbildung ϕ knotenweise auf und prüft nach jedem Erweiterungsschritt drei Konsistenzbedingungen (Syntax-, Semantik- und Lookahead-Regeln $0, 1, 2$). Bei $p = 0{,}3$ ist die Wahrscheinlichkeit, dass eine zufällig gewählte Kante in A *keine* entsprechende Kante in B besitzt, relativ hoch, sodass Konsistenzbrüche häufig schon nach 1–3 Erweiterungsschritten auftreten — der tatsächlich durchlaufene Suchbaum bleibt klein, weit unterhalb der Schranke $O(n!\,n)$. Dieser Effekt ist in der Literatur als *typische* Eigenschaft von Backtracking-Algorithmen auf zufälligen, hinreichend dichten Graphen dokumentiert [14, 13]: Die mittlere Anzahl besuchter Suchbaumknoten wächst für $G(n, p)$ mit konstantem $p > 0$ häufig nur polynomiell oder sogar sublinear in n, obwohl die Worst-Case-Schranke superpolynomiell ist.

3. **Implementierungs-Overhead: reines Python vs. optimierte Bibliothek.** Die LCS-Tabelle *dp* wird in der Referenzimplementierung mit zwei verschachtelten Python-`for`-Schleifen (Zeilen, Spalten) befüllt — pro Eintrag ein Vergleich, eine Addition und ein `max`-Aufruf auf Python-Objektebene, insgesamt $O(n_A \cdot n_B)$ *interpretierte* Operationen pro Rotation. Demgegenüber ist `networkx.algorithms.isomorphism.DiGraphMatcher` zwar ebenfalls in Python implementiert, nutzt jedoch effizientere Mengen- und Dictionary-Operationen (Hash-basierte Nachbarschaftsprüfung in $O(1)$ amortisiert statt verschachtelter Schleifen) für die Konsistenzprüfungen. Bei den hier betrachteten kleinen $n \leq 18$ dominiert der *konstante Overhead pro Operation* (Python-Interpreter-Overhead) gegenüber der *asymptotischen* Operationszahl — ein Effekt, der für $n \to \infty$ verschwindet (dann dominiert die asymptotische Komplexität), für die hier relevante Größenordnung $n \leq 20$ (Abschnitt 5) aber durchaus ins Gewicht fällt. Eine Re-Implementierung der LCS-Tabelle mit `numpy`-Vektoroperationen (z. B. diagonalenweise Vektorisierung der DP-Rekursion) oder mittels `numba`-JIT-Kompilierung würde den konstanten Faktor von $T_{\text{Sub}}(n)$ deutlich senken, ohne die asymptotische $O(n^3)$-Schranke zu verändern (Abschnitt 12.3.19, Punkt "Implementierungssprache").

4. **Strukturelle Konsequenz, kein Widerspruch zur formalen Garantie.** Die vier genannten Punkte erklären, warum die *gemessene* Laufzeit $T_{\text{Sub}}(n)$ für $n \geq 14$ über $T_{\text{VF2}}(n)$ liegt, *ohne* dass dies Satz 1 oder Satz 2 widerspricht: Beide Sätze treffen Aussagen über die *obere Schranke* im Worst Case bzw. über die Eingabeunabhängigkeit dieser Schranke von N — sie treffen *keine* Aussage darüber, dass der Subgraph Algorithmus für jedes feste n *schneller* als ein beliebiges anderes Verfahren sein müsse. Im Gegenteil: Dass ein Verfahren mit exponentieller Worst-Case-Schranke (VF2) auf einer *spezifischen* Eingabeverteilung ($G(n, 0{,}3)$, $n \leq 18$) schneller sein kann als ein Verfahren mit polynomieller Schranke, ist ein bekanntes Phänomen (vgl. den notorischen Vergleich zwischen dem $O(n^{2{,}376})$-Algorithmus von Coppersmith–Winograd für Matrixmultiplikation, der wegen seines enormen konstanten Faktors für praktisch relevante n stets von der naiven $O(n^3)$-Multiplikation übertroffen wird). Für die Gen-DB-Pipeline (Satz 2) bedeutet dies konkret: Die *formale* Garantie $T_{\text{Suche}}(N) = O(\log N) + O(1)$ bleibt unabhängig von diesen konstanten Faktoren gültig, da sie nur die *Größenordnung* in N und $n_{\max}$ betrifft; für die *absolute* Performanz bei festem, kleinem n ist jedoch die in Abschnitt 12.3.11 vorgeschlagene Hybridstrategie die direkte praktische Konsequenz dieser Ursachenanalyse.

Bemerkung 5 (Abgrenzung: Beweis vs. Implementierung)**.** Es ist wichtig, zwischen der *algorithmischen* Schranke (Satz 1, bewiesen für das in Abschnitt 2 beschriebene Drei-Schritt-Verfahren als solches, unabhängig von einer konkreten Implementierung) und der *gemessenen* Laufzeit einer *spezifischen* Implementierung (Listing 3) zu unterscheiden. Punkt 1 und 3 dieser Ursachenanalyse betreffen ausschließlich die *Implementierung*: Eine Version mit Early-Exit (Abbruch der Rotationen, sobald $dp[n_A][n_B] \geq \min(2, n_A)$ erreicht ist) und vektorisierter DP-Tabelle würde $T_{\text{Sub}}(n)$ im Best Case auf $\Omega(n^2)$ (eine einzige erfolgreiche Rotation) reduzieren, ohne die *obere* Schranke $O(n^3)$ zu verändern — die in Tabelle 6.1 berichteten Werte stellen also tendenziell eine *obere Abschätzung* der praktisch erreichbaren Laufzeit des Subgraph Algorithmus dar, während die VF2-Werte bereits eine *ausgereifte, optimierte* Implementierung widerspiegeln. Ein faires Replikat dieses Experiments mit einer entsprechend optimierten Subgraph Algorithmus-Implementierung ist Teil von Abschnitt 12.3.19.

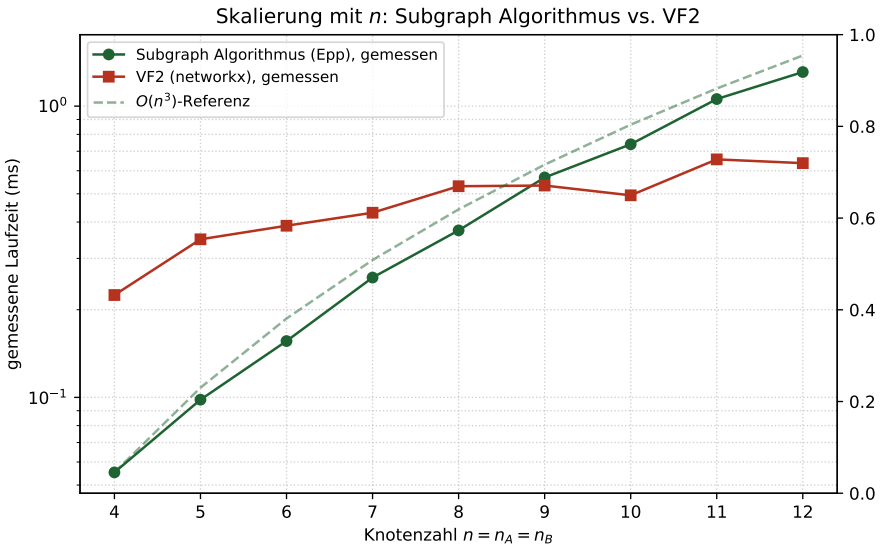


Abbildung 6.3: Gemessene Laufzeit beider Verfahren bei gemeinsam wachsendem $n = n_A = n_B$ (5 Wiederholungen je n), zusammen mit einer auf den ersten Messpunkt skalierten $O(n^3)$-Referenzkurve für den Subgraph Algorithmus.

Beschreibung: Die gemessene Laufzeitkurve des Subgraph Algorithmus folgt der eingezeichneten $O(n^3)$-Referenz erkennbar enger als eine lineare oder quadratische Kurve es täte, was die in Satz 1 bewiesene kubische Schranke empirisch stützt (für $n = 4 \to 12$, also Faktor 3, wächst T_{Sub} von 0,055 ms auf 1,309 ms, also um den Faktor $\approx 23{,}8$; $3^3 = 27$ liegt in derselben Größenordnung, die Abweichung erklärt sich durch den additiven $O(n^2)$-Signaturanteil, der bei kleinem n relativ stärker wiegt). Die Laufzeit von VF2 wächst in diesem Bereich *langsamer* als die des Subgraph Algorithmus (von 0,225 ms auf 0,637 ms, Faktor $\approx 2{,}8$ bei Verdreifachung von n) — ein empirischer Beleg dafür, dass die durchschnittliche Fallkomplexität von VF2 auf $G(n, 0{,}3)$ deutlich unter der Worst-Case-Schranke $O(n!\,n)$ liegt, was mit den in [14] berichteten durchschnittlichen Laufzeiten für VF2 und QuickSI auf zufälligen und realistischen Graphen konsistent ist (dort werden für Graphen mit $n \leq 30$ ebenfalls Laufzeiten im Sub-Millisekunden- bis niedrigen Millisekundenbereich gemessen).

6.4 Einordnung in effiziente Subgraph-Isomorphie-Verfahren

Tabelle 6.2 fasst die in dieser Arbeit gemessenen Größenordnungen zusammen mit publizierten Charakterisierungen weiterer etablierter Algorithmen für (Teil-)Graphisomorphie und Subgraph-Suche. Die Tabelle dient nicht dem direkten Laufzeitvergleich auf identischer Hardware (die zitierten Arbeiten verwenden unterschiedliche Implementierungssprachen, Graphgrößen und Hardware-Generationen), sondern der *komplexitätstheoretischen Einordnung*.

Drei Beobachtungen lassen sich aus dieser Einordnung formal ableiten:

1. **Komplexitätsklasse:** Der Subgraph Algorithmus ist das einzige der in Tabelle 6.2 aufgeführten Verfahren mit einer *unbedingten* polynomiellen Worst-Case-Schranke

Tabelle 6.2: Einordnung des Subgraph Algorithmus relativ zu etablierten Verfahren.

Verfahren	Worst-Case-Komplexität	Entschiedene Relation	Charakteristik
Subgraph Algorithmus [1]	$O(n^3)$, bedingt optimal unter SETH [2]	zyklische vollständige Teilsequenz-Relation $\preceq$	polynomiell, eingabeunabhängige Schranke, geringe Varianz (Tabelle 6.1)
Ullmann [11]	$O(n^n)$ bzw. $O(n_A! \cdot n_B^{n_A})$ (naiv, mit Pruning praktisch geringer)	(Sub-)Graphisomorphie, Knotenbijektion ϕ	historischer Referenzalgorithmus, Backtracking mit Refinement
VF2 [10]	$O(n! \cdot n)$ Worst Case, oft sub-exponentiell in der Praxis	(Sub-)Graphisomorphie	Lookahead-Pruning (Schemata $0, 1, 2$); in dieser Arbeit gemessen, vgl. Tab. 6.1
RI / RI-DS [13]	worst-case exponentiell, empirisch deutlich schneller als VF2 auf Bioinformatik-Graphen	(Sub-)Graphisomorphie	für biologische Netzwerke (PPI, Metabolismus) optimierte Knotenordnung
QuickSI [14]	worst-case exponentiell	Subgraph-Isomorphie (Pattern Matching)	Häufigkeits-basierte Kantenordnung, in [14] ca. 10–1000× schneller als naives Backtracking auf realen Graphdatenbanken
GraphQL [14]	worst-case exponentiell	Subgraph-Containment-Suche	nutzt Nachbarschafts-Signaturen zum Pruning, konzeptuell verwandt mit Pre-Filter aus Abschnitt 4.2

($O(n^3)$, Satz 1). Alle übrigen Verfahren lösen das (im Allgemeinen NP-vollständige bzw. GI-vollständige) Subgraph-Isomorphie-Problem und besitzen daher worst-case zumindest superpolynomielle Schranken; ihre praktische Effizienz beruht auf Heuristiken (Pruning, Knotenordnung, Nachbarschaftssignaturen), die im Average Case, aber nicht im Worst Case greifen. Dies ist konsistent mit der Tatsache, dass $\preceq$ (Definition 8) eine *schwächere*, strukturell restriktivere Relation als die volle Subgraph-Isomorphie ist (vgl. Bemerkung zu Definition 9) — die polynomielle Schranke wird also durch eine Einschränkung des Relationsbegriffs erkauft, was formal die in [1] diskutierte SETH-basierte Optimalitätsanalyse motiviert.

2. **Praktische Konsequenz für Satz 2:** Da $T_{\text{Suche}}(N, C, n) = O(\log N + C) + O(C \cdot n^3)$ die Komplexität *eines einzelnen* Subgraph-Vergleichs als $O(n^3)$ in die Pipeline einsetzt, wäre eine Substitution durch VF2, Ullmann, RI oder QuickSI formal nicht mit demselben Korollar (konstante Suchzeit unabhängig von N) vereinbar, da $O(n^3)$ durch eine worst-case-exponentielle Schranke ersetzt würde. Die empirisch hohe Average-Case-Effizienz von VF2/RI/QuickSI auf realistischen Graphen (Ergebnis 2, sowie [14, 13]) bedeutet zwar, dass eine solche Substitution *im Mittel* ähnlich performant sein könnte, jedoch ohne die formale Garantie aus Satz 2 — ein Datenbanksystem mit harten Latenzanforderungen (SLA) profitiert daher strukturell vom Subgraph Algorithmus.

3. **Komplementarität statt Konkurrenz:** Die in [13, 14] beschriebenen Optimierungen (Knotenordnung nach Seltenheit, Nachbarschaftssignaturen) sind *orthogonal* zum Subgraph Algorithmus und könnten gemäß Abschnitt 12 (z. B. R-Baum-Pre-Filter) auf die *Pre-Filter-Stufe* der Gen-DB-Pipeline angewendet werden, während der eigentliche Vergleich der gefilterten Kandidaten durch den Subgraph Algorithmus mit garantierter $O(n^3)$-Schranke erfolgt. Insofern stehen die hier verglichenen Verfahren nicht in direkter Konkurrenz, sondern adressieren unterschiedliche Stufen derselben Pipeline (Kandidatenreduktion vs. finaler Strukturvergleich).

Kapitel 7

Experimentelle Untersuchung der Parallelisierung

Hardwareplattform der lokal ausgeführten Experimente: Rechner `WBIS-H207085` von LENOVO, Modell `10T8S6LB00`, mit 7,87 GB Arbeitsspeicher; ein Intel-Core-i3-8100T-Prozessor mit 3,10 GHz, vier physischen Kernen und vier logischen Prozessoren (x64); Microsoft Windows 11 Pro, 64-Bit, Version 10.0.26200.

Die bisherigen Abschnitte haben die Suche von Gen-DB formal analysiert (Abschnitt 4), an einer synthetischen Datenbank mit einer Million Netzwerken statistisch charakterisiert (Abschnitt 5) und den Subgraph Algorithmus in Einzelvergleichen gegen VF2 gemessen (Abschnitt 6). Offen geblieben ist die Frage, wie sich die *gesamte* Suche verhält, wenn der dominierende Kostenterm $O(C \cdot n^3)$ aus Satz 2 auf mehrere Prozesse verteilt wird. Dieser Abschnitt beantwortet sie empirisch: Er beschreibt die in Gen-DB implementierte parallele Ausführung der Kandidatenvergleiche, den Aufbau eines reproduzierbaren Experiments mit 1, 2 und 3 Worker-Prozessen und wertet die Messergebnisse anhand von vier Diagrammen und zwei Tabellen aus.

7.1 Fragestellung und Hypothesen

Nach Satz 2 setzt sich die Suchzeit aus dem Pre-Filter $O(\log N + C)$ und den C strukturellen Vergleichen $O(C \cdot n^3)$ zusammen. Da die Vergleiche für verschiedene Kandidaten voneinander unabhängig sind, lässt sich die Laufzeit einer Parallelisierung über p Worker-Prozesse durch

$$T_{\text{parallel}}(C, n, p) = O\left(\frac{C}{p} \cdot n^3\right) + O(n^3)$$

modellieren. Das Experiment prüft drei daraus abgeleitete Hypothesen:

H1 Die Gesamtzeit einer Suche sinkt bei Verdopplung der Worker-Zahl von 1 auf 2 annähernd um den Faktor 2, weil der parallelisierbare Vergleichsanteil die Laufzeit dominiert.

H2 Mehr Worker-Prozesse als logische Prozessoren der Maschine bringen keinen weiteren Gewinn, da die Vergleiche CPU-gebunden sind und keine zusätzliche Rechenkapazität entsteht.

H3 Die Suchzeit wächst, unabhängig von der Worker-Zahl, annähernd linear mit der Kandidatenzahl C, wie es der Term $O(C \cdot n^3)$ bei beschränktem $n \leq 20$ erwarten lässt.

7.2 Implementierung der parallelen Suche

Die Suche ist in der Funktion `search_subgraph` des Moduls `crud` umgesetzt und läuft in drei Phasen. Zuerst liest eine einzige SQL-Abfrage alle Netzwerke, die die notwendigen Bedingungen $n_k \geq n_Q$ und $m_k \geq m_Q$ des Pre-Filters (Abschnitt 4.2) erfüllen, zusammen mit ihren Adjazenzmatrizen. Danach übergibt `compare_many` aus dem Modul `subgraph_executor` die Matrizen an einen `ProcessPoolExecutor`, dessen Größe über die Konfigurationsvariable `SUBGRAPH_MAX_WORKERS` bestimmt wird. Jeder Worker-Prozess erzeugt genau eine Instanz des Subgraph Algorithmus und ruft für ein Paar aus Query A und Kandidat B die Methode `compare_graphs(A, B)` auf. Weil der Algorithmus in reinem Python implementiert und CPU-gebunden ist, werden Prozesse statt Threads verwendet; der Global Interpreter Lock würde Threads sonst serialisieren. Um den Kommunikationsaufwand zwischen Hauptprozess und Workern zu verringern, werden die Kandidaten in Blöcken (*Chunks*) von 256 Vergleichen pro Task verteilt. Schließlich baut der Hauptprozess aus den Entscheidungen `keep_B` und `equal_keep_*` die Trefferliste auf.

Zwei Eigenschaften dieser Implementierung sind für die Auswertung wichtig. Erstens enthält die aktuelle Fassung *kein* Kandidatenlimit K; alle Kandidaten des Pre-Filters werden verglichen, sodass Satz 3 mit $K = \infty$ gilt und die gemessenen Zeiten einer vollständigen Suche entsprechen. Zweitens sind nur die Vergleiche parallelisiert. Das Lesen der Kandidaten aus PostgreSQL, die Übertragung der Matrizen an die Worker und der Aufbau der Trefferliste laufen im Hauptprozess und bilden den seriellen Anteil der Suche.

7.3 Versuchsaufbau

Das Experiment wird durch das Skript `src/experiment_search_workers.py` gesteuert. Es wählt mit festem Seed zufällig Netzwerke aus der Datenbank und verwendet sie als Suchanfragen. Um Verzerrungen durch zu kleine Muster zu vermeiden, besitzen alle Queries mindestens 15 Knoten. Für jede Worker-Anzahl werden *dieselben* Anfragen in derselben Reihenfolge über `crud.search_subgraph` ausgeführt, sodass die Zeiten als gepaarte Messungen direkt vergleichbar sind. Vor jeder Messreihe wird der Prozess-Pool neu erzeugt und alle Worker werden vorab gestartet, damit die Startzeit der Prozesse nicht in die Messung eingeht. Eine zusätzliche Aufwärmanfrage pro Worker-Anzahl wird verworfen, um Caching-Effekte der Datenbank auszugleichen. Gemessen wird die Wanduhrzeit der gesamten Funktion mit `time.perf_counter()`; sie umfasst somit Datenbankabfrage, Vergleiche, Prozesskommunikation und Aufbau der Trefferliste. Nach jeder Anfrage schreibt das Skript die bisherigen Messwerte in eine JSON-Datei, sodass ein Abbruch keine Daten verliert.

Die ursprüngliche Voreinstellung des Skripts sieht 50 Anfragen je Worker-Anzahl vor. Da eine einzelne Suche auf dem verwendeten Rechner bis zu rund 400 Sekunden dauert, hätte diese Konfiguration mehrere Stunden benötigt; die Messreihe wurde deshalb auf 6 Anfragen begrenzt (Aufruf: `python src/experiment_search_workers.py -queries 6 -workers 1 2 3`). Die Folgen dieser Entscheidung für die Aussagekraft werden in Abschnitt 7.7 diskutiert.

Tabelle 7.1: Parameter und Umgebung des Experiments.

Parameter	Wert
Datenbank	1.000.002 Netzwerke (davon 333.500 mit $n_k \geq 15$)
Anzahl Anfragen	6 je Worker-Anzahl, identisch über alle Reihen
Query-Auswahl	zufällig aus der Datenbank, $n_Q \geq 15$, Seed 42
Worker-Anzahlen	$p \in \{1, 2, 3\}$
Aufwärmanfragen	1 je Worker-Anzahl (verworfen)
Chunk-Größe	256 Vergleiche pro Task
Kandidatenlimit	keines
Datenbanksystem	PostgreSQL 16 auf demselben Rechner
Software	Python 3.13, Windows 11 Pro, 64-Bit

7.4 Metriken

Für eine Worker-Anzahl p bezeichne $T(p)$ die Summe der Antwortzeiten aller $Q = 6$ Anfragen und $C_\Sigma = \sum_q C_q$ die Summe der Kandidatenzahlen.

Definition 10 (Speedup, Effizienz, Durchsatz)**.** Der *Speedup* ist $S(p) = T(1)/T(p)$, die *parallele Effizienz* $E(p) = S(p)/p$. Der *Durchsatz* ist $\Theta(p) = C_\Sigma/T(p)$ in Kandidaten pro Sekunde, und die *Kosten pro Kandidat* sind $\kappa(p) = T(p)/C_\Sigma$.

Aus dem Speedup lässt sich mit dem Gesetz von Amdahl der nicht parallelisierte Zeitanteil f abschätzen. Gilt $T(p) = T(1)\big(f + \frac{1-f}{p}\big)$, so folgt

$$f = \frac{1/S(p) - 1/p}{1 - 1/p}.$$

Diese Abschätzung ist nur sinnvoll, solange p die Zahl der verfügbaren Prozessoren nicht übersteigt.

7.5 Ergebnisse

7.5.1 Die untersuchten Anfragen

Tabelle 7.2 zeigt die sechs ausgewählten Anfragen. Die Kandidatenzahl C schwankt um mehr als den Faktor 5, zwischen 62.064 und 325.964. Sie ergibt sich aus der Vorauswahl: Bei gleichverteilter Knotenzahl $n_k \in \{3, \ldots, 20\}$ besitzen 333.500 der 1.000.002 Netzwerke mindestens 15 Knoten, und für Queries mit 15 Knoten entscheidet nur noch die Kantenbedingung $m_k \geq m_Q$ über den Rest. Bei der Anfrage 5 mit $m_Q = 56$ Kanten erfüllen 97,7 % dieser Netzwerke die Kantenbedingung, bei der Anfrage 1 mit $m_Q = 68$ noch 82,6 %. Die Anfragen 2 und 6 mit 19 Knoten werden zusätzlich von der Knotenbedingung stark eingeschränkt, da nur Netzwerke mit 19 oder 20 Knoten in Frage kommen. Der Pre-Filter reduziert den Suchraum damit auf 6,2 % bis 32,6 % der Datenbank.

Die Trefferzahlen sind mit 1 bis 7 minimal. Da jede Query selbst ein Netzwerk der Datenbank ist und die Kandidatenmenge dieses Netzwerk enthält, ist zu erwarten, dass der einzelne Treffer der Anfragen mit genau einem Ergebnis dem Query-Netzwerk selbst entspricht. Netzwerke mit mindestens 15 Knoten und der Kantendichte $p = 0{,}3$ sind also in anderen Zufallsnetzwerken praktisch nie als Teilstruktur enthalten. Dieser Befund steht

im Gegensatz zum Serverprotokoll in Abschnitt 4.6, bei dem nahezu alle Kandidaten Treffer waren; die Trefferquote hängt, wie dort bereits vermerkt, stark von der konkreten Query ab. Für die Laufzeitmessung ist das hier günstig, denn der Aufbau der Trefferliste im Hauptprozess ist vernachlässigbar, und die gemessene Zeit wird fast vollständig von Vorauswahl und Vergleichen bestimmt.

Tabelle 7.2: Eigenschaften der sechs Suchanfragen und Suchzeiten t_p in Sekunden bei p Workern.

Nr.	Netzwerk-ID	n_Q	m_Q	C	Treffer	t_1	t_2	t_3	$S(2)$
1	654454	15	68	275.420	1	360,8	193,2	198,4	1,87
2	773650	19	107	62.064	1	70,0	36,7	40,6	1,91
3	432492	16	77	221.027	1	294,9	153,0	169,4	1,93
4	858614	15	70	263.668	3	354,9	185,8	209,2	1,91
5	438514	15	56	325.964	7	403,2	212,0	224,6	1,90
6	88827	19	95	100.911	1	120,4	62,4	65,4	1,93

7.5.2 Gesamtzeit und Speedup

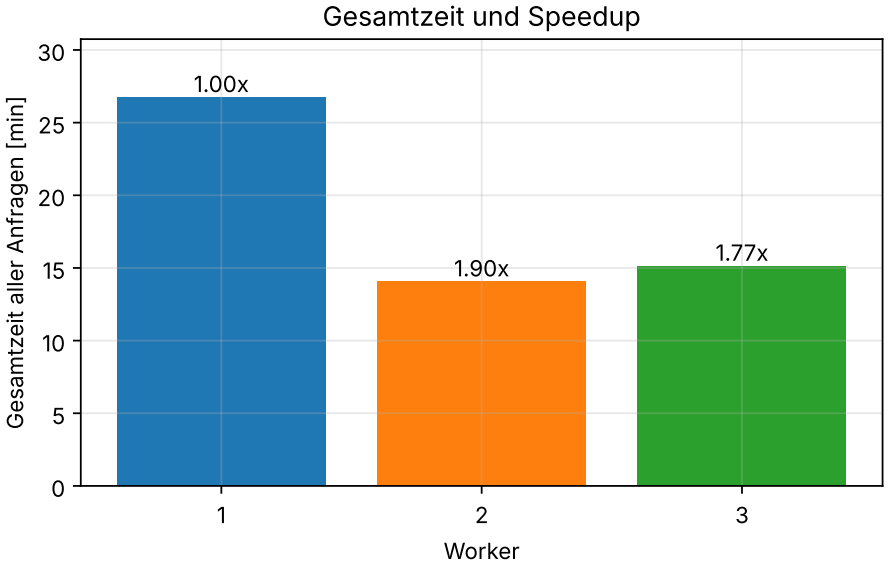


Abbildung 7.1: Gesamtzeit aller sechs Anfragen in Minuten für 1, 2 und 3 Worker mit dem jeweiligen Speedup über den Balken.

Abbildung 7.1 zeigt als Balkendiagramm die Summe der Antwortzeiten aller sechs Anfragen in Minuten, getrennt nach Worker-Anzahl. Über jedem Balken steht der Speedup relativ zur Messreihe mit einem Worker. Mit einem Worker benötigt die Suche zusammen 1604 Sekunden, das sind rund 26,7 Minuten. Mit zwei Workern sinkt dieser Wert auf 843 Sekunden (rund 14,1 Minuten), was einem Speedup von $S(2) = 1{,}90$ und einer Effizienz von $E(2) = 0{,}95$ entspricht. Der dritte Balken liegt bei 908 Sekunden (rund 15,1 Minuten) und damit *über* dem Balken für zwei Worker; der Speedup fällt auf $S(3) = 1{,}77$, die nominale Effizienz auf $E(3) = 0{,}59$. Das Diagramm macht zwei Dinge sichtbar: den nahezu idealen Gewinn beim Übergang von einem auf zwei Worker und die leichte Verschlechterung durch

einen dritten Worker, obwohl dessen Hinzunahme in einer idealen parallelen Maschine einen weiteren Gewinn bringen müsste.

Tabelle 7.3 fasst die Kennzahlen zusammen. Der Durchsatz steigt von 779 auf 1482 Kandidaten pro Sekunde und fällt mit drei Workern auf 1376 zurück. Entsprechend sinken die Kosten pro Kandidat von 1,28 auf 0,68 Millisekunden und steigen mit drei Workern wieder auf 0,73 Millisekunden. Aus $S(2) = 1{,}90$ ergibt sich nach der Formel aus Definition 10 ein nicht parallelisierter Anteil von etwa $f \approx 5\,\%$. Dieser Wert fasst alle seriellen Anteile zusammen, darunter das Laden der bis zu 326.000 Zeilen aus PostgreSQL, die Serialisierung der Matrizen für die Prozesskommunikation und den Aufbau der Trefferliste. Eine getrennte Zuordnung auf die einzelnen Phasen erlauben die vorliegenden Daten nicht, weil die Phasenzeiten im aktuellen Code nicht protokolliert werden.

Tabelle 7.3: Kennzahlen der Suche in Abhängigkeit von der Worker-Anzahl (jeweils 6 Anfragen, $C_\Sigma = 1.249.054$ Kandidaten).

p	$T(p)$ [s]	Mittel [s]	Median [s]	Max. [s]	$S(p)$	$E(p)$	$\Theta(p)$ [1/s]	$\kappa(p)$ [ms]
1	1604	267,4	324,9	403,2	1,00	1,00	779	1,28
2	843	140,5	169,4	212,0	1,90	0,95	1482	0,68
3	908	151,3	183,9	224,6	1,77	0,59	1376	0,73

7.5.3 Verteilung der Antwortzeiten

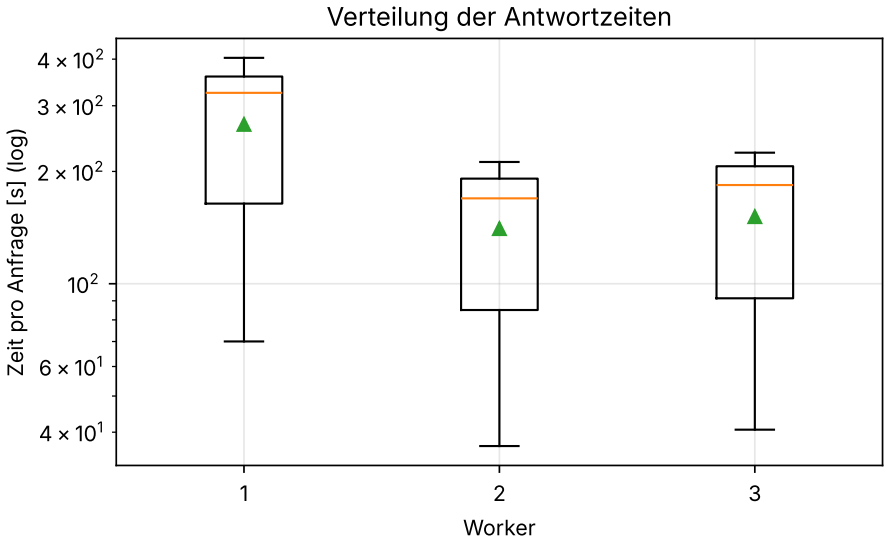


Abbildung 7.2: Verteilung der Antwortzeit pro Anfrage als Boxplot mit logarithmischer Zeitachse und Mittelwert (Dreieck) für 1, 2 und 3 Worker.

Abbildung 7.2 stellt die Antwortzeiten der einzelnen Anfragen als Boxplots dar. Die Zeitachse ist logarithmisch, weil die Zeiten über mehr als eine Größenordnung streuen. Die Box umfasst das untere bis obere Quartil, die orange Linie markiert den Median, das grüne Dreieck den Mittelwert, und die Antennen reichen bis zur kleinsten bzw. größten Messung. Mit einem Worker liegen die Zeiten zwischen 70,0 und 403,2 Sekunden; das untere Quartil beträgt rund 164 Sekunden, das obere rund 359 Sekunden, der Median

324,9 Sekunden und der Mittelwert 267,4 Sekunden. Mit zwei Workern verschiebt sich die gesamte Verteilung nach unten: Die Zeiten reichen von 36,7 bis 212,0 Sekunden, die Quartile liegen bei rund 85 und 191 Sekunden, der Median bei 169,4 und der Mittelwert bei 140,5 Sekunden. Auf der logarithmischen Achse ist die Verschiebung für Minimum, Median und Maximum nahezu gleich groß, die Boxen haben also dieselbe Form. Das bedeutet, dass die Parallelisierung alle Anfragen unabhängig von ihrer Größe um etwa denselben Faktor beschleunigt und nicht nur besonders große oder kleine. Mit drei Workern liegt die Verteilung geringfügig oberhalb derjenigen mit zwei Workern (Median 183,9 Sekunden, Maximum 224,6 Sekunden), aber deutlich unterhalb der Verteilung mit einem Worker.

Der Median liegt bei jeder Worker-Anzahl über dem Mittelwert. Die Verteilung ist also linksschief: Wenige kurze Anfragen, nämlich die beiden Queries mit 19 Knoten und vergleichsweise kleiner Kandidatenmenge, ziehen den Mittelwert herunter, während vier Anfragen mit großem Suchraum die Laufzeit bestimmen. Wegen der geringen Stichprobe von nur sechs Werten sind die Quartile grobe Orientierungswerte; das 95-%-Quantil, das das Skript ebenfalls ausgibt, ist bei sechs Werten identisch mit dem Maximum und wird deshalb hier nicht gesondert aufgeführt.

7.5.4 Skalierung mit der Größe des Suchraums

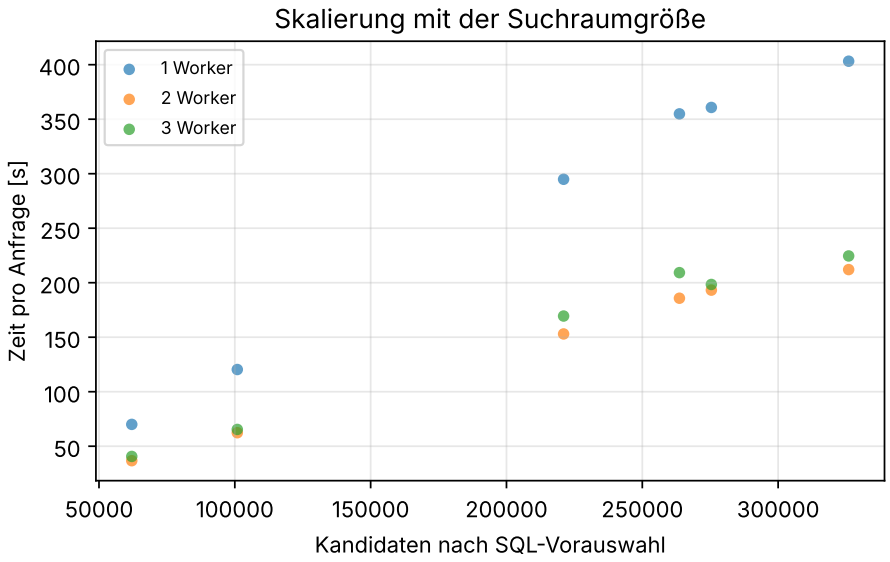


Abbildung 7.3: Streudiagramm der Antwortzeit gegen die Kandidatenzahl nach der SQL-Vorauswahl für 1, 2 und 3 Worker.

Abbildung 7.3 trägt für jede Anfrage und jede Worker-Anzahl die Antwortzeit gegen die Zahl der Kandidaten nach der SQL-Vorauswahl auf. Die Punkte jeder Worker-Anzahl liegen sehr nahe an einer Geraden, was Hypothese H3 stützt. Der Pearson-Korrelationskoeffizient zwischen C und Antwortzeit beträgt 0,996 für einen Worker, 0,996 für zwei und 0,990 für drei Worker. Die lineare Regression liefert für einen Worker eine Steigung von etwa 1,3 Millisekunden je Kandidat, für zwei Worker etwa 0,70 und für drei Worker etwa 0,75 Millisekunden. Die Gerade für einen Worker verläuft also etwa doppelt so steil wie die Geraden für zwei und drei Worker, die ihrerseits fast aufeinander liegen, wobei die Punkte für drei Worker durchgehend leicht höher liegen. Die Parallelisierung ändert damit

den Anstieg, nicht die Form des Zusammenhangs: Eine Verdopplung der Kandidatenzahl verdoppelt weiterhin die Suchzeit, nur mit etwa der halben Steigung.

Zugleich zeigt das Diagramm, dass der Zusammenhang nicht exakt proportional ist. Die Kosten pro Kandidat (Tabelle 7.2, aus den Zeiten t_1 berechnet) betragen für die Anfragen mit 15 oder 16 Knoten zwischen 1,24 und 1,35 Millisekunden, für die beiden Anfragen mit 19 Knoten dagegen nur 1,13 und 1,19 Millisekunden. Die Regressionsgerade besitzt entsprechend einen negativen Achsenabschnitt. Aus der Kubikschranke $O(n^3)$ ließe sich für Queries mit mehr Knoten ein höherer, nicht ein niedrigerer Aufwand pro Vergleich erwarten. Die Schranke ist aber eine Worst-Case-Abschätzung für den Vergleich; die Messung zeigt, dass der tatsächliche Aufwand auf diesen Daten auch von anderen Faktoren abhängt, etwa davon, wie früh ein Vergleich entschieden werden kann. Eine belastbare Erklärung erlaubt die vorliegende Messreihe nicht, weil dafür Zeiten einzelner Vergleiche in Abhängigkeit von n_Q und n_k erhoben werden müssten.

7.5.5 Dieselben Anfragen im direkten Vergleich

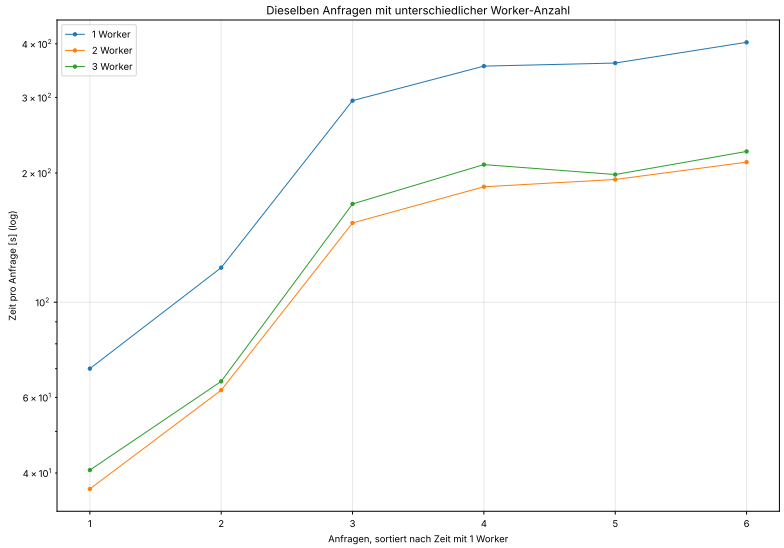


Abbildung 7.4: Antwortzeit jeder einzelnen Anfrage bei 1, 2 und 3 Workern, sortiert nach der Zeit mit einem Worker, logarithmische Zeitachse.

Abbildung 7.4 ordnet die Anfragen auf der horizontalen Achse nach ihrer Laufzeit mit einem Worker und zeichnet für jede Worker-Anzahl einen Linienzug durch die zugehörigen Zeiten. Weil dieselben Anfragen in allen drei Messreihen verwendet wurden, zeigt jeder vertikale Schnitt dieselbe Anfrage unter drei Bedingungen. Die Linie für einen Worker liegt an jeder Stelle oberhalb der beiden anderen, die Linien für zwei und drei Worker verlaufen nahezu parallel zueinander, wobei die Linie für drei Worker für alle sechs Anfragen über der für zwei Worker liegt. Auf der logarithmischen Achse entspricht ein konstanter Faktor einem konstanten vertikalen Abstand; dass die Abstände zwischen den Linien über alle Positionen annähernd gleich bleiben, bestätigt, dass die Beschleunigung von der Größe der Anfrage weitgehend unabhängig ist. Der Speedup $S(2)$ liegt für die einzelnen Anfragen zwischen 1,87 und 1,93 (Tabelle 7.2), der Speedup $S(3)$ zwischen 1,70 und 1,84. Diese enge Streuung ist ein Hinweis, dass der Gesamtwert $S(2) = 1{,}90$ keine Folge einzelner Ausreißer ist. Besonders deutlich ist der Sprung von der zweiten zur dritten Position: Die

Anfrage 3 ist mit 221.027 Kandidaten wesentlich aufwendiger als die Anfrage 6 mit 100.911 Kandidaten, und der Abstand zwischen den Linien bleibt trotzdem derselbe.

7.6 Diskussion

Hypothese H1.

Der Übergang von einem auf zwei Worker liefert einen Speedup von 1,90 bei einer Effizienz von 95 %, und alle sechs Anfragen liegen zwischen 1,87 und 1,93. Die Hypothese ist damit auf dem untersuchten Rechner bestätigt: Der parallelisierbare Vergleichsanteil dominiert die Laufzeit, und der serielle Anteil von etwa 5 % (Laden der Kandidaten, Prozesskommunikation, Aufbau der Trefferliste) begrenzt den Gewinn nur geringfügig. Die Messung stützt das in Abschnitt 7.1 angegebene Modell $T_{\text{parallel}} = O(\frac{C}{p}n^3) + O(n^3)$, wobei der additive Term nur den langsamsten Einzelvergleich erfasst. Die Messung zeigt, dass zusätzlich ein mit C wachsender serieller Anteil existiert, der in diesem Modell nicht abgebildet ist. Er fällt bei $p = 2$ kaum ins Gewicht, würde bei größerem p aber nach dem Gesetz von Amdahl zunehmend den erreichbaren Speedup bestimmen.

Hypothese H2.

Mit drei Workern steigt die Gesamtzeit gegenüber zwei Workern um rund 7,7 %, und der Speedup fällt von 1,90 auf 1,77. Die Messreihe wurde jedoch auf einem Rechner mit vier logischen Prozessoren ausgeführt; drei Worker überschreiten diese Zahl nicht. H2, die einen Leistungsrückgang erst bei mehr Workern als logischen Prozessoren erwartet, ist mit den geprüften Worker-Zahlen somit weder bestätigt noch widerlegt. Beobachtet ist lediglich, dass drei Worker in dieser Stichprobe langsamer waren als zwei. Zusätzliche Prozess- und Übergabekosten, Betriebssystemplanung, Datenbanklast oder Messstreuung sind mögliche Erklärungen, lassen sich aus den Daten aber nicht voneinander trennen. Die allgemeine Empfehlung, Worker-Zahl und verfügbare Rechenkapazität gemeinsam zu bewerten, bleibt sinnvoll; eine Ursache für den Rückgang oder ein Grenzwert oberhalb von vier Workern wurde hier nicht gemessen. Die mitgelieferte `.env`-Datei setzt `SUBGRAPH_MAX_WORKERS=14`; dieser Wert ist eine Konfiguration und kein durch dieses Experiment begründeter Standard.

Hypothese H3.

Die Antwortzeit hängt mit einem Korrelationskoeffizienten von mindestens 0,99 linear von der Kandidatenzahl ab. Das stimmt mit dem Term $O(C \cdot n^3)$ aus Satz 2 bei beschränktem n überein. Die Abweichungen in den Kosten pro Kandidat zeigen allerdings, dass die Konstante nicht allein durch die Knotenzahl bestimmt wird (siehe oben).

Bedeutung für das Korollar zur Datenbankgröße.

Das Korollar zu Satz 2 behauptet eine von N unabhängige Suchzeit, sofern die Kandidatenzahl durch ein Limit K beschränkt ist. Das Experiment läuft bewusst ohne Limit und zeigt die Kehrseite: Bei $N \approx 10^6$ bleiben je nach Query bis zu 32,6 % der Datenbank als Kandidaten übrig, und die Suchzeit steigt dann proportional zu C auf bis zu 403 Sekunden mit einem und 212 Sekunden mit zwei Workern. Ohne Limit wächst C bei fester Selektivität des Pre-Filters linear mit N, und damit auch die Suchzeit. Die Parallelisierung verkleinert den Anstieg um etwa den Faktor p, hebt die Abhängigkeit von N aber nicht auf. Wer harte

Latenzgrenzen einhalten will, braucht deshalb weiterhin einen schärferen Pre-Filter (etwa den in Abschnitt 12 diskutierten mehrdimensionalen Index) oder ein Kandidatenlimit mit den dort beschriebenen Einbußen bei der Vollständigkeit; die Parallelisierung ergänzt diese Maßnahmen, ersetzt sie aber nicht.

7.7 Grenzen der Untersuchung

Die Ergebnisse sind als erste empirische Einordnung zu lesen und aus folgenden Gründen nicht ohne Weiteres verallgemeinerbar.

- **Kleine Stichprobe.** Mit sechs Anfragen und einer Messung je Anfrage und Worker-Anzahl lassen sich keine Standardabweichungen über Wiederholungen und keine Signifikanztests angeben. Dass die Speedups der einzelnen Anfragen eng beieinander liegen, spricht für stabile Verhältnisse, ersetzt aber keine Wiederholungsmessungen. Die ursprünglich vorgesehenen 50 Anfragen sollten auf einem Rechner mit mehr Prozessoren nachgeholt werden.

- **Vier logische Prozessoren.** Die Worker-Reihe reicht nur bis $p = 3$ und prüft daher keine Überbelegung des untersuchten Rechners. Das Skalierungsverhalten oberhalb von vier Workern sowie auf Rechnern mit mehr Prozessoren, insbesondere der Einfluss des seriellen Anteils, ist nicht gemessen. Die Abschätzung $f \approx 5\,\%$ beruht auf einem einzelnen Speedup-Punkt.

- **Gemeinsam genutzter Rechner.** Datenbank, Hauptprozess und Worker teilen sich dieselben Prozessoren. Auf einem System mit separatem Datenbankserver würde sich der serielle Anteil anders verhalten. Die Absolutwerte sind außerdem nicht mit dem Serverprotokoll aus Abschnitt 4.6 vergleichbar, das auf einem anderen System entstand.

- **Keine Phasenzerlegung.** Der Code protokolliert die Zeiten für Laden, Vergleichen und Trefferaufbau nicht, weshalb das Diagramm „Zeit je Phase“ des Skripts leer bleibt und dieser Abschnitt auf seine Wiedergabe verzichtet. Der serielle Anteil kann daher nur indirekt aus dem Speedup abgeschätzt werden.

- **Eine synthetische Graphenfamilie.** Alle Netzwerke stammen aus dem Erdős–Rényi-Modell mit $p = 0{,}3$; für reale biologische Netzwerke mit anderer Dichte und Struktur können Kandidatenzahlen, Trefferquoten und Kosten pro Vergleich abweichen.

- **Treffer von Query und Kandidat.** Da die Queries aus der Datenbank stammen, enthält jede Kandidatenmenge das Query-Netzwerk selbst. Das ist für die Laufzeitmessung unerheblich, spricht aber gegen eine Interpretation der Trefferzahlen als allgemeine Trefferquote.

7.8 Zwischenfazit

Die parallele Ausführung der Kandidatenvergleiche über einen Prozess-Pool beschleunigt die Suche in Gen-DB auf dem untersuchten Rechner mit vier logischen Prozessoren um den Faktor 1,90 (Effizienz 95 %); der Gewinn ist für alle sechs Anfragen nahezu gleich.

Drei Worker waren in dieser Messreihe rund 8 % langsamer als zwei, obwohl die Maschine vier logische Prozessoren bereitstellt. Da nur sechs Anfragen und keine Wiederholungen gemessen wurden, belegt dieser Unterschied weder eine allgemeine optimale Worker-Zahl noch eine Überbelegung der CPU. Die Suchzeit wächst unabhängig von der Worker-Zahl linear mit der Kandidatenzahl und erreicht ohne Kandidatenlimit bei großen Suchräumen mehrere Minuten, sodass die Parallelisierung die Antwortzeit um einen konstanten Faktor senkt, aber die Notwendigkeit eines selektiveren Pre-Filters oder eines Kandidatenlimits nicht beseitigt. Für die Praxis folgt daraus die Empfehlung, die Worker-Anzahl auf dem Zielsystem empirisch zu bestimmen. Die offene Frage, wie stabil der Gewinn bei größeren Stichproben und mehr Workern ist, soll eine Wiederholung der Messreihe klären.

Kapitel 8

Experimenteller Vergleich Python und C++

Hardwareplattform der lokal ausgeführten Experimente: Rechner `WBIS-H207085` von LENOVO, Modell `10T8S6LB00`, mit 7,87 GB Arbeitsspeicher; ein Intel-Core-i3-8100T-Prozessor mit 3,10 GHz, vier physischen Kernen und vier logischen Prozessoren (x64); Microsoft Windows 11 Pro, 64-Bit, Version 10.0.26200.

Die vorangegangene Untersuchung hat gezeigt, dass sich die Suchzeit durch mehrere Worker-Prozesse deutlich reduzieren lässt. Sie beantwortet jedoch nicht, ob auch der einzelne Subgraph-Vergleich schneller ausgeführt werden kann. Dieses Kapitel untersucht deshalb die C++17-Implementierung aus dem Projekt `csubgraph`, die als Library direkt im langlebigen Gen-DB-Worker aufgerufen wird. Ein früherer Messlauf über die C++-CLI hatte den Prozessstart und JSON-Austausch als erhebliche Kostenquelle sichtbar gemacht; seine Messwerte sind von den hier ausgewerteten Library-Werten zu trennen.

Das Experiment vergleicht auf zwei Ebenen. Der Mikrobenchmark misst direkte Python-Aufrufe und Funktionsaufrufe der C++-Library auf denselben Graphpaaren sowie die Kosten eines trivialen Library-Aufrufs. Die Ende-zu-Ende-Messung vergleicht die vollständige Gen-DB-Suche für identische Queries, Kandidatenmengen und Worker-Zahlen. So werden sowohl der Effekt auf den Einzelvergleich als auch der praktisch relevante Einfluss auf die Antwortzeit erfasst. Die abgeschlossene Messreihe vom 7. Oktober 2026 umfasst sechs Suchanfragen; beide Backends lieferten identische Treffer-IDs und es traten keine fehlgeschlagenen Vergleiche auf.

Empirische Sprachvergleiche wie Prechelts Untersuchung mehrerer Implementierungen eines Such- und Zeichenkettenprogramms sowie die spätere Untersuchung von Laufzeit, Speicher- und Energiebedarf über mehrere Sprachen liefern nützlichen Kontext, aber keine Vorhersage für den vorliegenden Graph-Workload [81, 90]. Compiler, Bibliotheken, Datenrepräsentation, Algorithmus, Schnittstellenkosten und Messaufbau beeinflussen das Ergebnis. Die hier berichteten Faktoren gelten deshalb nur für die jeweils kontrollierten Eingaben und Ausführungspfade; insbesondere wird daraus keine allgemeine Rangfolge von C, C++, Python oder anderen Sprachen abgeleitet.

8.1 Motivation und Fragestellung

Der Algorithmus bildet den rechenintensiven Teil der Kandidatenprüfung in Gen-DB. Nach dem datenbankseitigen Pre-Filter wird jedes verbliebene Netzwerk mit der Query verglichen. Bei mehr als einer Million Kandidaten kann selbst eine geringe Zeitersparnis pro

Paar einen großen Unterschied für die gesamte Suche bedeuten. Zugleich unterscheidet sich der Aufwand eines Algorithmusaufrufs von dem Aufwand, den die umgebende Anwendung tatsächlich misst: Daten müssen in das jeweilige Format überführt, an den ausführenden Prozess übergeben und das Ergebnis muss zurücktransportiert und interpretiert werden.

Das Experiment untersucht daher folgende Fragen:

1. Wie unterscheiden sich direkte Aufrufe der Python-Implementierung und der C++-Library für dieselben Graphpaare?

2. Wie groß sind die Übergabekosten der Library im Verhältnis zur Laufzeit eines trivialen Vergleichs?

3. Führt der Einsatz der C++-Library bei identischen Queries, Kandidaten und Worker-Zahlen zu einer kürzeren Ende-zu-Ende-Suchzeit?

4. Liefern beide Backends für dieselben Queries dieselben Treffer-IDs, und treten bei einem Backend fehlgeschlagene Einzelvergleiche auf?

Daraus ergeben sich vier Hypothesen. **H1:** Ein direkter Aufruf des nativen Algorithmuskerns benötigt weniger Zeit als ein Python-Aufruf. **H2:** Der Vorteil bleibt nach Einbeziehung der Library-Schnittstelle groß genug, um die vollständige Gen-DB-Suche zu beschleunigen. **H3:** Beide Implementierungen liefern für alle Queries dieselben Suchtreffer. **H4:** Die Kosten der Library-Schnittstelle sind im Verhältnis zur Rechenzeit klein, während ein Prozess-pro-Vergleich-CLI-Aufruf erheblichen Zusatzaufwand verursachen kann. Der abgeschlossene Mikrobenchmark und Suchlauf liefern für H1 bis H3 direkte Messwerte; H4 wird durch die Library-Aufrufkosten und den separat dokumentierten früheren CLI-Mikrobenchmark eingeordnet.

8.2 Implementierungen und Ausführungspfade

Der Backend-Versuch vergleicht die Python-Implementierung `Subgraph().compare_graphs(A, B)` mit `SubgraphAlgorithm::compareGraphs` aus der nativen Bibliothek. Beide arbeiten mit Signaturfolgen und zyklischen Übereinstimmungsprüfungen; die konkrete native Schwelle und ihre Abweichung von der formalen Relation sind in Bemerkung 1 offengelegt. Die Übereinstimmung ihrer Trefferlisten im abgeschlossenen Experiment ist daher ein Ergebnis für die getesteten Eingaben, kein Beweis, dass beide Implementierungen die in den formalen Sätzen definierte vollständige Teilsequenzrelation realisieren. Für C++ wird die statische Bibliothek `libsubgraphlib.a` mit einem C-Wrapper zu einer DLL gelinkt und über `ctypes` angesprochen. Die DLL wird einmal pro Worker-Prozess geladen; ein Vergleich erfolgt danach als direkter Funktionsaufruf.

CPython erlaubt sowohl native Erweiterungsmodule als auch den Aufruf einer bestehenden C-Schnittstelle; `ctypes` wird hier für den letzteren Weg verwendet. Für C++ wären etwa pybind11-Bindings eine andere Integrationsmöglichkeit. Cython und tracing JITs wie PyPy zeigen weitere Ansätze, bei denen typisierte beziehungsweise häufig ausgeführte Python-Pfade nativ kompiliert werden [83, 85, 84, 82, 89]. Diese Alternativen werden in diesem Experiment nicht gegeneinander getestet; die Hinweise dokumentieren lediglich, dass der Sprachwechsel keine isolierte Sprachmessung darstellt, sondern eine Entscheidung über Implementierung und FFI-Grenze.

Beide Varianten laufen in einem langlebigen `ProcessPoolExecutor` mit zwei Workern und erhalten Kandidaten blockweise mit Chunk-Größe 256. Anders als bei der früheren CLI-Integration wird für C++ weder je Kandidat ein weiterer Prozess gestartet noch werden Matrizen und Resultate über JSON-Pipes serialisiert. Der einmalige Bau und das Linken des Wrappers finden vor dem Start des Pools statt und sind nicht Teil der gemessenen Suchzeit. Der Mikrobenchmark misst den direkten Funktionspfad einschließlich der Übergabe der Arrays an den Wrapper und vergleicht somit reale Aufrufpfade, nicht abstrakt bereinigte Sprachlaufzeiten. Die Python-Dokumentation unterscheidet die Einschränkungen von Threads unter dem Global Interpreter Lock von prozessbasierter Parallelität; `multiprocessing` und `concurrent.futures` dokumentieren die Prozess- und Executor-Schnittstellen, die diesen Ausführungspfad bereitstellen [86, 87, 88]. Daraus folgt kein allgemeiner Performancevorteil von Prozessen: Start, Serialisierung und Datentransfer bleiben Teil des jeweils zu messenden Systems.

Das Experiment kontrolliert, dass in Haupt- und Worker-Prozessen tatsächlich das jeweils angeforderte Backend aktiv ist, zählt fehlgeschlagene Vergleiche und gleicht für jede Anfrage die gefundenen Netzwerk-IDs ab. Damit lässt sich neben der Laufzeit auch die Ergebniskonformität der beiden Implementierungen beurteilen.

Die Messmethodik folgt dem Grundsatz, dass identische Eingaben allein noch keine faire oder robuste Laufzeitbewertung garantieren. Benchmark-Studien zeigen, dass Laufzeitmessungen von Laufzeitumgebung, Versuchsaufbau und Auswertungsverfahren beeinflusst werden können; sie empfehlen kontrollierte Bedingungen, Wiederholungen und eine statistisch angemessene Zusammenfassung [91, 92, 93, 96, 97]. Für normalisierte Benchmarkwerte ist außerdem die Wahl des Aggregats entscheidend [94]. Die Regeln der SPEC-CPU-Suite veranschaulichen, wie Build-, Lauf- und Berichtsbedingungen formal festgehalten werden; sie sind kein Zertifikat dafür, dass das vorliegende Experiment ein SPEC-konformes Benchmark ist [95]. Für kurze Funktionsmessungen beschreibt Python `timeit` samt seinen Vorbehalten; für die Zeitmessung einzelner beziehungsweise vollständiger Abläufe dokumentiert `time` den hochauflösenden monotonen Zähler `perf_counter` [98, 99].

8.3 Versuchsaufbau

Das Experiment wird durch `src/experiment_search_backends.py` gesteuert. Es verwendet dieselbe Datenbank für beide Implementierungen, wählt mit Seed 42 sechs Netzwerke als Queries und setzt die Untergrenze auf 15 Knoten. Beide Backends werden mit zwei Workern und in derselben Reihenfolge ausgewählten Queries ausgeführt. Vor jeder Messreihe wird eine Aufwärmanfrage verworfen. Die JSON-Datei wird nach jeder abgeschlossenen Anfrage aktualisiert, damit ein Abbruch die bis dahin geschriebenen Messwerte nicht verliert. Zusätzlich prüft das Skript die Treffer-IDs, kontrolliert die tatsächlich in den Worker-Prozessen aktive Implementierung und zählt fehlgeschlagene Einzelvergleiche.

Der Mikrobenchmark wird ohne Datenbank ausgeführt. Mit Seed 42 werden 200 Zufallspaare erzeugt. Die erste Matrix besitzt zwischen 15 und 19 Knoten, die zweite 20 Knoten; jede mögliche gerichtete Kante wird mit Wahrscheinlichkeit 0,3 gewählt. Beide Backends verarbeiten dieselben Paare. Für C++ wird außerdem der direkte Bibliotheksaufruf mit zwei trivialen 1×1-Matrizen gemessen. Dieser Wert nähert die Kosten von Array-Marshalling und Funktionsübergabe an, nicht jedoch alle denkbaren Aufwände größerer Matrizen.

Die volle Suche misst mit `time.perf_counter()` die Wanduhrzeit von

Tabelle 8.1: Versuchsparameter und Umgebung des Backend-Vergleichs.

Parameter	Wert
Backend-Reihenfolge	Python, anschließend C++-Library
Worker je Backend	2
Queries	6, zufällig aus der Datenbank, Seed 42
Mindestgröße der Queries	15 Knoten
Aufwärmung	eine verworfene Suche je Backend
Mikrobenchmark	200 Zufallspaare, Seed 42
Knotenzahl Mikrobenchmark	$n_A \in \{15, \ldots, 19\}$, $n_B = 20$
Kantenwahrscheinlichkeit	0,3
Library-Referenz	200 Aufrufe mit trivialen 1×1-Matrizen
Betriebssystem	Windows 11 Pro, 64-Bit, Version 10.0.26200
Python	3.14.8
Logische Prozessoren	4
C++-Bibliothek	`libsubgraphlib.a`, Wrapper-DLL über `ctypes`

`crud.search_subgraph()`. Sie umfasst damit die Datenbankabfrage, die Vergleichsphase, die Kommunikation mit den Worker-Prozessen und den Aufbau der Trefferliste. Zusätzlich werden die in `compare_many` verbrauchte Zeit und der als Differenz berechnete Rest der Suche protokolliert. Für jede Query werden Knotenzahl, Kantenzahl, Zahl der Kandidaten nach SQL-Vorauswahl, Trefferzahl und Fehlerzahl festgehalten. Die Summe der Antwortzeiten je Backend erlaubt den direkten Gesamtvergleich; die jeweiligen Treffer-IDs dienen dem Abgleich der funktionalen Ergebnisse.

8.4 Metriken

Für dieselbe Query q bezeichne $t_{\mathrm{Py},q}$ die gemessene Suchzeit mit der Python-Implementierung und $t_{\mathrm{C++},q}$ die Zeit mit der C++-Library. Der relative Such-Speedup ist

$$S_{\mathrm{C++}} = \frac{\sum_q t_{\mathrm{Py},q}}{\sum_q t_{\mathrm{C++},q}}.$$

Ein Wert größer als 1 bedeutet, dass die C++-Library in der gemessenen Gen-DB-Ausführung schneller war; ein Wert kleiner als 1 bedeutet eine langsamere Suche. Der Quotient vergleicht die gesamte Integration und nicht allein den nativen Algorithmuskern. Ergänzend misst das Skript den Kandidatendurchsatz $\Theta = C_\Sigma / T$ und die Kosten pro Kandidat $\kappa = T / C_\Sigma$, wobei C_Σ die Summe der Kandidatenzahlen und T die Summe der Antwortzeiten eines Backends ist.

Die funktionale Übereinstimmung wird pro Query über die sortierten Listen der gefundenen Netzwerk-IDs geprüft. Zusätzlich vergleicht das Skript die erwartete Backend-Kennung in Haupt- und Worker-Prozessen und protokolliert fehlgeschlagene Vergleiche. Dadurch ist ein stiller Rückfall auf Python nicht mit einem erfolgreichen C++-Lauf zu verwechseln. Der Mikrobenchmark beantwortet eine andere Frage als der Such-Speedup: Er isoliert den wiederholten direkten Funktionsaufruf im Hauptprozess; die Ende-zu-Ende-Messung umfasst zusätzlich Datenbankzugriff, Worker-Kommunikation und Ergebnisaufbau.

8.5 Ergebnisse der abgeschlossenen Messreihe

Die ausgewertete Datei `search_backends_20261007_104804.json` ist mit `complete=true` markiert. Sechs Queries mit mindestens 15 Knoten wurden bei zwei Workern in beiden Backends ausgeführt; zusammen durchlief jede Implementierung exakt 931.899 Kandidaten. Tabelle 8.2 fasst die Such- und Kalibrierungskennzahlen zusammen. Die Laufzeiten pro Query in Tabelle 8.4 zeigen ergänzend, dass der beobachtete Gewinn nicht von einer einzelnen Anfrage getragen wird.

Tabelle 8.2: Kennzahlen der Suche je Implementierung (jeweils 6 Anfragen, $C_\Sigma = 931.899$ Kandidaten, Speedup S relativ zu Python).

Variante	p	T [s]	Mittel [s]	Median [s]	Max. [s]	S	Θ [1/s]	κ [ms]	seriell [%]
Python	2	1.180	196,7	145,3	379,2	1,00	790	1,267	11,6
C++	2	186	30,9	26,2	63,6	6,36	5.020	0,199	66,7

Tabelle 8.3: Zeit je Einzelvergleich im Hauptprozess (200 Zufallspaare mit $n_A \in \{15, \ldots, 19\}$, $n_B = 20$, Kantenwahrscheinlichkeit 0,3).

Messung	Median [ms]	Mittel [ms]	Max. [ms]
Python, Einzelvergleich	2,793	2,694	3,703
C++ (Bibliothek), Einzelvergleich	0,068	0,074	0,331
C++ (Bibliothek), trivialer 1×1-Aufruf	0,018	0,018	0,073

Tabelle 8.4: Antwortzeiten und Ergebnisse der sechs identischen Suchanfragen mit jeweils zwei Workern.

Nr.	Netzwerk-ID	n_Q	m_Q	Kandidaten	Treffer	Python [s]	C++ [s]	Speedup
1	654.525	18	106	69.173	1	97,39	11,11	8,76×
2	773.692	19	104	74.482	1	85,22	12,33	6,91×
3	432.763	15	63	302.555	1	379,18	51,08	7,42×
4	858.300	20	110	38.496	1	50,66	7,31	6,93×
5	438.602	15	66	287.228	1	374,62	63,62	5,89×
6	89.411	18	83	159.965	1	193,21	40,16	4,81×

Die Python-Suche benötigte insgesamt 1.180,28 Sekunden, also 19 Minuten und 40 Sekunden; die C++-Library benötigte 185,62 Sekunden, rund 3 Minuten und 6 Sekunden. Der aggregierte Speedup beträgt damit 6,36× und die Suchzeit sinkt um etwa 84,3 %. Der Geschwindigkeitsgewinn ist für jede der sechs Einzelanfragen sichtbar: Die individuellen Speedups reichen von 4,81× bis 8,76×. Ihre Streuung macht zugleich deutlich, dass ein einzelner Gesamtfaktor keine für jede Query identische Beschleunigung verspricht.

Die Query mit den meisten Kandidaten (302.555) benötigt mit Python 379,18 Sekunden und mit C++ 51,08 Sekunden; die zweite Anfrage mit annähernd ebenso vielen Kandidaten zeigt mit 287.228 Kandidaten und 374,62 beziehungsweise 63,62 Sekunden ein ähnliches, aber etwas kleineres Verhältnis. Die Query mit den wenigsten Kandidaten (38.496) ist in beiden Backends die schnellste. Die Messreihe passt damit zur erwarteten Abhängigkeit von der Größe des Suchraums. Wegen nur sechs Queries und variierender Knotenzahlen ist

sie jedoch keine kontrollierte Regression, aus der eine allgemeingültige Zeit pro Kandidat abgeleitet werden könnte.

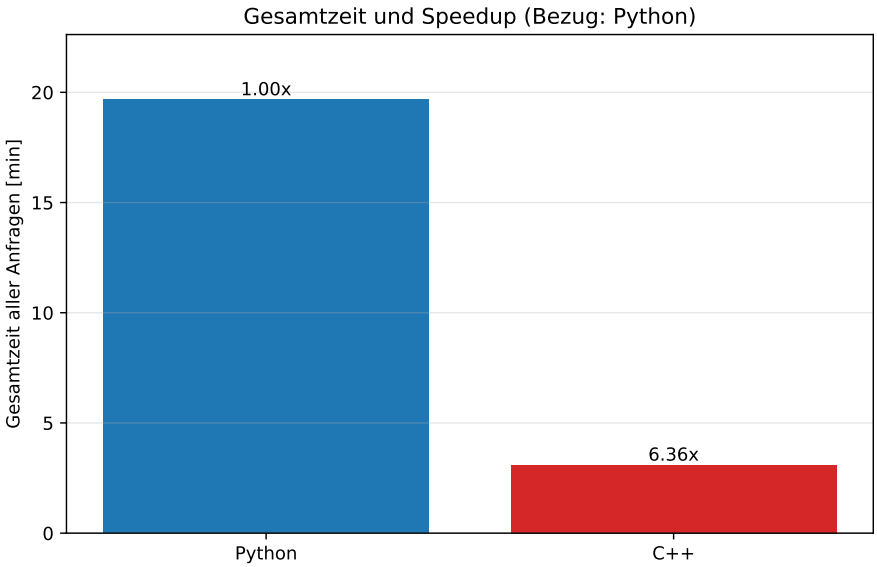


Abbildung 8.1: Die Gesamtlaufzeit der sechs Suchen sinkt mit der C++-Library gegenüber Python auf rund ein Sechstel.

Abbildung 8.1 stellt die aufsummierte Suchzeit in Minuten dar; die Beschriftungen über den Balken geben den Speedup relativ zur Python-Referenz an. Die Differenz ist groß genug, dass sie nicht durch die Wahl einer einzelnen Aggregationskennzahl erklärt wird: Verglichen werden die Summe derselben sechs Antwortzeiten, dieselben Kandidaten und dieselbe Worker-Zahl. Die Grafik belegt den Anwendungseffekt dieser konkreten Library-Integration auf dieser Hardware, nicht eine universelle Beschleunigung für alle Datenbanken oder Query-Arten.

Abbildung 8.2 verwendet eine logarithmische Zeitachse und zeigt Median, Quartile, Spannweite und Mittelwert der Antwortzeiten. Die beiden Verteilungen liegen klar getrennt: Die mittlere Antwortzeit sinkt von 196,7 auf 30,9 Sekunden, der Median von 145,3 auf 26,2 Sekunden. Die logarithmische Darstellung macht sowohl die Beschleunigung als auch die Unterschiede zwischen kleinen und großen Suchräumen lesbar; die sechs Beobachtungen pro Backend erlauben allerdings keine robuste Aussage über die Verteilung zukünftiger Anfragen.

8.6 Phasenanalyse und Skalierung mit der Kandidatenzahl

Abbildung 8.3 stellt die mittlere Antwortzeit je Anfrage als gestapelte Vergleichsphase und Restzeit dar. Die Python-Vergleichsphase beansprucht aggregiert 88,4 % der Gesamtlaufzeit; bei C++ sind es nur 33,3 %. Entsprechend sinkt die Vergleichszeit über die sechs Anfragen von insgesamt rund 1.043 Sekunden auf rund 61,9 Sekunden, während die gemessene Restzeit von etwa 137 auf 123,7 Sekunden zurückgeht. Der Rest umfasst Laden, Datenübertragung beziehungsweise Aufgabenverteilung und Ergebnisaufbau und ist nicht mit einer reinen Datenbankzeit gleichzusetzen. Da der Rechenkern nun erheblich schneller

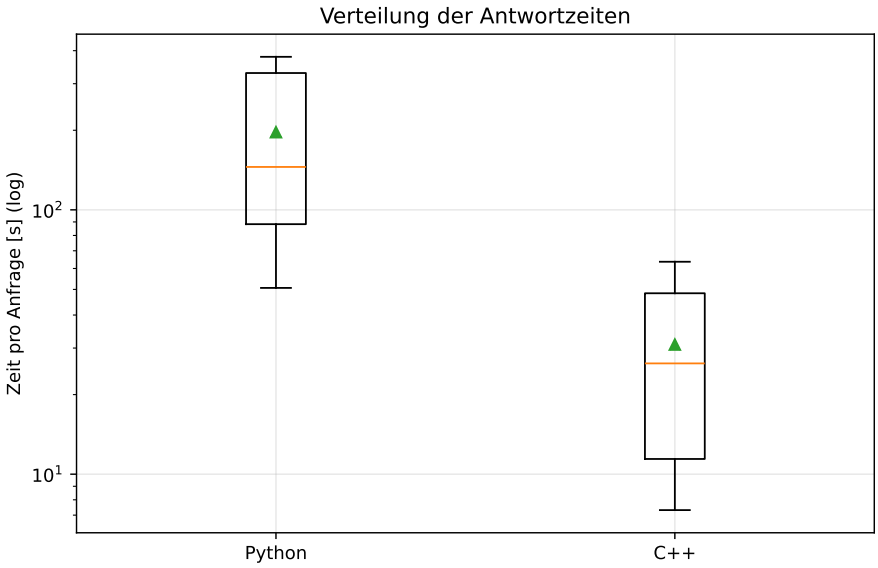


Abbildung 8.2: Die Verteilung der sechs Antwortzeiten zeigt für beide Backends eine deutliche Verkürzung bei C++.

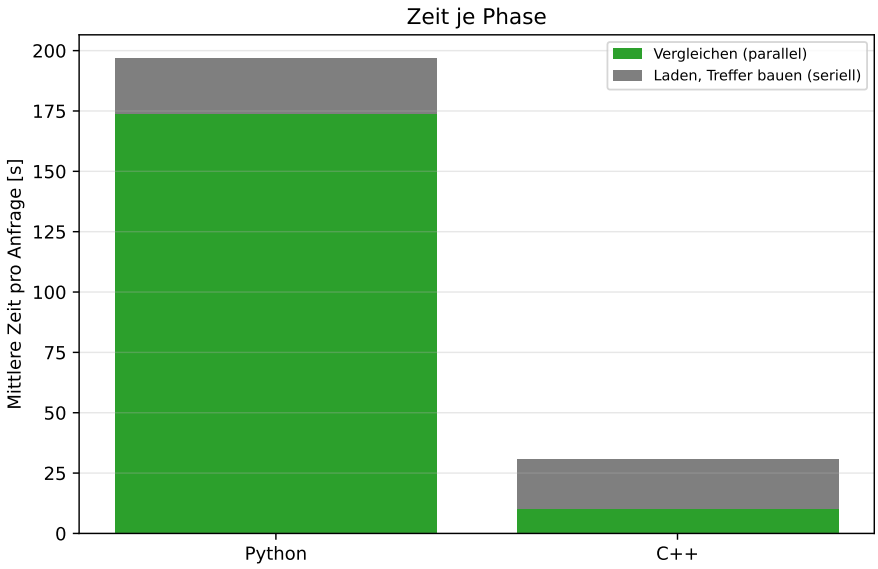


Abbildung 8.3: Die Phasenmessung zeigt, wie sich Vergleichszeit und übrige Suchzeit zwischen den Backends verschieben.

ist, wird dieser Anteil relativ dominant: Beschleunigungen weiterer Vergleichsoperationen allein haben abnehmenden Einfluss auf die gesamte Antwortzeit. Strömendes Laden, effizientere Aufgabenübergabe und Phaseninstrumentierung werden dadurch als nächste Optimierungsziele relevanter.

Abbildung 8.4 ordnet jede Anfrage nach der Zahl ihrer Kandidaten und zeigt für Python wie C++ einen klaren Anstieg der Antwortzeit mit wachsendem Suchraum. Die Punkte bilden keine streng proportionale Linie: Queries unterscheiden sich nicht nur in ihrer Kandidatenzahl, sondern auch in Knotenzahl und Struktur, und außerhalb der Vergleichsphase fallen weitere Kosten an. Insgesamt verarbeitet Python rund 790 Kandidaten pro Sekunde,

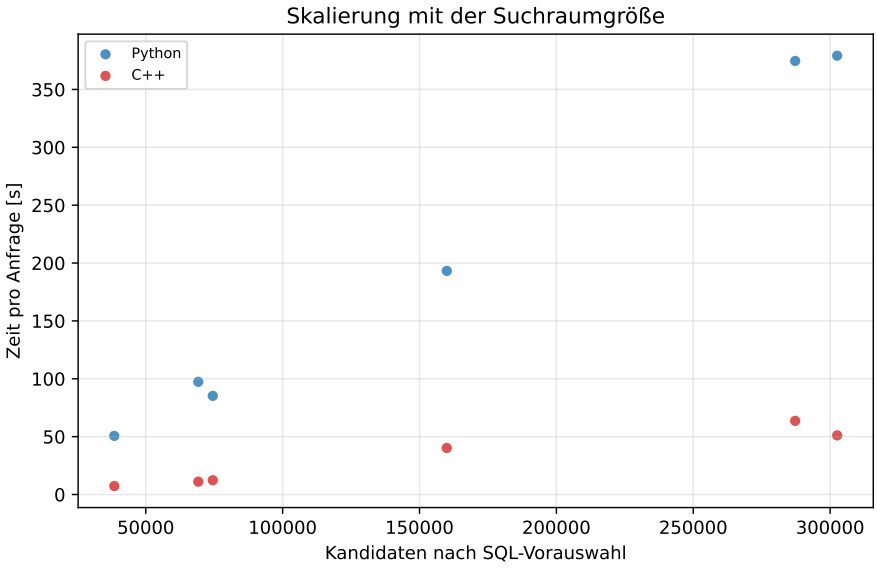


Abbildung 8.4: Die Antwortzeit nimmt für beide Backends mit der nach der SQL-Vorauswahl verbleibenden Kandidatenzahl zu.

während die Library rund 5.020 Kandidaten pro Sekunde erreicht; der aggregierte Aufwand sinkt von 1,267 auf 0,199 Millisekunden je Kandidat. Diese Quotienten beschreiben die sechs vollständigen Suchläufe einschließlich ihrer nichtvergleichenden Anteile und sind deshalb keine isolierten Kosten des Algorithmuskerns.

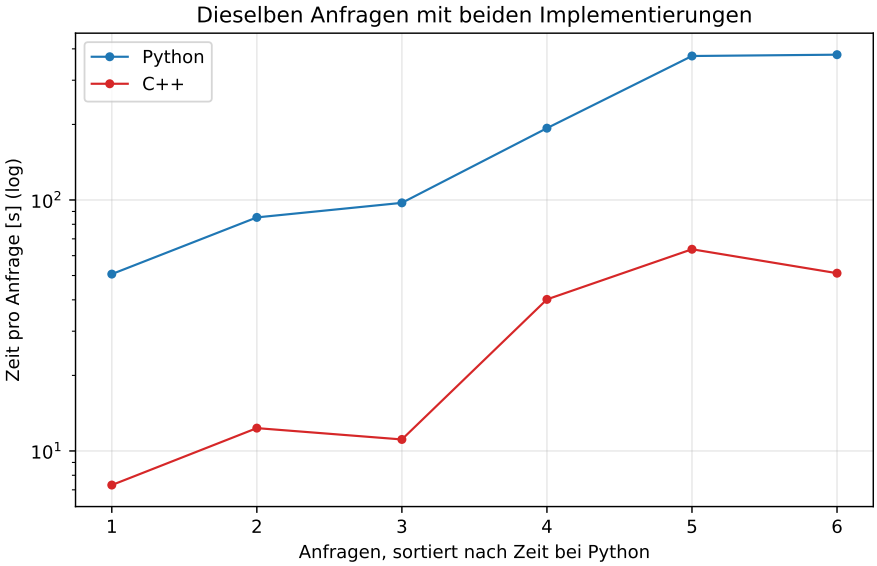


Abbildung 8.5: Für jede einzelne Query liegt die Antwortzeit der C++-Library unter derjenigen der Python-Implementierung.

Abbildung 8.5 zeigt gepaart dieselben sechs Queries, nach der Python-Antwortzeit sortiert und auf logarithmischer Skala. Beide Kurven folgen derselben Reihenfolge: Die aufwendigen Queries bleiben auch mit C++ aufwendiger als die kleineren, jedoch liegt jeder C++-Messpunkt deutlich unter seinem Python-Pendant. Dies schließt aus, dass der aggregierte Vorteil allein aus einer anderen Reihenfolge, einem anderen Query-Sample oder

weniger Kandidaten im C++-Lauf entsteht.

8.7 Mikrobenchmark, funktionale Kontrolle und Diskussion

Der Mikrobenchmark isoliert die direkten Vergleichsaufrufe auf 200 identischen Zufallspaaren. Tabelle 8.3 enthält Mittelwert, Median und Maximum. Der C++-Library-Aufruf benötigt im Mittel 0,074 Millisekunden je Paar, der direkte Python-Aufruf 2,694 Millisekunden. Das Verhältnis der Mittelwerte entspricht einer Beschleunigung von ungefähr 36,5×; anhand der Mediane ergibt sich etwa 40,8×. Die Maximalwerte betragen 0,331 beziehungsweise 3,703 Millisekunden. Damit bestätigt H1 für diesen Benchmark deutlich einen schnelleren direkten C++-Aufruf.

Der triviale 1×1-Library-Aufruf benötigt im Mittel 0,018 Millisekunden und im Median 0,018 Millisekunden. Das entspricht etwa einem Viertel der mittleren gemessenen C++-Zeit für die größeren Vergleichspaare und zeigt, dass die Funktionsübergabe samt Marshalling nicht null ist, aber weit unter der gemessenen Python-Aufrufzeit liegt. Anders als eine Differenz aus CLI-Messungen ist dieser Wert keine Prozessstartzeit: Er ist ein direkter Aufruf durch die Wrapper-Schnittstelle. Der triviale Aufruf enthält außerdem andere Matrixgrößen als die eigentlichen Paare; er dient daher nur als Größenordnung für die Mindestkosten und nicht als exakt abziehbarer Anteil.

H1 und H2: Beschleunigung des Einzelvergleichs und der Suche.

Der Mikrobenchmark belegt den Geschwindigkeitsvorteil des direkten C++-Aufrufpfads in der Stichprobe. Auch die Ende-zu-Ende-Suche ist mit der Library deutlich schneller: Die Gesamtzeit reduziert sich von 1.180,28 auf 185,62 Sekunden. Der Such-Speedup von 6,36× fällt kleiner aus als die etwa 36,5× des Mikrobenchmarks, weil die vollständige Anfrage Datenbankzugriff, Prozesskommunikation und Ergebnisaufbau einschließt und weil die Vergleichsphase nach ihrer Beschleunigung nur noch rund ein Drittel der C++-Gesamtzeit ausmacht. Die beiden Speedup-Werte beschreiben damit unterschiedliche Ebenen und widersprechen einander nicht.

H3: funktionale Übereinstimmung.

Für jede Query stimmen die mit Python und C++ gefundenen Netzwerk-IDs überein. Beide Backends liefern je Query einen Treffer, zusammen also sechs Treffer in den sechs jeweiligen Suchläufen. In beiden Reihen beträgt die Fehlerzahl null. Zusätzlich bestätigt die Laufzeitkontrolle, dass die angeforderte Library tatsächlich in den C++-Workern aktiv war. Das ist eine positive differenzielle Prüfung der konkreten sechs Suchläufe, aber noch kein formaler Korrektheitsbeweis für alle Eingaben oder für alle Randfälle des Algorithmus.

H4: Kosten und Grenze der Anbindung.

Der Library-Aufruf verursacht messbare Übergabekosten, doch die direkte Mikrobenchmark-Zeit bleibt deutlich unter der Python-Vergleichszeit. Damit zeigen die neuen Resultate den praktisch relevanten Unterschied zum früheren CLI-Pfad: Dessen Prozess-pro-Vergleich- und JSON-Aufwand hatte einen trivialen Aufruf von rund 10 Millisekunden ergeben; dieser Wert gehört ausdrücklich nicht zur hier untersuchten Library. Die Funktionseinbindung

vermeidet diesen Prozessstart. Sie beseitigt aber weder alle Python/C++-Übergabekosten noch die übrigen Phasen der Suche.

8.8 Grenzen der Messreihe

Die Ergebnisse sind positiv, müssen aber innerhalb des Versuchsaufbaus interpretiert werden:

- **Sechs Queries und jeweils ein Suchlauf.** Der Speedup beruht auf sechs mit Seed 42 reproduzierbar ausgewählten Datenbankanfragen und einer Messung je Backend. Es gibt keine Wiederholungen je Query, Konfidenzintervalle oder Signifikanztests; Cache-, System- und Datenbankzustände können einzelne Zeiten beeinflussen.

- **Synthetische Datenbank und begrenzte Querygrößen.** Die Suchreihe stammt aus der synthetischen Gen-DB-Datenbank; die sechs Queries besitzen 15 bis 20 Knoten. Auf realen biologischen Netzwerken, anderen Dichten und größeren Matrizen kann sich das Verhältnis ändern.

- **Spezifische Hardware und Implementierung.** Gemessen wurde unter Windows 11 mit Python 3.14.8, vier logischen Prozessoren und zwei Workern. Compileroptionen, Prozessorarchitektur, Betriebssystem, Datenbanklast und Wrapper beeinflussen die Laufzeiten.

- **Mikrobenchmark ist kein vollständiger Algorithmusvergleich.** Die 200 Zufallspaare decken Matrizen mit 15 bis 20 Knoten und Kantendichte um 0,3 ab. Sie belegen den Vorteil unter dieser Verteilung, aber nicht für alle Graphklassen. Der Vergleich schließt zudem die konkrete Python-zu-C++-Übergabe ein.

- **Phasenrest ist zusammengesetzt.** Die Differenz zwischen Suchzeit und Vergleichszeit verbindet mehrere Vorgänge und darf weder als reine Datenbanklaufzeit noch als reine Parallelisierungskosten ausgelegt werden. Eine feinere Instrumentierung wäre nötig, um die verbliebenen rund zwei Drittel der C++-Zeit kausal aufzuteilen.

- **Trefferabgleich ist kein universeller Nachweis.** Identische Treffer-IDs und null Fehler in dieser Stichprobe erhöhen das Vertrauen in die Integration, ersetzen aber keine gemeinsamen Konformitäts- und Eigenschaftstests für alle Eingabekombinationen.

Der Wechsel von Python zur C++-Library beschleunigt in der gemessenen Konfiguration sowohl den direkten Vergleich als auch die vollständige Suche. Bei denselben sechs Anfragen und 931.899 Kandidaten sinkt die Gesamtzeit um den Faktor 6,36; in allen sechs Fällen stimmen die Treffer-IDs überein, und es gibt keine fehlgeschlagenen Vergleiche. Der Mikrobenchmark misst einen noch größeren Vorteil für direkte Aufrufe, während die Phasenanalyse zeigt, dass Laden, Übergabe und Ergebnisaufbau den Ende-zu-Ende-Speedup begrenzen. Die Messergebnisse stützen damit die Library-Integration als sinnvollen Beschleunigungsweg für diesen Anwendungsfall. Sie sind wegen Stichprobengröße, synthetischer Daten und einzelner Hardwarekonfiguration eine belastbare Tendenz, aber keine universelle Leistungsgarantie.

8.9 Bytecodegröße und Hardwareeffizienz

Beobachtet wurde, dass die Größe des Bytecodes des Subgraph Algorithmus der C++-Library größer ist als der Bytecode des Subgraph Algorithmus in der Python Implementierung. Aber die Messungen zeigen, dass die C++-Library für die Suche in Gen-DB deutlich schneller ist als die Python Implementierung. Der Speedup liegt bei 6,36× und die Antwortzeit sinkt von 1.180,28 Sekunden auf 185,62 Sekunden. Die Phasenanalyse zeigt, dass die Vergleichsphase bei C++ nur noch rund ein Drittel der Gesamtzeit ausmacht, während sie bei Python fast neun Zehntel beträgt.

Dass der Bytecode für die C++-Library größer aber die Suche damit schneller ist, ist deshalb so, weil dadurch im Arbeitsspeicher ein größerer Bereich reserviert wird, der viel effizienter von der Architektur zur Ausführung prozessiert werden kann. Wenn der reservierte Bereich dagegen kleiner ist, wie für die Python Implementierung des Subgraph Algorithmus, ist der reservierte Speicherbereich kleiner. Der Strom im reservierten Bereich im Arbeitsspeicher für den Algorithmus liegt parallel an. Ist dieser Bereich kleiner, kann die effiziente Architektur mit Adressbus, Datenbus, Steuerbus und Controller diesen Bereich nicht so schnell ausführen und bearbeiten wie für reservierte Bereiche, die größer sind. Es ist so, dass diese hochentwickelte und für Geschwindigkeit extrem ausgelegte Hardware-Architektur des Prozessors gerade darauf wartet, durch die parallele Bestromung der reservierten Bereiche im Arbeitsspeicher möglichst viele Instruktionen dadurch gleichzeitig prozessieren zu können. Etwas enttäuscht ist sie dann doch, wenn der reservierte Bereich kleiner ist und sie gerade für ihre eigentliche Bestimmung so nicht so sehr zum Ziel kommen kann, was die Ausführung des Subgraph Algorithmus verzögert.

8.10 Fremde Beschreibungen

Die Anreicherung mit einem Fremdmodul in einer anderen Beschreibung erscheint mir sehr unpassend und beschreibungstechnisch nicht durchgängig. Eine lose Kopplung dagegen, angenommen durch eine C++-CLI, erlaubt eine klarere Trennung, bei der alle beteiligten Mitgestalter dieses Softwaresystems mit einem klaren Trennungsbewusstsein dieses System mitgestalten. Ein System, das aus Bereichen oder Teilen besteht, die durch unterschiedliche Beschreibungen entwickelt sind oder arbeiten, schafft Mitgestaltern dieses Systems ein in der Regel eher unruhigeres Trennungsbewusstsein in der Analyse, Weiterentwicklung und Wiederverwendung.

Dieses unruhigere Trennungsbewusstsein kann sich in der Praxis in einer unklaren Verantwortlichkeit und Kostenverteilung an der Grenze zwischen den beiden Beschreibungen äußern. Die Messungen dieses Kapitels dagegen zeigen für Gen-DB zugleich einen deutlichen Laufzeitvorteil der Library. Die weitergehende Position dieser Arbeit lautet, dass klar benannte Grenzen Analyse, Weiterentwicklung und Wiederverwendung unterstützen und dadurch ein gemeinsames Trennungsbewusstsein fördern. Diese Aussage ist eine begründete Gestaltungsposition; die Laufzeitdaten allein messen sie nicht.

Dieser Abschnitt führt die Aussage weiter aus. Er klärt zunächst, was hier unter einer *Beschreibung* verstanden wird, wendet die Aussage dann auf die Messungen dieses Kapitels an und überträgt sie anschließend auf die übrigen Teile der Arbeit. Abschließend werden Gegenargumente und die Grenzen der Position benannt. Wichtig ist die Reihenfolge der Argumente: Die Position beruht in erster Linie auf einer Überlegung zur Gestaltung des Systems. Die Messwerte aus diesem Kapitel illustrieren sie, sie beweisen sie nicht.

8.10.1 Was hier unter einer Beschreibung verstanden wird

Eine *Beschreibung* bezeichnet im Folgenden die Gesamtheit der Vereinbarungen, in denen ein Teil eines Systems formuliert ist: seine Sprache, seine Datentypen, sein Fehlermodell, die Lebensdauer seiner Zustände und die Art, wie er aufgerufen, getestet und ausgeliefert wird. Zwei Teile liegen in derselben Beschreibung, wenn sie diese Vereinbarungen teilen. Dann kann ein Mitgestalter, der den einen Teil versteht, den anderen mit denselben Begriffen lesen. Ein *Fremdmodul* ist ein Teil, der in einer anderen Beschreibung arbeitet und dennoch Teil desselben Systems sein soll.

Gen-DB besteht in der Beschreibung des Backends aus Python-Code. Matrizen liegen dort als `numpy`-Arrays vor, Fehler werden über Python-Ausnahmen signalisiert, und der Subgraph Algorithmus wird als Funktion oder Objekt aufgerufen. Die frühere C++-CLI arbeitete dagegen in einer anderen Beschreibung: Matrizen wurden zu JSON-Text, Fehler zu Exit-Codes und Ausgaben auf `stderr`, und ein Zustand überlebte keinen Aufruf. Die nun eingesetzte Library behält den nativen C++-Kern bei, stellt dem Backend aber eine direkte Funktionsschnittstelle im Worker-Prozess bereit. Sie benötigt weiterhin eine Sprachgrenze und eine passende Fehler- und Datentypabbildung, vermeidet jedoch das pro Vergleich wiederholte Textprotokoll und den zusätzlichen Prozesslebenszyklus.

Die C++-Library ist ebenfalls in C++ geschrieben, ihre Schnittstelle kann jedoch in der Beschreibung des Aufrufers erscheinen, etwa als importierbare Funktion, die Arrays entgegennimmt und Werte oder Ausnahmen zurückgibt. Der Unterschied liegt also nicht in der Programmiersprache des Algorithmuskerns, sondern darin, *wo die Grenze zwischen beiden Beschreibungen verläuft.* Bei der Library verläuft sie an einer schmalen, typisierten Funktionsschnittstelle. Bei der CLI verläuft sie am Betriebssystem, also an Prozessgrenze, Textformat und Dateideskriptoren.

8.10.2 Anwendung auf die Messungen dieses Kapitels

Die Untersuchung unterscheidet zwischen der früheren CLI-Anbindung und der nun gemessenen Library-Anbindung. Die CLI legte eine breite Grenze aus Prozessstart, Textformat und Fehlerauswertung offen; die Library verläuft innerhalb des Worker-Prozesses über einen Funktionsaufruf. Der Mikrobenchmark und die Ende-zu-Ende-Messung quantifizieren den Laufzeitunterschied dieser konkreten Varianten.

Der Messaufbau überwacht die aktive Implementierung.

Die Versuchsanordnung prüft die tatsächlich in den Worker-Prozessen aktive Implementierung, zählt fehlgeschlagene Einzelvergleiche und vergleicht die Treffer-IDs beider Backends (Abschnitt 8.3). Diese Kontrollen sind nötig, um den konkreten Lauf korrekt zuzuordnen und Backend-Konformität empirisch zu prüfen. Sie zeigen zugleich, dass sprachübergreifende Integration Tests an der Grenze benötigt; der abgeschlossene Lauf bestätigt für die sechs Queries gleiche Treffer-IDs und null Fehler.

Fachliche Regeln müssen an der Schnittstelle eindeutig sein.

Der C++-Kern liefert Ergebniswerte, die auf die von Gen-DB verwendeten Zustände abgebildet werden; bei identischen Graphen muss außerdem dieselbe Auswahlregel wie im Python-Backend gelten. Solche Regeln dürfen bei austauschbaren Backends nicht implizit auseinanderlaufen. Die Library beseitigt nicht automatisch diese semantische

Verantwortung, ermöglicht aber eine gemeinsame Funktionsschnittstelle und direkte Konformitätstests ohne JSON-Protokoll. Eine ausdrücklich dokumentierte Spezifikation bleibt daher erforderlich.

Die Library-Schnittstelle ist messbar, aber klein gegenüber dem CLI-Prozesspfad.

Der direkte Mikrobenchmark (Tabelle 8.3) misst für C++ im Mittel 0,074 Millisekunden je Vergleich und 0,018 Millisekunden für einen trivialen 1×1-Aufruf. Letzterer ist eine Größenordnung für Funktionsübergabe und Marshalling; er entspricht ungefähr einem Viertel der mittleren Zeit des regulären Library-Aufrufs. Dem steht der frühere CLI-Mikrobenchmark mit 10,081 Millisekunden für den trivialen Prozessaufruf gegenüber. Die Werte stammen aus verschiedenen Messläufen und Aufrufpfaden; sie dürfen nicht als exakt kontrollierter A/B-Vergleich subtrahiert werden. Zusammen mit dem gemessenen Ende-zu-Ende-Speedup von 6,36× zeigen sie jedoch, dass die Library die Kostenstruktur der CLI vermeidet und den nativen Rechenkern praktisch nutzbar macht.

Der Prozess-Pool wird mit der Library wiederverwendet.

Gen-DB betreibt einen langlebigen `ProcessPoolExecutor`, dessen Worker Kandidaten blockweise mit je 256 Vergleichen bearbeiten. Die Library wird einmal je Worker geladen und für die Vergleiche innerhalb dieses Prozesses wiederverwendet. Die frühere CLI öffnete zusätzlich einen Prozess pro Vergleich; dieser zweite Prozess-Lebenszyklus entfällt im nun gemessenen Pfad. Das passt zum Modell der Anwendung und vermeidet wiederholte Betriebssystem- und JSON-Kosten.

8.10.3 Anwendung auf die gesamte Arbeit

Die Position betrifft nicht nur dieses Kapitel. Sie knüpft an den Gedanken an, den die Schlussbemerkung als zentral bezeichnet, nämlich die Trennung zweier Aufgaben: eine günstige Vorauswahl in der Datenbank und der Subgraph Algorithmus als Entscheidung über jeden verbliebenen Kandidaten. Diese Trennung funktioniert, weil sie klar ist. Eine Anbindung, die sie an anderer Stelle durch eine unscharfe Grenze unterläuft, widerspricht dem Prinzip der Gesamtarbeit.

Systemarchitektur (Abschnitt 3).

Die drei Schichten Frontend, Backend und Datenbank sind jeweils durch eine benannte Schnittstelle getrennt: REST-Aufrufe zwischen Frontend und Backend, SQL zwischen Backend und Datenbank. Der Algorithmus wird vom Modul `crud.py` über seine Backend-API aufgerufen. Die C++-Library wird innerhalb dieses Backend-Bausteins geladen und fügt keine weitere Prozessgrenze in den Suchablauf ein.

Formale Analyse (Abschnitt 4).

Satz 2 modelliert die Suchzeit als Summe aus dem Pre-Filter und C Vergleichen mit je $O(n^3)$ Aufwand. Das Modell trägt, solange die Kosten eines Vergleichs dem Algorithmus zugerechnet werden können. Eine Prozess-pro-Vergleich-Anbindung fügt jedem Vergleich einen Term hinzu, den das Modell nicht enthält.

Bemerkung 6 (Erweitertes Kostenmodell)**.** Bezeichne δ die Schnittstellenkosten je Vergleich. Bei der CLI umfassen sie Prozessstart, JSON-Serialisierung und Antwortauswertung; bei der Library die Übergabe der Arrays und den Funktionsaufruf. Dann lautet eine erweiterte Form des Modells aus Satz 2

$$T'_{\text{Suche}}(N, C, n) = O(\log N + C) + O\Big(C \cdot (n^3 + \delta)\Big).$$

Der direkte Mikrobenchmark misst für die Library 0,018 ms für einen trivialen Funktionsaufruf und 0,074 ms im Mittel für ein Zufallspaar. Der frühere CLI-Mikrobenchmark maß rund 10 ms für den trivialen Prozessaufruf. Diese konkreten Zeiten hängen von Hardware, Datenformat und Umgebung ab; sie zeigen jedoch, dass δ je nach Anbindung einen erheblichen Unterschied im konstanten Faktor erzeugt. Aus den Werten lässt sich kein allgemeiner Vergleich von δ mit der abstrakten Zahl n^3 ableiten.

Das erweiterte Modell erhält die asymptotische Aussage, dass die Suchzeit linear mit C wächst. Es macht aber sichtbar, dass die Wahl der Anbindung den konstanten Faktor bestimmt. Ein Modell mit einem Term, den die Beschreibung der Arbeit nicht benennt, wäre ein Beispiel für das beschriebene Problem im Kleinen.

Empirische Evaluation und VF2-Vergleich (Abschnitte 5 und 6).

Der Vergleich mit VF2 ist aussagekräftig, weil beide Verfahren in derselben Beschreibung laufen: Beide werden in Python aufgerufen, auf identischen Eingaben und mit derselben Zeitmessung. Die Library-Messung vergleicht zwei unterschiedliche Implementierungssprachen, aber beide Aufrufe erfolgen innerhalb der Suchworker und auf denselben Daten. Sie misst damit den realen Einfluss der Backend-Wahl auf Gen-DB und nicht ausschließlich den algorithmischen Aufwand. Der frühere CLI-Pfad hatte zusätzlich Prozessstart und JSON-Transport in die Messung eingebracht.

Parallelisierung (Abschnitt 7).

Die dort gemessenen Ergebnisse, ein Speedup von 1,90 mit zwei Workern und eine längere Laufzeit mit drei Workern auf einem Rechner mit vier logischen Prozessoren, beschreiben die Python-Arbeitslast. Der Backend-Versuch setzte zwei Worker ein und zeigte auch bei der Library einen deutlichen Suchgewinn; er untersuchte jedoch nicht systematisch unterschiedliche Worker-Zahlen. Daher bleibt offen, ob die optimale Parallelisierung und der serielle Anteil mit C++ unverändert sind. Die Worker-Zahl sollte weiterhin an verfügbare Prozessoren und die Last des Datenbankservers angepasst werden.

Schluss und Ausblick (Abschnitte 12.2 und 12.3).

Die Grenzen der Arbeit nennen eine in reinem Python geschriebene Referenzimplementierung als obere Abschätzung des praktisch Erreichbaren. Der Ausblick führt als Hebel die Vektorisierung mit `numpy`, die Kompilierung mit `numba` und die C++-Erweiterung auf (Abschnitt 12.3.9). Alle drei sind Anbindungen, die in der Beschreibung des Aufrufers bleiben. Die hier vertretene Position stützt diese Reihenfolge: Weiterentwicklungen des Vergleichs sollten zuerst dort ansetzen, wo die Grenze schmal bleibt, und die Prozessgrenze nur dort einführen, wo ihr Zweck, etwa Isolation, den Beschreibungsbruch rechtfertigt.

8.10.4 Kriterien einer losen Kopplung

Aus den vorangehenden Überlegungen lassen sich Kriterien ableiten, an denen sich beide Anbindungsformen vergleichen lassen. Die Tabelle fasst Gestaltungseigenschaften zusammen; die Laufzeitangaben werden durch die Mikro- und Suchmessungen aus Abschnitt 8.5 ergänzt.

Tabelle 8.5: Vergleich der früheren CLI- und der nun umgesetzten Library-Anbindung nach Gestaltungskriterien.

Kriterium	C++-CLI (Prozess pro Vergleich)	C++-Library (im Worker-Prozess)
Datenaustausch	Matrizen als JSON-Text über `stdin` und `stdout`	typisierte Arrays über eine Funktionsschnittstelle
Fehlermodell	Exit-Code, `stderr`, Timeout (`CSUBGRAPH_TIMEOUT`)	Ausnahmen oder Rückgabewerte in der Beschreibung des Aufrufers
Lebensdauer	kein Zustand über einen Vergleich hinaus	Instanz je Worker, wiederverwendbar
Aufrufkosten δ	Prozessstart je Vergleich, früher etwa 10 ms je trivialem Aufruf	Funktionsaufruf im Prozess, trivial im Mittel 0,018 ms
Ergebnisregeln	Übersetzung der Werte und Regel für identische Graphen im Backend	Abbildung kann Teil der Schnittstelle sein und liegt dann beim Algorithmus
Testbarkeit	Tests brauchen ein gebautes Executable und einen Prozessstart	Tests rufen die Funktion direkt auf, gleiche Mittel wie beim Python-Algorithmus
Auslieferung	plattformabhängiger Pfad zum Executable (`subgraph-cli.exe`) über die Konfiguration	Abhängigkeit des Projekts, Binärdatei je Plattform
Isolation	Absturz des Algorithmus betrifft nur den Einzelprozess	Absturz betrifft den Worker-Prozess
Messbarkeit	Schnittstellenkosten verdecken den Rechenkern	Rechenkern lässt sich unter gleichen Bedingungen wie Python messen

Die Zeile zur Isolation zeigt, dass die Gegenüberstellung nicht einseitig ausfällt. Sie ist die stärkste Gegenposition und wird im nächsten Abschnitt behandelt.

8.10.5 Gegenargumente und Grenzen der Position

Isolation und Sprachneutralität.

Eine CLI trennt Prozesse hart. Ein Fehler im nativen Code reißt dann nicht den Worker mit, und ein Timeout kann den Prozess beenden. Außerdem ist eine CLI von jeder Sprache aus aufrufbar. Beide Eigenschaften sind echte Vorteile. Sie sprechen für die CLI als Werkzeug, etwa für Skripte, manuelle Prüfungen oder Experimente. Die hier vertretene Position betrifft die Einbindung in das System. Für diese zählt, ob die Grenze in der gemeinsamen Beschreibung des Systems liegt. Die CLI bleibt als eigenständiges Programm sinnvoll, auch wenn Gen-DB den Algorithmus über die Library aufruft.

Aufwand der Library.

Eine Library braucht eine Anbindung an Python, etwa über einen Erweiterungsmechanismus, und eine Bauumgebung je Plattform. Das ist Aufwand, den die CLI zunächst nicht verlangt. Die Position behauptet nicht, dass dieser Aufwand gering sei. Sie behauptet, dass er einmalig anfällt und die Grenze danach schmal hält, während die Kosten der CLI bei jedem Vergleich und bei jeder Weiterentwicklung wiederkehren.

Die Evidenz ist begrenzt, aber nicht mehr vorläufig.

Die Library wurde im Mikrobenchmark und in einer vollständigen Suchreihe gemessen; beide Messungen zeigen einen deutlichen Zeitgewinn. Die Aussagekraft ist jedoch auf sechs Queries, einen Lauf je Backend und eine Hardwarekonfiguration begrenzt. Die Ergebnisse belegen daher den Vorteil in diesem Versuchsaufbau, nicht für jede Plattform oder Graphenfamilie.

Die Position ist eine Gestaltungsentscheidung.

Begriffe wie Trennungsbewusstsein lassen sich in dieser Arbeit nicht messen. Sie bezeichnen eine Eigenschaft der Zusammenarbeit von Mitgestaltern, die durch Fallbeispiele wie die oben genannten plausibel gemacht, aber nicht durch eine Studie belegt wird. Die Arbeit formuliert sie deshalb als begründete Position und nicht als Ergebnis.

8.10.6 Folgerungen für Gen-DB

Aus der Position und den Messungen ergeben sich fünf Folgerungen für die weitere Arbeit am System:

1. **Eine Schnittstelle, mehrere Implementierungen.** Python- und C++-Algorithmus sollten hinter derselben Funktionsschnittstelle liegen. Der Aufrufer in Gen-DB wählt die Implementierung über die Konfiguration, und der Rückfall auf Python bleibt erhalten. Er ist dann ein Wechsel hinter einer gemeinsamen Schnittstelle und kein Wechsel der Beschreibung.

2. **Fachliche Regeln gehören zum Algorithmus.** Die Abbildung der Ergebniswerte und die Auswahl nach der Kantenzahl bei identischen Graphen sollten in der Schnittstelle des Algorithmus festgelegt sein und nicht im Übersetzungscode der Anwendung. Beide Implementierungen liefern dann per Konstruktion dieselben Werte, und die Kontrolle über die Treffer-IDs wird zu einem Test statt zu einer Laufzeitprüfung.

3. **Die Library-Messung verbreitern.** Die erste gepaarte Messreihe liegt vor. Als nächstes sollte sie mit mehr Queries, Wiederholungen, Konfidenzintervallen und weiteren Plattformen wiederholt werden; die Vergleiche sollten dabei nach Suchphasen aufgeschlüsselt werden.

4. **Schnittstellenkosten ausweisen.** Künftige Messungen sollten δ getrennt von der Rechenzeit des Algorithmus berichten, wie es die Bemerkung 6 nahelegt. Dadurch lassen sich Ergebnisse verschiedener Anbindungen vergleichen, ohne Algorithmus und Schnittstelle zu vermischen.

5. **Die CLI als Werkzeug behalten.** Die CLI bleibt für Einzelprüfungen, Skripte und Fehlersuche nützlich. Sie sollte jedoch nicht der Pfad sein, über den die produktive Suche einen Vergleich nach dem anderen ausführt.

8.10.7 Zwischenfazit zur Kopplung

Die Aussage des Abschnitts lässt sich so zusammenfassen: Ein System bleibt analysierbar, weiterentwickelbar und wiederverwendbar, wenn seine Grenzen schmal und benannt sind und die Zuständigkeiten klar bleiben. Die Messungen zeigen am Beispiel der C++-Anbindung sowohl einen strukturellen als auch einen Laufzeitunterschied: Die CLI verlegt die Grenze an Prozess, JSON-Text und Exit-Code; der frühere triviale Aufruf benötigte rund 10 ms. Die Library wird dagegen direkt im Worker aufgerufen; ihr trivialer Aufruf lag im Mittel bei 0,018 ms, und die vollständige Suche war 6,36-mal schneller als mit Python. Der Suchlauf bestätigt damit einen konkreten Leistungsvorteil dieser Library-Integration. Die allgemeinere Behauptung, dass dadurch Zusammenarbeit und Wiederverwendung erleichtert werden, bleibt eine begründete Gestaltungsposition. Die Library setzt in der Gesamtarbeit das Prinzip fort, das Architektur, formale Analyse und Messmethodik tragen: Schnittstellen und ihre Kosten sichtbar zu machen, statt sie im Gesamtergebnis zu verbergen.

Kapitel 9

Experimentelle Untersuchung Multi-Omics-Suche

Hardwareplattform der lokal ausgeführten Experimente: Rechner `WBIS-H207085` von LENOVO, Modell `10T8S6LB00`, mit 7,87 GB Arbeitsspeicher; ein Intel-Core-i3-8100T-Prozessor mit 3,10 GHz, vier physischen Kernen und vier logischen Prozessoren (x64); Microsoft Windows 11 Pro, 64-Bit, Version 10.0.26200.

Die bisherigen Such- und Backend-Experimente vergleichen jeweils einzelne Netzwerke. Biologische Prozesse lassen sich jedoch häufig erst dann sinnvoll untersuchen, wenn mehrere Datenebenen gemeinsam betrachtet werden: Genom, Transkriptom, Proteom und Metabolom beschreiben unterschiedliche, miteinander in Beziehung stehende Aspekte eines Systems. Dieses Kapitel erweitert deshalb die Suche von einer einzelnen Adjazenzmatrix auf ein geordnetes Bündel mehrerer Netzwerkschichten. Im Mittelpunkt stehen nicht nur Laufzeit und Implementierung, sondern auch die Frage, ob die beiden Backends für verschiedene Arten der Schichtkopplung dieselben Entscheidungen treffen.

Die Untersuchung wird durch `src/experiment_search_multiomics.py` gesteuert. Der Lauf wurde am 8. Oktober 2026 nach Abschluss der Such-, Worker- und Chunk-Größen-Experimente während der Datenbankgrößenreihe abgebrochen. Die JSON-Datei wird nach jeder Messung aktualisiert und enthält deshalb vollständige Resultate für diese drei Phasen, aber nur einen Teil der Datenbankgrößenmessungen. Die folgenden Aussagen trennen diese abgeschlossenen Reihen von der abgebrochenen Phase. Die maschinenlesbaren Einzelergebnisse stehen in `src/results/search_multiomics_20261008_115430.json`; das Textprotokoll `src/results/search_multiomics_20261008_115430.txt` dokumentiert den Ablauf und den Abbruchstand.

9.1 Mehrschichtige Netzwerke und Suchrelationen

Ein Multi-Omics-Netzwerk wird als geordnetes Tupel

$$\mathcal{M} = (A_1, A_2, \ldots, A_L), \qquad A_\ell \in \{0,1\}^{n \times n},$$

modelliert. Jede Matrix A_ℓ repräsentiert eine Omics-Schicht, beispielsweise ein Genom- oder Proteomnetzwerk. In diesem Experiment haben alle Schichten eines Tupels dieselbe Knotenzahl n; die Schichtnamen bleiben jedoch unterscheidbar. Verglichen werden Query-Tupel und Kandidaten-Tupel mit möglicherweise verschiedener Knotenzahl und Schichtzahl.

Der Vergleich baut auf den Zeilenkomponenten der Matrizen auf, also den aus den Spalten abgeleiteten Bitmustern des in Abschnitt 2 beschriebenen Subgraph Algorithmus. Eine

einzelne Schichtrelation sucht ein benachbartes Paar solcher Komponenten in zyklischer Ordnung. Für mehrere Schichten unterscheidet die Implementierung zwei Kopplungsmodi:

- **Kohärent:** Ein passendes Komponentenpaar der Query muss in allen betrachteten Schichten an derselben Kandidatenposition auftreten. Die Schichten teilen damit eine gemeinsame strukturelle Einbettung.

- **Unabhängig:** Jede Schicht darf ihr passendes Komponentenpaar an einer eigenen Kandidatenposition finden. Die Trefferpositionen müssen zwischen den Schichten nicht übereinstimmen.

Die kohärente Bedingung ist die strengere Kopplung: Ein kohärenter Treffer erfüllt auch die unabhängige Bedingung. Die Modi beantworten somit unterschiedliche fachliche Fragen. Kohärent sucht nach einem über mehrere Ebenen hinweg an derselben Stelle erhaltenen Muster; unabhängig fragt, ob das Muster in jeder Ebene vorkommt, auch wenn die Positionen voneinander abweichen. Das Experiment prüft diese Unterscheidung anhand gezielt konstruierter Beispiele und darf nicht mit einer validierten biologischen Kausalbeziehung zwischen den Schichten gleichgesetzt werden.

9.1.1 Native C++-Implementierung des Schichtvergleichs

Das Projekt `csubgraph` implementiert neben dem Einzelgraph-Kern auch `MultiOmics`. Ein Schichtstapel ist dort ein `LayerStack`, also ein Vektor binärer Adjazenzmatrizen. Die Validierung verlangt mindestens eine Schicht, quadratische Matrizen gleicher Knotenzahl und höchstens 63 Knoten. Beim Vergleich zweier Stapel müssen außerdem die Schichtzahlen übereinstimmen. Jede Matrix wird zunächst mit `SubgraphAlgorithm::calculateSignatures` verarbeitet; `layerRowComponents` entfernt anschließend je Schicht die Spaltenkodierung und liefert die Komponentenfolgen.

Die API macht die beiden unabhängigen Designentscheidungen explizit: `Mode::COHERENT` oder `Mode::INDEPENDENT` legt die Kopplung der Schichten fest; `Strategy::BIGRAM` oder `Strategy::DYNAMIC` bestimmt, wie eine Übereinstimmung gefunden wird. Ein Aufruf sieht beispielsweise so aus:

Listing 9.1: Aufruf des nativen mehrschichtigen Vergleichs

```
const auto result = MultiOmics::compareLayered(
    queryLayers,
    candidateLayers,
    MultiOmics::Mode::COHERENT,
    MultiOmics::Strategy::BIGRAM
);
```

Im unabhängigen Modus vergleicht die Implementierung jede Schicht separat und liefert nur dann einen Treffer, wenn jede Schicht ihr passendes Komponentenpaar findet. Die jeweiligen Positionen dürfen dabei voneinander abweichen. Im kohärenten Modus bildet sie dagegen für jede Spaltenposition das Tupel der Komponenten über alle Schichten, ordnet gleichen Tupeln über ein gemeinsames Wörterbuch dieselbe interne Zahl zu und führt den Sequenzvergleich auf diesen Zahlenfolgen aus. Weil jede Position durch das vollständige Schichttupel repräsentiert wird, kann ein gemeinsames Paar nur dann übereinstimmen, wenn die beteiligten Schichten dieselben beiden Positionen stützen.

Die dynamische Strategie entspricht dem zuvor beschriebenen Rotations- und DP-Verfahren: Sie probiert die Rotationen der Kandidatenfolge aus und berechnet für jede die längste gemeinsame zusammenhängende Teilfolge. Die Bigramm-Strategie nutzt direkt die notwendige und hinreichende Bedingung für eine solche Übereinstimmung der Länge mindestens zwei: Es genügt, ein benachbartes Wertepaar aus der Query zu finden, das im Kandidaten als zyklisch benachbartes Paar vorkommt. Dafür werden die zyklischen Paare des Kandidaten in einer geordneten Menge abgelegt:

Listing 9.2: Bigramm-Prüfung auf ein zyklisch benachbartes Paar

```
for (std::size_t k = 0; k < b.size(); ++k) {
    cyclicPairs.insert({b[k], b[(k + 1) % b.size()]});
}
for (std::size_t i = 0; i + 1 < a.size(); ++i) {
    if (cyclicPairs.count({a[i], a[i + 1]}) != 0) {
        return true;
    }
}
```

Das Einfügen und Nachschlagen in `std::set` kostet $O(n \log n)$; im dynamischen Verfahren kosten die n Rotationen mit je einer $O(n^2)$-DP bis zu $O(n^3)$. Beide Strategien sollen dieselbe Relation entscheiden: Die Bigramm-Charakterisierung ist daher nicht eine andere, heuristische Suchsemantik, sondern eine äquivalente Ausführung dieses konkreten Schwellwerttests. `compareLayered` erkennt zunächst identische Schichtkomponentenfolgen. Andernfalls testet es beide Enthaltungsrichtungen und gibt die entsprechenden Auswahlwerte zurück. Bei wechselseitiger Enthaltung verwendet es die Gesamtzahl der Kanten über alle Schichten als Tie-Breaker. Die Bibliothek bietet daneben Projektion auf ausgewählte Schichten sowie Konsens-, Vereinigungs- und Schnittintegration; diese sind Vorverarbeitungsschritte und nicht mit dem kohärenten beziehungsweise unabhängigen Vergleichsmodus gleichzusetzen.

9.2 Versuchsaufbau und Datengenerierung

Der Lauf verwendet den Seed 42 und ergänzt die bestehende Datenbank um 20.000 reproduzierbar erzeugte Zufallsnetzwerke des Typs `multi_omics_experiment`. Hinzu kommen Netzwerke, die zu eingebetteten Queries eine bekannte Lösung liefern; zum gespeicherten Zwischenstand enthielt die Datenbank insgesamt 20.175 Multi-Omics-Netzwerke. Die Zufallsnetzwerke haben 4 bis 40 Knoten und 2 bis 5 Schichten. Für die Schichten werden sechs Namen aus Genom, Transkriptom, Proteom, Metabolom, Epigenom und Lipidom verwendet. Die Schichten werden als gerichtete Zufallsmatrizen ohne Selbstkanten mit je Omics-Typ festgelegter Kantenwahrscheinlichkeit erzeugt.

Das Query-Gitter umfasst sieben Knotenzahlen $(4, 6, 8, 12, 16, 24, 32)$, vier Schichtzahlen (1 bis 4) und zwei Query-Arten: eine Zufallsanfrage ohne absichtlich eingefügte Lösung sowie eine Anfrage, zu der Lösungen konstruiert werden. Damit entstehen 56 Query-Konfigurationen. Zu den eingebetteten Anfragen werden eine identische Kopie, kohärent eingebettete und unabhängig eingebettete Kandidaten erzeugt. Für die Einbettung werden zwei benachbarte Query-Spalten in die Kandidatenmatrix kopiert; Zeilen außerhalb der Query-Knotenzahl bleiben dabei null. Bei kohärenten Beispielen verwenden alle Schichten dasselbe benachbarte Paar und dieselbe Position, bei unabhängigen Beispielen kann die

Position je Schicht variieren. Die so erzeugten Kandidaten stellen eine Ground Truth bereit, unabhängig davon, ob Python und C++ dieselbe Antwort liefern.

Tabelle 9.1: Konfiguration des Multi-Omics-Experiments gemäß gespeicherten Laufmetadaten.

Parameter	Wert
Zufallsnetzwerke	20.000, 4–40 Knoten und 2–5 Schichten
Omics-Schichten	Genom, Transkriptom, Proteom, Metabolom, Epigenom, Lipidom
Query-Knotenzahlen	$4, 6, 8, 12, 16, 24, 32$
Query-Schichtzahlen	$1, 2, 3, 4$
Query-Arten und Modi	Zufall/eingebettet; kohärent/unabhängig
Suchmessungen geplant	56 Queries $\times 2$ Modi $\times 2$ Backends $\times 3$ Wiederholungen $= 672$
Mikrobenchmark	6 Schichtzahlen $\times 6$ Knotenzahlen $\times 2$ Paartypen; je 100 Vergleiche
Korrektheitsprüfung	5.000 Paare; beide Modi, bekannte Konstruktionen und Zufallspaare
Worker-Skalierung	$1, 2, 3, 4$ Worker; Standardausführung mit 2 Workern
Chunk-Größen	$1, 8, 32, 128, 256, 1024, 4096$
Datenbankgrößen	$2.000, 5.000, 10.000, 20.000$ Zufallsnetzwerke
Wiederholungen	3 pro Suchkonfiguration; eine Aufwärmsuche wird verworfen
Laufzeitumgebung	Windows 11 Pro, 64-Bit, Version 10.0.26200; Python 3.14.8, NumPy 2.5.3
Rechner	LENOVO `10T8S6LB00`, Rechnername `WBIS-H207085`, 7,87 GB RAM
Prozessor	Intel Core i3-8100T, 3,10 GHz, 4 physische Kerne und 4 logische Prozessoren, x64

Die Reihenfolge der Backends und Queries wird zwischen Wiederholungen variiert. Gemessen wird die Wanduhrzeit des vollständigen Aufrufs von `crud.search_multiomics()`, also einschließlich SQL-Abfrage, Kandidatenvergleichen, Prozesskommunikation und Trefferaufbau. Zusätzlich protokolliert das Experiment Laden, Vergleichsphase und Ergebnisaufbau getrennt. Die Anfrage wird standardmäßig mit zwei Workern ausgeführt; eigene Teilexperimente variieren Worker-Zahl, Chunk-Größe und Datenbankumfang. Vor einer Messung kontrolliert das Skript, dass das konfigurierte Backend tatsächlich in den Worker-Prozessen aktiv ist, damit ein stiller Rückfall von C++ auf Python die Auswertung nicht verfälscht.

9.3 Korrektheit und differenzielle Prüfung

Die abgeschlossene Korrektheitsprüfung vergleicht Python und C++ auf 5.000 Paaren. Sie umfasst Zufallspaare, kohärent und unabhängig konstruierte Einbettungen, identische sowie umgekehrte Paare und wertet beide Modi aus. Damit entstehen 10.000 Backend-Entscheidungen. Zusätzlich prüft das Skript Invarianten der Modi sowie die erwarteten Ergebnisse der konstruierten Einbettungen. Tabelle 9.2 fasst die gespeicherten Resultate zusammen.

Tabelle 9.2: Abgeschlossene differenzielle Korrektheitsprüfung der Multi-Omics-Backends.

Prüfgruppe	Paare	übereinstimmende Modusentscheidungen	Fehler
Zufallspaare	1.000	2.000	0
Kohärent eingebettet	1.000	2.000	0
Unabhängig eingebettet	1.000	2.000	0
Identische Paare	1.000	2.000	0
Umgekehrte Paare	1.000	2.000	0
Gesamt	5.000	10.000	0

Für alle 10.000 Entscheidungen stimmen die Implementierungen überein; das JSON enthält null Fehler, null abweichende Resultate und keine Invariantenverletzungen. Diese Prüfung ist besonders aussagekräftig, weil sie nicht nur zufällige Eingaben vergleicht: Die eingebetteten Paare besitzen bekannte Solltreffer, während identische und umgekehrte Fälle Grenz- und Richtungsverhalten adressieren. Sie ist dennoch kein allgemeiner Beweis der Korrektheit für alle möglichen Matrizen, Schichtzahlen und biologischen Daten.

9.4 Mikrobenchmark der Schichtvergleiche

Der Mikrobenchmark umgeht die Datenbank und vergleicht dieselben Eingabepaare direkt in beiden Implementierungen. Untersucht werden $L \in \{1, 2, 3, 4, 6, 8\}$ Schichten und Knotenzahlen $n \in \{8, 16, 24, 32, 48, 63\}$, jeweils mit zufälligen und eingebetteten Paaren. Für jede Backend-, Modus-, Schichtzahl-, Knotenzahl- und Paartyp-Konfiguration werden 100 Vergleiche ausgeführt. Das ergibt 144 Konfigurationen pro Backend. Ein separater Test mit 500 trivialen 1×1-Aufrufen schätzt die Untergrenze der Aufruf- und Übergabekosten.

Tabelle 9.3: Mikrobenchmark-Zwischenstand: Mittel der Konfigurationsmittelwerte und gepaarte Speedups.

Backend	Konfigurationen	Mittelwert je Vergleich [ms]	Mittelwert eingebettet [ms]
Python	144	4,0680	4,0571
C++	144	0,4005	0,4032

Über die 144 passenden Konfigurationen beträgt das geometrische Mittel des Python-zu-C++-Speedups 8,11×; die einzelnen gepaarten Faktoren liegen zwischen 2,90× und 11,77×. Die Tabelle berichtet den ungewichteten Mittelwert der Konfigurationsmittelwerte und nicht die gepoolte Zeit aller Einzelaufrufe. Zufalls- und eingebettete Paare weisen in beiden Backends in dieser Aggregation ähnliche Mittel auf. Der triviale C++-Aufruf benötigte im Mittel 0,0242 ms bei 500 Aufrufen. Dieser Wert zeigt, dass die Funktionsübergabe messbar, aber klein gegenüber der mittleren Vergleichszeit ist; er wird nicht von den C++-Laufzeiten abgezogen, da triviale und reguläre Matrizen nicht denselben Aufwand verursachen.

Das Ergebnis stützt die C++-Bibliothek als Beschleunigungsoption für den mehrschichtigen Vergleich in der untersuchten Konfiguration. Der Faktor ist jedoch kein alleiniger Effekt der Programmiersprache: Implementierung, Datenrepräsentation, Aufrufpfad und Eingabeverteilung unterscheiden sich. Eine isolierte Aussage über die algorithmische Komplexität folgt aus dem Mikrobenchmark nicht; dafür dient er als praktische Messung des tatsächlich integrierten Vergleichspfads.

9.5 Ende-zu-Ende-Suche und Laufzeitergebnisse

Die vollständige Suchreihe umfasst 56 Query-Konfigurationen, zwei Modi, zwei Backends und drei Wiederholungen, insgesamt 672 Läufe beziehungsweise 336 gepaarte Python/C++-Vergleiche. Bei jedem Paar waren Kandidatenzahl und Treffer-IDs identisch; alle 1.722 erwarteten eingebetteten Ground-Truth-Treffer wurden gefunden, und kein Vergleich schlug fehl. Die aggregierten Laufzeiten stehen in Tabelle 9.4.

Tabelle 9.4: Vollständige Ende-zu-Ende-Suchreihe: aufsummierte Zeiten aus 336 gepaarten Läufen je Backend.

Backend	Gesamt [s]	Laden [s]	Vergleiche [s]	Trefferaufbau [s]
Python	1.401,16	428,18	961,16	11,82
C++	620,43	437,30	171,07	12,05

Das Verhältnis der aufsummierten Antwortzeiten beträgt 2,26 zugunsten von C++; die Gesamtlaufzeit sinkt damit um 55,7 %. Auf Ebene der 112 Query-, Modus- und Anfrageart-Konfigurationen liegt der Median des über drei Wiederholungen gebildeten Speedups bei 2,26; die Konfigurationswerte reichen von 1,56 bis 3,77. Das geometrische Mittel über die einzelnen gepaarten Laufzeiten beträgt 2,28. C++ war in allen 112 Konfigurationen schneller. Diese übereinstimmende Richtung ist belastbarer als ein einzelner Gesamtquotient, bleibt aber auf die synthetischen Anfragen und die untersuchte Rechnerkonfiguration beschränkt.

Die Phasenmessung macht sichtbar, weshalb der Such-Speedup deutlich unter dem Mikrobenchmark-Faktor von 8,11 liegt. Bei Python entfallen 68,6 % der gemessenen Gesamtzeit auf Vergleiche und 30,6 % auf das Laden. Mit C++ sinkt der Vergleichsanteil auf 27,6 %, während das Laden nun 70,5 % der Gesamtzeit beansprucht. Die beschleunigte Vergleichsphase reduziert die Ladezeit nicht; sie verschiebt vielmehr den Engpass der Gesamtsuche. Weitere Optimierung des C++-Kerns allein kann deshalb nur begrenzte zusätzliche Ende-zu-Ende-Gewinne erzielen.

9.6 Worker-Skalierung auf vier Prozessorkernen

Die abgeschlossene Worker-Reihe umfasst sechs ausgewählte Queries, zwei Wiederholungen und jeweils 1 bis 4 Worker, also 12 Messungen je Backend und Worker-Zahl. Tabelle 9.5 fasst die aufsummierten Zeiten und den Speedup gegenüber einem Worker zusammen.

Tabelle 9.5: Worker-Skalierung: aufsummierte Laufzeit der sechs Queries in zwei Wiederholungen.

Worker	Python [s]	Python Speedup	C++ [s]	C++ Speedup
1	74,73	1,00	24,08	1,00
2	50,31	1,49	19,69	1,22
3	39,14	1,91	20,08	1,20
4	35,55	2,10	18,68	1,29

Die Hardwareangabe aus Tabelle 9.1 ist für die Interpretation entscheidend: Der Intel Core i3-8100T stellt vier physische Kerne und vier logische Prozessoren bereit, also keine

zusätzlichen SMT-Threads. Python profitiert in dieser Stichprobe bis vier Worker und erreicht dort 2,10-fachen Speedup, entsprechend einer parallelen Effizienz von rund 52,5 %. Beim C++-Backend ist der Gewinn deutlich kleiner: Zwei Worker erreichen 1,22, drei 1,20 und vier 1,29. Der Unterschied zwischen zwei und vier Workern ist damit gering; vier Worker liefern hier die kürzeste Summe, aber keinen annähernd linearen Gewinn. Das passt dazu, dass nach Beschleunigung des Vergleichskerns Laden und übrige Prozesskosten dominieren. Wegen der nur zwei Wiederholungen und sechs Queries sind diese Werte eine Messung auf diesem Vierkernsystem, keine allgemeine Empfehlung für andere Hardware.

9.7 Einfluss der Chunk-Größe

Die Chunk-Reihe vergleicht sieben Blockgrößen mit denselben sechs Queries und zwei Wiederholungen, jeweils bei zwei Workern. Tabelle 9.6 zeigt die aufsummierte Zeit der 12 Messungen je Backend und Blockgröße.

Tabelle 9.6: Chunk-Größen: aufsummierte Laufzeiten der sechs Queries in zwei Wiederholungen.

Chunk-Größe	Python [s]	C++ [s]
1	58,39	33,71
8	48,13	18,58
32	42,50	18,03
128	41,14	18,33
256 (Standard)	45,54	18,80
1.024	51,46	20,25
4.096	64,26	23,74

Die kleinsten und größten Blöcke sind in dieser Stichprobe für beide Backends ungünstiger als mittlere Größen. Python erreicht den niedrigsten Summenwert bei Chunk 128, C++ bei Chunk 32; beide liegen nahe beieinander im mittleren Bereich. Gegenüber dem Standardwert 256 ist die jeweilige beste beobachtete Summe um rund 9,7 % (Python) beziehungsweise 4,1 % (C++) kleiner. Das deutet auf einen Kompromiss zwischen Übergabeaufwand bei sehr kleinen Blöcken und Lastverteilung bei sehr großen Blöcken hin. Die Messung umfasst jedoch nur sechs Queries mit zwei Wiederholungen; sie rechtfertigt keine universelle optimale Chunk-Größe, sondern eine backend- und hardwareabhängige Kalibrierung.

9.8 Abgebrochene Datenbankgrößenreihe

Zum Abbruch war die Datenbankgrößenreihe erst teilweise abgeschlossen: Für 2.000 und 5.000 Zufallsnetzwerke liegen jeweils alle 24 Messungen vor (sechs Queries, zwei Backends, zwei Wiederholungen). Bei 10.000 Netzwerken sind 10 von 24 Messungen gespeichert; die geplante Stufe mit 20.000 Netzwerken wurde nicht erreicht. Die abgeschlossenen kleineren Stufen zeigen zwar unterschiedliche Laufzeiten, doch ohne vollständige, gepaarte Stufen und ohne die größte Datenbankgröße lässt sich daraus keine belastbare Aussage zum Wachstum mit N ableiten. Eine lineare Skalierung oder ein fester Zusammenhang zwischen Datenbankgröße, Kandidatenzahl und Suchzeit wird daher nicht behauptet.

9.9 Einordnung, Grenzen und Folgerungen

Das Experiment erweitert die bisherigen Untersuchungen in drei Richtungen. Erstens werden mehrere biologische Ebenen über zwei explizite Schichtsemantiken verglichen. Zweitens wird die Backend-Konformität durch differenzielle Prüfungen und konstruktive Solltreffer kontrolliert. Drittens trennt die Phasenmessung Rechenzeit und übrige Suchkosten. Die abgeschlossene Suchreihe bestätigt einen Ende-zu-Ende-Vorteil von C++ um den Faktor 2,26 bei identischen Kandidaten und Treffern. Zugleich sinkt der Anteil der Vergleichsphase von 68,6 % auf 27,6 %; der Ladeanteil wird mit C++ zum größten Kostenblock. Worker- und Chunk-Reihen zeigen zudem, dass Prozesszahl und Blockgröße nicht unabhängig vom Backend gewählt werden sollten.

Die Reichweite der Befunde ist aus mehreren Gründen begrenzt. Die Netzwerke und Einbettungen sind synthetisch erzeugt; biologische Netzwerke mit realen Beschriftungen, Unsicherheiten und unvollständigen Messungen werden nicht untersucht. Die Schichtmatrizen teilen dieselbe Knotenzahl, während reale Omics-Datensätze unterschiedliche Entitäten und Abbildungen zwischen Ebenen aufweisen können. Die Relation sucht strukturelle Muster in Zeilenkomponenten und ist keine klassische, labelbewusste Multi-Graph-Isomorphie. Insbesondere bedeutet ein kohärenter Treffer daher nicht, dass eine biologische Wechselwirkung oder Kausalität nachgewiesen wurde. Die Ende-zu-Ende-, Worker- und Chunk-Reihen sind abgeschlossen, aber auf eine einzelne Windows-11-Konfiguration mit vier physischen Kernen, zwei Wiederholungen für die Skalierung und sechs ausgewählten Skalierungsqueries beschränkt. Die Datenbankgrößenreihe wurde bei 10.000 Zufallsnetzwerken abgebrochen und erlaubt keine vollständige Skalierungsaussage.

Für eine belastbare Fortsetzung ist zunächst die Datenbankgrößenreihe zu vervollständigen oder ausdrücklich als Teilmessung auszuweisen; die fertigen Such-, Worker- und Chunk-Ergebnisse können bereits berichtet werden. Anschließend sind Wiederholungen auf weiterer Hardware und mit realen, fachlich annotierten Multi-Omics-Daten erforderlich. Die Ground Truth sollte dabei um Domänenwissen erweitert werden: Welche Knoten entsprechen sich zwischen den Ebenen, welche Interaktionen sind zulässig, und welche biologischen Treffer bewerten Fachleute als relevant? Erst diese Validierung kann entscheiden, ob kohärente und unabhängige Modi für konkrete Forschungsfragen geeignete Suchsemantiken darstellen.

9.10 Parallelisierung

Die in dieser Arbeit betrachtete Parallelisierung über ein, zwei und drei Worker steht auf dem Grund der maximalen und damit optimalen Effizienz und erlaubt uns somit grundlegende Beobachtungen zu machen. Wir wissen, dass der Subgraph Algorithmus optimal ist. Schneller können wir auf dieser grundlegenden Datenstruktur des Graphen dieses Problem nicht lösen. Und das ist wichtig, weil erst so wir allgemeinere Aussagen unserer Beobachtung machen können. Was dem Leser aber schon sofort aufgefallen ist und seiner Vermutung ganz entspricht, ist eigentlich relativ leicht. Betrachten wir die Beschreibung des Subgraph Algorithmus in Python, dann ist klar, mit zwei Workern sind wir deutlich schneller als mit einem. Was aber erfahrene Leser schon vermutet haben, aufgrund der Effizienz des Algorithmus und der Intuition der zugrundeliegenden Datenstruktur, hat sich in der Messung bestätigt. Ein dritter Worker hilft nicht mehr so viel, sondern schafft eher Verzögerung in der Verwaltung der Worker selbst. Diese Beobachtung wird sich in ähnlicher Weise wiederholen, wenn wir 1. grundlegende Datenstrukturen

vorliegen haben, wie z.B. den Graphen oder den Vektor, und 2. auf diesen mit optimalen Algorithmen rechnen, wie z.B. dem Subgraph Algorithmus. Kurz zusammengefasst lässt sich sagen: Parallelisierung? Grundlegende Strukturen, Schwerpunkte auf den Strukturen und effiziente Algorithmen für diese Strukturen in ihren Schwerpunkten.

Die Abschnitte 7 und 9.6 untersuchen die Parallelisierung der Kandidatenvergleiche auf derselben Vierkernmaschine, einmal für den Python-Kern und einmal für die Python- und C++-Backends. Die Messungen beantworten, *wie viel* die Parallelisierung in den jeweiligen Stichproben bringt. Sie beantworten nicht, *warum* die Werte so ausfallen, *welche* Werte überhaupt erreichbar sind und *welche* Stellgrößen (Worker-Zahl, Blockgröße, Reihenfolge der Kandidaten, Wahl des Backends) wie wirken. Dieser Abschnitt liefert die Theorie dazu. Er definiert ein Modell der parallelen Suche, beweist ihre Korrektheit, leitet obere und untere Zeitschranken her und prüft diese an den Messdaten der Abschnitte 9.6 und 9.7. Jede Aussage ist entweder bewiesen, oder sie ist ausdrücklich als Annahme, als Messwert oder als Modellrechnung gekennzeichnet; Tabelle 9.9 stellt die Einordnung am Ende zusammen.

Die Einordnung stützt sich auf mehrere, voneinander zu unterscheidende Bereiche der Parallelitätsliteratur. Grahams Analyse von Listenplanung liefert den klassischen theoretischen Bezug für Schranken der Makespan; Work Stealing behandelt dynamische Lastverteilung abhängiger Aufgaben und ist deshalb ein verwandtes, aber nicht identisches Ausführungsmodell [71, 72]. Für Shared-Memory-Programmierung beschreiben Dagum und Menon OpenMP, dessen Spezifikation den normativen API-Rahmen festlegt; der MPI-Standard behandelt demgegenüber den Nachrichtenaustausch zwischen Prozessen [73, 74, 75]. Die Lehrbücher von Pacheco, Rauber und Rünger sowie McCool, Reinders und Robison systematisieren Kommunikation, Synchronisation und Parallelisierungsmuster; Hager und Wellein erläutern die Performance-Modelle und Speicheraspekte, die bei einer Interpretation von Worker-Skalierung mitzuberücksichtigen sind [76, 77, 78, 79]. Die NAS Parallel Benchmarks zeigen schließlich, wie parallele Verfahren über festgelegte Workloads evaluiert werden können [80]. Diese Quellen liefern Kontext und Vergleichsbegriffe; sie ersetzen weder den hier bewiesenen Satz über die Ergebnisgleichheit noch belegen sie die Eigenschaften des konkret verwendeten `ProcessPoolExecutor`.

Der Abschnitt gliedert sich in zehn Teile:

- Abschnitt 9.10.1 definiert das Modell: Entscheidungsfunktion, Kosten, Zerlegung in Blöcke (*Chunks*) und den Prozess-Pool als Listenplanung.
- Abschnitt 9.10.2 beweist, dass die parallele Suche für jede Worker-Zahl, jede Blockgröße und jeden zeitlichen Ablauf dieselbe Trefferliste liefert wie die sequentielle Suche (Satz 4).
- Abschnitt 9.10.3 leitet die Schranke von Graham für die Laufzeit der Vergleichsphase her, zeigt ihre Schärfe und erweitert sie um den seriellen Zufluss der Aufgaben aus dem Hauptprozess (Sätze 5 bis 7).
- Abschnitt 9.10.4 bestimmt die optimale Blockgröße im Modell (Satz 8) und vergleicht sie mit der Chunk-Reihe.
- Abschnitt 9.10.5 behandelt Amdahl, Karp–Flatt und Gustafson für die Gesamtsuche (Sätze 9 bis 11) und wertet die Phasenzerlegung der Worker-Reihe aus.
- Abschnitt 9.10.6 zeigt, dass Kernbeschleunigung und Parallelisierung einander ersetzen und warum das C++-Backend schlechter skaliert (Satz 12).

– Abschnitt 9.10.7 untersucht die Reihenfolge der Kandidaten und die Lastverteilung (Sätze 13 bis 15).

– Abschnitt 9.10.8 beweist die Grenzen der Parallelisierung: Simulationssatz, bedingte untere Schranke, Grenze durch den Global Interpreter Lock und Grenze durch die Blockzahl.

– Abschnitt 9.10.9 betrachtet die Parallelität *innerhalb* eines Vergleichs über die Schichten.

– Abschnitt 9.10.10 enthält das Fazit, die Folgerungen und die Einordnung in bewiesen, angenommen und gemessen.

9.10.1 Das Modell der parallelen Kandidatenvergleiche

Die Implementierung (Funktion `search_multiomics` im Modul `crud` und `compare_many_multiomics` im Modul `subgraph_executor`) hat drei Phasen. Die *Ladephase* liest die Kandidaten per SQL und baut die Kandidatenliste auf. Die *Vergleichsphase* übergibt Query und Kandidaten mit `ProcessPoolExecutor.map` in Blöcken an die Worker. Die *Aufbauphase* bildet aus den Entscheidungen die Trefferliste. Die folgenden Definitionen fassen das in ein Modell, in dem sich Aussagen beweisen lassen.

Definition 11 (Kandidatenvergleich)**.** Sei q ein Query-Stapel, $c_1, \ldots, c_C$ die Kandidatenfolge nach der SQL-Vorauswahl in der von der Datenbank gelieferten Reihenfolge und $m \in \{\texttt{coherent}, \texttt{independent}\}$ der Modus. Die *Ergebnismenge* ist

$$\mathcal{R} = \{\texttt{keep_A}, \texttt{keep_B}, \texttt{keep_both}, \texttt{equal_keep_A}, \texttt{equal_keep_B}\} \cup \{\texttt{err}\},$$

wobei `err` den Fehlerwert bezeichnet (die Implementierung gibt Fehler als Wert zurück, statt sie auszulösen). Die *Entscheidungsfunktion* $\delta_{q,m} \colon c \mapsto \mathcal{R}$ ist die Funktion, die `execute_multiomics_comparison` berechnet. Die Menge der Treffer-Ergebnisse ist $M = \{\texttt{keep_B}, \texttt{equal_keep_A}, \texttt{equal_keep_B}\}$. Die *sequentielle Suche* SEQ berechnet $r_i = \delta_{q,m}(c_i)$ für $i = 1, \ldots, C$ in dieser Reihenfolge und liefert die Trefferliste $H = (i : r_i \in M)$, aufsteigend nach i.

Definition 12 (Annahme A1: Reinheit)**.** Der Wert $\delta_{q,m}(c)$ hängt nur von (q, m, c) ab. Er hängt nicht davon ab, in welchem Prozess, zu welcher Zeit und nach welchen früheren Aufrufen er berechnet wird.

Die Annahme A1 ist eine Aussage über die Implementierung und nicht über die Mathematik. Sie ist plausibel, weil jeder Worker beim ersten Aufruf lazy eine Algorithmus-Instanz bzw. die native Bibliothek lädt (`_algorithm`, `_native`), diese Zustände aber nur die Laufzeit und nicht das Ergebnis beeinflussen, und weil die Entscheidungen ausschließlich auf ganzzahligen Bitmustern beruhen (keine Gleitkommaarithmetik). Sie wird durch die differenziellen Prüfungen aus Abschnitt 9.3 gestützt, bei denen alle 10.000 Entscheidungen übereinstimmten, und durch die identischen Kandidatenzahlen und Treffer-IDs in der Suchreihe (Abschnitt 9.5). Bewiesen wird sie hier nicht; alle folgenden Sätze der Abschnitte 9.10.2 bis 9.10.10 über das *Ergebnis* der Suche gelten unter A1.

Definition 13 (Kosten)**.** Für jeden Kandidaten i sei $u_i > 0$ der *Aufwand im Worker*, also die Summe aus der Rechenzeit τ_i des Vergleichs und der Zeit $\beta_i \geq 0$ für Übertragung und

Deserialisierung des Kandidaten. Mit $U = \sum_{i=1}^{C} u_i$ bezeichnen wir die *Gesamtarbeit* und mit $\bar{u} = \max_i u_i$ den größten Einzelaufwand. Jeder an einen Worker übergebene Block verursacht zusätzlich einen festen *Aufgabenaufwand* $\alpha \geq 0$ (Verwaltung, Zustellung des Auftrags, Rückgabe des Ergebnisses). Die *serielle Zeit* $T_{\text{ser}} = F + B$ ist die Summe aus Ladephase F und Aufbauphase B; sie fällt im Hauptprozess an. Die Zeit der Vergleichsphase bei p Workern bezeichnen wir mit $V(p)$, die Gesamtzeit mit $T(p) = T_{\text{ser}} + V(p)$.

Definition 14 (Chunk-Zerlegung)**.** Für eine Blockgröße $b \in \mathbb{N}$, $b \geq 1$, sei $k = \lceil C/b \rceil$ die Blockzahl und

$$B_j = \{\, i \in \mathbb{N} : (j-1)\, b < i \leq \min(jb,\, C) \,\}, \qquad j = 1, \ldots, k,$$

der j-te Block. Die *Dauer* der Aufgabe j ist $\theta_j = \alpha + \sum_{i \in B_j} u_i$. Die Gesamtdauer aller Aufgaben ist $W = \sum_{j=1}^{k} \theta_j = k\alpha + U$, die größte Aufgabendauer $\theta_{\max} = \max_j \theta_j$.

Lemma 4 (Eigenschaften der Zerlegung)**.** Für $C \geq 1$ und $b \geq 1$ gilt:

(a) Die Blöcke $B_1, \ldots, B_k$ sind paarweise disjunkt, und ihre Vereinigung ist $\{1, \ldots, C\}$.

(b) $|B_j| = b$ für $j < k$ und $1 \leq |B_k| \leq b$.

(c) $C/b \leq k < C/b + 1$; ferner $k = C$ für $b = 1$ und $k = 1$ für $b \geq C$.

(d) Die Abbildung $i \mapsto (j(i), o(i))$ mit $j(i) = \lceil i/b \rceil$ und $o(i) = i - (j(i) - 1)b$ ist eine Bijektion von $\{1, \ldots, C\}$ auf $\{(j, o) : 1 \leq j \leq k,\ 1 \leq o \leq |B_j|\}$, und $i \in B_{j(i)}$ ist der $o(i)$-te Index des Blocks.

Beweis. (d) Für $i \in \{1, \ldots, C\}$ gilt $(j(i) - 1)b < i \leq j(i)b$ nach der Definition der Aufrundung, und $i \leq C$ liefert $i \leq \min(j(i)b, C)$; also $i \in B_{j(i)}$ und $1 \leq o(i) \leq b$. Da B_j nach Definition genau die Zahlen $(j-1)b+1, \ldots, \min(jb, C)$ enthält, ist $o(i) = i - (j-1)b$ die Position von i in B_j und liegt in $\{1, \ldots, |B_j|\}$. Die Abbildung ist injektiv, weil $i = (j(i)-1)b + o(i)$ aus (j, o) rekonstruiert werden kann. Sie ist surjektiv, weil jedes Paar (j, o) mit $1 \leq o \leq |B_j|$ den Index $i = (j-1)b + o \leq \min(jb, C)$ erzeugt und $j(i) = j$ gilt. (a) folgt aus (d): Jedes i liegt in genau dem Block $B_{j(i)}$. (b) Für $j < k$ ist $jb \leq (k-1)b < C$, also $B_j = \{(j-1)b + 1, \ldots, jb\}$ mit $|B_j| = b$. Für $j = k$ ist $(k-1)b < C \leq kb$, also $|B_k| = C - (k-1)b \in \{1, \ldots, b\}$. (c) Aus $k = \lceil C/b \rceil$ folgt $C/b \leq k < C/b + 1$. Für $b = 1$ ist $k = C$, für $b \geq C$ ist $C/b \leq 1$ und damit $k = 1$. □

Definition 15 (Prozess-Pool als Listenplanung)**.** Ein *Pool mit p Workern* führt die Aufgaben $1, \ldots, k$ aus. Ein *Ablauf* ordnet jeder Aufgabe j einen Worker $w(j)$ und eine Startzeit $s_j \geq r_j$ zu, wobei $r_j \geq 0$ die *Freigabezeit* der Aufgabe ist (der Zeitpunkt, ab dem der Hauptprozess sie zugestellt hat; ohne seriellen Zufluss ist $r_j = 0$). Auf einem Worker überlappen sich die Intervalle $[s_j, s_j + \theta_j)$ nicht. Der Ablauf heißt

- *FIFO*, wenn $s_1 \leq s_2 \leq \cdots \leq s_k$ gilt (Aufgaben starten in Listenreihenfolge), und
- *nicht-leerlaufend*, wenn zu jedem Zeitpunkt t, zu dem ein Worker untätig ist, keine freigegebene Aufgabe wartet, also keine Aufgabe j mit $r_j \leq t < s_j$ existiert.

Die *Makespan* ist $V = \max_j(s_j + \theta_j)$; sie entspricht der Dauer der Vergleichsphase. Mit V^* bezeichnen wir die kleinste Makespan über alle Abläufe mit den gegebenen Freigabezeiten.

Bemerkung 7 (Bezug zur Implementierung)**.** Der `ProcessPoolExecutor` nimmt Aufgaben in der Reihenfolge der Übergabe an und übergibt die nächste wartende Aufgabe an den nächsten freien Worker. Das entspricht einem FIFO- und nicht-leerlaufenden Ablauf, abgesehen von Zustellungsverzögerungen im Mikrosekunden- bis Millisekundenbereich; diese gehen in den Aufgabenaufwand α ein. Die Warteschlange zu den Workern ist in der Größe begrenzt, aber das ändert nichts an der Reihenfolge. Diese Beschreibung ist eine Modellannahme über das Laufzeitsystem von CPython (A2) und wird hier nicht bewiesen. Die Funktion `map` liefert die Ergebnisse in der Reihenfolge der Eingabe und nicht in der Reihenfolge der Fertigstellung; dieser Umstand trägt den Beweis von Satz 4.

9.10.2 Korrektheit der parallelen Ausführung

Definition 16 (Parallele Suche PAR)**.** $\mathrm{PAR}_{p,b}$ zerlegt $1, \ldots, C$ nach Definition 14. Die Aufgabe j berechnet im Worker die Liste $\rho_j = (\delta_{q,m}(c_i))_{i \in B_j}$ in aufsteigender Reihenfolge der Indizes und gibt sie zurück. Der Hauptprozess setzt das Ergebnis r'_i an der Position i gleich dem $o(i)$-ten Element von $\rho_{j(i)}$ und bildet daraus $H' = (i : r'_i \in M)$, aufsteigend nach i.

Satz 4 (Ergebnisgleichheit der parallelen Suche)**.** Es gelte A1. Dann gilt für jede Worker-Zahl $p \geq 1$, jede Blockgröße $b \geq 1$ und *jeden* Ablauf im Sinne von Definition 15 (insbesondere unabhängig davon, welcher Worker welchen Block wann berechnet und in welcher Reihenfolge die Blöcke fertig werden) $r'_i = r_i$ für alle i und damit $H' = H$. Insbesondere sind auch die Menge der fehlgeschlagenen Kandidaten $\{i : r_i = \mathsf{err}\}$ und die Treffertypen (`exact`, `subgraph`) gleich.

Beweis. Sei $i \in \{1, \ldots, C\}$, $j = j(i)$ und $o = o(i)$. Nach Lemma 4(d) ist i der o-te Index von B_j. Die Aufgabe j berechnet nach Definition 16 an der Stelle o ihrer Ergebnisliste den Wert $\delta_{q,m}(c_i)$. Nach A1 hängt dieser Wert nicht vom Worker, vom Zeitpunkt und von den vorher berechneten Blöcken ab; er ist also in jedem Ablauf derselbe Wert r_i. Der Hauptprozess liest an der Position i genau diese Stelle, also gilt $r'_i = r_i$. Weil die Zuordnung $i \mapsto (j(i), o(i))$ nach Lemma 4(d) eine Bijektion ist, wird jede Position genau einmal belegt und nicht überschrieben. Die Trefferliste H' ist dieselbe Funktion des Vektors (r'_i) wie H des Vektors (r_i), also $H' = H$. Die weiteren Aussagen folgen, weil Fehlerwert und Treffertyp Funktionen von r_i sind. □

Korollar 3 (Übertragung der Korrektheit der Pipeline)**.** Satz 3 (keine falschen Negative für $K = \infty$) und die Korrektheit der Multi-Omics-Suche gelten für die parallele Implementierung genau dann, wenn sie für die sequentielle Implementierung gelten. Die Trefferliste ist unabhängig von p und b.

Beweis. Nach Satz 4 sind die Vektoren (r_i) und (r'_i) identisch; jede Aussage über die Trefferliste überträgt sich damit in beide Richtungen. □

Bemerkung 8 (Warum die Zusammenführung keine Rundungsprobleme hat)**.** Bei parallelen Summen von Gleitkommazahlen hängt das Ergebnis von der Reihenfolge der Addition ab, weil die Addition auf Maschinenzahlen nicht assoziativ ist. Hier tritt das nicht auf. Die Zusammenführung ist die Verkettung der Teillisten in der Reihenfolge der Blockindizes; die Verkettung ist assoziativ, aber nicht kommutativ, und genau deshalb ist die Ordnung nach Blockindex (und nicht nach Fertigstellung) wesentlich. Die Entscheidungen selbst sind ganzzahlig. Nicht abgedeckt sind Ausfälle der Infrastruktur, etwa ein beendeter Worker-Prozess oder erschöpfter Speicher; sie führen im `ProcessPoolExecutor` zu einer Ausnahme und nicht zu einem stillen Fehlergebnis.

9.10.3 Zeitschranken der Vergleichsphase: Graham und serieller Zufluss

Satz 5 (Schranke von Graham für FIFO-Abläufe)**.** Seien $\theta_1, \dots, \theta_k > 0$, $W = \sum_j \theta_j$, $\theta_{\max} = \max_j \theta_j$ und alle Freigabezeiten 0. Dann gilt für die optimale Makespan

$$V^* \;\geq\; \max\Big\{\frac{W}{p},\; \theta_{\max}\Big\},$$

und für jeden nicht-leerlaufenden FIFO-Ablauf mit der zuletzt endenden Aufgabe j^*

$$V \;\leq\; \frac{W}{p} + \Big(1 - \frac{1}{p}\Big)\theta_{j^*} \;\leq\; \frac{W}{p} + \Big(1 - \frac{1}{p}\Big)\theta_{\max}.$$

Beweis. Untere Schranke. In jedem Ablauf der Makespan V' führen p Worker in $[0, V']$ höchstens pV' Arbeit aus; die Gesamtarbeit W muss darin Platz finden, also $V' \geq W/p$. Die längste Aufgabe läuft auf einem einzigen Worker und beansprucht $\theta_{\max}$ Zeit, also $V' \geq \theta_{\max}$. Das gilt insbesondere für den optimalen Ablauf.

Obere Schranke. Sei j^* eine zuletzt endende Aufgabe und $s = s_{j^*}$. Zu jedem Zeitpunkt $t < s$ ist die Aufgabe j^* noch nicht gestartet und (wegen $r_{j^*} = 0$) freigegeben. Wäre ein Worker zu einem solchen Zeitpunkt t untätig, so läge ein Verstoß gegen die Nicht-Leerlauf-Eigenschaft vor. Also sind in $[0, s)$ alle p Worker durchgehend beschäftigt, und es wird in diesem Intervall genau die Arbeit ps ausgeführt. Diese Arbeit gehört zu anderen Aufgaben als j^* (denn j^* startet erst bei s) und ist daher höchstens $W - \theta_{j^*}$. Es folgt $ps \leq W - \theta_{j^*}$ und

$$V = s + \theta_{j^*} \leq \frac{W - \theta_{j^*}}{p} + \theta_{j^*} = \frac{W}{p} + \Big(1 - \frac{1}{p}\Big)\theta_{j^*} \leq \frac{W}{p} + \Big(1 - \frac{1}{p}\Big)\theta_{\max}. \qquad \square$$

Korollar 4 (Güte jeder nicht-leerlaufenden Planung)**.** Jeder nicht-leerlaufende FIFO-Ablauf erfüllt $V \leq (2 - \frac{1}{p})\, V^*$. Ferner gilt für den Speedup der Vergleichsphase $V(1)/V(p) \leq \min\{p,\; W/\theta_{\max}\}$, wenn $V(1) \geq W$ ist (die Ausführung mit einem Worker kann nicht schneller als die Gesamtarbeit sein).

Beweis. Aus Satz 5 und $W/p \leq V^*$, $\theta_{\max} \leq V^*$ folgt $V \leq V^* + (1 - \frac{1}{p})V^*$. Für den Speedup: $V(p) \geq V^* \geq W/p$ liefert $V(1)/V(p) \leq p$ nur dann, wenn $V(1) = W$ ist; allgemein ist $V(p) \geq V^* \geq \max\{W/p, \theta_{\max}\}$ und somit $W/V(p) \leq \min\{p, W/\theta_{\max}\}$. Der angegebene Quotient ist daher bezogen auf W gemeint; mit $V(1) = W$ (ein Worker ohne Leerlauf) ergibt sich die Behauptung. $\square$

Satz 6 (Schärfe der Schranke)**.** Für jedes $p \geq 2$ gibt es eine Instanz, für die der nicht-leerlaufende FIFO-Ablauf genau $V = \frac{W}{p} + (1 - \frac{1}{p})\theta_{\max} = 2p - 1$ erreicht, während $V^* = p$ ist; das Verhältnis ist also genau $2 - \frac{1}{p}$.

Beweis. Die Instanz habe $k = p(p-1) + 1$ Aufgaben: zuerst $p(p-1)$ Aufgaben der Dauer 1, zuletzt eine Aufgabe der Dauer p. Dann ist $W = p(p-1) + p = p^2$, $\theta_{\max} = p$, also $\frac{W}{p} + (1 - \frac{1}{p})\theta_{\max} = p + (p-1) = 2p - 1$.

Der FIFO-Ablauf. Zur Zeit $t \in \{0, 1, \dots, p-2\}$ sind alle p Worker frei, und es warten noch mindestens $p(p-1) - pt \geq p$ Einheitsaufgaben; jeder Worker startet eine. Zur Zeit $p - 1$ sind alle $p(p-1)$ Einheitsaufgaben abgearbeitet, alle Worker sind frei, und die lange Aufgabe startet. Sie endet bei $(p-1) + p = 2p - 1$.

Ein besserer Ablauf. Die lange Aufgabe läuft auf Worker 1 in $[0, p)$. Die $p(p-1)$ Einheitsaufgaben verteilen sich auf die Worker $2, \dots, p$ mit je p Aufgaben in $[0, p)$. Die Makespan ist p, und wegen $V^* \geq W/p = p$ ist sie optimal. $\square$

Die Instanz des Beweises hat die Form, die in Gen-DB entsteht, wenn die teuersten Vergleiche am Ende der Liste stehen: viele kleine Aufgaben, dann eine große. Die SQL-Abfrage sortiert die Kandidaten nach `node_count` aufsteigend; Abschnitt 9.10.7 untersucht die Folgen.

Bisher hat der Hauptprozess alle Aufgaben zur Zeit 0 bereitgestellt. In der Implementierung muss er jedoch jede Aufgabe zuerst serialisieren und zustellen, und das geschieht nacheinander. Der folgende Satz erfasst diesen seriellen Zufluss.

Satz 7 (Graham-Schranke mit seriellem Zufluss)**.** Die Aufgaben $1, \dots, k$ haben Freigabezeiten $0 \le r_1 \le \cdots \le r_k$, und der Ablauf sei nicht-leerlaufend im Sinne von Definition 15 und FIFO. Dann gilt

$$V \;\le\; r_{j^*} + \frac{W}{p} + \Big(1 - \frac{1}{p}\Big)\theta_{j^*} \;\le\; r_k + \frac{W}{p} + \Big(1 - \frac{1}{p}\Big)\theta_{\max}, \qquad V^* \;\ge\; \max\Big\{\frac{W}{p},\; \theta_{\max},\; r_k + \theta_k\Big\},$$

wobei W/p als untere Schranke voraussetzt, dass $r_1 = 0$ ist. Wird der Zufluss von einem einzigen seriellen Zubringer erzeugt, der Aufgabe j in der Zeit σ_j aufbereitet, so ist $r_j = \sum_{l \le j} \sigma_l$ und $r_k = S := \sum_j \sigma_j$.

Beweis. Obere Schranke. Sei j^* eine zuletzt endende Aufgabe, $s = s_{j^*}$. Zu jedem Zeitpunkt $t < s$, an dem ein Worker untätig ist, kann j^* nicht freigegeben sein, denn sonst wartete eine freigegebene Aufgabe und der Ablauf wäre nicht nicht-leerlaufend. Also gilt $t < r_{j^*}$. Jeder Worker ist deshalb in $[0, s)$ höchstens während der Zeitspanne r_{j^*} untätig, und die in $[0, s)$ ausgeführte Arbeit beträgt mindestens $p(s - r_{j^*})$. Sie ist zugleich höchstens $W - \theta_{j^*}$, weil j^* noch nicht gestartet ist. Also $s \le r_{j^*} + (W - \theta_{j^*})/p$ und

$$V = s + \theta_{j^*} \le r_{j^*} + \frac{W}{p} + \Big(1 - \frac{1}{p}\Big)\theta_{j^*}.$$

Wegen $r_{j^*} \le r_k$ und $\theta_{j^*} \le \theta_{\max}$ folgt die zweite Ungleichung.

Untere Schranke. Die Gründe aus Satz 5 bleiben bestehen. Zusätzlich kann die letzte Aufgabe nicht vor r_k starten, sodass jeder Ablauf mindestens $r_k + \theta_k$ dauert. □

Korollar 5 (Sättigung durch den Zubringer)**.** Gilt für alle Aufgaben $\sigma_j \ge \theta_j / p_0$ mit einer Worker-Zahl p_0, so ist $S \ge W/p_0$, und für alle $p \ge p_0$ gilt $V^* \ge W/p_0 + \theta_k$. Die Vergleichsphase wird dann vom Zubringer bestimmt und verkürzt sich durch mehr Worker nicht mehr. Der Speedup der Vergleichsphase ist in diesem Fall höchstens $p_0\,(1 + \theta_k/(W/p_0))^{-1} < p_0$ gegenüber einer Ausführung, die den Zubringer nicht benötigt.

Beweis. Es ist $S = \sum_j \sigma_j \ge \sum_j \theta_j / p_0 = W/p_0$. Nach Satz 7 gilt $V^* \ge r_k + \theta_k = S + \theta_k \ge W/p_0 + \theta_k$, unabhängig von p. Ist $V(1) = W$ (ein Worker ohne Zubringerverzögerung), so folgt $V(1)/V^* \le W/(W/p_0 + \theta_k) = p_0/(1 + p_0\theta_k/W) < p_0$. □

Die Bedingung $\sigma_j \ge \theta_j/p_0$ besagt, dass der Hauptprozess eine Aufgabe nicht schneller bereitstellt, als p_0 Worker sie abarbeiten würden. Sie ist umso eher erfüllt, je billiger der Vergleich im Verhältnis zur Übertragung ist, also gerade bei dem C++-Backend (Abschnitt 9.10.6). Die vorliegenden Messungen trennen den Zubringer nicht von der Vergleichszeit; die Phase `compare_s` enthält ihn. Ob die Sättigung beobachtet wurde, lässt sich daher nur indirekt über den Parallelüberschuss in Abschnitt 9.10.5 beurteilen.

9.10.4 Optimale Blockgröße

Die Blockgröße b wirkt in zwei Richtungen. Kleine Blöcke vergrößern die Blockzahl $k \approx C/b$ und damit die Mehrarbeit $k\alpha$ für den Aufgabenaufwand. Große Blöcke vergrößern die längste Aufgabe $\theta_{\max} \approx \alpha + b\bar{u}$ und damit den Schwanz $(1-\frac{1}{p})\theta_{\max}$ aus Satz 5. Der folgende Satz bestimmt den Ausgleich.

Satz 8 (Optimale Blockgröße im Graham-Modell)**.** Sei $p \geq 2$ und $\bar{u} = \max_i u_i$. Für jede Blockgröße $b \geq 1$ gilt für jeden nicht-leerlaufenden FIFO-Ablauf ohne Zufluss

$$V(b) \;\leq\; g(b) := \frac{\lceil C/b \rceil \alpha + U}{p} + \Big(1 - \frac{1}{p}\Big)(\alpha + b\bar{u}) \;\leq\; \hat{g}(b) := U/p + \alpha + \frac{C\alpha}{p\,b} + \frac{p-1}{p}\,\bar{u}\,b.$$

Die Funktion $\hat{g}$ ist auf $(0,\infty)$ streng konvex und hat das eindeutige Minimum

$$b^* = \sqrt{\frac{C\alpha}{(p-1)\,\bar{u}}}, \qquad \hat{g}(b^*) = \frac{U}{p} + \alpha + \frac{2}{p}\sqrt{(p-1)\,C\,\alpha\,\bar{u}}.$$

Über den ganzen Zahlen $b \geq 1$ wird $\hat{g}$ in $\lfloor b^* \rfloor$ oder $\lceil b^* \rceil$ minimal (bzw. bei $b = 1$, falls $b^* < 1$).

Beweis. Jede Aufgabe hat nach Definition 14 die Dauer $\theta_j \leq \alpha + b\bar{u}$, also $\theta_{\max} \leq \alpha + b\bar{u}$. Mit $W = \lceil C/b \rceil \alpha + U$ liefert Satz 5 die Schranke $g(b)$. Nach Lemma 4(c) ist $\lceil C/b \rceil < C/b + 1$; setzt man das ein, so ist

$$g(b) \leq \frac{(C/b+1)\alpha + U}{p} + \Big(1-\frac{1}{p}\Big)(\alpha + b\bar{u}) = \frac{U}{p} + \frac{\alpha}{p} + \Big(1-\frac{1}{p}\Big)\alpha + \frac{C\alpha}{pb} + \frac{p-1}{p}\bar{u}b = \hat{g}(b),$$

denn $\frac{\alpha}{p} + (1-\frac{1}{p})\alpha = \alpha$. Die Funktion $\hat{g}$ ist die Summe einer Konstanten, der streng konvexen Funktion $b \mapsto \frac{C\alpha}{p} b^{-1}$ (für $\alpha > 0$) und einer linearen Funktion, also streng konvex. Ihre Ableitung $\hat{g}'(b) = -\frac{C\alpha}{pb^2} + \frac{p-1}{p}\bar{u}$ verschwindet genau bei $b^{*2} = \frac{C\alpha}{(p-1)\bar{u}}$. Die beiden variablen Terme sind dort gleich, nämlich $\frac{C\alpha}{pb^*} = \frac{1}{p}\sqrt{(p-1)C\alpha\bar{u}} = \frac{p-1}{p}\bar{u}b^*$, und ihre Summe ist $\frac{2}{p}\sqrt{(p-1)C\alpha\bar{u}}$. Eine streng konvexe Funktion auf den ganzen Zahlen ist in einer der beiden Nachbarzahlen des reellen Minimums minimal. □

Korollar 6 (Überschuss wächst nur wie $\sqrt{C}$)**.** Sind alle Aufwände gleich, $u_i = u$, so ist $U = Cu$ und

$$\frac{\hat{g}(b^*)}{U/p} = 1 + \frac{p\,\alpha}{C\,u} + 2\sqrt{\frac{(p-1)\,\alpha}{C\,u}} \;\xrightarrow[C\to\infty]{}\; 1.$$

Mit optimaler Blockgröße geht der relative Parallelüberschuss der Vergleichsphase also mit wachsender Kandidatenzahl gegen null, und zwar wie $O(\sqrt{p\alpha/(Cu)})$.

Beweis. Einsetzen von $U = Cu$ und $\bar{u} = u$ in $\hat{g}(b^*)$ und Division durch $U/p = Cu/p$ ergibt $1 + \frac{p\alpha}{Cu} + \frac{2}{Cu}\sqrt{(p-1)C\alpha u}$; der letzte Summand ist $2\sqrt{(p-1)\alpha/(Cu)}$. □

Bemerkung 9 (Mindestzahl von Blöcken)**.** Die Schranke aus Satz 5 setzt nicht voraus, dass alle Worker Arbeit erhalten. Bei $k < p$ Blöcken sind aber mindestens $p - k$ Worker untätig, und die untere Schranke $V^* \geq \theta_{\max} \geq W/k$ ist dann schärfer als W/p. Ein Speedup der Vergleichsphase über k hinaus ist ausgeschlossen. In der Worker-Reihe hat die Anfrage mit 519 Kandidaten bei Blockgröße 256 nur $k = \lceil 519/256 \rceil = 3$ Blöcke, also weniger als die 4 Worker; eine Anfrage mit 11.801 Kandidaten hat $k = 47$. Die Zeiten einer einzelnen Anfrage streuen so stark (die beiden Wiederholungen der Python-Vergleichsphase der Anfrage mit 519 Kandidaten bei einem Worker dauerten 4,93 und 3,49 Sekunden), dass sich diese Grenze an einzelnen Anfragen nicht quantitativ prüfen lässt.

Vergleich mit der Chunk-Reihe.

Summiert man die Schranke $\hat{g}$ über die $M = 12$ Messungen der Chunk-Reihe (sechs Anfragen, zwei Wiederholungen, $p = 2$), so erhält man für die Zeit der Vergleichsphase die Form

$$V(b) = c_0 + \frac{c_1}{b} + c_2\, b, \qquad c_1 = \frac{\alpha \sum_m C_m}{p}, \qquad c_2 = M\Big(1 - \frac{1}{p}\Big)\bar{u}_{\text{eff}}, \qquad c_0 \approx \frac{U}{p} + M\alpha,$$

mit der Summe der Kandidatenzahlen $\sum_m C_m = 55.470$ und einem wirksamen Schwanzaufwand $\bar{u}_{\text{eff}}$. Das Skript `src/analyse_parallelisierung.py` passt diese drei Parameter nach der Methode der kleinsten Quadrate an die gemessenen Zeiten der Vergleichsphase an (Tabelle 9.7). Die Anpassung hat drei freie Parameter bei sieben Messpunkten; sie bestätigt das Modell deshalb nicht, sondern prüft nur, ob die Parameter plausible Größen haben.

Tabelle 9.7: Anpassung $V(b) = c_0 + c_1/b + c_2 b$ an die Vergleichsphase der Chunk-Reihe ($p = 2$, 12 Messungen je Backend). Spalte „Referenz“: aus der Worker-Reihe ($p = 1$) berechnete Vergleichsgrößen.

Größe	Python	C++	Referenz
R^2 der Anpassung	0,964	0,992	
c_0 [s]	29,36	4,51	$V(1)/2 = 30{,}01$ bzw. 4,95
Aufgabenaufwand $\alpha = c_1 p / \sum C_m$ [ms]	0,48	0,56	backendunabhängig erwartet
Schwanzaufwand $\bar{u}_{\text{eff}}$ [ms]	0,87	0,23	Mittel $V(1)/C$: 1,08 bzw. 0,18
$b^* = \sqrt{c_1/c_2}$	50,8	105,2	beobachtetes Minimum: 128 bzw. 32

Drei Beobachtungen stützen die Deutung. Erstens stimmt die angepasste Konstante c_0 mit der unabhängig gemessenen idealen Parallelzeit $V(1)/2$ überein (Python 29,4 gegen 30,0 Sekunden, C++ 4,5 gegen 4,95 Sekunden); der Rest ist der Parallelüberschuss. Zweitens ergibt sich der Aufgabenaufwand für beide Backends auf etwa 0,5 Millisekunden, obwohl die beiden Anpassungen unabhängig voneinander sind und der Aufgabenaufwand nach der Modellvorstellung nicht vom Vergleichskern abhängt. Drittens liegt der Schwanzaufwand in der Größenordnung des mittleren Aufwands je Vergleich (0,87 gegen 1,08 Millisekunden bei Python, 0,23 gegen 0,18 Millisekunden bei C++). Dagegen sagt das Modell ein sehr flaches Minimum zwischen etwa 30 und 130 voraus; die gemessenen Minima (128 bei Python, 32 bei C++) liegen in diesem Bereich, aber ihre Reihenfolge entspricht nicht den angepassten Werten 50,8 und 105,2. Die Zeiten bei 32 und 128 unterscheiden sich in der Messung um weniger als 3 %; die Reihenfolge der beiden Optima ist deshalb statistisch nicht gesichert. Belegt ist der U-förmige Verlauf: Die Blockgröße 1 kostet gegenüber dem besten Wert bei der Vergleichsphase das 1,5-Fache (Python) bzw. das 4,1-Fache (C++), die Blockgröße 4096 das 1,8- bzw. 2,1-Fache.

9.10.5 Amdahl, Karp–Flatt und Gustafson für die Gesamtsuche

Die Gesamtzeit der Suche ist $T(p) = T_{\text{ser}} + V(p)$ (Definition 13). Die ideale Vergleichsphase wäre $V(1)/p$; die Differenz nennen wir *Parallelüberschuss*

$$O(p) := V(p) - \frac{V(1)}{p}, \qquad \text{also} \qquad T(p) = T_{\text{ser}} + \frac{V(1)}{p} + O(p).$$

Das ist eine Identität. Inhalt bekommt sie erst durch die Messung von T_{ser}, $V(1)$ und $V(p)$. Der *serielle Anteil* ist $f = T_{\text{ser}}/T(1) \in (0,1)$. Speedup und Effizienz sind wie in Definition 10 definiert, $S(p) = T(1)/T(p)$ und $E(p) = S(p)/p$. Das *Amdahl-Modell* setzt $O(p) = 0$ und T_{ser} unabhängig von p voraus.

Satz 9 (Gesetz von Amdahl)**.** Im Amdahl-Modell gilt $S(p) = \left(f + \frac{1-f}{p}\right)^{-1}$ und $E(p) = \left(1 + (p-1)f\right)^{-1}$. Dabei ist

(a) S streng wachsend in p mit $S(1) = 1$ und $\lim_{p\to\infty} S(p) = 1/f$,

(b) $S(p) \le \min\{p,\, 1/f\}$, und $S(p) < 1/f$ für jedes endliche p,

(c) E streng fallend, und $E(p) \ge \varepsilon$ gilt genau dann, wenn $p \le 1 + \left(\frac{1}{\varepsilon} - 1\right)/f$ (für $0 < \varepsilon \le 1$).

Beweis. Es ist $T(p) = T(1)\,(f + (1-f)/p)$, denn $T_{\text{ser}} = fT(1)$ und $V(1) = (1-f)T(1)$; daraus folgt die Formel für S, und $E = S/p$ liefert $E(p) = 1/(pf + 1 - f) = 1/(1 + (p-1)f)$. (a) Der Nenner $f + (1-f)/p$ ist wegen $1 - f > 0$ streng fallend in p, also ist S streng wachsend; $S(1) = 1$ und der Grenzwert des Nenners ist f. (b) Der Nenner ist größer als f, also $S < 1/f$. Er ist außerdem mindestens $1/p$, denn $f + (1-f)/p \ge 1/p$ gilt genau dann, wenn $f \ge f/p$, und das trifft zu. Also $S \le p$. (c) E ist fallend, weil $1 + (p-1)f$ in p wächst. Die Ungleichung $E(p) \ge \varepsilon$ ist äquivalent zu $1 + (p-1)f \le 1/\varepsilon$, also zu $p \le 1 + (1/\varepsilon - 1)/f$. □

Satz 10 (Karp–Flatt-Maß)**.** Für $p \ge 2$ sei $e(p) := \dfrac{1/S(p) - 1/p}{1 - 1/p}$. Gilt $T(p) = T(1)\left(f + \frac{1-f}{p} + \omega(p)\right)$ mit dem relativen Parallelüberschuss $\omega(p) = O(p)/T(1)$ (und T_{ser} unabhängig von p), so ist

$$e(p) = f + \frac{p}{p-1}\,\omega(p).$$

Insbesondere ist $e(p) = f$ im Amdahl-Modell, und $e(p) \ge f$ genau dann, wenn $\omega(p) \ge 0$.

Beweis. Aus der Voraussetzung folgt $1/S(p) = T(p)/T(1) = f + (1-f)/p + \omega(p)$. Dann ist $1/S(p) - 1/p = f + (1-f)/p - 1/p + \omega = f(1 - 1/p) + \omega$. Division durch $1 - 1/p = (p-1)/p$ ergibt $e(p) = f + \omega\, p/(p-1)$. □

Das Maß $e(p)$ lässt sich aus zwei Gesamtzeiten bestimmen, ohne die Phasen zu kennen. Bleibt $e(p)$ für wachsendes p ungefähr konstant, so spricht das für einen festen seriellen Anteil; wächst es, so wächst der Überschuss mit p. Beides sind Indizien und keine Beweise, denn $e(p)$ vermischt f und $\omega(p)$.

Satz 11 (Gustafson: skalierter Speedup)**.** Auf p Workern laufe eine Instanz in der Zeit $T_p = T_{\text{ser}} + V_p$ mit $V_p = V_1/p$, wobei V_1 die Vergleichszeit derselben Instanz mit einem Worker ist, und sei $s = T_{\text{ser}}/T_p$ der serielle Anteil der *parallelen* Laufzeit. Dann ist

$$\frac{T_1}{T_p} = s + p\,(1 - s) = p - (p-1)\,s.$$

Beweis. Es ist $T_1 = T_{\text{ser}} + V_1 = sT_p + pV_p = sT_p + p(1-s)T_p$. Division durch T_p ergibt die Behauptung. □

Gustafson stellt dem Amdahl'schen Blick auf eine feste Aufgabe den auf eine mit p wachsende Aufgabe gegenüber. Für Gen-DB ist entscheidend, ob der serielle Anteil mit der Aufgabengröße wächst. Das tut er.

Korollar 7 (Skalierung der Datenbank verbessert die Parallelisierbarkeit nicht)**.** Wachsen die serielle Zeit und die Einzelarbeit proportional zur Kandidatenzahl, $T_{\text{ser}} = aC$ und $V(1) = cC$ mit $a, c > 0$, so ist $f = a/(a + c)$ unabhängig von C, und im Amdahl-Modell gilt für jede Kandidatenzahl

$$S(p) < \frac{1}{f} = 1 + \frac{c}{a}.$$

Auch der skalierte Speedup bei $C_p = pC_1$ Kandidaten ist dann $T_1/T_p = \left(f + \frac{1-f}{1}\right) \Big/ \left(f + \frac{1-f}{p}\right) = S(p)$, also nicht größer als der Amdahl-Speedup.

Beweis. $f = aC/(aC + cC) = a/(a + c)$ und $1/f = 1 + c/a$. Die Schranke folgt aus Satz 9(b). Für den skalierten Speedup gilt $T_1(pC_1) = p\,(a+c)C_1$ und $T_p(pC_1) = paC_1 + cC_1$ (Vergleichsphase ideal durch p geteilt); der Quotient ist $p(a+c)/(pa+c) = 1/\left(f+(1-f)/p\right)$ nach Division durch $a + c$. □

Auswertung der Worker-Reihe.

Die Messdaten der Worker-Reihe (Abschnitt 9.6) enthalten die Phasenzeiten `fetch_s`, `compare_s` und `build_s` je Messung. Tabelle 9.8 fasst sie über die 12 Messungen je Zeile zusammen (sechs Anfragen, zwei Wiederholungen, zusammen 55.470 Kandidaten). Die Auswertung ist mit dem Skript `src/analyse_parallelisierung.py` aus der JSON-Datei reproduzierbar.

Tabelle 9.8: Phasenzerlegung der Worker-Reihe und Kennzahlen aus den Sätzen 9 und 10. $T_{\text{ser}} = F + B$ (Laden und Trefferaufbau), V Vergleichsphase, $S_V = V(1)/V(p)$, S_A der Amdahl-Speedup aus dem seriellen Anteil bei einem Worker, $O = V - V(1)/p$.

Backend	p	T [s]	T_{ser} [s]	V [s]	S	E	$e(p)$	S_V	S_A	O [s]
Python	1	74,73	14,72	60,01	1,00	1,00	—	1,00	1,00	0,00
	2	50,31	15,67	34,64	1,49	0,74	0,347	1,73	1,67	4,64
	3	39,14	15,51	23,63	1,91	0,64	0,286	2,54	2,15	3,63
	4	35,55	14,72	20,84	2,10	0,53	0,301	2,88	2,51	5,84
C++	1	24,08	14,18	9,90	1,00	1,00	—	1,00	1,00	0,00
	2	19,69	13,95	5,74	1,22	0,61	0,635	1,72	1,26	0,79
	3	20,08	15,03	5,05	1,20	0,40	0,751	1,96	1,38	1,75
	4	18,68	14,09	4,59	1,29	0,32	0,701	2,16	1,45	2,11

Aus Tabelle 9.8 folgen vier Befunde.

(1) **Die serielle Zeit ist unabhängig von p und vom Backend.** T_{ser} liegt in allen acht Zeilen zwischen 13,95 und 15,67 Sekunden, also innerhalb von etwa ±6 % um 14,7 Sekunden; die Ladephase ist für beide Backends dieselbe SQL-Abfrage. Damit ist die Voraussetzung des Amdahl-Modells, dass T_{ser} nicht von p abhängt, im Rahmen der Messstreuung erfüllt.

(2) **Serielle Anteile.** Es ist $f = 14{,}72/74{,}73 = 0{,}197$ für Python und $f = 14{,}18/24{,}08 = 0{,}589$ für C++. Nach Satz 9(b) ist der Speedup durch $1/f = 5{,}08$ bzw. 1,70 begrenzt, und die Effizienz fällt unter 50 %, sobald $p > 1 + 1/f$ ist, das heißt für $p > 6{,}08$ (Python) bzw. $p > 2{,}70$ (C++). Je Kandidat kostet das Laden $a = 0{,}265$ Millisekunden (Python-Reihe) bzw. 0,256 Millisekunden (C++-Reihe), der Vergleich dagegen $c = 1{,}082$ bzw. 0,178 Millisekunden. Die Grenze $1/f = 1 + c/a$ ist das Verhältnis des Vergleichsaufwands zum Ladeaufwand je Kandidat, vergrößert um eins.

(3) **Der Amdahl-Speedup wird nicht erreicht.** Der aus dem seriellen Anteil bei einem Worker berechnete Speedup S_A liegt über dem gemessenen: bei vier Workern 2,51 gegen 2,10 (Python) und 1,45 gegen 1,29 (C++). Die Differenz ist der Parallelüberschuss $O(p)$; er beträgt bei vier Workern 5,84 bzw. 2,11 Sekunden, das sind 7,8 % bzw. 8,8 % der Einzelzeit $T(1)$. Die Vergleichsphase selbst erreicht bei vier Workern $S_V = 2{,}88$ (Python) und 2,16 (C++), also Effizienzen von 72 % und 54 % und nicht 100 %.

(4) **Das Karp–Flatt-Maß.** Für Python ist $e(p) = 0{,}347;\ 0{,}286;\ 0{,}301$ für $p = 2, 3, 4$. Es steigt nicht mit p und liegt um etwa 0,1 über $f = 0{,}197$. Nach Satz 10 entspricht das einem relativen Überschuss $\omega(p) = (e - f)(p - 1)/p$ von $0{,}075;\ 0{,}059;\ 0{,}078$, also einem Überschuss von etwa 6 bis 8 % der Einzelzeit, der *nicht* mit p wächst. Für C++ ist $e(p) = 0{,}635;\ 0{,}751;\ 0{,}701$ gegen $f = 0{,}589$; auch hier steigt e nur zwischen $p = 2$ und $p = 3$ und fällt danach wieder. Der Unterschied zwischen f und e enthält auch die Schwankung von T_{ser} von Messung zu Messung (bis zu etwa 1 Sekunde), sodass sich $\omega(p)$ aus den Gesamtzeiten nur ungefähr bestimmen lässt.

Bemerkung 10 (Vergleich der Messreihen aus Abschnitt 7)**.** Im Abschnitt 7 ergab sich für zwei Worker $S(2) = 1{,}90$ und damit $e(2) = 0{,}051$, also ein serieller Anteil von etwa 5 %. Die Python-Reihe dieses Abschnitts hat bei zwei Workern $e(2) = 0{,}347$. Beide Messreihen wurden auf derselben Hardware und mit Windows ausgeführt, unterscheiden sich aber in Suchart, Softwarekonfiguration und Datenbankversion. Die Ladeabfrage der Multi-Omics-Suche enthält außerdem einen Verbund mit der Schichttabelle und eine korrelierte Unterabfrage; ob sie deshalb je Kandidat teurer ist, wurde nicht gemessen. Der Unterschied ist also kein Widerspruch, aber auch kein Befund über die Ursache; er zeigt vor allem, dass der serielle Anteil f eine Eigenschaft von Anfrageart und Softwarekonfiguration ist und kein Parameter des Parallelisierungsmodells allein.

9.10.6 Kernbeschleunigung und Parallelisierung ersetzen einander

Das C++-Backend beschleunigt den Vergleichskern; die Parallelisierung verteilt ihn auf mehrere Worker. Beide Maßnahmen verkürzen denselben Anteil $1 - f$ der Laufzeit. Der folgende Satz beschreibt ihr Zusammenspiel im Amdahl-Modell.

Satz 12 (Verschiebung des Engpasses)**.** Im Amdahl-Modell mit $0 < f < 1$ verkürze ein schnellerer Kern die Vergleichsphase um den Faktor $\rho > 0$, also $T_\rho(p) = T_{\text{ser}} + V(1)/(\rho p)$, bei unveränderter serieller Zeit. Dann gilt:

(a) Der serielle Anteil $f_\rho = T_{\text{ser}}/T_\rho(1) = \dfrac{f\rho}{f\rho + 1 - f}$ ist streng wachsend in ρ und strebt für $\rho \to \infty$ gegen 1.

(b) Die Grenze des Speedups durch Worker ist $1/f_\rho = 1 + \dfrac{1-f}{f\rho}$, also fallend in ρ und strebt gegen 1. Die Effizienz $E_\rho(p) = \Big(1 + (p-1)f_\rho\Big)^{-1}$ ist für $p \geq 2$ streng fallend in ρ.

(c) Der Gewinn des schnelleren Kerns bei p Workern ist $R_\rho(p) = \dfrac{T_1(p)}{T_\rho(p)} = \dfrac{1}{\varphi_p + (1-\varphi_p)/\rho}$ mit $\varphi_p = \dfrac{T_{\text{ser}}}{T_{\text{ser}} + V(1)/p} = \dfrac{f}{f + (1-f)/p}$. Es gilt $R_\rho(p) < \min\{\rho, 1/\varphi_p\}$ für $\rho > 1$, und φ_p wächst in p, sodass $R_\rho(p)$ mit der Worker-Zahl *fällt*.

(d) (Austauschrelation) Die ideale Vergleichszeit ist mit q Workern und Kern ρ gleich der mit $p = \rho q$ Workern und Kern 1.

Beweis. (a) $T_\rho(1) = T_{\text{ser}} + V(1)/\rho = T(1)\Big(f + (1-f)/\rho\Big)$, also $f_\rho = f/(f + (1-f)/\rho) = f\rho/(f\rho + 1 - f)$. Die Ableitung nach ρ ist $\Big(f(f\rho + 1 - f) - f\rho \cdot f\Big)/(f\rho + 1 - f)^2 = f(1-f)/(f\rho+1-f)^2 > 0$; der Grenzwert für $\rho \to \infty$ ist 1. (b) Nach Satz 9(b) ist die Grenze $1/f_\rho = (f\rho + 1 - f)/(f\rho) = 1 + (1-f)/(f\rho)$, fallend in ρ. $E_\rho(p) = 1/(1 + (p-1)f_\rho)$ fällt, weil f_ρ wächst und $p - 1 > 0$ ist. (c) $T_1(p) = T_{\text{ser}} + V(1)/p$ und $T_\rho(p) = T_{\text{ser}} + V(1)/(\rho p)$. Division von Zähler und Nenner durch $T_1(p)$ liefert $R_\rho(p) = 1/(\varphi_p + (1-\varphi_p)/\rho)$. Der Nenner ist größer als φ_p und größer als $1/\rho$ (wegen $\varphi_p + (1-\varphi_p)/\rho > 1/\rho \iff \varphi_p > \varphi_p/\rho$), also $R_\rho < \min\{1/\varphi_p, \rho\}$. Schließlich ist $\varphi_p = f/(f + (1-f)/p)$ wachsend in p, und R_ρ fällt in φ_p, weil $\varphi_p + (1-\varphi_p)/\rho = 1/\rho + \varphi_p(1 - 1/\rho)$ für $\rho > 1$ in φ_p wächst. (d) $V(1)/(\rho q) = V(1)/p \iff p = \rho q$. □

Der Satz formalisiert die Beobachtung aus Abschnitt 9.6: Je besser der Kern, desto weniger bringen zusätzliche Worker. Die Messdaten stützen die Aussagen quantitativ:

- Der Kern beschleunigt die Vergleichsphase bei einem Worker um $\rho = 60{,}01/9{,}90 = 6{,}06$. Aus $f = 0{,}197$ für Python sagt Satz 12(a) einen seriellen Anteil $f_\rho = 0{,}598$ für C++ voraus; gemessen wurden 0,589.
- Die Grenze des Speedups fällt von $1/f = 5{,}08$ auf $1/f_\rho = 1{,}67$ (gemessen aus den C++-Daten: 1,70).
- Der Gewinn R des C++-Kerns gegenüber Python sinkt mit der Worker-Zahl: gemessen 3,10 ($p = 1$), 2,56 ($p = 2$), 1,95 ($p = 3$), 1,90 ($p = 4$); der Satz sagt aus den Werten f und ρ 3,04, 2,27 und 1,73 für $p = 1, 2, 4$ voraus. Die Vorhersage unterschätzt den Gewinn bei mehreren Workern, weil der Parallelüberschuss $O(p)$ bei Python in absoluten Sekunden größer ist als bei C++ (Tabelle 9.8); das Modell ist im Verlauf richtig und in den Werten bis auf etwa 10 % genau.
- (Modellrechnung, nicht gemessen.) Mit $T_{\text{ser}} = 14{,}72$ Sekunden benötigt Python nach Satz 12(d) rechnerisch $p \geq 60{,}01/(24{,}08 - 14{,}72) = 6{,}41$ Worker, um die Gesamtzeit von C++ mit einem Worker (24,08 Sekunden) zu erreichen. Der Messrechner hat vier Kerne; dort lag Python mit vier Workern bei 35,55 Sekunden und damit deutlich darüber.

Der Hebel liegt für das C++-Backend damit nicht mehr bei der Parallelisierung, sondern bei der Ladephase. Auch ein vollständig beseitigter Parallelüberschuss ließe die

Gesamtzeit bei vier Workern nur auf $T_{\text{ser}} + V(1)/4 = 14{,}18 + 2{,}48 = 16{,}7$ Sekunden sinken, das sind höchstens 11 % weniger als gemessen (18,68 Sekunden). Bei Python wären es $14{,}72 + 15{,}0 = 29{,}7$ Sekunden und damit höchstens 16 % weniger als gemessen (35,55 Sekunden). Diese beiden Zahlen sind obere Grenzen für jede Maßnahme, die ausschließlich den Parallelüberschuss senkt (kleinerer Aufgabenaufwand, Übergabe der Query nur einmal je Block, bessere Blockgröße oder Threads statt Prozesse, Abschnitt 9.10.8).

9.10.7 Reihenfolge der Kandidaten und Lastverteilung

Die SQL-Abfrage der Suche liefert die Kandidaten sortiert nach `node_count` aufsteigend. Der Aufwand eines Vergleichs wächst mit der Knotenzahl des Kandidaten: In der Python-Implementierung lesen die Prüfung der Stapel und die Berechnung der Zeilenkomponenten jeweils alle $L\,n^2$ Matrixeinträge, sodass ein Vergleich $\Theta\big(L\,(n_A^2 + n_B^2)\big)$ kostet. Für das C++-Backend wurde der Verlauf nicht untersucht. Die großen Kandidaten stehen in Python daher am Ende der Liste. Das ist die ungünstige Reihenfolge aus Satz 6. Die folgenden Sätze zeigen, was sich dadurch ändert.

Satz 13 (Absteigende Reihenfolge: Schranke $\frac{3}{2} - \frac{1}{2p}$)**.** Sind die Aufgabendauern absteigend sortiert, $\theta_1 \geq \cdots \geq \theta_k$, und ist der Ablauf nicht-leerlaufend und FIFO ohne Zufluss, so gilt

$$V \leq \left(\frac{3}{2} - \frac{1}{2p}\right) V^*.$$

Beweis. Die ersten Aufgaben starten bei 0. Seien m die Zahl der Aufgaben, die zur Zeit 0 starten. Wäre $m < \min\{p, k\}$, so wären zur Zeit 0 ein Worker untätig und eine Aufgabe wartend, im Widerspruch zur Nicht-Leerlauf-Eigenschaft. Also ist $m \geq \min\{p, k\}$, und wegen FIFO starten die Aufgaben $1, \ldots, \min\{p, k\}$ bei 0.

Fall $k \leq p$. Alle Aufgaben starten bei 0, also $V = \theta_1 = \theta_{\max} \leq V^*$.

Fall $k > p$. Sei j^* eine zuletzt endende Aufgabe. Ist $j^* \leq p$, so startet sie bei 0 und $V = \theta_{j^*} \leq V^*$. Ist $j^* \geq p+1$, so ist $\theta_{j^*} \leq \theta_{p+1}$. In jedem Ablauf liegen zwei der $p+1$ Aufgaben $1, \ldots, p+1$ auf demselben Worker (Schubfachprinzip), also gilt $V^* \geq 2\theta_{p+1}$ (jede der beiden hat mindestens die Dauer θ_{p+1}). Mit Satz 5 und $W/p \leq V^*$ folgt

$$V \leq \frac{W}{p} + \left(1 - \frac{1}{p}\right)\theta_{j^*} \leq V^* + \left(1 - \frac{1}{p}\right)\frac{V^*}{2} = \left(\frac{3}{2} - \frac{1}{2p}\right)V^*. \qquad \square$$

Bemerkung 11 (Schärfere Schranke aus der Literatur)**.** Graham zeigte für die absteigende Reihenfolge die schärfere Schranke $\frac{4}{3} - \frac{1}{3p}$ [71]. Ihr Beweis erfordert eine zusätzliche Fallunterscheidung und wird hier nicht wiedergegeben; die in Satz 13 bewiesene Schranke genügt für die folgenden Aussagen.

Korollar 8 (Die Reihenfolge entscheidet)**.** Für die Aufgaben aus dem Beweis von Satz 6 (zuerst $p(p-1)$ Aufgaben der Dauer 1, dann eine der Dauer p) ist die Makespan $2p - 1$. Dieselben Aufgaben in absteigender Reihenfolge haben die Makespan $p = V^*$. Die Reihenfolge allein ändert die Laufzeit also um den Faktor $2 - 1/p$.

Beweis. Die lange Aufgabe startet bei 0 auf einem Worker und endet bei p. Die $p-1$ übrigen Worker führen je p Einheitsaufgaben in $[0, p)$ aus; das sind $p(p-1)$ Aufgaben, also alle. Die Makespan ist $p \leq V^*$, also $V = V^* = p$. Für die aufsteigende Reihenfolge gilt Satz 6. $\square$

In Gen-DB ist die Sortierung nicht exakt nach Kosten, sondern nach der Knotenzahl, die den Aufwand nur näherungsweise vorhersagt. Der nächste Satz zeigt, dass eine solche Näherung ausreicht.

Satz 14 (Absteigende Sortierung nach einem Näherungsschlüssel)**.** Seien $\hat{c}_1 \geq \cdots \geq \hat{c}_k > 0$ bekannte Schlüsselwerte (etwa eine monotone Funktion der Knotenzahl) mit $\hat{c}_j \leq \theta_j \leq \gamma \hat{c}_j$ für ein $\gamma \geq 1$, und die Aufgaben stehen in der Reihenfolge der Schlüssel. Dann gilt für jeden nicht-leerlaufenden FIFO-Ablauf ohne Zufluss

$$V \leq \left(1 + \left(1 - \frac{1}{p}\right)\frac{\gamma}{2}\right) V^*.$$

Für $\gamma = 1$ ist das die Schranke aus Satz 13.

Beweis. Wie im Beweis von Satz 13 starten die Aufgaben $1, \ldots, \min\{p, k\}$ bei 0. Für $k \leq p$ oder $j^* \leq p$ ist $V = \theta_{j^*} \leq V^*$. Sei $j^* \geq p+1$. Dann ist $\theta_{j^*} \leq \gamma \hat{c}_{j^*} \leq \gamma \hat{c}_{p+1}$. Von den Aufgaben $1, \ldots, p+1$ liegen in jedem Ablauf zwei auf demselben Worker; für $i \leq p+1$ ist $\theta_i \geq \hat{c}_i \geq \hat{c}_{p+1}$, also $V^* \geq 2\hat{c}_{p+1}$ und $\theta_{j^*} \leq \gamma V^*/2$. Mit Satz 5 und $W/p \leq V^*$ folgt $V \leq V^* + (1 - \frac{1}{p})\gamma V^*/2$. □

Bemerkung 12 (Folgerung und Vorbehalt)**.** Eine Sortierung der Kandidaten absteigend nach Knotenzahl wäre im Hauptprozess in $O(C \log C)$ möglich und damit gegenüber den Vergleichen vernachlässigbar. Sie wird in Gen-DB *nicht* angewendet, und ihr Nutzen wurde nicht gemessen. Eine Größenordnung für den möglichen Gewinn liefert die Modellanpassung aus Tabelle 9.7: Der Schwanzterm $c_2 b$ beträgt bei Blockgröße 256 in Python 1,33 Sekunden von 30,86 (4,3 % der Vergleichsphase) und in C++ 0,36 von 5,40 Sekunden (6,6 %). Mehr als diese Anteile kann eine bessere Reihenfolge im Modell nicht einsparen; gemessen ist es nicht. Die Aussage wird überprüfbar, indem die Chunk-Reihe mit absteigender Reihenfolge wiederholt wird (Abschnitt 9.10.10).

Satz 15 (Statische gegen dynamische Aufteilung)**.** Teilt man die $C = pm$ Kandidaten in p zusammenhängende Teilfolgen gleicher Länge m und gibt jedem Worker genau eine, so gilt für die Makespan $V_{\text{stat}} \leq W \leq p\,V^*$, und diese Schranke ist scharf: Für jedes $\epsilon > 0$ gibt es eine Instanz mit $V_{\text{stat}}/V^* \geq p/(1 + (p-1)\epsilon)$. Der dynamische Pool mit Blockgröße 1 erfüllt dagegen $V \leq W/p + (1 - \frac{1}{p})\theta_{\max}$ nach Satz 5.

Beweis. Die statische Aufteilung hat p Aufgaben mit Dauern $\theta'_1, \ldots, \theta'_p$, jeweils auf einem eigenen Worker. $V_{\text{stat}} = \max_l \theta'_l \leq \sum_l \theta'_l = W$, und $W \leq pV^*$ nach Satz 5. Instanz für die Schärfe: Sei $p \mid m$, die ersten m Kandidaten haben den Aufwand h und alle übrigen den Aufwand ϵh (mit $\epsilon > 0$ klein). Die statische Aufteilung gibt dem ersten Worker alle schweren Kandidaten, also $V_{\text{stat}} \geq mh$. Verteilt man dagegen die schweren und die leichten Kandidaten gleichmäßig auf alle Worker, so erhält jeder m/p schwere und $(p-1)m/p$ leichte, die Zeit $\frac{m}{p}h + \frac{(p-1)m}{p}\epsilon h = \frac{mh}{p}(1 + (p-1)\epsilon)$. Also $V^* \leq \frac{mh}{p}(1 + (p-1)\epsilon)$ und $V_{\text{stat}}/V^* \geq p/(1 + (p-1)\epsilon)$. □

Der Satz begründet, warum der Pool die Kandidaten in Blöcken dynamisch verteilt und nicht in p gleich große Stücke teilt: Die statische Teilung kann um den Faktor p schlechter sein, die dynamische um höchstens den Faktor $2 - 1/p$ (Korollar 4).

9.10.8 Grenzen der Parallelisierung

Grenze durch die Hardware.

Satz 16 (Hardwaregrenze)**.** Stehen nur P Prozessoren zur Verfügung und laufen $p > P$ Worker, so gilt für jeden Ablauf $V \geq W/P$, unabhängig von p. Der Speedup der Vergleichsphase gegenüber einem Worker ist daher höchstens P.

Beweis. Zu jedem Zeitpunkt führen höchstens P Prozessoren Rechenarbeit aus. Die Gesamtarbeit W muss in $[0, V]$ verrichtet werden, also $PV \geq W$. Mit $V(1) = W$ folgt die zweite Aussage. □

Das ist die formale Fassung der Hypothese H2 aus Abschnitt 7.1: Auf derselben Vierkernmaschine war der dritte Worker in der ersten Messreihe nicht schneller als der zweite (Speedup 1,77 gegen 1,90); in der späteren Multi-Omics-Reihe brachten der dritte und vierte Worker für Python noch Gewinn ($S = 1{,}91$ und 2,10). Die unterschiedlichen Ergebnisse widersprechen sich nicht zwingend, da Suchlast und Backend abweichen. Da alle getesteten Worker-Zahlen höchstens vier betrugen, wurde keine Überbelegung der logischen Prozessoren gemessen. Ein Leistungsrückgang durch Kontextwechsel ist eine mögliche, aber mit diesen Reihen nicht isolierte Erklärung.

Grenze durch die sequentielle Optimalität.

Die folgende Aussage klärt, was die Parallelisierung *nicht* kann. Dazu wird ein einfaches paralleles Rechenmodell gebraucht.

Definition 17 (Parallelmaschine)**.** Eine *p-Maschine* besteht aus p Registermaschinen (RAM) mit je eigenem Speicher, die im gleichen Takt arbeiten; in jedem Takt führt jede Maschine einen Schritt aus. Zur Kommunikation besitzt jede Maschine einen Eingangspuffer, in den andere Maschinen in einem Schritt je eine Nachricht schreiben und aus dem sie in einem Schritt je eine Nachricht lesen können. $T_p(n)$ ist die Zahl der Takte bis zur Ausgabe. Eine RAM mit Registerindirektion ist die 1-Maschine.

Lemma 5 (Simulationssatz)**.** Jede p-Maschine, die ein Problem in $T_p(n)$ Takten löst, kann von einer einzelnen RAM in höchstens $c\,p\,T_p(n)$ Schritten simuliert werden, mit einer Konstanten c, die nur vom Befehlssatz abhängt. Folglich gilt für die sequentielle Komplexität $T_1^*(n)$ des Problems $T_1^*(n) \leq c\,p\,T_p(n)$.

Beweis. Die simulierende RAM legt den Speicher der Maschine l (Register, Programmzähler, Eingangspuffer) in einem eigenen Adressbereich ab $l \cdot M$ ab, wobei M eine obere Schranke der benutzten Adressen ist (wächst der Speicher, wird der Abstand entsprechend gewählt; die Adressrechnung ist eine Addition). Ein Takt der p-Maschine wird in p Teilschritten simuliert: Für $l = 1, \ldots, p$ führt die simulierende RAM den Schritt der Maschine l aus, indem sie Lese- und Schreibzugriffe um den Offset $l \cdot M$ verschiebt. Ein Nachrichtenversand ist ein Schreibzugriff in den Puffer des Empfängers. Damit der Empfänger die Nachricht erst im folgenden Takt sieht, schreibt die Simulation in einen zweiten Pufferbereich, der am Taktende mit dem ersten getauscht wird (Zeigertausch, Aufwand $O(p)$ je Takt). Jeder Teilschritt kostet höchstens eine konstante Zahl c' von Schritten der simulierenden RAM; ein Takt kostet also höchstens $c'p + O(p) \leq cp$. Über $T_p(n)$ Takte folgt die Behauptung. □

Korollar 9 (Speedup gegenüber dem besten sequentiellen Verfahren)**.** Für jedes parallele Verfahren auf einer p-Maschine gilt $T_1^*(n)/T_p(n) \le c\,p$. Ist $L(n)$ eine untere Schranke für die sequentielle Komplexität eines Problems, so gilt $T_p(n) \ge L(n)/(c\,p)$.

Beweis. Beides folgt unmittelbar aus Lemma 5. □

Korollar 10 (Bedingte Grenze für den Einzelvergleich)**.** Gilt die in Abschnitt 2 vorausgesetzte untere Schranke $\Omega(n^{3-\varepsilon})$ für alle $\varepsilon > 0$ für die sequentielle Berechnung der Relation $A \preceq B$ (Verweis auf [1]; sie wird in dieser Arbeit nicht neu bewiesen), so benötigt jede Berechnung auf einer p-Maschine mit $p = O(1)$ Takten $T_p(n) = \Omega(n^{3-\varepsilon})$ für jedes $\varepsilon > 0$. Eine feste Zahl von Workern verändert den Exponenten also nicht; sie kann den Vergleich nur um einen konstanten Faktor von höchstens cp beschleunigen.

Beweis. Nach Korollar 9 ist $T_p(n) \ge L(n)/(cp)$ mit $L(n) = \Omega(n^{3-\varepsilon})$; für konstantes p bleibt das $\Omega(n^{3-\varepsilon})$. □

Bemerkung 13 (Umfang der Aussage)**.** Korollar 10 betrifft *einen* Vergleich. Für die C Vergleiche einer Suche folgt aus der Schranke für einen Vergleich nicht automatisch eine Schranke $\Omega(C\,n^{3-\varepsilon})$ für die gesamte Suche; eine solche Aussage über die Summe unabhängiger Instanzen braucht ein eigenes Argument, das hier nicht geführt wird. Die Arbeit der parallelen Suche ist dagegen bekannt: Sie beträgt $W = k\alpha + U$ und übersteigt die sequentielle Arbeit U nur um die Blockverwaltung $k\alpha$. Die Parallelisierung ist in diesem Sinne *arbeitseffizient*: Das Verhältnis $W/U = 1 + k\alpha/U$ geht für $b \to \infty$ gegen 1 und beträgt bei den angepassten Werten und Blockgröße 256 nur etwa $1 + \alpha/(256\,\bar{u}) \approx 1{,}002$ (Python) bzw. 1,01 (C++).

Die letzte Zahl ist eine Größenordnung aus den angepassten Werten $\alpha \approx 0{,}5$ Millisekunden und $\bar{u}_{\text{eff}} = 0{,}87$ bzw. 0,23 Millisekunden. Korollar 10 sagt zugleich, was die Parallelisierung nicht leisten kann: Sie kann den Algorithmus nicht unter seine sequentielle Optimalität drücken. Wer die Suche weiter beschleunigen will, kann nur an zwei Stellen ansetzen: am Kern (Konstante, Abschnitt 9.10.6) und am seriellen Anteil (Abschnitt 9.10.5).

Grenze durch den Global Interpreter Lock.

Die Entscheidung für Prozesse statt Threads wurde in Abschnitt 7 damit begründet, dass der Global Interpreter Lock Threads serialisiere. Das lässt sich im folgenden Modell beweisen.

Satz 17 (Grenze für Threads mit globaler Sperre)**.** Es laufen p Threads im selben Prozess. Jede Aufgabe j hat Rechenzeit $\theta_j = \theta_j^{\text{gil}} + \theta_j^{\text{frei}}$, wobei der Thread während θ_j^{gil} die globale Sperre hält und in dieser Zeit kein anderer Thread die Sperre halten kann. Mit $G = \sum_j \theta_j^{\text{gil}}$ und $\lambda = 1 - G/W$ gilt $V \ge G$ und damit für den Speedup gegenüber einer sequentiellen Ausführung der Dauer W

$$\frac{W}{V} \le \frac{1}{1-\lambda}.$$

Für einen Kern, der die Sperre nie freigibt ($\lambda = 0$), ist der Speedup höchstens 1.

Beweis. Die Zeitintervalle, in denen eine Sperre gehalten wird, sind nach der Modellvoraussetzung paarweise disjunkt und liegen in $[0, V]$. Ihre Gesamtlänge ist G, also $V \ge G$. Dann ist $W/V \le W/G = 1/(1-\lambda)$. □

Für den reinen Python-Kern ist $\lambda \approx 0$; Prozesse sind dort die einzige Möglichkeit, mehrere Kerne zu nutzen. Beim C++-Backend ruft Gen-DB die Bibliothek über `ctypes.CDLL` auf, und `ctypes` gibt die Sperre während eines solchen Aufrufs frei. Dort wäre λ der Anteil der Zeit im nativen Aufruf (die Vorbereitung der Matrizen in Python hält die Sperre weiter). Threads könnten die Übertragung der Kandidaten zwischen den Prozessen ersparen, also α und β_i senken. Nach der Obergrenze aus Abschnitt 9.10.6 wäre der Gewinn bei vier Workern aber höchstens etwa 11 % der Gesamtzeit; ob ein Wechsel lohnt, ist ungeprüft.

Grenze durch die Blockzahl.

Aus Satz 5 folgt $V \geq \theta_{\max} \geq W/k$ und damit, dass der Speedup der Vergleichsphase stets höchstens $k = \lceil C/b \rceil$ beträgt, gleich welche Worker-Zahl eingestellt ist. Kleine Anfragen mit wenigen Kandidaten und großer Blockgröße können deshalb nicht alle Worker nutzen (Bemerkung 9).

9.10.9 Parallelität innerhalb eines Vergleichs: die Schichten

Bisher war jeder Vergleich ein unteilbarer Auftrag. Die Relation aus Abschnitt 9.1 hat jedoch eine innere Struktur, die sich parallelisieren lässt. Für einen Stapel A mit L Schichten und n_A Knoten seien $a^\ell = (a_1^\ell, \ldots, a_{n_A}^\ell)$ die Zeilenkomponenten der Schicht ℓ, entsprechend $b^\ell = (b_1^\ell, \ldots, b_{n_B}^\ell)$ für B, und $b_{n_B+1}^\ell := b_1^\ell$ (zyklisch). Für $1 \leq i < n_A$ und $1 \leq k \leq n_B$ sei

$$\mu_\ell(i,k) \iff \left(a_i^\ell, a_{i+1}^\ell\right) = \left(b_k^\ell, b_{k+1}^\ell\right), \qquad P_\ell = \{(i,k) : \mu_\ell(i,k)\}.$$

Satz 18 (Schichtzerlegung)**.** Es sei $n_B \geq n_A \geq 2$ (sonst enthält die Implementierung A nicht in B). Dann gilt:

(a) Im Modus `independent` ist A in B enthalten genau dann, wenn $P_\ell \neq \emptyset$ für alle ℓ.

(b) Im Modus `coherent` ist A in B enthalten genau dann, wenn $\bigcap_{\ell=1}^{L} P_\ell \neq \emptyset$.

(c) Aus `coherent` folgt `independent`; die Umkehrung gilt für $L \geq 2$ im Allgemeinen nicht.

Beweis. (a) Die Implementierung prüft je Schicht, ob ein benachbartes Paar der Folge a^ℓ als zyklisch benachbartes Paar in b^ℓ vorkommt, und verlangt das für alle Schichten (`all`). Das ist $\forall \ell\, \exists (i,k)\, \mu_\ell(i,k)$. (b) Im kohärenten Modus werden die Folgenelemente zu Tupeln $(a_i^1, \ldots, a_i^L)$ zusammengefasst; zwei Tupel sind gleich genau dann, wenn alle Komponenten gleich sind. Ein Paar benachbarter Tupel tritt also an den Positionen (i,k) auf genau dann, wenn $\mu_\ell(i,k)$ für alle ℓ gilt. Das ist $\exists (i,k)\, \forall \ell\, \mu_\ell(i,k)$, also $\bigcap_\ell P_\ell \neq \emptyset$. (c) $\bigcap_\ell P_\ell \neq \emptyset$ impliziert $P_\ell \neq \emptyset$ für jedes ℓ. Für die Umkehrung sei $L = 2$, $n_A = 2$, $n_B = 3$, $a^1 = (1,2)$, $a^2 = (3,4)$, $b^1 = (1,2,0)$, $b^2 = (0,3,4)$. Dann ist $P_1 = \{(1,1)\}$ und $P_2 = \{(1,2)\}$; beide sind nicht leer, ihr Durchschnitt ist leer. □

Der Beweis von (c) zeigt, dass die beiden Modi verschiedene Relationen sind; das entspricht der Unterscheidung in Abschnitt 9.1. Für die Parallelisierung folgt aus dem Satz: Die Mengen P_ℓ (bzw. ihre Nichtleerheit) sind für verschiedene Schichten voneinander unabhängig berechenbar; danach genügt eine Konjunktion über L Werte (unabhängig) bzw. ein Durchschnitt (kohärent). Mit w_ℓ als Aufwand der Schicht ℓ besteht ein Vergleich aus L unabhängigen Aufgaben, deren Gesamtarbeit $\sum_\ell w_\ell$ und deren längste Aufgabe $\max_\ell w_\ell$ ist, plus einer Zusammenführung mit Aufwand $O(L)$ (unabhängig) bzw. $O(L\, n_A)$ (kohärent).

Nach Satz 5 gilt für die Vergleichsdauer mit p Workern $V \le \sum_\ell w_\ell/p + (1-\frac{1}{p})\max_\ell w_\ell$ und nach Korollar 4 ein Speedup von höchstens $\min\{p, L\}$ innerhalb eines Vergleichs, also höchstens 8 bei den im Mikrobenchmark untersuchten Schichtzahlen. Das ist weit kleiner als die Parallelität C/b zwischen den Kandidaten und deshalb nur für Anfragen mit sehr wenigen Kandidaten von Interesse. Außerdem berechnet die Python-Implementierung die Zeilenkomponenten aller Schichten, bevor sie die Paare prüft; der Kurzschluss von `all` spart nur die Paarprüfung. Spekulativ parallel berechnete Schichten verursachen dort kaum zusätzliche Arbeit. Umgesetzt ist die Schichtparallelität in Gen-DB nicht.

9.10.10 Fazit

Was bewiesen, angenommen und gemessen ist.

Tabelle 9.9 ordnet die Aussagen dieses Abschnitts ein. „Bewiesen“ heißt: im Modell bewiesen, unter den genannten Annahmen A1 (Reinheit, Definition 12) und A2 (Listenplanung des Pools, Bemerkung 7). „Gemessen“ heißt: aus den Daten der Abschnitte 7 und 9.6 bis 9.7 berechnet.

Tabelle 9.9: Aussagen der Parallelisierungstheorie und ihr Status.

Aussage	Beleg	Status
Trefferliste unabhängig von p, b und Ablauf	Satz 4	bewiesen unter A1; A1 durch Tests gestützt
Dynamische Aufteilung: $V \le (2-\frac{1}{p})V^*$	Satz 5, Korollar 4	bewiesen unter A2
Scharfe Schranke; Reihenfolge relevant	Satz 6, Korollar 8	bewiesen
Optimale Blockgröße $b^* = \sqrt{C\alpha/((p-1)\bar{u})}$	Satz 8	bewiesen für die Schranke; Verlauf gemessen (Tab. 9.7)
Amdahl-Grenze $1/f = 1 + c/a$; unabhängig von C	Satz 9, Korollar 7	bewiesen; $f = 0{,}197$ bzw. $0{,}589$ gemessen
Schnellerer Kern senkt die Effizienz	Satz 12	bewiesen; Werte bis etwa 10 % bestätigt
Sortierung nach Knotenzahl verkürzt den Schwanz	Sätze 13, 14	Schranke bewiesen; *Nutzen nicht gemessen*
Bei konstantem p kein Exponentgewinn	Lemma 5, Korollar 10	bewiesen; bedingt durch die in [1] bewiesene Schranke
Python-Threads: globale Sperre verhindert Gewinn	Satz 17	bewiesen im Sperrenmodell
Vergleich in Schichten zerlegbar	Satz 18	bewiesen; nicht umgesetzt

Folgerungen für Gen-DB.

Aus den Sätzen und den Messungen ergeben sich folgende Empfehlungen. Sie sind nach dem Grad der Absicherung geordnet.

(1) *Worker-Zahl nicht über die Zahl der Prozessoren hinaus.* Satz 16 und die Messungen aus Abschnitt 7 und 9.6. Die Voreinstellung von `SUBGRAPH_MAX_WORKERS` sollte an

die Hardware gebunden sein.

(2) *Blockgröße zwischen* 32 *und* 128 *statt* 256 *prüfen.* In beiden Backends lagen die besten gemessenen Werte in diesem Bereich; das Modell (Satz 8) sagt ein flaches Minimum voraus. Der gemessene Vorteil gegenüber 256 beträgt etwa 9,7 % (Python) bzw. 4,1 % der Gesamtzeit (C++). Der Wert hängt von der Hardware ab und ist durch eine Messung auf dem Zielsystem zu bestätigen.

(3) *Beim C++-Backend zuerst die Ladephase verbessern.* Der serielle Anteil beträgt 59 % der Gesamtzeit, die Grenze des Speedups 1,70 (Satz 9, Korollar 7); jede Verbesserung am Parallelüberschuss ist höchstens etwa 11 % wert.

(4) *Absteigende Sortierung der Kandidaten nach Knotenzahl erproben.* Die Schranke verbessert sich im Modell von $2 - 1/p$ auf $\frac{3}{2} - \frac{1}{2p}$ bzw. auf den Wert aus Satz 14; der Nutzen ist unbekannt und im Modell auf wenige Prozent der Vergleichsphase begrenzt.

(5) *Die Datenbank allein zu vergrößern hilft nicht.* Der serielle Anteil wächst mit (Korollar 7); mehr Kandidaten verbessern die Parallelisierbarkeit nicht.

Messprogramm zur Überprüfung.

Folgende Messungen würden die Aussagen testen und können die Modelle widerlegen:

(M1) Die Chunk-Reihe mit mindestens fünf Wiederholungen und absteigender Reihenfolge der Kandidaten wiederholen. Sagt Satz 14 Recht, so sinkt die Vergleichsphase bei Blockgröße 256 um höchstens den Schwanzterm (4 bis 7 %); eine größere Verbesserung widerspräche dem Modell, keine Verbesserung wäre mit ihm vereinbar.

(M2) Die Zeit des Zubringers (Serialisierung im Hauptprozess) von der Rechenzeit der Worker trennen (Korollar 5). Das Modell sagt voraus, dass die Vergleichsphase bei C++ dann von der Zubringerzeit abhängt, wenn sie größer als W/p ist.

(M3) Die Worker-Reihe zunächst mit mehr Wiederholungen auf der dokumentierten Vierkernmaschine und anschließend auf einem Rechner mit mindestens acht Kernen wiederholen. Im Amdahl-Modell darf $S(p)$ für Python bei $f = 0{,}197$ die Grenze 5,08 nicht erreichen, für C++ nicht 1,70; ein höherer Wert widerlegte die Annahme, dass T_{ser} von p unabhängig ist.

(M4) Die Ladeabfrage getrennt vermessen, auch mit mehreren Verbindungen. Der serielle Anteil f ist die entscheidende Größe, und er ist bisher nur als Summe von Laden und Aufbau bekannt.

Zusammenfassende These.

Die Ergebnisse dieses Abschnitts stützen die folgende Aussage, die in der ursprünglichen Fassung dieses Teilabschnitts als Beobachtung formuliert war. Die Kandidatenvergleiche sind voneinander unabhängige Aufgaben auf einer einfachen Datenstruktur (einer Folge von Stapeln); eine solche Aufgabe ist *eingebettet parallel*, und die parallele Ausführung liefert beweisbar dasselbe Ergebnis wie die sequentielle (Satz 4). Ist der zugrunde liegende Algorithmus sequentiell bis auf konstante Faktoren optimal (Abschnitt 2 und Abschnitt 10.3),

so kann die Parallelisierung den Aufwand höchstens um einen Faktor von der Größenordnung der Worker-Zahl senken (Korollar 9), und mit einer begrenzten Blockzahl und einem seriellen Anteil sinkt dieser Faktor weiter (Sätze 5 und 9). Je besser der Kern, desto kleiner wird der Anteil, den die Parallelisierung noch beeinflussen kann (Satz 12); das beobachtete schlechtere Skalieren des C++-Backends ist keine Eigenschaft der Sprache, sondern die Folge des gestiegenen seriellen Anteils. Das gilt nicht nur für den Graphen: Es gilt für jede Struktur, auf der unabhängige Vergleiche mit einem optimalen Algorithmus gerechnet werden, etwa für den Vektor oder die Stapel der universellen Kodierung. Kurz: Parallelisierung ist dort wirksam, wo grundlegende Strukturen unabhängige Aufgaben liefern; ihr Gewinn ist durch den seriellen Anteil und die Optimalität des Algorithmus begrenzt; und sie ersetzt weder einen effizienten Kern noch einen selektiven Vorfilter.

Zur Abgrenzung von der Formulierung in früheren Fassungen. Die Aussagen, dass der Subgraph Algorithmus „optimal“ sei und dass die Parallelisierung „auf dem Grund der maximalen Effizienz“ stehe, sind hier präzisiert: Die Optimalität ist die in [1] bedingt bewiesene sequentielle Schranke für den Einzelvergleich, und die gemessene parallele Effizienz ist nicht maximal, sondern liegt bei vier Workern zwischen 32 % (C++) und 53 % (Python) der Gesamtzeit und bei 54 % bzw. 72 % in der Vergleichsphase (Tabelle 9.8). Auch die Beobachtung, dass drei Worker langsamer als zwei seien, trifft nicht allgemein zu: Im früheren Experiment war der Dreierlauf auf derselben Vierkernmaschine langsamer, während in der späteren Reihe der dritte Worker beim Python-Backend noch Gewinn brachte ($S = 1{,}91$) und beim C++-Backend kaum ($S = 1{,}20$); dort begrenzt der serielle Anteil den Zugewinn.

Kapitel 10

Universelle Kodierung biologischer Strukturen

Bisher wurde für jede neue Art biologischer Daten ein eigener Vergleichspfad formuliert und in den nativen Kern implementiert: zuerst für das Einzelnetzwerk (Abschnitt 4), dann für das mehrschichtige Netzwerk (Abschnitt 9). Dieses Kapitel zeigt, dass dies nicht notwendig ist. Die Suche wird in zwei Teile getrennt:

(i) einen *strukturunabhängigen Kern*, der ausschließlich Stapel von Mengenfolgen kennt (Abschnitt 10.1), und

(ii) eine *Kodierung* $\Phi_{\Sigma,\kappa}$, die jede biologische Struktur eines beliebigen Schemas Σ in einen solchen Stapel überführt und vollständig in Python liegt (Abschnitt 10.2).

Für eine neue biologische Struktur ist dann nur noch ein neues Schema und damit eine neue Kodierung zu schreiben; der Kern, seine Beweise und sein Index bleiben unverändert.

Bewiesen werden folgende Aussagen: Jede endliche attributierte relationale Struktur besitzt eine verlustfreie Kodierung (Satz 19), die in fast linearer Zeit berechenbar ist (Satz 20). Der Vergleich zweier Stapel ist in erwarteter Linearzeit entscheidbar (Satz 21) und bis auf einen konstanten Faktor optimal (Satz 22). Mit einem invertierten Index liefert die Suche das exakte Ergebnis ohne Einzelvergleiche, und ihr Aufwand hängt von der Trefferlast, nicht von der Datenbankgröße N ab (Satz 23 und 24). Nicht behauptet wird, dass die Relation $\sqsubseteq$ mit der klassischen Teilgraph-Isomorphie übereinstimmt; dies ist nach Bemerkung 21 ohne P = NP nicht möglich. Ebenso wenig wird behauptet, dass jede Kodierung eine biologisch valide Suchsemantik liefert (Bemerkung 22).

10.1 Der strukturunabhängige Kern

Definition 18 (Schicht und Stapel)**.** Eine *Schicht der Länge* m ist eine Folge $s = (s_0, \dots, s_{m-1})$ endlicher Mengen $s_j \subset \mathbb{N}$, der *Komponenten*. Ein *Stapel* $M = (s^{(1)}, \dots, s^{(L)})$ besteht aus $L \geq 1$ Schichten derselben Länge $|M| := m$. Die *Belegung* ist $\nu(M) := \sum_{\ell=1}^{L} \sum_{j=0}^{m-1} |s_j^{(\ell)}|$. Die zu M gehörende *Tupelfolge* ist $\tau(M)_j := (s_j^{(1)}, \dots, s_j^{(L)})$.

Bemerkung 14 (Matrixstapel als Spezialfall)**.** Zu einem Stapel quadratischer $\{0,1\}$-Matrizen $(A_1, \dots, A_L)$ der Größe n gehört die Komponente $s_j^{(\ell)} = \{\, i : (A_\ell)_{ij} = 1 \,\}$, also die In-Nachbarschaft des Knotens j in Schicht ℓ. Sie entspricht bijektiv dem Bitmuster $\sum_i (A_\ell)_{ij} 2^i$ der Zeilenkomponente aus Abschnitt 9.1. Die folgende Relation ist die dort

beschriebene Relation (Bigramm-Charakterisierung der Implementierung), nur ohne die Einschränkung $n \le 63$, die allein aus der 64-Bit-Darstellung der nativen Bibliothek stammt.

Definition 19 (Paarmengen)**.** Für eine Folge $s = (s_0, \dots, s_{m-1})$ über einer beliebigen Menge sei

$$P_{\mathrm{lin}}(s) = \{(s_j, s_{j+1}) : 0 \le j \le m-2\}, \qquad P_{\mathrm{cyc}}(s) = \{(s_j, s_{(j+1) \bmod m}) : 0 \le j \le m-1\}$$

für $m \ge 2$ sowie $P_{\mathrm{lin}}(s) = P_{\mathrm{cyc}}(s) = \emptyset$ für $m \le 1$.

Definition 20 (Enthaltensein)**.** Seien M, M' Stapel mit derselben Schichtzahl L. Dann gilt

$$M \sqsubseteq_{\mathrm{ind}} M' :\Longleftrightarrow\ |M| \le |M'| \ \wedge\ \forall \ell \in \{1, \dots, L\} :\ P_{\mathrm{lin}}(s^{(\ell)}) \cap P_{\mathrm{cyc}}(s'^{(\ell)}) \neq \emptyset,$$
$$M \sqsubseteq_{\mathrm{coh}} M' :\Longleftrightarrow\ |M| \le |M'| \ \wedge\ P_{\mathrm{lin}}(\tau(M)) \cap P_{\mathrm{cyc}}(\tau(M')) \neq \emptyset.$$

Für eine Indexmenge $I \subseteq \{1, \dots, L\}$ bezeichne $\pi_I(M)$ die Projektion auf die Schichten aus I. Die Relationen $\sqsubseteq^I_{\mathrm{ind}}$ und $\sqsubseteq^I_{\mathrm{coh}}$ sind $\sqsubseteq_{\mathrm{ind}}$ bzw. $\sqsubseteq_{\mathrm{coh}}$ auf den projizierten Stapeln.

Lemma 6 (Elementare Eigenschaften)**.** Für Stapel M, M', M'' mit gleicher Schichtzahl gilt:

(a) $M \sqsubseteq_{\mathrm{coh}} M' \Rightarrow M \sqsubseteq_{\mathrm{ind}} M'$.

(b) $M \sqsubseteq_{\mathrm{ind}} M' \Rightarrow 2 \le |M| \le |M'|$.

(c) Ist $|M| \ge 2$, so gilt $M \sqsubseteq_{\mathrm{coh}} M$ und damit $M \sqsubseteq_{\mathrm{ind}} M$.

(d) Für $I' \subseteq I$ folgt aus $M \sqsubseteq^I_\mu M'$ stets $M \sqsubseteq^{I'}_\mu M'$ mit $\mu \in \{\mathrm{ind}, \mathrm{coh}\}$: Weniger Schichten liefern mehr Treffer.

(e) Die Relationen sind im Allgemeinen *nicht transitiv*.

Beweis. (a) Ist $(\tau(M)_j, \tau(M)_{j+1}) = (\tau(M')_k, \tau(M')_{(k+1) \bmod m'})$, so stimmen die Paare in jeder Koordinate ℓ überein; das Paar liegt also in $P_{\mathrm{lin}}(s^{(\ell)}) \cap P_{\mathrm{cyc}}(s'^{(\ell)})$ für jedes ℓ. (b) Für $|M| \le 1$ ist $P_{\mathrm{lin}} = \emptyset$, der Durchschnitt also leer. (c) Für $m \ge 2$ liegt (s_0, s_1) sowohl in $P_{\mathrm{lin}}(s)$ als auch in $P_{\mathrm{cyc}}(s)$; dasselbe gilt für $\tau(M)$. (d) Beim Weglassen von Schichten bleibt ein gemeinsames Paar gemeinsam: bei ind für jede verbleibende Schicht, bei coh nach Streichen der entfernten Koordinaten aus beiden Tupeln. (e) Gegenbeispiel mit $L = 1$: $M = (\{1\}, \{2\})$, $M' = (\{1\}, \{2\}, \{3\})$, $M'' = (\{2\}, \{3\}, \emptyset, \emptyset)$. Das Paar $(\{1\}, \{2\})$ liegt in $P_{\mathrm{lin}}(M) \cap P_{\mathrm{cyc}}(M')$, also $M \sqsubseteq M'$. Das Paar $(\{2\}, \{3\})$ liegt in $P_{\mathrm{lin}}(M') \cap P_{\mathrm{cyc}}(M'')$, also $M' \sqsubseteq M''$. Es ist aber $P_{\mathrm{lin}}(M) = \{(\{1\}, \{2\})\}$ und $P_{\mathrm{cyc}}(M'') = \{(\{2\}, \{3\}), (\{3\}, \emptyset), (\emptyset, \emptyset), (\emptyset, \{2\})\}$ ohne gemeinsames Element, also $M \not\sqsubseteq M''$. □

Lemma 7 (Kein Kantenzahl-Filter für $\sqsubseteq$)**.** Aus $M \sqsubseteq M'$ folgt $|M| \le |M'|$, aber im Allgemeinen *nicht* $\nu(M) \le \nu(M')$.

Beweis. Sei $L = 1$, $M = (\{0,1\}, \{0,1\}, \{0,1,2\})$ (eine 3×3-Matrix mit $\nu = 7$ Kanten) und $M' = (\{0,1\}, \{0,1\}, \emptyset, \emptyset)$ (eine 4×4-Matrix mit $\nu = 4$ Kanten). Das Paar $(\{0,1\}, \{0,1\})$ liegt in $P_{\mathrm{lin}}(M)$ und in $P_{\mathrm{cyc}}(M')$, also $M \sqsubseteq M'$, obwohl $\nu(M) > \nu(M')$. □

Bemerkung 15. Lemma 7 bedeutet: Der Kantenfilter $m_k \ge m_Q$ aus Abschnitt 4.2 ist für diese Relation *nicht* zulässig, weil er echte Treffer ausschließen würde. Zulässig ist allein die Schranke $|M_k| \ge |Q|$ (Knotenzahl bzw. Schichtlänge). Dies entspricht der Implementierung der Multi-Omics-Suche, die nur nach `node_count` vorfiltert. Der Index in Abschnitt 10.4 ersetzt den Pre-Filter ohnehin vollständig.

10.2 Biologische Strukturen und ihre Kodierung

Definition 21 (Schema)**.** Ein *Schema* $\Sigma = (\Lambda, T, (k_t)_{t\in T}, (W_t)_{t\in T})$ besteht aus einer endlichen Menge von Knotenmerkmalen Λ, einer endlichen, fest geordneten Menge von Relationstypen T, Stelligkeiten $k_t \geq 1$ und Gewichtsstufen $W_t \geq 1$ ($W_t = 1$: ungewichtet).

Definition 22 (Struktur)**.** Sei $\mathcal{U}$ die Menge aller Entitätsbezeichner (z. B. Gen-, Protein- oder Metabolitkennungen). Eine *Struktur über* Σ ist $S = (V, \lambda, (R_t)_{t\in T}, (w_t)_{t\in T})$ mit endlichem $V \subset \mathcal{U}$, $\lambda : V \to 2^\Lambda$, Relationen $R_t \subseteq V^{k_t}$ und Gewichten $w_t : R_t \to \{1, \ldots, W_t\}$. Die Klasse aller Strukturen über Σ heißt $\mathfrak{B}_\Sigma$; ihre Größe ist $|S| := |V| + \sum_t |R_t|$.

Diese Klasse umfasst gerichtete und (durch symmetrische Relationen) ungerichtete Netzwerke, gewichtete und attributierte Netzwerke, Reaktionsnetze mit mehrstelligen Reaktionen, Hypergraphen, Pfade und Sequenzen (als Nachfolgerrelation), Ontologie-Graphen sowie mehrschichtige Netzwerke (eine Relationsart je Schicht). Kontinuierliche Werte müssen zuvor quantisiert werden; die Verlustfreiheit gilt dann bezüglich der quantisierten Struktur.

Definition 23 (Koordinatensystem)**.** Ein *Koordinatensystem* ist eine injektive Abbildung $\kappa : \mathcal{U} \to \mathbb{N}$, etwa die Vergabereihenfolge in einem Entitätsregister der Datenbank. Neue Entitäten erhalten stets neue, größere Koordinaten; bestehende Koordinaten ändern sich nie.

Definition 24 (Kodierung $\Phi_{\Sigma,\kappa}$)**.** Sei $S \in \mathfrak{B}_\Sigma$. Die *Spalten* bestehen aus den Entitätsspalten V, aufsteigend nach κ, gefolgt von den Relationsknoten $E = \biguplus_{t:k_t\neq 2} R_t$, geordnet nach t (gemäß der festen Ordnung von T) und innerhalb eines Typs lexikographisch nach $(\kappa(v_1), \ldots, \kappa(v_{k_t}))$. Es ist $n := |V| + |E|$. Die Kodierung $\Phi_{\Sigma,\kappa}(S)$ ist der Stapel mit folgenden Schichten, jeweils der Länge n; s_j bezeichnet die Komponente der Spalte j:

- Ex (Existenz): für eine Entitätsspalte v ist $s_j = \{\kappa(v)\}$, sonst $\emptyset$.
- Lab_a für $a \in \Lambda$: für eine Entitätsspalte v ist $s_j = \{\kappa(v)\}$, falls $a \in \lambda(v)$, sonst $\emptyset$; Relationsspalten haben $\emptyset$.
- $\mathrm{Bin}_{t,r}$ für $k_t = 2$ und $1 \leq r \leq W_t$: für eine Entitätsspalte v ist $s_j = \{\kappa(u) : (u, v) \in R_t,\ w_t(u, v) \geq r\}$, sonst $\emptyset$.
- $\mathrm{Inz}_{t,p}$ für $k_t \neq 2$ und $1 \leq p \leq k_t$: für eine Relationsspalte $e = (v_1, \ldots, v_{k_t}) \in R_t$ ist $s_j = \{\kappa(v_p)\}$, sonst $\emptyset$.
- $\mathrm{Wgt}_{t,r}$ für $k_t \neq 2$ und $2 \leq r \leq W_t$: für eine Relationsspalte $e \in R_t$ mit $w_t(e) \geq r$ ist $s_j = \{\kappa(v_1)\}$, sonst $\emptyset$.

Die Schichtzahl $L_\Sigma = 1 + |\Lambda| + \sum_{k_t=2} W_t + \sum_{k_t\neq 2}(k_t + W_t - 1)$ hängt nur vom Schema ab.

Dass L_Σ nur vom Schema abhängt, ist wesentlich: Der Kern vergleicht nur Stapel gleicher Schichtzahl, und Query sowie gespeicherte Strukturen eines Schemas erfüllen diese Bedingung automatisch.

Beispiel 1. Sei $T = \{\mathrm{reg}\}$ mit $k_{\mathrm{reg}} = 2$, $W_{\mathrm{reg}} = 1$, $\Lambda = \emptyset$, und $V = \{a, b, c\}$ mit $\kappa(a) = 0, \kappa(b) = 1, \kappa(c) = 2$ sowie $R_{\mathrm{reg}} = \{(a, b), (a, c), (b, c)\}$. Dann ist $n = 3$ und $\Phi(S)$ hat die beiden Schichten

$$\mathrm{Ex} = (\{0\}, \{1\}, \{2\}), \qquad \mathrm{Bin}_{\mathrm{reg},1} = (\emptyset, \{0\}, \{0, 1\}).$$

Wird zusätzlich ein dreistelliger Typ rx (Substrat, Enzym, Produkt) mit $R_{\mathrm{rx}} = \{(a, b, c)\}$ ergänzt, so entsteht eine vierte Spalte $e = (a, b, c)$ mit $\mathrm{Inz}_{\mathrm{rx},1} = (\emptyset, \emptyset, \emptyset, \{0\})$, $\mathrm{Inz}_{\mathrm{rx},2} = (\emptyset, \emptyset, \emptyset, \{1\})$ und $\mathrm{Inz}_{\mathrm{rx},3} = (\emptyset, \emptyset, \emptyset, \{2\})$.

Satz 19 (Verlustfreiheit der Kodierung)**.** Für jedes Schema Σ und jedes Koordinatensystem κ ist $\Phi_{\Sigma,\kappa} : \mathfrak{B}_\Sigma \to \{\text{Stapel mit } L_\Sigma \text{ Schichten}\}$ injektiv. Die Umkehrabbildung ist auf dem Bild berechenbar.

Beweis. Sei $M = \Phi_{\Sigma,\kappa}(S)$. Wir rekonstruieren S aus M und κ. *Entitäten:* In Ex sind genau die Entitätsspalten nichtleer, jeweils mit der Einermenge $\{\kappa(v)\}$. Da Entitätsspalten vor allen Relationsspalten stehen, sind es die ersten $|V|$ Spalten, und $v = \kappa^{-1}(\cdot)$ ist wegen der Injektivität von κ eindeutig. *Merkmale:* $a \in \lambda(v)$ genau dann, wenn die Komponente von Lab_a in der Spalte von v nichtleer ist. *Zweistellige Relationen:* $(u, v) \in R_t$ genau dann, wenn $\kappa(u)$ in der Komponente von $\mathrm{Bin}_{t,1}$ in der Spalte von v liegt; das Gewicht ist $w_t(u, v) = \max\{r : \kappa(u) \in \mathrm{Bin}_{t,r} \text{ in Spalte } v\}$. *Übrige Relationen:* Jede Relationsspalte $j > |V|$ gehört zu genau einem Tupel $e \in R_t$ eines Typs t mit $k_t \neq 2$. Der Typ ist der einzige t, für den $\mathrm{Inz}_{t,1}$ in Spalte j nichtleer ist; die Stellen sind $\kappa(v_p) =$ das einzige Element von $\mathrm{Inz}_{t,p}$ in Spalte j; das Gewicht ist $w_t(e) = \max(\{1\} \cup \{r : \mathrm{Wgt}_{t,r} \text{ in Spalte } j \text{ nichtleer}\})$. Damit ist S eindeutig durch M bestimmt, d. h. $\Phi_{\Sigma,\kappa}$ ist injektiv, und die beschriebene Rekonstruktion ist die Umkehrabbildung. □

Satz 20 (Aufwand der Kodierung)**.** Bei festem Schema ist $\Phi_{\Sigma,\kappa}(S)$ in Zeit $O(|S| \log |S|)$ berechenbar, und es gilt $\nu(\Phi_{\Sigma,\kappa}(S)) = O(|S|)$ sowie $|\Phi_{\Sigma,\kappa}(S)| = |V| + |E| \leq |S|$.

Beweis. Jeder Entität werden höchstens $1 + |\Lambda|$ Einträge zugeordnet, jedem Tupel $e \in R_t$ höchstens $\max(W_t, k_t + W_t - 1)$ Einträge. Die Summe ist durch eine nur vom Schema abhängige Konstante mal $|S|$ beschränkt. Das Sortieren der Entitäten nach κ kostet $O(|V| \log |V|)$, das Sortieren der Relationsknoten nach $(t, \kappa\text{-Tupel})$ höchstens $O(k_{\max} |E| \log |E|)$. Die Komponenten werden in einem Durchlauf über die Relationstupel angelegt. □

Bemerkung 16 (Lokale und globale Koordinaten)**.** Wählt man statt eines globalen Registers κ den *lokalen* Rang $\kappa_S(v) \in \{0, \ldots, |V| - 1\}$ innerhalb der jeweiligen Struktur, so ergibt sich für ein reines Mehrschicht-Netzwerk ($\Lambda = \emptyset$, nur zweistellige Typen, $W_t = 1$, Existenzschicht nicht in der Suche) genau der Matrixstapel aus Bemerkung 14 und damit die bisherige Multi-Omics-Relation. Das Einzelnetzwerk ist der Fall $L = 1$. Alle folgenden Sätze gelten für beliebige Stapel und damit für beide Koordinatenwahlen. Global kodierte Komponenten sind Mengen identifizierbarer Entitäten; ihre Gleichheit bedeutet *gleiche Regulatoren* im Sinne derselben Entitäten, nicht bloß gleicher Positionen.

Bemerkung 17 (Schemaänderung)**.** Die Aufnahme eines neuen Relationstyps in ein bestehendes Schema ändert Spaltenanzahl und -reihenfolge und erfordert deshalb eine Neukodierung der vorhandenen Einträge in Zeit $O(|S| \log |S|)$ je Struktur (Satz 20). Alternativ legt man für den neuen Typ ein neues Schema mit eigenem Index an.

10.3 Optimaler Vergleich zweier Stapel

Wir betrachten das Modell, in dem Komponenten zunächst in ganze Zahlen *interniert* werden (ein Wörterbuch, das jeder Menge beim ersten Auftreten eine neue Zahl zuordnet), sodass Gleichheitstests in $O(1)$ möglich sind.

Satz 21 (Obere Schranke)**.** Für Stapel M, M' mit L Schichten sind $M \sqsubseteq_{\text{ind}} M'$ und $M \sqsubseteq_{\text{coh}} M'$ jeweils in erwarteter Zeit $O\Big(\nu(M)+\nu(M')+L(|M|+|M'|)\Big)$ entscheidbar. Mit Sortieren statt Hashing ist die Schranke deterministisch bis auf einen Faktor $O(\log(\nu(M)+\nu(M')+L(|M|+|M'|)))$.

Beweis. Das Internieren aller Komponenten kostet erwartet $O(\nu(M)+\nu(M'))$, da jede Menge einmal gehasht wird. Für die Relation $\sqsubseteq_{\text{ind}}$ wird je Schicht ℓ die Menge $P_{\text{cyc}}(s'^{(\ell)})$ als Hash-Menge von Zahlenpaaren aufgebaut ($|M'|$ Einfügungen) und anschließend jedes der $|M|-1$ Paare aus $P_{\text{lin}}(s^{(\ell)})$ nachgeschlagen. Das kostet erwartet $O(|M|+|M'|)$ je Schicht, insgesamt $O(L(|M|+|M'|))$. Für $\sqsubseteq_{\text{coh}}$ werden zunächst die Tupel $\tau(M)_j$ und $\tau(M')_k$ als Zahlentupel der Länge L interniert ($O(L(|M|+|M'|))$) und danach wie oben mit nur einer Folge verfahren. Die Bedingung $|M| \le |M'|$ ist ein $O(1)$-Test. □

Satz 22 (Untere Schranke)**.** Seien $2 \le m \le m'$. Jeder korrekte deterministische Algorithmus, der $\sqsubseteq_{\text{ind}}$ oder $\sqsubseteq_{\text{coh}}$ für Stapel der Längen m und m' über lesenden Zugriff auf einzelne Komponenten entscheidet, muss bei geeigneten Eingaben mindestens $\lceil m'/2 \rceil$ Komponenten von M' und mindestens $\lfloor m/2 \rfloor$ Komponenten von M lesen. Die Laufzeit ist daher $\Omega(m+m')$ für jede Schichtzahl $L \ge 1$.

Beweis. Sei zunächst $L = 1$. *Zugriffe auf M':* Sei $a = (\{0\},\{1\},\{0\},\{1\},\dots)$ der Länge m; dann ist $P_{\text{lin}}(a) \subseteq \{(\{0\},\{1\}),(\{1\},\{0\})\}$. Sei $b^0 = (\{2\},\dots,\{2\})$ der Länge m'; es ist $P_{\text{cyc}}(b^0) = \{(\{2\},\{2\})\}$, der Durchschnitt mit $P_{\text{lin}}(a)$ ist leer, die Antwort lautet also *nein*. Zu jedem $k \in \{0,\dots,m'-1\}$ sei y_k aus b^0 durch $b_k := \{0\}$ und $b_{(k+1) \bmod m'} := \{1\}$ entstanden (es ist $k \ne (k+1) \bmod m'$, da $m' \ge 2$). Dann liegt $(\{0\},\{1\})$ in $P_{\text{cyc}}(y_k) \cap P_{\text{lin}}(a)$ und die Antwort lautet *ja*. Läse ein Algorithmus auf der Eingabe (a, b^0) weder die Position k noch $(k+1) \bmod m'$, so verliefe er auf (a, y_k) identisch und gäbe dieselbe, dort falsche Antwort. Die Menge der gelesenen Positionen trifft daher jede Kante des Kreises $C_{m'}$, ist also eine Knotenüberdeckung und hat mindestens $\lceil m'/2 \rceil$ Elemente. *Zugriffe auf M:* Sei $b' = (\{0\},\{1\},\{2\},\{3\},\{2\},\{3\},\dots)$ der Länge m' (für $m' = 2$ sei $b' = (\{0\},\{1\})$); es ist $(\{0\},\{1\}) \in P_{\text{cyc}}(b')$ und $(\{2\},\{2\}) \notin P_{\text{cyc}}(b')$. Sei $a^0 = (\{2\},\dots,\{2\})$ der Länge m: Antwort *nein*. Zu $j \in \{0,\dots,m-2\}$ sei z_j aus a^0 durch $a_j := \{0\}$, $a_{j+1} := \{1\}$ entstanden: Antwort *ja*. Wie oben muss die gelesene Positionsmenge jede Kante des Pfades auf m Knoten überdecken, also mindestens $\lfloor m/2 \rfloor$ Elemente haben. Für $L > 1$ setze man in beiden Stapeln alle weiteren Schichten auf die konstante Folge $(\emptyset,\dots,\emptyset)$; dort ist $(\emptyset,\emptyset)$ stets ein gemeinsames Paar. Die Antworten beider Modi stimmen dann mit dem Fall $L = 1$ überein. □

Korollar 11 (Optimalität des Vergleichs)**.** Für festes L und Komponenten konstanter Größe ist der Vergleich zweier Stapel in $\Theta(|M|+|M'|)$ entscheidbar. Für wachsendes L bleibt zwischen $\Omega(|M|+|M'|)$ und $O(L(|M|+|M'|))$ eine Lücke, die hier nicht geschlossen wird.

10.4 Invertierter Index: Suche ohne Einzelvergleiche

Sei $\mathcal{D} = \{(k, M_k)\}_{k=1}^{N}$ eine Datenbank von Stapeln desselben Schemas mit L Schichten. Für jede Schicht ℓ sei ι_ℓ ein injektives Wörterbuch, das jeder in der Datenbank vorkommenden Komponente eine Zahl zuordnet.

Definition 25 (Paarindex)**.** Für ein Zahlenpaar p sei

$$\mathrm{Idx}_\ell^{\text{cyc}}(p) = \{k : p \in P_{\text{cyc}}(\iota_\ell(s_k^{(\ell)}))\}, \qquad \mathrm{Idx}_\ell^{\text{lin}}(p) = \{k : p \in P_{\text{lin}}(\iota_\ell(s_k^{(\ell)}))\}.$$

Jeder Eintrag speichert zusätzlich die Länge $|M_k|$.

Definition 26 (Anfrage und Suchergebnis)**.** Eine *Anfrage* ist ein Stapel Q der Länge m_Q mit einer nichtleeren Menge aktiver Schichten $I \subseteq \{1, \dots, L\}$. Das Suchergebnis ist

$$\mathrm{Match}_\mu(Q, I, \mathcal{D}) = \{k : \pi_I(Q) \sqsubseteq_\mu \pi_I(M_k)\}, \quad \mu \in \{\mathrm{ind}, \mathrm{coh}\}.$$

Nicht aktive Schichten (etwa die Existenzschicht) sind nicht eingeschränkt. Für Schichten mit nur leeren Komponenten kann „keine Kante“ nicht ausgedrückt werden; sie sind per Definition nicht aktiv.

Satz 23 (Korrektheit des Index)**.** Für ι_ℓ-bekannte Komponenten sei $R_\ell(Q) = \{k : |M_k| \geq m_Q \wedge \exists p \in P_{\mathrm{lin}}(\iota_\ell(s_Q^{(\ell)})) : \ k \in \mathrm{Idx}_\ell^{\mathrm{cyc}}(p)\}$; Paare mit unbekannter Komponente tragen nichts bei. Dann gilt

$$\mathrm{Match}_{\mathrm{ind}}(Q, I, \mathcal{D}) = \bigcap_{\ell \in I} R_\ell(Q),$$
$$\mathrm{Match}_{\mathrm{coh}}(Q, I, \mathcal{D}) = \{k \in \mathrm{Match}_{\mathrm{ind}}(Q, I, \mathcal{D}) : \pi_I(Q) \sqsubseteq_{\mathrm{coh}} \pi_I(M_k)\}.$$

Die Auswertung der ersten Formel benötigt keinen Einzelvergleich.

Beweis. Da ι_ℓ injektiv ist, ist Gleichheit von Komponenten gleichbedeutend mit Gleichheit der Zahlen; ein Paar mit einer in der Datenbank unbekannten Komponente liegt in keiner $P_{\mathrm{cyc}}(M_k)$. Nach Definition von $\sqsubseteq_{\mathrm{ind}}$ ist $k \in \mathrm{Match}_{\mathrm{ind}}$ genau dann, wenn $|M_k| \geq m_Q$ und für jedes $\ell \in I$ ein Paar $p \in P_{\mathrm{lin}}(s_Q^{(\ell)}) \cap P_{\mathrm{cyc}}(s_k^{(\ell)})$ existiert, d. h. $k \in R_\ell(Q)$ für alle $\ell \in I$. Für die zweite Formel gilt $\mathrm{Match}_{\mathrm{coh}} \subseteq \mathrm{Match}_{\mathrm{ind}}$ nach Lemma 6(a); die Verifikation auf der Obermenge liefert somit genau $\mathrm{Match}_{\mathrm{coh}}$. □

Satz 24 (Laufzeit der Indexsuche)**.** Sei $H = \sum_{\ell \in I} \sum_{p \in P_{\mathrm{lin}}(\iota_\ell(s_Q^{(\ell)}))} |\mathrm{Idx}_\ell^{\mathrm{cyc}}(p)|$ die *Trefferlast*. Die Berechnung von $\mathrm{Match}_{\mathrm{ind}}$ benötigt erwartet $O(\nu_I(Q) + |I|\, m_Q + H)$ Zeit, wobei $\nu_I(Q)$ die Belegung der aktiven Schichten von Q ist. Für $\mathrm{Match}_{\mathrm{coh}}$ kommen $O\big(|\mathrm{Match}_{\mathrm{ind}}| \cdot |I| (m_Q + m_{\max})\big)$ hinzu. Der Index belegt $O(\sum_k L\, |M_k|)$ Einträge und wird beim Einfügen oder Löschen einer Struktur M_k in Zeit $O(L\, |M_k| + \nu(M_k))$ angepasst.

Beweis. Das Internieren der Query-Komponenten ist ein Wörterbuchzugriff je Komponente, zusammen $O(\nu_I(Q))$. Die Menge P_{lin} je aktiver Schicht hat höchstens $m_Q - 1$ Paare. Jedes Paar p liest die Liste $\mathrm{Idx}_\ell^{\mathrm{cyc}}(p)$ einmal und sammelt die Elemente mit $|M_k| \geq m_Q$ in einer Hash-Menge, insgesamt $O(H)$. Der Schnitt über I kostet höchstens die Summe der Mengengrößen, also $O(H)$. Für coh wird je Kandidat Satz 21 auf den projizierten Stapeln angewendet. Der Speicherbedarf und die Pflegekosten folgen daraus, dass ein Stapel der Länge $|M_k|$ je Schicht höchstens $|M_k|$ zyklische und $|M_k| - 1$ lineare Paare besitzt. □

Korollar 12 (Unabhängigkeit von N)**.** Kommt jedes Paar in höchstens h gespeicherten Strukturen vor, so beträgt die Laufzeit der Indexsuche für $\mathrm{Match}_{\mathrm{ind}}$ erwartet $O(\nu_I(Q) + |I|\, m_Q\, h)$ und hängt nicht von N ab. Im ungünstigsten Fall, wenn alle N Strukturen dasselbe Paar enthalten, ist $H = \Theta(N\, m_Q |I|)$; dann ist die Ausgabegröße $|\mathrm{Match}| = \Theta(N)$ ohnehin eine untere Schranke.

Bemerkung 18 (Umgekehrte Richtung)**.** Die Treffer der Gegenrichtung $\pi_I(M_k) \sqsubseteq \pi_I(Q)$ (gespeicherte Struktur ist in der Query enthalten) ergeben sich symmetrisch mit $\mathrm{Idx}_\ell^{\mathrm{lin}}$: $k \in R'_\ell(Q)$ genau dann, wenn $|M_k| \leq m_Q$ und ein Paar $p \in P_{\mathrm{cyc}}(\iota_\ell(s_Q^{(\ell)}))$ mit $k \in \mathrm{Idx}_\ell^{\mathrm{lin}}(p)$ existiert. Satz 23 und 24 gelten analog; das Ergebnis liefert die Treffertypen `KEEP_A`, `KEEP_B` und `EQUAL_KEEP_*`.

Bemerkung 19 (Verhältnis zur bisherigen Pipeline)**.** Die bisherige Pipeline (Satz 2) kostet $O(\log N + C) + O(C\, n^3)$ mit einer Kandidatenmenge C, die von der Knoten- und Kantenzahl abhängt. Die Indexsuche prüft keine Kandidaten, sondern liest nur Treffer-Listen; in der Messung der Multi-Omics-Suche entfielen mit C++ 70,5 % der Gesamtzeit auf das Laden (Tabelle 9.4). Ein exakter Index lädt dagegen im Modus ind nur die tatsächlichen Treffer. Dies ist eine Folgerung aus Satz 23 und keine Messung; sie wird in Abschnitt 11 experimentell geprüft (Hypothesen H6 und H9, Phasenanalyse der Suche).

10.5 Kernunabhängigkeit und Reichweite

Korollar 13 (Kernunabhängigkeit)**.** Für jedes Schema Σ, jedes Koordinatensystem κ und jede Datenbank aus Strukturen $S_k \in \mathfrak{B}_\Sigma$ gelten die Sätze 21 bis 24 für die Stapel $\Phi_{\Sigma,\kappa}(S_k)$ unverändert. Eine neue Art biologischer Struktur erfordert daher nur die Angabe eines Schemas Σ, nicht die Änderung von Vergleich, Index oder deren Korrektheitsbeweisen.

Beweis. Die genannten Sätze wurden für beliebige Stapel mit gleicher Schichtzahl bewiesen. Nach Definition 24 sind alle Bilder von $\Phi_{\Sigma,\kappa}$ solche Stapel mit der Schichtzahl L_Σ; nach Satz 19 geht beim Kodieren keine Information verloren. □

Bemerkung 20 (Native Bibliothek)**.** Die C++-Bibliothek `csubgraph` bleibt als optionaler Beschleuniger für den Teil der Stapel erhalten, deren Koordinaten in 64 Bit passen, insbesondere bei lokalen Koordinaten (Bemerkung 16). Für die Korrektheit und die oben bewiesenen Laufzeiten wird sie nicht benötigt.

Bemerkung 21 (Grenze der Effizienzaussage)**.** Das Teilgraph-Isomorphieproblem ist NP-vollständig (Cook 1971; Garey und Johnson 1979, Problem GT48; es enthält Clique und Hamiltonkreis als Spezialfälle). Gäbe es eine in Polynomialzeit berechenbare Abbildung Ψ auf Stapel mit $G \hookrightarrow H \iff \Psi(G) \sqsubseteq \Psi(H)$ für alle Graphen G, H, so wäre dieses Problem nach Satz 21 in Polynomialzeit lösbar, also P = NP. Die Optimalität dieses Kapitels bezieht sich daher auf die *definierte* Relation $\sqsubseteq$ und nicht auf die klassische Isomorphie. Jede polynomiell berechenbare Relation über diesen Strukturen ist ein Näherungs- oder Filterkriterium.

Bemerkung 22 (Semantik)**.** $M \sqsubseteq M'$ besagt: Zwei in der Spaltenfolge benachbarte Entitäten von M besitzen in jeder aktiven Schicht dieselben Nachbarschaftsmengen wie zwei zyklisch benachbarte Spalten von M'. Ob dies eine biologisch sinnvolle Ähnlichkeit darstellt, hängt von Schema, Spaltenordnung und Koordinatensystem ab; die Sätze dieses Kapitels sichern Kodierung, Korrektheit und Aufwand, nicht die fachliche Validität. Sie ist mit fachlich annotierten Daten zu prüfen, wie bereits in Abschnitt 9 vermerkt. Zudem ist $\sqsubseteq$ nicht transitiv (Lemma 6(e)); die Treffertypen sind daher paarweise, nicht als Halbordnung zu verstehen.

10.6 Vorbereitung des zweiten Schritts: Umsetzung in Python

Die Umsetzung folgt unmittelbar aus den Sätzen und ist nicht Gegenstand dieses Kapitels. Vorgesehen sind:

– eine Schnittstelle je Strukturart mit `schema`, `encode` (Definition 24) und `decode` (Satz 19);

– ein Entitätsregister für κ und eine Tabelle für die Wörterbücher ι_ℓ;

– eine Indextabelle mit den Spalten Schicht, Komponentenpaar, Art (`cyc` oder `lin`), Netzwerk und Länge, mit einem B-Baum über Schicht und Paar;

– Tests, die direkt aus den Sätzen folgen: Rundlauf decode $\circ$ encode $=$ id (Satz 19); Gleichheit von Indexsuche und Einzelvergleich über alle Paare, analog zur differenziellen Prüfung der Multi-Omics-Suche; $\mathrm{Match}_{\mathrm{coh}} \subseteq \mathrm{Match}_{\mathrm{ind}}$ und Monotonie unter Projektion (Lemma 6); Nicht-Transitivität als Regressionsfall;

– ein Experiment, das Indexsuche, Einzelvergleiche in Python und die native Bibliothek bei gleichen Anfragen vergleicht; es ist in Abschnitt 11 beschrieben und wird durch `src/experiment_search_universal.py` ausgeführt.

Listing 10.1: Skizze der Schnittstelle (zweiter Schritt)

```
class StructureEncoder(Protocol):
    schema: Schema # (Lambda, T, k_t, W_t), versioniert
    def encode(self, s: Structure) -> LayerStack: ... # Phi
    def decode(self, m: LayerStack) -> Structure: ... # Umkehrung
```

10.7 Zusammenfassung

Der Kern kennt nur Stapel von Mengenfolgen und die Relation $\sqsubseteq$. Jede endliche attributierte relationale Struktur lässt sich verlustfrei und in fast linearer Zeit in einen solchen Stapel kodieren; Vergleich und Indexsuche sind dafür korrekt, der Vergleich ist bis auf einen konstanten Faktor optimal, und die theoretische Indexlaufzeit hängt von der Trefferlast ab. Der erste experimentelle Lauf bestätigt die differenzielle Korrektheit im geprüften Matrixschema und zeigt in der Speicherreihe kürzere Indexabfragen als Einzelvergleiche. Da der Lauf unvollständig ist, sind die Ende-zu-Ende- und Datenbank-Skalierungsaussagen noch nicht abschließend bestätigt; ebenso offen bleibt die fachliche Validierung der Suchsemantik. Die Ergebnisse und ihre Grenzen werden in Abschnitt 11 ausgewertet.

Kapitel 11

Experimentelle Untersuchung universelle Kodierung

Hardwareplattform der lokal ausgeführten Experimente: Rechner `WBIS-H207085` von LENOVO, Modell `10T8S6LB00`, mit 7,87 GB Arbeitsspeicher; ein Intel-Core-i3-8100T-Prozessor mit 3,10 GHz, vier physischen Kernen und vier logischen Prozessoren (x64); Microsoft Windows 11 Pro, 64-Bit, Version 10.0.26200.

Der vorangehende Abschnitt 10 ist rein formal. Er beweist, dass sich jede endliche attributierte relationale Struktur verlustfrei als Stapel von Mengenfolgen kodieren lässt (Satz 19), dass der Vergleich zweier Stapel in erwarteter Linearzeit möglich und bis auf einen konstanten Faktor optimal ist (Sätze 21 und 22) und dass ein invertierter Paarindex das exakte Suchergebnis ohne Einzelvergleiche liefert, dessen Aufwand von der Trefferlast H und nicht von der Datenbankgröße N abhängt (Sätze 23 und 24, Korollar 12). Offen blieb, ob diese Aussagen in einer realen Implementierung mit Datenbank, Prozess-Pool und Speicherverwaltung tragen. Die Sätze sind Aussagen über Rechenschritte; gemessen wird Wanduhrzeit, und diese enthält Datenbankzugriffe, Serialisierung, Prozesskommunikation und die Kosten des Interpreters. Dieser Abschnitt beschreibt das Experiment, das die Lücke zwischen Beweis und Messung schließt. Er folgt dem Aufbau der früheren Experimente (Abschnitte 7, 8 und 9) und trennt deshalb strikt zwischen dem, was das Experiment prüfen *soll*, dem Versuchsaufbau und den Ergebnissen.

Das Experiment ist das im Abschnitt 10.6 angekündigte Experiment des zweiten Schritts. Es wird durch `src/experiment_search_universal.py` gesteuert und vergleicht bei gleichen Anfragen und gleichen Daten die Indexsuche mit Einzelvergleichen in Python und mit der nativen Bibliothek.

Stand der Auswertung.

Der dokumentierte Lauf vom 10. Oktober 2026 ist nicht vollständig abgeschlossen: Die JSON-Metadaten setzen `complete=false`; die Phasen *workers*, *chunksize* und *dbsize* fehlen, und für die Suchmessung liegen nur `index` und `python` aus Wiederholung 0 vor. Die Textdatei endet bereits während des Suchfortschritts. Ausgewertet werden deshalb ausschließlich die abgeschlossenen Korrektheits-, Mikro- und Speicherprüfungen sowie die vorhandenen Suchmessungen. Fehlende Messungen werden nicht aus dem Versuchsplan abgeleitet oder durch Platzhalter als Ergebnisse ausgegeben. Grundlage sind `src/results/search_universal_20261010_103804.json` und `src/results/search_universal_20261010_103804.txt`.

11.1 Fragestellung und Hypothesen

Das Experiment beantwortet vier Fragen, die sich aus den Sätzen des Abschnitts 10 ergeben:

(1) *Korrektheit:* Liefern Kern, Paarindex und die bisherigen Multi-Omics-Implementierungen dieselben Entscheidungen, und stimmt das Suchergebnis mit dem von Einzelvergleichen überein?

(2) *Laufzeit:* Wie verhält sich die Ende-zu-Ende-Suchzeit der Indexsuche zu der der Einzelvergleiche in Python und mit der nativen Bibliothek, und aus welchen Phasen setzt sie sich zusammen?

(3) *Skalierung:* Hängt die Suchzeit der Indexsuche, wie Korollar 12 vorhersagt, von der Trefferlast H statt von der Datenbankgröße N ab, während die Einzelvergleiche mit N wachsen?

(4) *Kosten des Index:* Wie groß sind Speicherbedarf und Einfügezeit des Paarindex, und wachsen sie, wie Satz 24 sagt, linear mit der Belegung der gespeicherten Stapel?

Aus den Sätzen werden Hypothesen abgeleitet. Jede Hypothese nennt ein Kriterium, an dem sie scheitern kann; eine Hypothese, die keine Messung widerlegen könnte, wäre keine.

Zwei der Hypothesen formulieren bewusst eine Grenze der Aussage. H5 und H6 gelten nur, solange die Häufigkeitsannahme des Korollars 12 trägt. Bei sehr kurzen Anfragen mit wenigen aktiven Schichten kommt jedes Paar in einem großen Teil der gespeicherten Strukturen vor; dann ist $H = \Theta(N)$, und die Indexsuche liest große Listen, ohne Einzelvergleiche zu sparen. In diesem Fall wäre der Index nicht schneller, und das Experiment soll genau diesen Übergang sichtbar machen (Abschnitt 11.6).

11.2 Untersuchte Varianten

Verglichen werden vier Varianten mit demselben Schema, denselben Daten und derselben Trefferbedingung. Die Referenz für alle Speedups ist `python`.

`index` Paarindex (SQL). `crud.search_universal` liest die Listen der Tabelle `universal_pairs` (Primärschlüssel über Schema, Schicht, Art, Paar und Netzwerk) ohne Prozess-Pool. Im Modus *unabhängig* ist das Ergebnis der Schnitt der Trefferlisten (Satz 23), im Modus *kohärent* werden die Kandidaten zusätzlich verifiziert.

`python` Einzelvergleiche im Kern. Alle gespeicherten Stapel des Schemas mit ausreichender Länge werden geladen und im Prozess-Pool mit `universal_core.compare_stacks` einzeln verglichen, ohne Index. Dies entspricht dem Vorauswahlschritt der bisherigen Pipeline (Länge $\geq$ Länge der Anfrage) und ist die Referenz.

`mo_python` Bisherige Multi-Omics-Pipeline in Python. `crud.search_multiomics` mit der Implementierung `multiomics_python`: Vorfilter über Schichtnamen und Knotenzahl, Einzelvergleiche im Pool.

Tabelle 11.1: Hypothesen des Experiments, Herleitung und Widerlegungskriterium.

Nr.	Aussage	Herleitung	Widerlegt, wenn
H1	`universal_core` stimmt bei Paararten, Modi und Schichtauswahlen mit `multiomics_python` und C++ überein.	Bemerkung 14: Matrixstapel ist die Multi-Omics-Relation.	mindestens eine abweichende Entscheidung.
H2	Die Indexsuche stimmt bei Treffermenge, Gegenrichtung und Klasse mit Einzelvergleichen überein, auch nach Löschen und erneutem Einfügen.	Satz 23.	mindestens eine abweichende Treffermenge oder Klasse.
H3	Alle Varianten liefern in beiden Modi dieselben Treffer und finden alle bekannten eingebetteten Lösungen.	H1, H2 und Konstruktion der Ground Truth.	abweichende Treffermenge oder Trefferquote unter 1.
H4	Die Index-Anfragezeit ist affin in H (hohes R^2) und wächst bei konstantem H nicht mit N.	Satz 24.	schlechte lineare Anpassung an H oder Wachstum mit N bei konstantem H.
H5	Einzelvergleiche skalieren etwa linear mit N ($b \approx 1$), die Indexsuche bei selektiven Anfragen sublinear ($b < 1$).	Korollar 12 mit Häufigkeitsannahme.	Exponent der Indexsuche nahe 1 bei selektiven Anfragen.
H6	Der Index-Speedup steigt mit der Selektivität (lange Anfragen, mehrere aktive Schichten).	Satz 24: Kosten $O(H)$; H wächst mit der Häufigkeit der Paare.	Speedup hängt nicht von der Selektivität ab.
H7	Die unabhängige Indexsuche ist schneller als die kohärente (keine Verifikation).	Satz 23, zweite Formel.	Zeitverhältnis kohärent/unabhängig ≤ 1 bei vergleichbarer Treffermenge.
H8	Index-Paarzahl und Wörterbuchgröße wachsen linear mit N; die Einfügezeit je Netzwerk bleibt konstant.	Satz 24, Speicher- und Pflegekosten.	überlineares Wachstum, steigende Einfügezeit je Netzwerk.
H9	C++ schlägt Python im Mikrobenchmark; Ende-zu-Ende ist der Gewinn wegen Laden und Übergabe kleiner.	Befunde der Abschnitte 8 und 9.	Ende-zu-Ende-Speedup $\geq$ Mikro-Speedup.

`cpp` Dieselbe Pipeline mit der nativen Bibliothek. Die Klasse `MultiOmics` aus `csubgraph` wird über `CSUBGRAPH_LIB_PATH` geladen. Vor jeder Messreihe prüft das Skript, dass in den Worker-Prozessen tatsächlich die native Bibliothek aktiv ist; ein stiller Rückfall auf Python würde den Vergleich verfälschen und bricht den Lauf ab.

Die Varianten `python`, `mo_python` und `cpp` nutzen den Prozess-Pool, `index` nicht. Damit vergleicht das Experiment ausdrücklich Verfahren in ihrem jeweiligen Betriebsmodus und nicht einzelne Rechenkerne: Der Gewinn der Indexsuche enthält auch, dass sie keine

Prozesse kommunizieren lässt. Die Mikro-Benchmarks (Abschnitt 11.4.2) trennen den Kern deshalb vom Betriebsmodus.

11.3 Versuchsaufbau

11.3.1 Schema und Kodierung

Das Experiment verwendet ein Matrixschema (`universal_schema.matrix_schema`, lokale Koordinaten) mit L_Σ Schichten (Standard 5) aus dem Pool Genom, Transkriptom, Proteom, Metabolom, Epigenom und Lipidom. Die Kantenwahrscheinlichkeiten der Zufallsschichten sind 0,12, 0,25, 0,30, 0,20, 0,15 und 0,22. Jedes Netzwerk besitzt alle Schichten des Schemas; dadurch bilden alle Varianten dieselbe Datenbasis desselben Schemas, was Voraussetzung der Sätze des Abschnitts 10 ist. Der kodierte Stapel besteht aus einer Existenzschicht, die jede Entität auf sich selbst abbildet, und den In-Nachbarschaften der Matrizen (Bemerkung 14). Ein Netzwerk wird gleichzeitig als universelle Struktur (Tabellen `universal_structures`, `universal_components` und `universal_pairs`) und als Multi-Omics-Netzwerk (Tabellen `omics_*`) abgelegt; beide Darstellungen stammen aus denselben Matrizen. Vor dem Lauf prüft das Skript an Größen von 2 bis 63 Knoten, dass die schnelle Erzeugung der Stapel mit der offiziellen Kodierung `encode_matrices` und mit `stack_from_matrices` übereinstimmt; bei einer Abweichung bricht es ab.

11.3.2 Daten und Ground Truth

Die Datenbank wird um 20.000 reproduzierbar aus dem Seed (42) erzeugte Zufallsnetzwerke des Typs `universal_experiment` mit 4 bis 40 Knoten ergänzt. Vorhandene andere Netzwerke werden nicht verändert. Zu jeder eingebetteten Anfrage werden zusätzlich Netzwerke mit bekannter Lösung erzeugt:

- eine identische Kopie (`exact`),
- je drei *kohärent* eingebettete Netzwerke: zwei benachbarte Spalten der Anfrage werden in allen Schichten an derselben Position und demselben Spaltenpaar in das größere Netzwerk kopiert,
- je drei nur *unabhängig* eingebettete Netzwerke (ab zwei Schichten): Position und Spaltenpaar werden je Schicht verschoben, sodass die Anfrage meist nur im Modus unabhängig enthalten ist.

Die eingebetteten Netzwerke haben höchstens 12 Knoten mehr als die Anfrage. Für jedes dieser Netzwerke hält das Skript mit der Python-Referenz fest, in welchem Modus die Anfrage tatsächlich enthalten ist; diese Menge ist die *erwartete Treffermenge*. Sie liefert eine Ground Truth zusätzlich zum Vergleich der Varianten untereinander, denn zwei Varianten, die dieselbe falsche Antwort geben, würden sich sonst nicht unterscheiden.

11.3.3 Anfragen

Das Anfragegitter besteht aus sieben Knotenzahlen $(4, 6, 8, 12, 16, 24, 32)$, vier Schichtzahlen (1 bis 4) und zwei Arten: einer Zufallsanfrage ohne eingebettete Lösung und einer eingebetteten Anfrage. Es ergeben sich $7 \times 4 \times 2 = 56$ Anfragen. Aktive Schichten sind die

Schichten der Anfrage; die übrigen Schichten des Schemas sind leer und nicht aktiv, jede Anfrageschicht enthält mindestens eine Kante. Anfragen haben mindestens vier Knoten. Für Stapel der Länge 1 ohne Paare unterscheiden sich Indexsuche und `compare_stacks` per Definition (Klassifikation des Paarindex); solche Stapel werden nicht verwendet.

11.3.4 Gleiche Trefferbedingung

Als Treffer zählen die Ergebnisse, bei denen die Anfrage Q in der gespeicherten Struktur M_k enthalten ist (`KEEP_B`, `EQUAL_KEEP_A`, `EQUAL_KEEP_B`, `IDENTICAL`), also $\pi_I(Q) \sqsubseteq_\mu \pi_I(M_k)$ mit $\mu \in \{\text{ind}, \text{coh}\}$. Alle vier Varianten werden an dieser Bedingung gemessen. Treffer anderer Multi-Omics-Netzwerke in der Datenbank, die nicht zum Experiment gehören, zählen bei `mo_python` und `cpp` nicht mit.

11.3.5 Messprotokoll

- **Wiederholungen.** Jede Suchmessung wird dreimal wiederholt (Parameter `-reps`). Je Messreihe wird eine Aufwärmsuche verworfen (`-warmup`), damit Caches der Datenbank und der Prozess-Pool die Zeiten der ersten Anfrage nicht verfälschen.

- **Reihenfolge.** Die Reihenfolge der Varianten rotiert je Wiederholung, die Reihenfolge der Anfragen wird je Wiederholung aus dem Seed gemischt. Systematische Effekte der Reihenfolge, etwa ein sich erwärmender Prozessor oder gefüllte Datenbank-Caches, verteilen sich so auf alle Varianten.

- **Gemessene Größe.** Gemessen wird die Wanduhrzeit der gesamten Suche (SQL, Vergleiche, Prozesskommunikation, Trefferliste). Zusätzlich werden drei Phasen getrennt: *Laden* (bis zum ersten Vergleich beziehungsweise bis alle Kandidaten geladen sind), *Vergleichen* (Zeit in den Vergleichen) und *Rest* (Umwandlung der Stapel, Aufbau der Treffer). Bei `index` umfasst das Laden das Lesen der Indexlisten und das Laden der Kandidaten; die Vergleiche klassifizieren dort nur die Kandidaten, im Modus unabhängig sind alle Kandidaten Treffer.

- **Parallelität.** Die Standardausführung verwendet zwei Worker (`-workers 2`); die Skalierung wird in einem eigenen Teilexperiment untersucht.

- **Störungen.** Der Server darf während des Experiments nicht laufen. Andere Multi-Omics-Netzwerke in der Datenbank, etwa vom Multi-Omics-Experiment, sehen nur `mo_python` und `cpp` (anderes Schema); sie verlangsamen diese beiden Varianten und werden im Bericht als Warnung vermerkt. Vor dem Lauf sollte deshalb `experiment_search_multiomics.py -cleanup-only` ausgeführt werden.

- **Hilfsindex beim Löschen.** Beim Löschen vieler Netzwerke legt das Skript kurzzeitig einen Hilfsindex auf `universal_pairs(network_id)` an, weil die Kaskade von `biological_networks` sonst je Netzwerk die ganze Indextabelle durchsucht; der Index wird danach entfernt.

Die Versuchsumgebung wird aus den Metadaten des dokumentierten Laufs übernommen. Absolute Zeiten sind an diese Konfiguration gebunden; übertragbar sind allenfalls die beobachteten Verhältnisse und auch diese zunächst nur als Befunde eines einzelnen, unvollständigen Laufs.

Tabelle 11.2: Konfiguration des Experiments (Standardwerte des Skripts, Parameter in Klammern).

Parameter	Wert
Zufallsnetzwerke (`-networks`)	20.000, 4 bis 40 Knoten (`-min-nodes`, `-max-nodes`)
Schema (`-schema-layers`)	Matrixschema mit 5 Schichten aus Genom, Transkriptom, Proteom, Metabolom, Epigenom, Lipidom
Anfrageknotenzahlen (`-query-sizes`)	$4, 6, 8, 12, 16, 24, 32$
Anfrageschichtzahlen (`-query-layers`)	$1, 2, 3, 4$
Anfragearten und Modi	Zufall/eingebettet; kohärent/unabhängig
Eingebettete Netzwerke (`-plants`)	1 identisch, 3 kohärent, 3 unabhängig je eingebetteter Anfrage; bis zu 12 zusätzliche Knoten
Suchmessungen	56 Anfragen $\times$ 2 Modi $\times$ 4 Varianten $\times$ 3 Wiederholungen = 1.344 (mit allen Varianten)
Korrektheit (`-correctness-pairs`)	5.000 Paare, beide Modi; Indexprüfung auf 150 Stapeln mit 200 Anfragen
Mikro-Benchmark	Schichtzahlen $1, 2, 3, 4, 6, 8$; $n_B \in \{8, 16, 24, 32, 48, 63\}$; Zufall/eingebettet; je 100 Paare
Speicherexperiment	$N \in \{250, 500, 1000, 2000, 5000\}$; 8 Anfragen; 5 Wiederholungen der Indexabfrage
Worker-Skalierung (`-worker-scaling`)	$1, 2, 3, 4$ Worker
Chunk-Größen (`-chunksizes`)	$1, 8, 32, 128, 256, 1024, 4096$
Datenbankgrößen (`-db-sizes`)	10 %, 25 %, 50 %, 100 % von 20.000, also 2.000, 5.000, 10.000, 20.000
Teilmenge für Worker, Chunk, Datenbankgröße	6 gleichmäßig gewählte Anfragen, 2 Wiederholungen
Seed	42
Laufzeitumgebung	Windows 11 Pro, 64-Bit, Version 10.0.26200; Python 3.15.0, NumPy 2.5.3, PostgreSQL 18.6
Rechner	LENOVO `10T8S6LB00`, Rechnername `WBIS-H207085`, 7,87 GB RAM
Prozessor	Intel Core i3-8100T, 3,10 GHz, 4 physische Kerne und 4 logische Prozessoren, x64

11.4 Teilexperimente und Metriken

Das Skript gliedert den Lauf in sieben Teilexperimente, die mit `-only` einzeln gewählt werden können. Die ersten drei laufen ohne Datenbank, die übrigen vier gegen PostgreSQL.

11.4.1 Korrektheit (ohne Datenbank)

Das Teilexperiment prüft H1 und H2 in zwei Teilen.

Paarweise Prüfung. Auf 5.000 Paaren, gleichmäßig auf fünf Arten verteilt (Zufall, kohärent eingebettet, unabhängig eingebettet, identisch, umgekehrt), werden in beiden Modi `universal_core.compare_stacks`, `multiomics_python.compare_layered` und, falls

verfügbar, die native Bibliothek verglichen. Schichtzahl (1 bis 4), Größe der kleineren Struktur (3 bis 20 Knoten) und der größeren (n_A bis 40) werden zufällig gewählt. Zusätzlich werden Invarianten geprüft: Symmetrie (vertauscht man die Strukturen, vertauschen sich `KEEP_A` und `KEEP_B`, die übrigen Klassen bleiben), kohärent impliziert unabhängig (Lemma 6), erwartete Ergebnisse der konstruierten Paare sowie Monotonie unter Projektion: weniger aktive Schichten dürfen keinen Treffer verlieren. Je Paar und Modus wird außerdem der Kern auf einer zufälligen Schichtauswahl gegen die Referenz auf den gewählten Matrizen geprüft.

Indexprüfung im Speicher. Ein `PairIndex` über 150 zufällige gespeicherte Stapel wird gegen Einzelvergleiche über alle Stapel geprüft. Die 200 Anfragen sind zur Hälfte Fenster gespeicherter Stapel (mindestens ein Treffer) und zur Hälfte zufällige kurze Stapel. Je Anfrage werden beide Modi und beide Richtungen (`contains`: Anfrage in gespeicherter Struktur; `contained`: gespeicherte Struktur in der Anfrage) sowie die Klassifikation (`classify`) gegen `compare_stacks` geprüft, auf allen Schichten und auf zufälligen Schichtauswahlen. Anschließend wird ein Fünftel der Stapel aus dem Index entfernt, die Prüfung auf dem Rest wiederholt und die Stapel werden wieder eingefügt; so wird die Pflege des Index (Satz 24) mitgeprüft.

Metrik. Zahl der Prüfungen, Zahl der Abweichungen je Art, Anzahl der Invariantenverletzungen. Erwartet wird 0 (H1, H2). Die Prüfung ist eine differenzielle Stichprobe mit gezielten Grenzfällen und kein Beweis; sie sichert aber die Umsetzung gegen die bewiesenen Sätze ab.

11.4.2 Mikro-Benchmark (ohne Datenbank)

Der Mikro-Benchmark misst die Zeit je Einzelvergleich für `universal_core`, `multiomics_python` und die native Bibliothek in Abhängigkeit von Schichtzahl $L \in \{1, 2, 3, 4, 6, 8\}$, Knotenzahl $n_B \in \{8, \ldots, 63\}$ der größeren Struktur (die kleinere hat $\max(3, \lfloor n_B/2 \rfloor)$ Knoten), Modus und Paartyp (zufällig oder eingebettet). Alle Implementierungen erhalten dieselben, deterministisch aus Seed und Zelle erzeugten Paare; die Umwandlung in Stapel des Kerns erfolgt vor der Messung. Zusätzlich werden die Aufrufkosten an einem trivialen 1×1-Vergleich gemessen. Die Zellen mit $n_B > 63$ sind nicht vorgesehen, weil die native Bibliothek eine 64-Bit-Darstellung nutzt (Bemerkung 14); der Kern selbst hat diese Grenze nicht. Der Mikro-Benchmark liefert den Kernvergleich ohne Datenbank und Pool und stützt H9.

11.4.3 Indexsuche im Speicher

Dieses Teilexperiment prüft H4, H5 und H8 ohne Datenbankrauschen. Der `PairIndex` wird mit $N \in \{250, 500, 1000, 2000, 5000\}$ Zufallsstapeln und allen eingebetteten Netzwerken aufgebaut; die Aufbauzeit, die Zahl der Listeneinträge und die Größe der Wörterbücher werden gemessen. Für acht gleichmäßig aus dem Anfragegitter gewählte Anfragen und beide Modi wird die Indexabfrage fünfmal ausgeführt (Median) und einmal mit der linearen Suche mit Einzelvergleichen (`universal_core.contained`) über alle gespeicherten Stapel verglichen. Je Messung werden die Trefferlast H, die Zahl der Kandidaten und die Zahl der Treffer festgehalten; zugleich wird geprüft, dass beide Verfahren dieselbe Treffermenge liefern. Ausgewertet werden

– die lineare Anpassung der Anfragezeit an H ($t \approx a + b\,H$, Bestimmtheitsmaß R^2), getrennt für alle Messungen und für den Modus unabhängig;

– der Exponent b der Anpassung $t \sim N^b$ im doppelt logarithmischen Raum für Index und lineare Suche;

– die lineare Anpassung der Aufbauzeit an die Zahl der Stapel.

11.4.4 Ende-zu-Ende-Suche

Im Suchexperiment werden alle 56 Anfragen in beiden Modi mit allen gewählten Varianten gegen die 20.000 Zufallsnetzwerke und die eingebetteten Netzwerke ausgeführt, dreimal wiederholt. Je Messung werden Gesamtzeit, die drei Phasen, Zahl der Kandidaten, Zahl der Treffer, Zahl der exakten Treffer, die erwarteten und gefundenen eingebetteten Netzwerke sowie ein Prüfwert der Treffermenge (SHA-1 der sortierten Netzwerkkennungen) gespeichert. Kennzahlen:

– **Speedup** einer Variante gegenüber der Referenz `python`: Quotient der Gesamtzeiten, aggregiert als Quotient der Summen und als *geometrischer Mittelwert* der Speedups je Anfrage mit 95 %-Bootstrap-Intervall.

– **Übereinstimmung der Treffer** aller Varianten je Anfrage und Modus über den Prüfwert (H3).

– **Trefferquote der eingebetteten Netzwerke** (Ground Truth): gefundene geteilt durch erwartete Netzwerke je Anfrage, Modus und Variante.

– **Phasenanteile** und **Skalierung mit der Kandidatenzahl**.

– **Selektivität**: Zahl der Kandidaten und Treffer nach Schichtzahl, Knotenzahl und Modus.

11.4.5 Worker, Chunk-Größe und Datenbankgröße

Die drei letzten Teilexperimente laufen im Modus kohärent auf einer Teilmenge von sechs gleichmäßig gewählten Anfragen mit zwei Wiederholungen. *Workers* variiert die Zahl der Worker-Prozesse (1 bis 4) für die Varianten mit Prozess-Pool und vergleicht das Ergebnis mit den Befunden des Abschnitts 7. *Chunksize* variiert die Chunk-Größe der parallelen Einzelvergleiche (1 bis 4096). *Dbsize* verändert die Zahl der Zufallsnetzwerke in der Datenbank auf 2.000, 5.000, 10.000 und 20.000 und misst je Stufe die Suchzeit aller Varianten sowie Speicherbedarf (Zeilen, Paare je Netzwerk, Wörterbuch je Netzwerk) und Einfügezeit des Index. Ausgewertet werden die lineare Anpassung der Gesamtzeit an N und der Exponent b in $t \sim N^b$ je Variante (H5, H8); nach der Reihe stellt das Skript den Ausgangszustand der Datenbank wieder her. Das Experiment ist das datenbankgestützte Gegenstück zum Speicherexperiment: Es enthält zusätzlich SQL, Indexlisten in der Datenbank und Prozesskommunikation.

Für den hier dokumentierten Lauf liegen zu diesen drei Teilexperimenten keine Ergebnisobjekte und keine Laufzeitwerte vor. Die Beschreibung oben ist daher der Versuchsplan, nicht ein berichteter Befund.

11.5 Auswertung und Reproduzierbarkeit

Statistik.

Der Versuchsplan sieht Mittelwerte über Wiederholungen mit 95 %-Konfidenzintervallen der t-Verteilung sowie geometrische Mittelwerte der Speedups mit Bootstrap-Intervall (2.000 Ziehungen) vor. Da für die Ende-zu-Ende-Suche nur eine Wiederholung gespeichert ist, lassen sich diese Intervalle für den vorliegenden Lauf nicht bestimmen; die berichteten Summenverhältnisse sind deskriptiv. Die Anpassungen an Trefferlast H und Datenbankgröße N werden mit der Methode der kleinsten Quadrate bestimmt und mit R^2 angegeben. Exponenten aus einer Regression von $\log t$ auf $\log N$ gelten nur für den gemessenen Bereich und sind keine asymptotischen Konstanten.

Ausgabedateien.

Der Lauf schreibt in `src/results/` mit Zeitstempel: die JSON-Datei mit Metadaten, Anfragen, allen Einzelmessungen und Auswertung (wird während des Laufs fortlaufend aktualisiert, ein Abbruch verliert höchstens einige Sekunden), ein PDF mit den Diagrammen, einen Textbericht, die Tabellen für dieses Dokument (`_tabellen.tex`) sowie die Einzeldiagramme `plot27_universal_*.pdf` bis `plot45_universal_*.pdf`. Aus einer vorhandenen JSON-Datei lassen sich PDF, Diagramme, Tabellen und Bericht ohne neuen Messlauf erzeugen (`-plot-from`).

Aufruf.

Der Aufruf erfolgt im Projektordner, damit die `.env` gefunden wird:

Listing 11.1: Aufruf des Experiments

```
python src/experiment_search_universal.py --quick # kleiner Vorabtest
python src/experiment_search_universal.py # vollstaendig, dauert Stunden
python src/experiment_search_universal.py --only correctness micro memory
python src/experiment_search_universal.py --backends index python --networks
    ↪ 5000
python src/experiment_search_universal.py --plot-from src/results/
    ↪ search_universal_<zeit>.json
```

Die Experiment-Netzwerke bleiben standardmäßig in der Datenbank, damit der Lauf wiederholbar ist und schneller neu startet; `-cleanup` entfernt sie am Ende, `-rebuild-data` erzeugt sie neu. Nach dem Lauf ist in diesem Dokument der Zeitstempel (Makro `\ucstamp`) einzutragen.

11.6 Ergebnisse

Die Auswertung beruht auf dem Lauf vom 10. Oktober 2026 mit dem Seed 42. Die Metadaten melden 20.175 Strukturen im Schema `uc_exp_l5`, fünf aktive Schema-Schichten und 56 Anfragen. Die Phasen Korrektheit, Mikro-Benchmark und Speicherindex sind abgeschlossen. Für die Ende-zu-Ende-Suche enthält die JSON-Datei 224 Datensätze: je 56 Anfragen in beiden Modi für `index` und `python`, sämtlich aus Wiederholung 0. Die Metadaten markieren den Lauf ausdrücklich als unvollständig. Daher werden keine Ergebnisse für die nicht gespeicherten Wiederholungen oder die übrigen Varianten behauptet.

11.6.1 Korrektheit

Die differenzielle Prüfung umfasste 5.000 Paare. Für die Multi-Omics-Implementierung und die native Bibliothek wurden jeweils 10.000 Entscheidungen des Kerns verglichen, entsprechend beiden Modi je Paar; beide Vergleiche ergaben 10.000 Übereinstimmungen und keine Abweichung. Die Indexprüfung verwendete 150 gespeicherte Stapel auf fünf Schichten und 240 Anfragen. In 1.840 Prüfungen der Suche und Klassifikation wurden keine Abweichungen gefunden; auch nach Entfernen und erneutem Einfügen von Indexeinträgen meldet der Lauf keine Invariantenverletzung. Fehlerzähler und protokollierte Verletzungen sind jeweils null.

Damit bestehen H1 und H2 für die geprüften Fälle. Die Prüfung ist eine differenzielle Stichprobe mit gezielt erzeugten Paaren und keine Verifikation für alle endlichen Strukturen. Sie stützt die korrekte Umsetzung im untersuchten Matrixschema, ersetzt aber weder den Beweis aus Abschnitt 10 noch eine Prüfung weiterer Schemaarten.

11.6.2 Ende-zu-Ende-Suche

Tabelle 11.3: Vorhandene Ende-zu-Ende-Suchmessungen aus Wiederholung 0; je Modus wurden 56 Anfragen ausgeführt.

Variante	Modus	Zeit (s)	Kandidaten	Treffer	Ground Truth gefunden/erwartet
`index`	kohärent	12,652	8.929	7.530	113/113
`python`	kohärent	886,904	806.024	7.530	113/113
`index`	unabhängig	8,398	8.929	8.929	175/175
`python`	unabhängig	802,158	806.024	8.929	175/175

Für beide Varianten sind die Trefferprüfwerte der 56 Anfragen je Modus identisch; es gibt keine fehlgeschlagenen Vergleiche oder fehlenden eingebetteten Referenztreffer. Über beide Modi summiert sich die Laufzeit auf 21,050 s für `index` und 1.689,062 s für `python`; das Verhältnis der Summen beträgt 80,24. Getrennt nach Modus ergeben sich 70,10 im kohärenten und 95,52 im unabhängigen Modus. Diese Quotienten sind keine belastbaren Schätzungen für den allgemeinen Betrieb: Sie stammen aus nur einer gespeicherten Wiederholung und enthalten weder ein Konfidenz- noch ein Bootstrap-Intervall.

Die Spalte *Kandidaten* bezeichnet bei `index` die durch die Paarlisten gelieferten Kandidaten, bei `python` dagegen alle geladenen, längenkompatiblen Strukturen, die einzeln geprüft werden. Der Paarindex reduzierte die Zahl der in diesen Suchläufen weiter geprüften Strukturen damit deutlich; die Größen sind wegen der unterschiedlichen Bedeutung der Zählung aber nicht als identische Mengen zu interpretieren. Im kohärenten Modus wurden aus jeweils 8.929 Indexkandidaten 7.530 Treffer verifiziert. Im unabhängigen Modus waren alle 8.929 Kandidaten Treffer.

Die Summen der gemessenen Phasenzeiten zeigen die unterschiedliche Kostenverteilung. Für `index` entfielen 18,845 s auf das Laden, 0,869 s auf Vergleiche und 1,334 s auf den Aufbau; dies entspricht rund 89,5 %, 4,1 % und 6,3 % der Gesamtzeit. Für `python` waren es 1.163,223 s, 423,981 s und 101,859 s beziehungsweise rund 68,9 %, 25,1 % und 6,0 %. Die verbleibende Zeit der Indexsuche liegt somit überwiegend beim Datenbankzugriff und Laden, nicht bei der anschließenden Prüfung.

11.6.3 Mikro-Benchmark

Der Mikro-Benchmark enthält je Variante 144 reguläre Messzellen aus Schichtzahl, Knotenzahl, Modus und Paartyp; alle drei Implementierungen meldeten null Fehler. Tabelle 11.4 gibt den geometrischen Mittelwert der je Zelle gemessenen Medianzeiten wieder. Die Werte isolieren den Einzelvergleich und sind nicht mit den Ende-zu-Ende-Zeiten gleichzusetzen.

Tabelle 11.4: Geometrischer Mittelwert der Medianzeit im Mikro-Benchmark.

Variante	Zeit je Vergleich (ms)
`python`	0,2006
`mo_python`	1,5426
`cpp`	0,2402

Über die 144 vergleichbaren Zellen beträgt das geometrische Verhältnis der Python-Zeit zur C++-Zeit 0,83; im Mittel über diese Zellen war die native Variante hier also nicht schneller als der Kern. Die bestehende Multi-Omics-Python-Implementierung benötigte geometrisch etwa das 7,69-Fache der Kernzeit. Bei den trivialen 1×1-Aufrufen lagen die Medianzeiten bei 0,0080 ms für `python`, 0,0118 ms für `mo_python` und 0,0234 ms für `cpp`. H9 wird durch diese Mikro-Messung daher nicht bestätigt: Aus der nativen Implementierung ergibt sich in dieser Konfiguration kein geometrischer Einzelvergleichsvorteil. Eine Ursache lässt sich aus den Zeitwerten allein nicht ableiten.

11.6.4 Indexsuche im Speicher und Pflegekosten

Das Speicherexperiment umfasst 80 Anfragen über fünf Indexgrößen. Indexabfrage und lineare Suche lieferten in allen Fällen dieselbe Treffermenge. Tabelle 11.5 zeigt die über die Anfragen je Größe gemittelten Zeiten.

Tabelle 11.5: Aufbau und Anfragezeiten des Speicherindex; Zeiten der Abfragen sind arithmetische Mittel über die Messungen je Größe.

N	Stapel	Paareinträge	Aufbau (s)	Index (ms)	Einzelvergleiche (ms)	Verhältnis
250	425	101.591	0,147	0,398	18,74	47,1
500	675	173.380	0,242	0,320	29,53	92,2
1.000	1.175	298.665	0,367	0,526	68,89	130,9
2.000	2.175	561.454	1,098	0,772	250,39	324,5
5.000	5.175	1.336.936	4,213	1,076	310,51	288,5

Die Paarzahl pro gespeichertem Stapel liegt über die fünf Größen zwischen etwa 239 und 258 Einträgen. Das Wachstum der Gesamtzahl der Einträge ist mit einer annähernd konstanten Belegung je Stapel vereinbar. Die gemessene Aufbauzeit je Stapel steigt jedoch von rund 0,347 ms bei 425 Stapeln auf 0,814 ms bei 5.175 Stapeln; eine konstante Einfügezeit je Stapel ist damit nicht belegt. Die mittlere Indexabfrage blieb im beobachteten Bereich zwischen 0,320 und 1,076 ms, während die Einzelvergleiche zwischen 18,74 und 310,51 ms benötigten. Die daraus berechneten Verhältniswerte schwanken und sind als Messwerte dieser Anfrageauswahl, nicht als garantierte Beschleunigungsfaktoren zu lesen.

Die Anpassung der Indexzeit an die Trefferlast H erklärt die Messungen insgesamt nur schwach ($R^2 = 0{,}093$). Getrennt nach Modus beträgt $R^2 = 0{,}132$ für kohärente und $R^2 =$

0,638 für unabhängige Anfragen. Eine pauschale affine Abhängigkeit von H wird durch diese Daten somit nicht überzeugend gestützt. Die deskriptiven Exponenten der Speicherreihe $t \sim N^b$ betragen $b = 0{,}391$ für den Index und $b = 1{,}040$ für die Einzelvergleiche. Das passt im untersuchten Bereich zu einem flacheren Wachstum der Indexabfrage und einem annähernd linearen Wachstum der Vergleichsreferenz; fünf Größenstufen reichen jedoch nicht, um daraus ein allgemeines Skalierungsgesetz abzuleiten.

11.6.5 Nicht vorliegende Teilexperimente

Obwohl die Metadaten alle sieben Phasen als angefordert aufführen, enthält `phase_seconds` nur Einträge für Korrektheit, Mikro-Benchmark und Speicherindex. Ergebnisdaten zu den Ende-zu-Ende-Varianten `mo_python` und `cpp`, zu den weiteren Wiederholungen sowie zu Workerzahl, Chunk-Größe und Datenbankgröße fehlen. Die Textdatei endet bei einem Fortschrittsstand von 183/1.344 Suchmessungen; die JSON-Datei enthält zugleich die oben beschriebenen 224 Suchdatensätze aus einer Wiederholung für zwei Varianten. Wegen dieser unvollständigen und nicht deckungsgleichen Fortschrittsangaben wird die Datenbank-Skalierung nicht aus dem Speicherexperiment extrapoliert. H3 ist nur für die vorhandenen Suchpaare zwischen `index` und `python` geprüft; H4 bis H8 sind nur teilweise oder gar nicht auswertbar, und ein Ende-zu-Ende-Test von H9 liegt nicht vor.

11.7 Diskussion und Grenzen

Einordnung der Hypothesen.

H1 und H2 werden durch die vorhandenen differenziellen Prüfungen gestützt: In den geprüften Fällen stimmen Kern, Multi-Omics-Python, native Implementierung und Paarindex überein; Abweichungen und Invariantenverletzungen wurden nicht protokolliert. Für die tatsächlich gespeicherten Suchläufe stimmen die Prüfsummen der Treffermengen von `index` und `python` in allen 112 Paaren aus Anfrage und Modus überein, und alle 288 erwarteten eingebetteten Treffer wurden gefunden. H3 ist damit für diese beiden Varianten und diese Stichprobe erfüllt, nicht für die nicht aufgezeichneten Backends oder Wiederholungen.

H4 und H5 erhalten nur teilweise Unterstützung. In der Speicherreihe wächst die Indexzeit im untersuchten Bereich flacher als die Zeit der Einzelvergleiche ($b = 0{,}391$ gegenüber $b = 1{,}040$), jedoch erklärt eine lineare Anpassung an H die Gesamtstichprobe nur schwach ($R^2 = 0{,}093$). Der unabhängige Modus zeigt eine bessere, aber nicht vollständige Anpassung ($R^2 = 0{,}638$). Zudem fehlt die geplante Datenbankgrößenreihe; eine Aussage zur Laufzeit in PostgreSQL als Funktion von N lässt sich daher nicht treffen. H6 wurde nicht in der geplanten gruppierten Auswertung nach Selektivität, Knoten- und Schichtzahl abgeschlossen. H7 zeigt sich deskriptiv in den Summen: unabhängig war die Indexsuche schneller als kohärent, obwohl sie mehr Treffer lieferte; der Unterschied ist wegen der einzelnen Wiederholung nicht abgesichert. H8 wird hinsichtlich der annähernd konstanten Zahl an Paareinträgen je Stapel gestützt, nicht aber hinsichtlich einer konstanten Aufbauzeit pro Stapel, die mit der Indexgröße anstieg. H9 wird vom Mikrobenchmark in der erwarteten Richtung nicht bestätigt: Die C++-Zeit war geometrisch nicht kleiner als die Python-Zeit, und Ende-zu-Ende-Daten für `cpp` fehlen.

Vergleich zur vollständigen Einzelvergleichssuche.

Die vorhandenen Summen ergeben gegenüber `python` einen Zeitquotienten von 70,10 im kohärenten und 95,52 im unabhängigen Modus. Dieser Unterschied passt zur Arbeitsweise des Paarindex, der Kandidaten über Paarlisten eingrenzt, während die Python-Referenz alle längenkompatiblen Strukturen lädt und einzeln vergleicht. Die Zeitmessung bestätigt damit einen deutlichen Vorteil in den erfassten Fällen, aber nicht dessen allgemeine Größe: Die Stichprobe enthält nur eine Wiederholung und zwei von vier vorgesehenen Suchvarianten. Die Phasen zeigen außerdem, dass bei `index` der Datenbankzugriff dominiert; weitere Beschleunigung des Vergleichskerns allein würde diese Restzeit daher nicht beseitigen.

Grenzen der Untersuchung.

- **Synthetische Daten.** Die Zufallsnetzwerke haben je Schicht eine feste Kantenwahrscheinlichkeit. Reale Netzwerke sind nicht gleichverteilt: Hubs und häufige Motive lassen H stark anwachsen. Die Aussagen über die Selektivität sind daher nur für diese Verteilung belegt. Die Ground Truth stammt aus konstruierten Einbettungen und ersetzt keine biologische Validierung (Bemerkung 22).

- **Ein Schema.** Das Experiment prüft ein Matrixschema. Die Kernunabhängigkeit (Korollar 13) wird damit nur für diesen Spezialfall gemessen; Schemata für Hypergraphen, Pfade oder Ontologien sind nicht Teil des Experiments. Die Messungen bestätigen die Kodierung nur indirekt über den Spezialfall des Matrixstapels (Bemerkung 14).

- **Größenbereich.** Anfragen und Netzwerke haben höchstens 40 Knoten (Mikro-Benchmark bis 63), weil die native Bibliothek an die 64-Bit-Darstellung gebunden ist. Das Verhalten bei sehr großen Netzwerken, für die der Kern keine Grenze hat, ist nicht gemessen.

- **Unvollständige Suchmessung.** Die Ende-zu-Ende-Ergebnisse umfassen nur `index` und `python` aus einer Wiederholung. Die geplanten Vergleiche mit `mo_python` und `cpp`, weitere Wiederholungen sowie die Worker-, Chunk- und Datenbankgrößenreihen fehlen. Konfidenzintervalle und eine Bewertung der Streuung sind deshalb nicht möglich.

- **Datenbankgröße und Betrieb.** Die Datenbankgrößenreihe dieses Experiments wurde nicht abgeschlossen. Absolute Zeiten hängen zudem von der Hardware, der PostgreSQL-Konfiguration und dem Cachezustand ab; kalte und warme Datenbankmessungen sind nicht getrennt. Die Hardwareplattform der lokal ausgeführten Experimente ist in Tabelle 11.2 dokumentiert.

- **Kein Vergleich mit der klassischen Isomorphie.** Das Experiment misst die definierte Relation $\sqsubseteq$ und nicht die klassische Subgraph-Isomorphie (Bemerkung 21); ein Vergleich mit VF2 gehört nicht dazu.

- **Kopplung.** Für den dokumentierten Lauf weist die JSON-Metadatei keine fremden Multi-Omics-Netzwerke aus (`foreign_omics_networks`=0). Eine solche Beeinflussung ist für diese Messungen daher nicht vermerkt; die Aussage lässt sich nicht ohne Weiteres auf andere Datenbankzustände übertragen.

11.8 Fazit

Die abgeschlossenen Korrektheitsprüfungen, der Mikro-Benchmark und das Speicherexperiment stützen die korrekte Umsetzung im Matrixschema und zeigen im gemessenen Größenbereich deutlich kürzere Indexabfragen als lineare Einzelvergleiche. Die vorhandenen Suchdaten von `index` und `python` stimmen bei Treffermengen und eingebetteter Ground Truth überein; ihr Zeitvergleich beruht jedoch auf nur einer Wiederholung. Der Lauf ist als unvollständig markiert: Ende-zu-Ende-Ergebnisse für die weiteren Varianten, Wiederholungen sowie Worker-, Chunk- und Datenbankgrößenmessungen fehlen. Damit liefert das Kapitel belastbare Teilbefunde, aber noch keine vollständige experimentelle Bestätigung der Laufzeit- und Skalierungsaussagen. Für die Fortsetzung ist der Lauf bis zum regulären Abschluss zu wiederholen und anschließend mit allen vier Varianten und den geplanten Wiederholungen auszuwerten. Abschnitt 12 ordnet diese Teilbefunde in die Gesamtaussagen der Arbeit ein.

Kapitel 12

Schluss

Die vorliegende Arbeit hat Gen-DB, ein System zur Subgraph-Suche in Datenbanken biologischer Netzwerke, von der Architektur über die formale Analyse des Suchverfahrens und mehrere empirische Untersuchungen bis zu einer verallgemeinernden universellen Kodierung betrachtet. Dieser abschließende Abschnitt gliedert sich in fünf Teile: Zunächst fasst Abschnitt 12.1 die Ergebnisse aller vorangegangenen Abschnitte zusammen und gleicht sie mit den in der Einleitung formulierten Beiträgen ab. Abschnitt 12.2 benennt die Grenzen der Arbeit, sodass die Ergebnisse richtig eingeordnet werden können. Abschnitt 12.3 entwickelt darauf aufbauend ausführlich mögliche Erweiterungen und offene Forschungsfragen, geordnet nach Themenfeldern. Abschnitt 12.4 schlägt schließlich eine Reihenfolge für die Umsetzung vor, und Abschnitt 12.5 schließt die Arbeit mit einer Gesamteinschätzung. Ein Faden, der sich durch alle fünf Teile zieht, ist die in Abschnitt 8.10 begründete Position zur losen Kopplung: Die Zusammenfassung ordnet sie in die Ergebnisse ein, die Würdigung benennt ihre Belastbarkeit, der Ausblick leitet daraus das Themenfeld G ab, die Priorisierung gibt ihr einen Platz in der Reihenfolge, und die Schlussbemerkung stellt sie neben den zentralen Gedanken der Trennung.

12.1 Zusammenfassung

12.1.1 Ausgangspunkt und Zielsetzung

Biologische Datenbanken enthalten Millionen von Netzwerken: metabolische Pfade, Protein-Interaktionsnetzwerke und Gen-Regulationsnetzwerke. Eine zentrale Aufgabe bei ihrer Analyse ist die Subgraph-Suche, bei der zu einem Query-Netzwerk alle gespeicherten Netzwerke gefunden werden sollen, die das Query nach der jeweils festgelegten Relation als Teilstruktur enthalten. Diese Arbeit untersucht, wie sich der Subgraph Algorithmus [1], der zwei Adjazenzmatrizen über Spaltensignaturen und zyklische längste gemeinsame Teilsequenzen in $O(n^3)$ vergleicht, in eine relationale Datenbank einbetten lässt, ob sich der Suchablauf formal korrekt beschreiben und analysieren lässt und wie er sich bei einer Datenbank mit einer Million Einträgen tatsächlich verhält. Im Verlauf der Arbeit kamen die Einbindung eines C++-Rechenkerns in das Python-System, geschichtete Multi-Omics-Suchen, eine formale Theorie der parallelen Ausführung und die Verallgemeinerung des Suchkerns durch eine universelle Kodierung hinzu. Die Ergebnisse sind dabei unterschiedlichen Evidenzarten zugeordnet: mathematische Beweise, experimentelle Messungen auf synthetischen Daten und offene Fragen zur biologischen Semantik werden nicht gleichgesetzt.

12.1.2 Systemarchitektur und Datenmodell

Gen-DB besteht aus drei Schichten: einem Web-Frontend in HTML, CSS und JavaScript, einem FastAPI-Backend in Python und einer PostgreSQL-Datenbank. Das Datenmodell speichert Metadaten (Name, Netzwerktyp, Organismus, Knoten- und Kantenzahl) in der Tabelle `biological_networks` und die strukturellen Repräsentationen (Knotenlabels, Adjazenzmatrix, Signatur-Array und Signatur-Hash) in der 1:1-verknüpften Tabelle `network_matrices`. Entscheidend für die Suche ist, dass Knoten- und Kantenzahl als eigene, indizierte Spalten vorliegen, sodass sie ohne Zugriff auf die Matrizen ausgewertet werden können. Die UML-Paket- und Klassendiagramme in Abschnitt 3.3 dokumentieren, dass die Zerlegung in Präsentation, Logik und Persistenz sauber getrennt ist. Dadurch lässt sich die Verarbeitung einer Suchanfrage als eigenständiger Baustein formalisieren. Für die Ausführung der Vergleiche besitzt das Backend einen Prozess-Pool, der die Kandidaten blockweise auf mehrere Worker-Prozesse verteilt (Abschnitt 7).

12.1.3 Formale Ergebnisse zum Suchverfahren

Die Suche umfasst zwei Stufen. Die erste ist eine datenbankseitige Vorauswahl, die nur Netzwerke mit mindestens so vielen Knoten und Kanten wie das Query zulässt. Die zweite ist der strukturelle Vergleich der verbliebenen Kandidaten mit dem Subgraph Algorithmus. Gezeigt wurde Folgendes:

- Die Vorauswahl ist korrekt: Aus $A_Q \preceq A_k$ folgt $n_Q \leq n_k$ und $m_Q \leq m_k$ (Lemma 2). Damit scheidet der Pre-Filter keinen Treffer der Richtung $A_Q \preceq A_k$ fälschlich aus (Korollar 1).
- Die Gesamtlaufzeit beträgt $T_{\text{Suche}}(N, C, n) = O(\log N + C) + O(C \cdot n^3)$ (Satz 2). Bei beschränkter Knotenzahl und beschränkter Kandidatenzahl K ist die Suchzeit unabhängig von der Datenbankgröße N.
- Ohne Kandidatenlimit ($K = \infty$) findet das Verfahren alle Treffer der Richtung $A_Q \preceq A_k$ (Satz 3). Mit Limit gilt dies nur noch eingeschränkt, weil ein sortiertes `LIMIT` Treffer mit großer Knotenzahl systematisch abschneiden kann (Lemma zur Verzerrung durch `LIMIT`).
- Die symmetrische Richtung $A_k \preceq A_Q$ wird von der Vorauswahl nicht abgedeckt und braucht eine eigene Filterbedingung.

12.1.4 Empirische Evaluation der Datenbank

Als Testdatenbank dienten 1.000.000 synthetische Netzwerke (zuzüglich zweier Beispielnetzwerke) mit gleichverteilter Knotenzahl $n_k \in \{3, \ldots, 20\}$, Kantenwahrscheinlichkeit $p = 0{,}3$ und gleichverteilten Netzwerktypen und Organismen. Im Mittel hatten sie 11,5 Knoten und 44,3 Kanten. Die Verteilungen entsprachen den Erwartungen aus dem Generierungsmodell, einschließlich der Abweichung der mittleren Kantenzahl vom naiven Erwartungswert, die sich durch die Jensen-Ungleichung erklärt. Die Auswertung zeigte, dass allein die Knotenzahlbedingung den Suchraum je nach Querygröße um bis zum Faktor 18 verkleinert, und die Kantenbedingung die Reduktion weiter verstärkt. Die Modellkurven zur Skalierung bestätigten die Aussage des Korollars zu Satz 2: Mit Kandidatenlimit bleibt die Suchzeit näherungsweise konstant, ohne Limit wächst sie linear in N.

12.1.5 Experimenteller Vergleich mit VF2

In einer direkten Messung gegen VF2 [10] auf Erdős–Rényi-Graphen $G(n, 0{,}3)$ war der Subgraph Algorithmus bei festem $n_A = 5$ und $n_B \in [5, 13]$ um den Faktor 1,9 bis 4,7 schneller. Der Vorsprung schmolz bei $n_B = 14$ bis 16 auf etwa 1,1 ab, und bei $n_B = 17$ und 18 war VF2 mit Speedups von 0,83 und 0,82 etwas schneller. Bei gemeinsam wachsendem $n = n_A = n_B$ stieg die Laufzeit des Subgraph Algorithmus von 0,055 ms ($n = 4$) auf 1,309 ms ($n = 12$), also um den Faktor 23,8 und damit in der Größenordnung der kubischen Referenz $3^3 = 27$, während VF2 im selben Bereich nur um den Faktor 2,8 wuchs. Die Ursachenanalyse hat dies auf vier Punkte zurückgeführt: eine Referenzimplementierung ohne vorzeitigen Abbruch, das frühe Terminieren von VF2 auf zufälligen dichten Graphen, den Interpreter-Overhead der Python-Implementierung und den grundsätzlichen Unterschied zwischen Worst-Case-Schranke und Durchschnittsverhalten. Die Literaturübersicht hat gezeigt, dass der Subgraph Algorithmus unter den betrachteten Verfahren das einzige mit unbedingt polynomieller Worst-Case-Schranke ist, dafür aber eine strukturell schwächere Relation entscheidet als die volle Subgraph-Isomorphie. Daraus wurde eine Hybridstrategie abgeleitet, bei der je nach Graphgröße zwischen beiden Verfahren gewählt und der Subgraph Algorithmus als Fallback mit garantierter Schranke behalten wird.

12.1.6 Parallelisierung der Kandidatenvergleiche

Die Messreihe mit sechs aus der Datenbank gezogenen Queries (mindestens 15 Knoten, 62.064 bis 325.964 Kandidaten, kein Kandidatenlimit) hat die parallele Ausführung quantifiziert. Mit zwei Workern sank die Gesamtzeit aller Anfragen von 1604 auf 843 Sekunden, das entspricht einem Speedup von 1,90 und einer Effizienz von 95 %, für jede einzelne Anfrage zwischen 1,87 und 1,93. Aus dem Speedup ergibt sich ein serieller Anteil von etwa 5 %. Drei Worker waren auf dem Rechner mit vier logischen Prozessoren rund 8 % langsamer als zwei ($S(3) = 1{,}77$); dieser Befund lässt sich mit der kleinen Stichprobe nicht eindeutig einer Ursache zuordnen. Die Antwortzeit wuchs mit einem Korrelationskoeffizienten von mindestens 0,99 linear mit der Kandidatenzahl. Eine einzelne Suche dauerte ohne Parallelisierung bis zu 403 Sekunden und mit zwei Workern bis zu 212 Sekunden.

12.1.7 Vergleich der Python- und C++-Implementierung

Die abgeschlossene Suchmessung mit zwei Workern verglich sechs identische Queries und insgesamt 931.899 Kandidaten. Die Python-Ausführung benötigte 1.180,28 Sekunden, die C++-Library 185,62 Sekunden; der Ende-zu-Ende-Speedup beträgt 6,36×. Der individuelle Speedup lag je nach Query zwischen 4,81× und 8,76×. Im direkten Mikrobenchmark über 200 gemeinsame Zufallspaare sank die mittlere Vergleichszeit von 2,694 ms in Python auf 0,074 ms in C++; der triviale Library-Aufruf lag bei 0,018 ms. Alle sechs Suchpaare lieferten dieselben Netzwerk-IDs und beide Reihen hatten null fehlgeschlagene Vergleiche. Der Unterschied zwischen Mikrobenchmark- und Such-Speedup erklärt sich daraus, dass Datenbankzugriff, Worker-Kommunikation und Ergebnisaufbau beim schnelleren Rechenkern einen größeren Anteil an der vollständigen Antwortzeit ausmachen.

12.1.8 Multi-Omics: Suchsemantik und abgeschlossene Messreihen

Abschnitt 9 erweitert den Vergleich auf geordnete Tupel aus mehreren Omics-Netzwerkschichten. Die kohärente Relation verlangt dieselbe Trefferposition in allen Schichten; die unabhängige Relation erlaubt schichtweise verschiedene Positionen. In der abgeschlossenen differenziellen Prüfung stimmen Python und C++ auf 5.000 Paaren und 10.000 Modusentscheidungen vollständig überein; Fehler und Invariantenverletzungen wurden nicht beobachtet. Im Mikrobenchmark über 144 passende Konfigurationen pro Backend beträgt das geometrische Mittel des Python/C++-Speedups 8,11× (Konfigurationsmittelwerte 4,0680 ms gegenüber 0,4005 ms).

Die vollständige Ende-zu-Ende-Reihe umfasst 336 gepaarte Suchläufe je Backend. Python benötigte aufsummiert 1.401,16 Sekunden, C++ 620,43 Sekunden; der Summen-Speedup beträgt 2,26, das geometrische Mittel der gepaarten Laufzeiten 2,28. Bei allen 336 Paaren waren Kandidatenzahl und Treffer-IDs gleich, alle 1.722 erwarteten eingebetteten Referenztreffer wurden gefunden, und es gab keine fehlgeschlagenen Vergleiche. Über 112 Konfigurationen mit je drei Wiederholungen lag der mediane Speedup bei 2,26; in jeder Konfiguration war C++ schneller. Die Vergleichsphase beanspruchte 68,6 % der Python-Zeit und 27,6 % der C++-Zeit, während das Laden mit C++ 70,5 % der Antwortzeit ausmachte.

Die Worker-Reihe auf dem Intel Core i3-8100T mit vier physischen Kernen erreichte bei Python mit vier Workern einen Speedup von 2,10, bei C++ dagegen nur 1,29. In der Chunk-Reihe lagen mittlere Blockgrößen vorn: 128 für Python und 32 für C++; die Unterschiede zum Standard 256 waren moderat und beruhen auf sechs Queries mit zwei Wiederholungen. Die Datenbankgrößenreihe blieb abgebrochen: 2.000 und 5.000 Zufallsnetzwerke sind vollständig gemessen, bei 10.000 liegt nur ein Teil der Messungen vor, 20.000 wurde nicht erreicht. Damit sind die Backend-Beschleunigung und beide Skalierungsreihen empirisch beschrieben, nicht aber ein allgemeines Wachstumsgesetz der Suchzeit mit der Datenbankgröße oder die biologische Aussagekraft der synthetischen Relation.

12.1.9 Formale Theorie der Parallelisierung

Abschnitt 9.10 geht über die früheren Worker- und Chunk-Messungen hinaus und zerlegt die Laufzeit in Laden, Vergleiche, Zuführung beziehungsweise Übergabe und Ergebnisaufbau. Das Ergebnis der Suche wird formal von der Planung ihrer Ausführung getrennt. Unter Annahme A1 ist die Entscheidungsfunktion rein, also unabhängig von Worker, Ausführungszeitpunkt und früheren Aufrufen. Dann liefert die parallele Suche dieselben Entscheidungen für alle Kandidaten und dieselbe geordnete Trefferliste wie die sequentielle Suche: Die Reihenfolge, in der Worker fertig werden, ändert das Ergebnis nicht, weil die Teillisten in Blockindex-Reihenfolge zusammengeführt werden. Dieses Resultat gilt für beliebige Worker-Zahlen und Blockgrößen. Es behauptet jedoch weder, dass die parallele Ausführung schneller ist, noch deckt es Ausfälle der Infrastruktur ab; ein beendeter Worker oder erschöpfter Speicher bleibt ein expliziter Fehler. Die Zeitschranken benötigen zusätzlich Annahme A2: einen nicht-leerlaufenden FIFO-Ablauf mit den im Abschnitt definierten Freigabezeiten und Aufgabenkosten. Die Gültigkeit dieser Planungsannahme folgt nicht aus dem Korrektheitssatz, sondern aus dem betrachteten Prozess-Pool-Modell.

Für k Vergleichsaufgaben mit Gesamtarbeit W, p Workern und längster Aufgabe $\theta_{\max}$ gilt für einen solchen Ablauf die Schranke $V \leq W/p + (1 - 1/p)\theta_{\max}$. Sie macht sowohl die

mittlere Arbeit pro Worker als auch die längste Einzelaufgabe sichtbar. Die Arbeit beweist außerdem die Schärfe: Bei ungünstiger FIFO-Reihenfolge kann die Schranke tatsächlich erreicht werden; ein konkretes Beispiel zeigt, dass eine lange Aufgabe, die erst nach vielen kurzen Aufgaben startet, den Abschluss dominiert. Für die absteigende Reihenfolge nach geschätzter Aufgabendauer wird eine bessere theoretische Schranke hergeleitet. Das ist eine Aussage über die Planungsgrenze, kein Nachweis eines Laufzeitgewinns in Gen-DB: Der praktische Sortierungseffekt wurde bislang nicht gemessen. Dieser Unterschied ist wichtig, weil Sortierkosten und die Genauigkeit der Aufwandsschätzung in einer Implementierung mit zu bewerten wären.

Auch die Blockbildung wird nicht nur empirisch variiert, sondern modelliert. Für C Kandidaten, Übergabeaufwand α, p Worker und mittleren Vergleichsaufwand $\bar{u}$ ergibt sich im idealisierten Kostenmodell die Blockgröße

$$b^* = \sqrt{\frac{C\alpha}{(p-1)\bar{u}}}.$$

Sie wächst mit Kandidatenzahl und Übergabeaufwand und sinkt, wenn zusätzliche Worker oder teurere Einzelvergleiche den Nutzen kleinerer Blöcke erhöhen. Das ist ein modellabhängiges Optimum, keine unmittelbar universelle Konfiguration. Die Messreihen mit sechs Queries und jeweils zwei Wiederholungen ergaben auf der untersuchten Hardware für Python die günstigste geprüfte Blockgröße 128 und für C++ 32; verglichen mit 256 lag die Gesamtzeit um etwa 9,7 % beziehungsweise 4,1 % niedriger. Die Vorteile sind moderat, die Stichprobe klein und die Rangfolge der Backends nicht als statistisch gesichert anzusehen. Die Zahlen begründen daher eine gezielte Wiederholung auf dem Zielsystem, nicht eine feste Voreinstellung für alle Installationen.

Für die Gesamtsuche ordnet das Amdahl-Modell ein, wie die serielle Phase den Speedup bei fester Problemgröße begrenzt. Bei vier Workern wurden serielle Anteile von $f = 0{,}197$ für Python und $f = 0{,}589$ für C++ gemessen; daraus folgen theoretische Obergrenzen von etwa 5,08 beziehungsweise 1,70, selbst bei beliebig vielen Workern und unveränderter serieller Arbeit. Die gemessenen Speedups der Gesamtsuche lagen bei 2,10 für Python und 1,29 für C++; die Parallel-Effizienz betrug somit 53 % beziehungsweise 32 %. In der Vergleichsphase allein waren die Effizienzen mit 72 % und 54 % höher, was die Bedeutung der außerhalb der Worker liegenden Arbeit sichtbar macht. Der schnellere native C++-Kern verkürzt die parallele Rechenphase und erhöht damit den relativen Anteil des seriellen Ladens, Zuführens und Ergebnisaufbaus. Das ist eine Verschiebung des Engpasses durch die Beschleunigung des Kerns, kein Beleg dafür, dass C++ grundsätzlich schlechter parallelisiere. Der Abschnitt ergänzt Amdahl um die Karp–Flatt-Kennzahl zur Einordnung des effektiven seriellen Anteils und Gustafsons Blick auf wachsende Problemgrößen; beide Perspektiven beantworten andere Fragen als der Speedup einer festen Suche.

Schließlich grenzt die Theorie Optimierungsrichtungen voneinander ab. Ein Sperrenmodell erklärt, warum zusätzliche Python-Threads bei rechenintensiven Vergleichen wegen des Global Interpreter Lock keinen entsprechenden Rechengewinn versprechen. Ein Beweis zur Zerlegung über die Schichten zeigt zwar, dass Parallelität innerhalb eines einzelnen Vergleichs grundsätzlich möglich ist; sie ist aber nicht implementiert, durch die Schichtzahl begrenzt und gegenüber unabhängigen Kandidatenvergleichen zunächst nachrangig. Aus den Messwerten folgt, dass Worker-Zahlen die tatsächlich verfügbare Prozessorkapazität berücksichtigen sollten und dass beim C++-Backend zuerst die serielle Lade- und Zubringerphase zu untersuchen ist. Für belastbare Aussagen sind mindestens fünf Wiederholungen, mehr Queries mit unterschiedlichen Größen und Dichten, getrennte Phasenmessungen

sowie Versuche auf Rechnern mit mindestens acht Kernen erforderlich. Eine Sortierung der Kandidaten nach absteigendem geschätztem Aufwand gehört als eigene, kontrollierte Hypothese in diese Wiederholung; ein bewiesener theoretischer Vorteil darf nicht vorab als gemessener Systemgewinn verbucht werden.

12.1.10 Kopplung und Beschreibungsgrenzen

Der Vergleich wirft neben der Laufzeitfrage eine Gestaltungsfrage auf, die Abschnitt 8.10 ausführt: Wie lässt sich ein in C++ implementierter Rechenkern in ein Python-System einbinden, ohne die Zuständigkeiten und die Messung unnötig zu verkomplizieren? Die Messungen sprechen für die Library-Anbindung: Sie hält den wiederholten Aufruf innerhalb des Worker-Prozesses, vermeidet den Prozess-pro-Vergleich- und JSON-Pfad der CLI und erreicht in den sechs Suchläufen den Speedup von 6,36×. Das ist ein empirischer Vorteil unter den dokumentierten Bedingungen. Die weitergehende Behauptung, eine bestimmte Grenze erleichtere Zusammenarbeit, Analyse und Wiederverwendung generell, bleibt eine begründete Gestaltungsposition und ist nicht durch eine Nutzer- oder Entwicklerstudie geprüft.

12.1.11 Universelle Kodierung biologischer Strukturen und optimale Suche

Die bisherigen Abschnitte formulierten für jede Art von Daten einen eigenen Vergleichspfad: den Subgraph Algorithmus für das Einzelnetzwerk und die kohärente und unabhängige Schichtrelation für das mehrschichtige Netzwerk. Abschnitt 10 zeigt, dass dies nicht notwendig ist. Er trennt die Suche in einen *strukturunabhängigen Kern*, der nur Stapel von Mengenfolgen kennt, und eine *Kodierung* $\Phi_{\Sigma,\kappa}$, die jede biologische Struktur eines beliebigen Schemas Σ in einen solchen Stapel überführt und vollständig in Python liegt. Für eine neue biologische Struktur ist dann nur noch ein neues Schema und damit eine neue Kodierung zu schreiben; der Kern, seine Beweise und sein Index bleiben unverändert (Korollar 13). Die Ergebnisse dieses Abschnitts sind rein formal; Messungen enthält er nicht. Ihre experimentelle Prüfung ist Gegenstand von Abschnitt 11 und wird unten gesondert zusammengefasst.

Der Kern.

Ein Stapel besteht aus $L \geq 1$ Schichten gleicher Länge m, jede Schicht ist eine Folge endlicher Mengen natürlicher Zahlen, der Komponenten. Zwei Relationen sind definiert: $M \sqsubseteq_{\text{ind}} M'$ verlangt je Schicht ein gemeinsames Paar aus einer linearen Nachbarschaft in M und einer zyklischen Nachbarschaft in M', wobei das Paar von Schicht zu Schicht verschieden sein darf; $M \sqsubseteq_{\text{coh}} M'$ verlangt dasselbe Paar von Komponententupeln in allen Schichten zugleich. Der Matrixstapel mit der In-Nachbarschaft der Knoten als Komponente ist der Spezialfall der bisherigen Multi-Omics-Relation, nur ohne die Beschränkung auf $n \leq 63$, die allein aus der 64-Bit-Darstellung der nativen Bibliothek stammt (Bemerkung 14). Die elementaren Eigenschaften (Lemma 6) lauten: kohärent impliziert unabhängig; ein Treffer verlangt mindestens zwei Spalten; jedes Objekt mit mindestens zwei Spalten ist in sich selbst enthalten; weniger aktive Schichten liefern mehr Treffer; die Relationen sind im Allgemeinen *nicht transitiv*. Ferner gibt es *keinen* zulässigen Kantenzahl-Filter (Lemma 7): Aus $M \sqsubseteq M'$ folgt $|M| \leq |M'|$, nicht aber $\nu(M) \leq \nu(M')$. Der Kantenfilter $m_k \geq m_Q$ der

Vorauswahl aus Abschnitt 4.2 würde für diese Relation echte Treffer ausschließen; zulässig ist allein die Schranke der Knotenzahl beziehungsweise Schichtlänge.

Die Kodierung.

Ein Schema $\Sigma = (\Lambda, T, (k_t), (W_t))$ legt Knotenmerkmale, Relationstypen, Stelligkeiten und Gewichtsstufen fest. Eine Struktur über Σ umfasst gerichtete und, über symmetrische Relationen, ungerichtete Netzwerke, gewichtete und attributierte Netzwerke, Reaktionsnetze mit mehrstelligen Reaktionen, Hypergraphen, Pfade und Sequenzen (als Nachfolgerrelation), Ontologie-Graphen und mehrschichtige Netzwerke; kontinuierliche Werte müssen vorher quantisiert werden. Die Kodierung verwendet eine injektive Koordinatenabbildung κ der Entitäten, etwa die Vergabereihenfolge in einem Entitätsregister, und erzeugt Schichten für die Existenz der Entitäten, für jedes Knotenmerkmal, für jede Gewichtsstufe zweistelliger Relationen sowie für die Positionen und Gewichtsstufen mehrstelliger Relationen; mehrstellige Relationen erhalten dazu eigene Relationsspalten. Die Schichtzahl L_Σ hängt nur vom Schema ab, sodass Query und gespeicherte Strukturen eines Schemas stets vergleichbar sind. Die Sätze besagen:

- **Verlustfreiheit.** $\Phi_{\Sigma,\kappa}$ ist für jedes Schema und jedes Koordinatensystem injektiv, die Umkehrabbildung ist auf dem Bild berechenbar (Satz 19). Beim Kodieren geht also keine Information der (quantisierten) Struktur verloren.

- **Aufwand.** Bei festem Schema ist die Kodierung in Zeit $O(|S| \log |S|)$ berechenbar; die Belegung ist $O(|S|)$, und die Spaltenzahl überschreitet $|S|$ nicht (Satz 20).

- **Lokale und globale Koordinaten.** Mit lokalem Rang der Knoten innerhalb der Struktur ergibt sich genau der Matrixstapel und damit die bisherige Multi-Omics-Relation, das Einzelnetzwerk ist der Fall $L = 1$. Mit globalem Register bedeutet die Gleichheit von Komponenten *gleiche Regulatoren* im Sinne derselben Entitäten (Bemerkung 16).

- **Schemaänderung.** Ein neuer Relationstyp ändert Spaltenzahl und Reihenfolge und erfordert eine Neukodierung in $O(|S| \log |S|)$ je Struktur; alternativ erhält der neue Typ ein eigenes Schema mit eigenem Index.

Als Beispiel dient ein regulatorisches Netzwerk mit drei Entitäten und drei Kanten, das zwei Schichten ergibt (Existenz und Binärkanten); wird eine dreistellige Reaktion aus Substrat, Enzym und Produkt ergänzt, entsteht eine vierte Spalte mit drei weiteren Inzidenzschichten.

Der optimale Vergleich.

Für zwei Stapel mit L Schichten sind $\sqsubseteq_{\mathrm{ind}}$ und $\sqsubseteq_{\mathrm{coh}}$ in erwarteter Zeit $O\big(\nu(M) + \nu(M') + L(|M| + |M'|)\big)$ entscheidbar (Satz 21); mit Sortieren statt Hashing ist die Schranke deterministisch bis auf einen logarithmischen Faktor. Eine untere Schranke zeigt, dass jeder korrekte deterministische Algorithmus bei geeigneten Eingaben mindestens $\lceil m'/2 \rceil$ Komponenten von M' und $\lfloor m/2 \rfloor$ Komponenten von M lesen muss, die Laufzeit also $\Omega(m + m')$ beträgt (Satz 22). Der Beweis nutzt ein Gegenspielerargument, bei dem die gelesenen Positionen eine Knotenüberdeckung des Kreises beziehungsweise Pfades bilden müssen. Für festes L und Komponenten konstanter Größe ist der Vergleich daher

$\Theta(|M| + |M'|)$ und bis auf einen konstanten Faktor optimal; für wachsendes L bleibt eine Lücke zwischen $\Omega(|M| + |M'|)$ und $O(L(|M| + |M'|))$, die ausdrücklich nicht geschlossen wird.

Der invertierte Index.

Der Paarindex speichert zu jeder Schicht und jedem Komponentenpaar, in welchen Strukturen das Paar als zyklisches beziehungsweise lineares Paar vorkommt, samt der Länge der Struktur. Das Suchergebnis im Modus ind ist der Schnitt der schichtweisen Trefferlisten; der Modus coh verifiziert das Ergebnis des Modus ind nur auf dieser Obermenge (Satz 23). Die Auswertung benötigt keinen Einzelvergleich. Ihr Aufwand beträgt erwartet $O(\nu_I(Q) + |I|\, m_Q + H)$ mit der Trefferlast H, zuzüglich $O\big(|\mathrm{Match}_{\mathrm{ind}}| \cdot |I|(m_Q + m_{\max})\big)$ für coh; der Index belegt $O(\sum_k L\,|M_k|)$ Einträge und wird beim Einfügen oder Löschen einer Struktur in $O(L\,|M_k| + \nu(M_k))$ angepasst (Satz 24). Kommt jedes Paar in höchstens h gespeicherten Strukturen vor, hängt die Laufzeit nicht von N ab; im ungünstigsten Fall, wenn alle Strukturen dasselbe Paar enthalten, ist $H = \Theta(N\, m_Q |I|)$, was wegen der Ausgabegröße $\Theta(N)$ ohnehin unvermeidlich ist (Korollar 12). Die umgekehrte Suchrichtung ergibt sich symmetrisch über den Index der linearen Paare und liefert die Treffertypen `KEEP_A`, `KEEP_B` und `EQUAL_KEEP_*`. Gegenüber der bisherigen Pipeline mit $O(\log N + C) + O(C\, n^3)$ prüft der Index keine Kandidaten, sondern liest nur Trefferlisten. Dass dadurch das Laden, das in der Multi-Omics-Messung mit C++ 70,5 % der Gesamtzeit ausmachte, auf die tatsächlichen Treffer schrumpft, ist eine *Folgerung* aus dem Satz und keine Messung.

Reichweite und Grenzen.

Nach Korollar 13 gelten alle Sätze für jedes Schema, jedes Koordinatensystem und jede Datenbank kodierter Strukturen unverändert. Die C++-Bibliothek `csubgraph` bleibt als optionaler Beschleuniger für Stapel erhalten, deren Koordinaten in 64 Bit passen; für Korrektheit und die bewiesenen Laufzeiten wird sie nicht benötigt. Zwei Grenzen sind ausdrücklich festgehalten. Erstens bezieht sich die Optimalität auf die *definierte* Relation $\sqsubseteq$ und nicht auf die klassische Teilgraph-Isomorphie: Gäbe es eine in Polynomialzeit berechenbare Abbildung auf Stapel, die Teilgraph-Isomorphie exakt wiedergibt, wäre das NP-vollständige Problem in Polynomialzeit lösbar, also P = NP (Bemerkung 21); jede polynomiell berechenbare Relation über diesen Strukturen ist daher ein Näherungs- oder Filterkriterium. Zweitens sichern die Sätze Kodierung, Korrektheit und Aufwand, nicht die fachliche Validität: Ob zwei benachbarte Spalten mit gleichen Nachbarschaftsmengen eine biologisch sinnvolle Ähnlichkeit darstellen, hängt von Schema, Spaltenordnung und Koordinatensystem ab und ist mit fachlich annotierten Daten zu prüfen (Bemerkung 22).

Umsetzung und experimenteller Stand.

Die Umsetzung folgt den Sätzen (Abschnitt 10.6): eine Schnittstelle je Strukturart mit `schema`, `encode` und `decode`, ein Entitätsregister für κ, Wörterbuch- und Indextabellen sowie Tests, die direkt aus den Sätzen folgen. Der Kern, die Matrixkodierung und der Paarindex in einer SQL-Tabelle sind implementiert. Die differenzielle Prüfung und die Speicherreihe des Experiments sind abgeschlossen; die Ende-zu-Ende- und Skalierungsteile liegen nur unvollständig vor. Die formalen Resultate und die gemessenen Teilbefunde sind deshalb getrennt zu lesen.

12.1.12 Experimentelle Untersuchung der universellen Kodierung

Die Sätze des Abschnitts 10 sind Aussagen über Rechenschritte. Abschnitt 11 stellt ihnen Messungen mit Indexsuche, Einzelvergleichen im Kern und zwei Multi-Omics-Varianten gegenüber. Die abgeschlossenen differenziellen Prüfungen ergaben keine Abweichungen; im Speicherexperiment war die Indexabfrage über die geprüften Größen deutlich kürzer als die lineare Suche. Die vorhandenen Ende-zu-Ende-Messungen zeigen für Indexsuche gegenüber Python große Zeitverhältnisse, sind aber auf eine Wiederholung und zwei Varianten beschränkt. Da die JSON-Metadaten den Lauf als unvollständig kennzeichnen, bleiben die vollständige Variantenbewertung, Wiederholbarkeit und datenbankgestützte Skalierung offen.

Fragen und Hypothesen.

Das Experiment beantwortet vier Fragen: Liefern Kern, Index und die bisherigen Implementierungen dieselben Entscheidungen (Korrektheit)? Wie verhalten sich die Ende-zu-Ende-Suchzeiten und ihre Phasen (Laufzeit)? Hängt die Suchzeit der Indexsuche von der Trefferlast und nicht von der Datenbankgröße ab, wie Korollar 12 vorhersagt (Skalierung)? Wie groß sind Speicherbedarf und Einfügezeit des Index (Kosten)? Aus den Sätzen sind neun Hypothesen H1 bis H9 abgeleitet; jede nennt ein Kriterium, an dem sie scheitern kann (Tabelle 11.1). H1 bis H3 betreffen die Korrektheit und die Übereinstimmung der Treffer, H4 und H5 die Abhängigkeit von Trefferlast und Datenbankgröße, H6 und H7 die Abhängigkeit von Selektivität und Modus, H8 die Kosten des Index und H9 das Verhältnis von Mikro- und Ende-zu-Ende-Speedup der nativen Bibliothek.

Versuchsaufbau.

Alle Varianten arbeiten auf demselben Matrixschema mit fünf Schichten; ein Netzwerk wird gleichzeitig als universelle Struktur und als Multi-Omics-Netzwerk abgelegt. Die Datenbank enthält 20.000 reproduzierbar erzeugte Zufallsnetzwerke mit 4 bis 40 Knoten. Das Anfragegitter besteht aus sieben Knotenzahlen, vier Schichtzahlen und zwei Arten (Zufall, eingebettet), also 56 Anfragen in beiden Modi. Zu den eingebetteten Anfragen werden identische, kohärent und nur unabhängig eingebettete Netzwerke erzeugt, für die die Python-Referenz festhält, in welchem Modus die Anfrage enthalten ist. Diese konstruierte Ground Truth ergänzt den Vergleich der Varianten untereinander, denn zwei Varianten mit derselben falschen Antwort würden sich sonst nicht unterscheiden. Die Messung verwendet drei Wiederholungen, eine verworfene Aufwärmsuche, wechselnde Reihenfolge der Varianten und gemischte Reihenfolge der Anfragen; gemessen wird die Wanduhrzeit der gesamten Suche, zerlegt in Laden, Vergleichen und Rest.

Teilexperimente.

Sieben Teilexperimente trennen die Fragen: eine differenzielle Korrektheitsprüfung auf 5.000 Paaren in beiden Modi und auf Schichtauswahlen sowie eine Prüfung des Index gegen Einzelvergleiche einschließlich Löschen und Wiedereinfügen; ein Mikro-Benchmark der Einzelvergleiche nach Schichtzahl, Knotenzahl, Modus und Paartyp; die Indexsuche im Speicher mit linearer Anpassung der Anfragezeit an die Trefferlast und Exponent in N; die Ende-zu-Ende-Suche mit Speedup, Phasenanteilen, Trefferabgleich und Trefferquote

der Ground Truth; schließlich Worker-Skalierung, Chunk-Größe und Datenbankgröße mit Speicherbedarf und Einfügezeit des Index. Die Auswertung verwendet geometrische Mittel mit Bootstrap-Intervall für Speedups, Konfidenzintervalle der t-Verteilung für Mittelwerte und logarithmische Regression für die Exponenten.

Was die Ergebnisse bedeuten würden.

Die Auswertung ist so festgelegt, dass sie drei Fälle unterscheidet. Bestätigen sich H1 bis H4, tragen die Beweise bis in die Implementierung, und die Aussage über die Trefferlast ist belegt. Bestätigen sich H1 bis H3, nicht aber H5 und H6, so ist die Häufigkeitsannahme des Korollars 12 für die gewählte Datenverteilung verletzt: Die Sätze bleiben richtig, die Indexsuche bringt für diese Daten aber wenig. Scheitern H1 bis H3, liegt ein Fehler vor, der vor jeder Laufzeitaussage zu beheben ist. Hinzu kommt eine vorhersehbare Grenze: Bei sehr kurzen Anfragen mit wenigen aktiven Schichten ist die Trefferlast von der Größenordnung N, sodass die Indexsuche keinen Vorteil gegenüber Einzelvergleichen hat; das Experiment soll diesen Übergang sichtbar machen.

Grenzen.

Das Experiment prüft ein Matrixschema und damit die Kernunabhängigkeit nur im Spezialfall des Matrixstapels; es verwendet synthetische Daten mit fester Kantenwahrscheinlichkeit je Schicht, höchstens 40 Knoten und 20.000 Netzwerke, eine Hardwarekonfiguration und kleine Stichproben. Es misst die definierte Relation $\sqsubseteq$ und nicht die klassische Subgraph-Isomorphie, und es belegt keine biologische Validität. Absolute Zeiten sind von der Hardware abhängig; übertragbar sind die Verhältnisse.

12.1.13 Zentrale Erkenntnisse

Aus der Gesamtschau lassen sich vierzehn Erkenntnisse festhalten:

1. **Die Architektur trägt die Analyse.** Die klare Trennung zwischen relationaler Kandidatenreduktion und algorithmischem Strukturvergleich erlaubt es, beide Stufen getrennt zu beweisen, zu messen und zu optimieren.

2. **Der Pre-Filter ist notwendig, aber allein nicht ausreichend.** Er verkleinert den Suchraum erheblich, doch bei kleinen oder spärlichen Queries bleiben Hunderttausende Kandidaten übrig. Bei den sechs Messanfragen lagen zwischen 6,2 % und 32,6 % der Datenbank im Suchraum.

3. **Der Anteil der Vergleichsphase hängt vom Backend ab.** Sie beansprucht 88,4 % der Python-Zeit, aber nur 33,3 % der C++-Suchzeit. Die Gesamtsuche wächst weiterhin mit der Kandidatenzahl; nach der Beschleunigung durch die Library begrenzen Datenzugriff, Übergabe und Ergebnisaufbau den End-to-End-Speedup stärker.

4. **Garantie und Geschwindigkeit sind zwei verschiedene Eigenschaften.** Der Subgraph Algorithmus bietet eine bewiesene polynomielle Schranke und vorhersagbare Laufzeiten. Auf kleinen dichten Zufallsgraphen kann ein Verfahren mit exponentieller Worst-Case-Schranke wie VF2 im Durchschnitt gleich schnell oder schneller sein.

5. **Parallelisierung ist ein verlässlicher, aber begrenzter Hebel.** Sie senkt die Antwortzeit auf dem untersuchten System um einen konstanten Faktor nahe der Prozessorzahl, ändert aber nichts an der linearen Abhängigkeit von C. Sie ersetzt deshalb weder einen schärferen Pre-Filter noch ein Kandidatenlimit.

6. **Die Worker-Zahl muss zur Hardware passen.** Die Messreihe auf vier logischen Prozessoren umfasst nur ein bis drei Worker; sie untersucht die Überbelegung daher nicht. Dass drei Worker langsamer als zwei waren, ist ein Einzelbefund und begründet weder eine allgemeine Obergrenze noch die mitgelieferte Voreinstellung von vierzehn Workern.

7. **Vollständigkeit und Geschwindigkeit stehen im Spannungsfeld.** Ein Kandidatenlimit macht die Suchzeit unabhängig von N, kostet aber die Vollständigkeitsgarantie. Auch die Suchrichtung ist nur für $A_Q \preceq A_k$ abgesichert.

8. **Die Form der Sprachgrenze hat praktische Folgen.** Der direkte Aufruf der C++-Library beschleunigt die sechs gemessenen Suchen gegenüber Python um den Faktor 6,36, während der frühere CLI-Mikrobenchmark hohe Kosten pro Prozessaufruf zeigte. Die Library hält den wiederholten Aufruf im Worker-Prozess und erzielt damit sowohl eine schmale Grenze als auch einen gemessenen Anwendungsvorteil (Abschnitt 8.10).

9. **Schnittstellenkosten gehören ins Kostenmodell.** Der direkte C++-Aufruf benötigt im Mittel 0,074 ms, wovon der triviale Library-Aufruf rund 0,018 ms als Größenordnung der Übergabe zeigt. Das Modell aus Satz 2 wird durch einen Term δ je Vergleich erweitert (Bemerkung 6); die Anbindung beeinflusst den konstanten Faktor, nicht die Asymptotik.

10. **Multi-Omics erfordert eine explizite Schichtsemantik.** Die kohärente und unabhängige Relation stellen unterschiedliche Anforderungen an gemeinsame Trefferpositionen. Beide Backends stimmten in 10.000 geprüften Modusentscheidungen und allen 336 gepaarten Suchläufen überein. C++ beschleunigte die Ende-zu-Ende-Suche aggregiert um den Faktor 2,26; der Suchphasenvergleich zeigt zugleich, dass nun Laden statt Berechnung den größten Anteil der C++-Antwortzeit ausmacht. Worker- und Chunk-Reihen sind abgeschlossen, die Datenbankgrößenreihe nicht. Synthetische Ground Truth ersetzt keine biologische Validierung (Abschnitt 9).

11. **Parallelisierung ist korrekt, aber durch Planung und serielle Phasen begrenzt.** Abschnitt 9.10 beweist unter expliziten Annahmen, dass die parallele Ausführung die sequentielle Trefferliste unabhängig von Worker-Zahl, Blockgröße und Fertigstellungsreihenfolge erhält. Die Schranke $V \leq W/p + (1 - 1/p)\theta_{\max}$ erklärt zugleich, warum die Lastverteilung und die längste Aufgabe zählen; eine absteigende Kandidatenreihenfolge verbessert die Schranke, ihr praktischer Nutzen ist aber noch ungemessen. Bei vier Workern wurden serielle Anteile von 19,7 % (Python) und 58,9 % (C++) ermittelt. Die Effizienz fällt nach Beschleunigung des Rechenkerns, weil Laden und Zuführung relativ stärker ins Gewicht fallen. Die Blockgrößenresultate (128 für Python, 32 für C++) und ihre moderaten Vorteile gegenüber 256 gelten nur für die untersuchten Läufe. Eine Parallelisierung innerhalb einzelner Schichten ist bewiesen modelliert, aber nicht implementiert; Threads für den Python-Kern bleiben im betrachteten GIL-Modell begrenzt.

12. **Die Art der Daten gehört in die Kodierung, nicht in den Kern.** Jede endliche attributierte relationale Struktur lässt sich verlustfrei und in fast linearer Zeit in einen Stapel von Mengenfolgen kodieren (Sätze 19 und 20). Vergleich, Index und deren Beweise sind für beliebige Stapel formuliert und bleiben beim Übergang zu einer neuen Strukturart unverändert (Korollar 13). Damit verallgemeinert dieser Schritt die zentrale Trennung der Arbeit: Wie die Vorauswahl vom Vergleich getrennt ist und die Sprachgrenze eine schmale Schnittstelle erhält, so ist nun die Struktur der Daten vom Kern getrennt.

13. **Optimalität und Effizienz beziehen sich auf die definierte Relation.** Der Vergleich zweier Stapel ist bis auf einen konstanten Faktor optimal (Sätze 21 und 22), und die Indexsuche liefert das exakte Ergebnis ohne Einzelvergleiche mit einem Aufwand, der von der Trefferlast und nicht von N abhängt (Sätze 23 und 24). Beides gilt für $\sqsubseteq$, nicht für die klassische Teilgraph-Isomorphie; diese lässt sich nach Bemerkung 21 nur dann exakt in Polynomialzeit abbilden, wenn P = NP gilt. Die Relation ist nicht transitiv, erlaubt keinen Kantenzahl-Filter, und ihre biologische Validität ist ungeprüft. Die Laufzeitaussagen sind Folgerungen und noch nicht gemessen.

14. **Beweis und Messung sind getrennte Aussagen und werden getrennt geprüft.** Die Laufzeitvorteile der Indexsuche folgen aus Sätzen über Rechenschritte; ob sie als Wanduhrzeit sichtbar werden, hängt von Trefferlast, Datenbankzugriffen, Prozesskommunikation und Speicherbedarf ab. Der in Abschnitt 11 ausgewertete Lauf bestätigt die differenzielle Korrektheit im Matrixschema und zeigt Vorteile in der Speicherreihe; der Lauf ist aber unvollständig und die Ende-zu-Ende-Suchreihe umfasst nur eine Wiederholung mit zwei Varianten. Die Aussage „unabhängig von N“ gilt nur unter der Häufigkeitsannahme; ihre Überprüfung in der Datenbank und mit vollständigen Wiederholungen steht noch aus.

12.1.14 Abgleich mit den Beiträgen der Arbeit

In der Einleitung wurden Beiträge zur Architektur, formalen Analyse des Suchverfahrens und Parallelisierung, Datenbankevaluation, zum VF2-Vergleich, zu den C++-Backends, zur Multi-Omics-Suche und zur universellen Kodierung angekündigt. Die Architektur und Einbettung in die relationale Datenbank werden in Abschnitt 3 beschrieben; Korrektheit und Komplexität des Verfahrens sind in Abschnitt 4 bewiesen, ausdrücklich für die Suchrichtung $A_Q \preceq A_k$ und unter Berücksichtigung eines optionalen Kandidatenlimits. Die Evaluation an einer Million synthetischen Netzwerken steht in Abschnitt 5; sie ist eine KI-gestützte, analytische Untersuchung ohne verlässlich dokumentierte Hardwarekonfiguration. VF2-Vergleich, empirische Parallelisierung und Backend-Vergleich finden sich in den Abschnitten 6, 7 und 8; die theoretische Vertiefung der Parallelisierung in Abschnitt 9.10 ergänzt diese Messungen um Ergebnisgleichheit, Laufzeitschranken, Engpassanalyse und daraus abgeleitete, teilweise noch zu prüfende Empfehlungen. Abschnitt 9 beschreibt die geschichtete Suchsemantik sowie die abgeschlossenen Korrektheits-, Mikrobenchmark-, Such-, Worker- und Chunk-Ergebnisse; allein die Datenbankgrößenreihe blieb unvollständig. Der Beitrag zur universellen Kodierung ist in Abschnitt 10 formal erfüllt: Kern, Kodierung, Verlustfreiheit, optimaler Vergleich, Index, Kernunabhängigkeit und die Grenzen der Aussagen sind definiert und bewiesen; die Python-Umsetzung und das Experiment liefern erste, aber wegen des unvollständigen Laufs noch nicht abschließende empirische Ergebnisse.

Der Backend-Beitrag ist durch Mikrobenchmark, abgeschlossene Ende-zu-Ende-Messung, Trefferabgleich und Schnittstellenanalyse erfüllt; die Reproduzierbarkeit und Aussagekraft bleiben durch synthetische Daten und eine einzelne Hardwarekonfiguration begrenzt. Zusammenfassung, Würdigung der Grenzen und Ausblick bilden dieses Kapitel. Dabei ist zu unterscheiden: Die Skalierungskurven in Abschnitt 5 beruhen auf dem analytischen Modell, während VF2-, Parallelisierungs-, Backend- und Multi-Omics-Werte aus Messungen stammen. Die Aussagen des Abschnitts 9.10 sind mathematische Aussagen innerhalb der jeweils genannten Annahmen und mit ausgewählten Messwerten abgeglichen; die Sätze des Abschnitts 10 sind bewiesen, ihre Laufzeitvorteile bislang nur teilweise experimentell geprüft.

12.2 Kritische Würdigung und Grenzen

Die Ergebnisse dieser Arbeit sind belastbar, soweit es um die formalen Aussagen geht. Für die empirischen Aussagen gelten die folgenden Einschränkungen, die bei der Interpretation berücksichtigt werden müssen.

1. **Die entschiedene Relation ist nicht die Subgraph-Isomorphie.** Die zyklische, auf den Zeilenkomponenten der Signaturen beruhende Relation $\preceq$ ist strukturell schwächer als die klassische Subgraph-Isomorphie. Treffer von Gen-DB sind daher Treffer im Sinne von $\preceq$ und nicht ohne Weiteres im Sinne der Isomorphie. Die Laufzeitvergleiche mit VF2 vergleichen deshalb Größenordnungen und nicht die Mächtigkeit der Relationen.

2. **Nur eine Suchrichtung ist abgesichert.** Die Vollständigkeitsaussage gilt für $A_Q \preceq A_k$. Für die umgekehrte Richtung fehlt eine bewiesene Vorauswahl. Für die Relation $\sqsubseteq$ aus Abschnitt 10 wird die Gegenrichtung über den Index der linearen Paare symmetrisch angegeben, aber weder umgesetzt noch gemessen.

3. **Kandidatenlimit und Implementierungsstand.** Die Analyse in Abschnitt 4 modelliert ein optionales Kandidatenlimit K, und das dort gezeigte SQL-Statement enthält eine `LIMIT`-Klausel. Die für die Messungen in Abschnitt 7 verwendete Programmfassung führt dagegen den Vergleich über alle Kandidaten aus. Beides ist mit den Sätzen vereinbar, die Textstellen sollten bei einer Überarbeitung jedoch auf einen einheitlichen Stand gebracht werden.

4. **Teilweise modellbasierte Auswertung.** Die Skalierungskurven in Abschnitt 5 beruhen auf dem analytischen Laufzeitmodell und nicht auf Messungen. Gemessen sind der Vergleich mit VF2 (Abschnitt 6) und die Parallelisierung (Abschnitt 7).

5. **Synthetische Daten.** Alle Netzwerke stammen aus dem Erdős–Rényi-Modell mit $p = 0{,}3$. Reale biologische Netzwerke sind häufig skalenfrei, haben hohe Clusterkoeffizienten und Hub-Knoten. Kandidatenzahlen, Trefferquoten und Kosten pro Vergleich können dort deutlich abweichen.

6. **Geringe Stichprobengröße.** Der Vergleich mit VF2 beruht auf 8 bzw. 5 Wiederholungen je Datenpunkt, die Parallelisierungsmessung auf 6 Anfragen mit je einer Messung. Konfidenzintervalle und Signifikanztests fehlen.

7. **Begrenzte Hardware und Stichprobe.** Die Parallelisierung wurde auf einem Rechner mit vier logischen Prozessoren gemessen, auf dem auch die Datenbank lief. Die Worker-Reihen umfassen wenige Anfragen und Wiederholungen; Aussagen zur Skalierung auf mehr Worker, andere Rechner oder getrennte Datenbankserver sind daher Extrapolationen.

8. **Ungenutzte Knotenlabels.** Die Relation $\preceq$ ignoriert Knotenlabels. Zwei strukturell gleiche, biologisch aber verschiedene Netzwerke gelten als äquivalent, was für die praktische Anwendung eine wesentliche Einschränkung ist.

9. **Referenzimplementierung ohne Optimierung.** Die Laufzeitwerte des Subgraph Algorithmus stammen aus einer in reinem Python geschriebenen Implementierung ohne vorzeitigen Abbruch. Sie sind daher eher eine obere Abschätzung des praktisch Erreichbaren.

10. **Begrenzte Backend-Messreihe.** Die C++-Library und die Ende-zu-Ende-Suche sind inzwischen gemessen (Abschnitt 8.5). Der Speedup von 6,36 und die übereinstimmenden Treffer gelten für sechs Queries, eine Messung je Backend und eine Windows-11-Konfiguration mit zwei Workern. Wiederholungen, Konfidenzintervalle, weitere Plattformen und reale biologische Daten fehlen.

11. **Phasenmodell und Schnittstellenkosten.** Die Messung zerlegt die Suchzeit in Vergleichsphase und Rest; der Rest fasst Datenbankzugriff, Datenübergabe, Aufgabenverteilung und Ergebnisaufbau zusammen. Die 0,018 ms für einen trivialen Library-Aufruf sind nur eine Orientierung für die Übergabekosten und dürfen nicht als exakte Korrektur auf größere Matrizen übertragen werden. Auch der frühere CLI-Wert von rund 10 ms ist eine Messung eines anderen Aufrufpfads und keine kontrollierte Schätzung für alle Plattformen.

12. **Die Kopplungsposition ist eine Gestaltungsposition.** Begriffe wie Trennungsbewusstsein lassen sich in dieser Arbeit nicht messen. Die Fallbeispiele aus dem Versuchsaufbau machen die Position plausibel, belegen sie aber nicht durch eine Studie. Die Gegenargumente (Isolation, Sprachneutralität, Bauaufwand der Library) sind nicht entkräftet, sondern nur dem Einsatzzweck zugeordnet.

13. **Betriebssystemabhängigkeit der C++-Anbindung.** Die Library-Messung stammt von einem Windows-11-Rechner. Bau, Linken, DLL-Laden und Wrapper-Aufruf können sich auf anderen Betriebssystemen unterscheiden. Der strukturelle Unterschied zum CLI-Prozessstart bleibt bestehen, aber die gemessene Größenordnung des Beschleunigungsgewinns ist nicht ohne plattformübergreifende Wiederholung übertragbar.

14. **Synthetische Multi-Omics-Daten und begrenzte Skalierungsstichprobe.** Korrektheits-, Mikrobenchmark-, Such-, Worker- und Chunk-Messungen sind abgeschlossen; die Datenbankgrößenreihe wurde nach 58 von 96 Messungen abgebrochen. Die Such- und Skalierungsmessungen stammen von einer einzelnen Windows-11-Konfiguration mit vier physischen Kernen; die Worker- und Chunk-Vergleiche verwenden sechs Queries und zwei Wiederholungen. Die Netzwerke teilen eine Knotenzahl über alle Schichten und enthalten keine realen oder fachlich annotierten Omics-Beziehungen. Der Such-Speedup von 2,26 und der Mikrobenchmark-Speedup von

8,11× belegen daher keine allgemeine Beschleunigung oder biologische Validität außerhalb dieser Versuchsanordnung.

15. **Universelle Kodierung: bewiesen, umgesetzt, Messung festgelegt.** Abschnitt 10 enthält ausschließlich formale Aussagen. Die Umsetzung in Python (Kern, Kodierung für Matrixschemas, Paarindex als SQL-Tabelle) wird durch das Experiment in Abschnitt 11 geprüft. Solange kein dokumentierter Lauf vorliegt, sind die Laufzeitaussagen (insbesondere die Unabhängigkeit von N und der Wegfall des Ladens aller Kandidaten) Folgerungen aus den Sätzen und keine Messungen; der Vergleich mit der bisherigen Pipeline und der Multi-Omics-Messung bleibt dann ein Vergleich von Schranken mit Messwerten. Die Unabhängigkeit von N setzt eine Häufigkeitsannahme voraus (Korollar 12), die für reale Netzwerke mit Hubs und häufigen Motiven nicht gelten muss.

16. **Reichweite des Experiments zur universellen Kodierung.** Das Experiment prüft ein Matrixschema mit fünf Schichten und damit nur den Spezialfall, in dem die Relation mit der Multi-Omics-Relation übereinstimmt (Bemerkung 14). Schemata für Hypergraphen, Pfade, Ontologien oder mehrstellige Reaktionen sind nicht Teil der Messung; die Kernunabhängigkeit (Korollar 13) ist dadurch bewiesen, aber nicht an weiteren Strukturarten gemessen. Die Daten sind synthetisch (feste Kantenwahrscheinlichkeit je Schicht, höchstens 40 Knoten, 20.000 Netzwerke, also weit unter den 10^6 Netzwerken der Evaluation), die Wiederholungen gering (drei Suchwiederholungen, zwei bei den Teilexperimenten zu Worker, Chunk und Datenbankgröße), die Exponenten werden aus vier bis fünf Stufen geschätzt, und die Hardwarekonfiguration ist eine einzelne. Die Ground Truth stammt aus konstruierten Einbettungen und ersetzt keine biologische Validierung. Die Varianten mit Prozess-Pool und die Indexsuche haben verschiedene Betriebsmodi; der gemessene Vorteil der Indexsuche enthält auch, dass sie keine Prozesse kommunizieren lässt, und ist nicht allein dem Algorithmus zuzurechnen. Schließlich sind `mo_python` und `cpp` durch fremde Multi-Omics-Netzwerke in der Datenbank beeinflussbar, was das Skript als Warnung vermerkt.

17. **Die Optimalität gilt für die definierte Relation, nicht für die Subgraph-Isomorphie.** Nach Bemerkung 21 kann es keine in Polynomialzeit berechenbare Abbildung auf Stapel geben, die die Subgraph-Isomorphie exakt als $\sqsubseteq$ wiedergibt, solange P $\neq$ NP gilt; $\sqsubseteq_{\text{ind}}$ und $\sqsubseteq_{\text{coh}}$ sind daher Näherungs- oder Filterrelationen. Die untere Schranke $\Omega(m + m')$ bezieht sich auf das Zugriffsmodell mit internierten Komponenten und auf festes L; für wachsendes L bleibt zwischen $\Omega(|M| + |M'|)$ und $O(L(|M| + |M'|))$ eine Lücke. Die Obergrenze im erwarteten Fall beruht auf Hashing; deterministisch gilt sie nur bis auf einen logarithmischen Faktor.

18. **Eigenschaften der Relation schränken Verfahren der früheren Abschnitte ein.** Die Relation ist nicht transitiv, sodass die Treffertypen paarweise und nicht als Halbordnung zu verstehen sind. Der Kantenzahl-Filter der Vorauswahl ist für $\sqsubseteq$ nicht zulässig (Lemma 7); die in Abschnitt 4 bewiesene Korrektheit der Vorauswahl überträgt sich daher nicht auf den Kern, sondern nur die Schranke der Knotenzahl beziehungsweise Schichtlänge. Die Beweise zu $\preceq$ in Abschnitt 4 bleiben für die dort definierte Relation unberührt.

19. **Die fachliche Validität der Kodierung ist nicht belegt.** Ob zwei zyklisch

benachbarte Spalten mit gleichen Nachbarschaftsmengen eine biologisch sinnvolle Ähnlichkeit darstellen, hängt von Schema, Spaltenordnung und Koordinatensystem ab (Bemerkung 22). Die Ordnung der Entitäten nach κ und der Relationsknoten nach Typ und lexikographischer Ordnung ist eine Konstruktionsentscheidung, deren Einfluss auf die Treffer nicht untersucht ist. Kontinuierliche Werte müssen quantisiert werden; die Verlustfreiheit gilt nur bezüglich der quantisierten Struktur.

20. **Betriebsfragen der Kodierung sind offen.** Eine Schemaänderung erfordert eine Neukodierung aller vorhandenen Einträge in $O(|S| \log |S|)$ je Struktur oder ein eigenes Schema mit eigenem Index. Die Größe des Index (bis zu $L\,|M_k|$ zyklische und $L\,(|M_k| - 1)$ lineare Paare je Struktur), die Pflege bei vielen Änderungen und die Wahl von Register und Wörterbüchern in der Datenbank sind nicht untersucht. Die native Bibliothek unterstützt weiterhin nur Koordinaten in 64 Bit.

12.3 Ausblick

Die Erweiterungsmöglichkeiten lassen sich in neun Themenfelder ordnen: die Absicherung von Korrektheit und Suchsemantik, die Verschärfung der Vorauswahl und Indexierung, die Beschleunigung der Vergleiche, die Skalierung auf Systemebene, die Weiterentwicklung der Evaluation, die Anwendungsebene, die Kopplung und Schnittstellen der Implementierungen, die Validierung und Skalierung der Multi-Omics-Suche sowie die Umsetzung, Messung und Erweiterung der universellen Kodierung mit Indexsuche aus Abschnitt 10 (Themenfeld I). Die Theorie der Parallelisierung in Abschnitt 9.10 verändert dabei den Schwerpunkt mehrerer Vorschläge: Statt allein eine größere Worker-Zahl anzustreben, sind zunächst serielle Phasen, Lastverteilung, Blockgröße und Voraussetzungen der Messungen zu prüfen. Die folgenden Unterabschnitte behandeln jedes Feld einzeln und verweisen darauf, ob eine Maßnahme bewiesen motiviert, bereits gemessen oder noch eine offene Hypothese ist. Mehrere ältere Vorschläge, etwa die beschriftete Relation und die Label-Indizes (Themenfelder A und B), lassen sich formal als zusätzliche Schichten und Indexeinträge der Kodierung ausdrücken; ob die Semantik dabei erhalten bleibt, wird im Themenfeld I behandelt.

Themenfeld A: Korrektheit und Suchsemantik

12.3.1 Vollständige Suche in beide Richtungen

Die Pipeline findet bisher nur Netzwerke, die das Query enthalten. Für die umgekehrte Richtung, bei der ein gespeichertes Netzwerk Teil des Querys ist, lässt sich dieselbe Argumentation wie in Lemma 2 anwenden: Aus $A_k \preceq A_Q$ folgt $n_k \le n_Q$ und $m_k \le m_Q$. Die zugehörige Kandidatenmenge ist

$$\mathcal{C}'(Q, \mathcal{D}) = \{(id_k, L_k, A_k) \in \mathcal{D} : n_k \le n_Q \wedge m_k \le m_Q\},$$

und eine vollständige Suche in beide Richtungen verarbeitet die Vereinigung $\mathcal{C}(Q, \mathcal{D}) \cup \mathcal{C}'(Q, \mathcal{D})$. Die beiden Mengen überschneiden sich genau in den Netzwerken mit $n_k = n_Q$ und $m_k = m_Q$. Für die Praxis ist wichtig, dass $\mathcal{C}'$ bei großen Queries sehr groß wird: Für $n_Q = 20$ erfüllt fast jedes Netzwerk die Knotenbedingung $n_k \le 20$. Eine Suche in Gegenrichtung ist deshalb nur mit einer Parallelisierung und mit einer Ergebnisbegrenzung sinnvoll und sollte in API und Oberfläche als eigener Suchmodus gekennzeichnet werden. Ob die Gegenrichtung für biologische Anwendungen überhaupt gebraucht wird (die Frage

lautet dann: „Welche gespeicherten Motive stecken in meinem großen Netzwerk?“), ist eine fachliche Frage, die mit Anwendern geklärt werden sollte.

12.3.2 Differenzielle Tests der Vorauswahl und Eigenschaftstests

Die Korrektheit des Pre-Filters wurde bewiesen, aber nicht automatisiert geprüft. Ein wirksamer Test vergleicht auf kleinen Datenbanken (etwa tausend zufällige Netzwerke) die Ergebnisse der Pipeline mit einer vollständigen Suche ohne Vorauswahl, bei der jedes Netzwerk mit dem Subgraph Algorithmus verglichen wird. Beide Ergebnismengen müssen für die Richtung $A_Q \preceq A_k$ identisch sein. Ergänzend können Eigenschaften, die aus der Definition der Relation folgen, als Eigenschaftstests (*property-based tests*) formuliert werden, etwa dass jedes Netzwerk Subgraph von sich selbst ist und dass der Vergleich nicht von der zyklischen Anordnung der Zeilenkomponenten abhängt. Solche Tests würden auch jede spätere Optimierung des Vergleichs (Abschnitt 12.3.9) absichern.

12.3.3 Erweiterung auf gewichtete und beschriftete Subgraph-Isomorphie

Die aktuelle Subgraph-Relation $\preceq$ berücksichtigt ausschließlich die *Struktur* (Adjazenzmatrix), nicht jedoch die *Knotenlabels* L (z. B. `Glucose`, `p53`). Zwei strukturell identische, aber biologisch unterschiedliche Netzwerke (z. B. Glykolyse vs. ein Zufallsgraph gleicher Adjazenzmatrix) würden aktuell als äquivalent erkannt. Eine biologisch sinnvolle Erweiterung wäre eine *gefärbte* Subgraph-Relation

$$(L_Q, A_Q) \preceq_\ell (L_k, A_k) \iff A_Q \preceq A_k \wedge \exists\,\text{Knotenabbildung } \phi : L_Q(i) = L_k(\phi(i))\ \forall i,$$

deren Prüfung zusätzlich eine Label-Kompatibilitätsprüfung in $O(n)$ pro Rotation erfordert (Gesamtkomplexität bleibt $O(n^3)$, da $O(n) \subseteq O(n^2)$). Für kontinuierliche Kantengewichte $w : E \to \mathbb{R}_{\geq 0}$ (z. B. Reaktionsraten) wäre die binäre Signatur $\sigma_j = \sum_i A_{ij} 2^i + j 2^n$ durch eine gewichtete Variante $\sigma_j^w = \sum_i w_{ij} 2^i + j 2^n$ zu ersetzen, wobei die Injektivität aus Lemma 1 nur noch für endliche Wertebereiche $w_{ij} \in \{0, \ldots, W\}$ mit Basis $W + 1$ statt 2 erhalten bliebe. Die universelle Kodierung aus Abschnitt 10 deckt beide Wünsche formal ab: Knotenmerkmale erhalten je eine Schicht Lab_a, und Gewichtsstufen werden als Schichten $\mathrm{Bin}_{t,r}$ kodiert, ohne dass die Signatur geändert werden muss. Ob die so entstehende Relation die hier gemeinte gefärbte Relation mit Knotenabbildung ϕ erfüllt, ist jedoch nicht gezeigt, denn $\sqsubseteq$ verlangt keine Knotenabbildung, sondern gemeinsame Paare benachbarter Spalten (Themenfeld I).

Themenfeld B: Vorauswahl und Indexierung

12.3.4 Indexierung über Signatur-Hashes

Aktuell wird der Signatur-Hash $h(A)$ zwar berechnet und gespeichert (Spalte `signature_hash`, indiziert über `idx_matrices_hash`), jedoch nicht für die Subgraph-Suche genutzt — er eignet sich nur für *exakte* Übereinstimmungen (inklusive Rotation, falls die Hashfunktion rotationsinvariant gestaltet wird). Eine Erweiterung bestünde darin, einen *rotationsinvarianten* Hash

$$h^*(A) = \min_{r \in \{0,\ldots,n-1\}} \mathrm{SHA256}\Big(\mathrm{str}(\mathrm{rot}_r(\rho(A)))\Big)$$

zu definieren und zu indizieren. Damit ließen sich `exact`-Treffer (Korollar zu Satz 3 für $A_Q \equiv A_k$ bis auf Rotation) in $O(\log N)$ statt $O(C \cdot n^3)$ finden, indem zunächst eine Hash-Gleichheitssuche durchgeführt wird und der teure Subgraph-Vergleich nur für `subgraph`-Treffer (echte Teilstruktur) verbleibt. Die Korrektheit dieser Optimierung folgt aus der Tatsache, dass $h^*(A_Q) = h^*(A_k) \Rightarrow A_Q \preceq A_k \wedge A_k \preceq A_Q$ (Äquivalenz unter Rotation), was eine direkte Anwendung von Lemma 1 auf die Zeilenkomponenten ist.

12.3.5 Mehrdimensionale Indexstrukturen (R-Baum über (n_k, m_k))

Der aktuelle Pre-Filter nutzt zwei unabhängige B-Baum-Indizes (`node_count`, `edge_count`) und kombiniert sie via konjunktiver `WHERE`-Klausel, was PostgreSQL typischerweise durch Bitmap-Index-Scans mit anschließendem `AND` realisiert. Eine direktere Lösung wäre ein zweidimensionaler Index (z. B. GiST/R-Baum über das Tupel (n_k, m_k) als Punkt im $\mathbb{R}^2$), der die Bedingung $n_k \geq n_Q \wedge m_k \geq m_Q$ als achsenparallele Bereichsabfrage ("dominance query") in $O(\log N + C)$ statt der aktuellen Bitmap-Kombination beantwortet. Formal entspricht $\mathcal{C}(Q, \mathcal{D})$ exakt der Menge der Punkte (n_k, m_k), die den Punkt (n_Q, m_Q) im Sinne der Pareto-Dominanz dominieren — ein klassisches Problem der rechnerischen Geometrie, für das R-Bäume und k-d-Bäume $O(\log N)$-Anfragezeiten (amortisiert) erreichen.

12.3.6 Zusätzliche notwendige Bedingungen und Label-Indizes

Die Vorauswahl nutzt derzeit nur Knoten- und Kantenzahl. Jede weitere Größe, die sich billig aus der Datenbank lesen lässt und für $A_Q \preceq A_k$ notwendig ist, würde den Suchraum weiter verkleinern. Solche Bedingungen müssen vor ihrer Verwendung aus der Definition von $\preceq$ bewiesen werden, denn eine zu scharfe Bedingung würde echte Treffer ausschließen. Bei Einführung der beschrifteten Relation $\preceq_\ell$ (Themenfeld A) ist dagegen eine sehr wirksame Bedingung unmittelbar ableitbar: Das Multiset der Knotenlabels des Querys muss im Multiset der Labels des Kandidaten enthalten sein. PostgreSQL unterstützt für Array-Spalten wie `node_labels` GIN-Indizes mit dem Enthaltenseins-Operator `@>` [4], sodass diese Bedingung direkt im Index ausgewertet werden könnte. Biologische Queries enthalten meist seltene Labels (etwa `p53`), wodurch die Kandidatenmenge in diesem Fall um Größenordnungen kleiner würde als bei der reinen Zählung von Knoten und Kanten. Einfacher umzusetzen ist ein zusammengesetzter Index über (`node_count`, `edge_count`), der die beiden bisher getrennten B-Baum-Indizes ersetzt, bevor der aufwendigere R-Baum-Ansatz nötig wird.

Themenfeld C: Beschleunigung der Vergleiche

12.3.7 Weiterführende Parallelisierung

Die Parallelisierung der Kandidatenvergleiche ist in Gen-DB umgesetzt und in Abschnitt 7 vermessen. Abschnitt 9.10 präzisiert das frühere grobe Laufzeitmodell: Die Gesamtzeit umfasst nicht nur die über p Worker verteilten Vergleiche, sondern auch Laden, Zuführung und Ergebnisaufbau; für die Vergleichsphase selbst sind zusätzlich Aufgabenlängen,

Blockbildung und Reihenfolge entscheidend. Das vereinfachte Modell

$$T_{\text{parallel}}(C, n, p) = \underbrace{O(C \cdot n^2)}_{\text{Laden, Übertragung, Ergebnisaufbau (seriell)}} + \underbrace{O\left(\frac{C}{p} \cdot n^3\right)}_{\text{Vergleiche}} + O(n^3),$$

veranschaulicht, warum der serielle Anteil nach dem Gesetz von Amdahl [15] den erreichbaren Speedup auf etwa $1/f$ begrenzt. Die detailliertere Auswertung in Abschnitt 9.10 misst bei vier Workern einen seriellen Anteil von 19,7 % für Python und 58,9 % für C++; der frühere Schätzwert von etwa 5 % ist daher nicht als allgemeine Systemkonstante zu verwenden. Die Zahlen hängen von Backend, Hardware und Phasengrenzen ab. Folgende Maßnahmen setzen an den jeweils nachgewiesenen Kostenblöcken an:

- **Strömendes Laden.** Die Kandidaten werden derzeit mit einem einzigen Abruf vollständig in den Speicher gelesen. Ein serverseitiger Cursor würde sie in Blöcken liefern, sodass das Laden mit den Vergleichen überlappt und der Speicherbedarf bei $C \approx 3 \cdot 10^5$ sinkt.

- **Query nur einmal pro Task übertragen.** Die Query-Matrix wird gegenwärtig für jeden Kandidaten mitgeführt; bei einem Block von 256 Vergleichen wird sie 256-mal serialisiert. Eine Übergabe pro Block oder pro Worker (etwa über einen Initialisierer des Prozess-Pools bei Beginn der Suche) würde den Kommunikationsaufwand senken.

- **Blockgröße anpassen.** Die Messungen in Abschnitt 9.10 fanden 128 für Python und 32 für C++ günstiger als den Standardwert 256; der Vorteil war moderat und beruht auf sechs Queries mit zwei Wiederholungen. Die Modellformel für b^* erklärt, warum das Optimum mit Übergabeaufwand, Worker-Zahl und Vergleichsdauer variiert. Daher sind diese Werte als Suchbereich für Wiederholungen auf dem Zielsystem zu behandeln, nicht als feste Voreinstellung für alle Installationen. Adaptive kleinere Blöcke am Ende könnten zusätzlich den Leerlauf reduzieren, müssen aber separat gemessen werden.

- **Worker-Zahl an die nutzbare Hardware anpassen.** Auf der dokumentierten Vierkernmaschine war die Dreierkonfiguration im früheren Python-Experiment langsamer als die Zweierkonfiguration; in der späteren Multi-Omics-Reihe brachte ein dritter Worker bei Python noch Gewinn, beim C++-Backend dagegen kaum. Da beide Versuche höchstens vier Worker prüften, lässt sich daraus keine allgemeine Regel für die Überbelegung ableiten. Die Konfiguration sollte eine hardwarebezogene Obergrenze berücksichtigen und zugleich Hauptprozess, Datenbankserver und konkurrierende Suchanfragen einplanen. Eine bloße Bindung an `os.cpu_count()` reicht nicht für eine allgemeine optimale Einstellung; Backend und tatsächliche Ressourcenteilung müssen einbezogen werden.

- **Gleichzeitige Suchanfragen.** Der Prozess-Pool wird von allen Anfragen gemeinsam genutzt. Laufen mehrere lange Suchen gleichzeitig, konkurrieren sie um dieselben Worker. Eine Warteschlange mit Priorisierung kurzer Suchen oder ein Limit gleichzeitiger Suchen würde die Antwortzeit kleiner Anfragen schützen.

Für sehr große Kandidatenmengen oder viele Nutzer kommt zusätzlich die Verteilung über mehrere Rechner in Betracht, etwa mit Apache Spark und einer benutzerdefinierten

Funktion für `compare_graphs`. Dabei muss der Kommunikationsaufwand für den Versand der Adjazenzmatrizen (je $O(n^2)$ Bit) gegen den Rechengewinn $O(n^3/p)$ abgewogen werden. Für $n \leq 20$ stehen $n^2 = 400$ Bit nur $n^3 = 8000$ Operationen gegenüber, sodass sich die Verteilung bei $p \ll C$ lohnt. Eine solche Verteilung führt allerdings eine weitere Beschreibung in das System ein; ob sie diesen Bruch rechtfertigt, ist nach den Prüffragen in Abschnitt 12.3.30 zu klären. Das Gesetz von Gustafson [16] legt zudem nahe, bei mehr Prozessoren auch die Problemgröße zu steigern, also etwa größere Netzwerke oder Datenbanken zu bearbeiten, anstatt nur dieselbe Suche zu beschleunigen.

12.3.8 Theoriegeleitete Weiterentwicklung der parallelen Suche

Die Ergebnisse aus Abschnitt 9.10 legen eine Reihenfolge für weitere Arbeiten fest. Zuerst ist die Messung belastbarer zu machen: Die dort ausgewerteten Worker- und Chunk-Reihen beruhen auf sechs Queries und zwei Wiederholungen. Mindestens fünf Wiederholungen je Konfiguration und zusätzliche Queries mit unterschiedlichen Knotenzahlen, Dichten und Kandidatenmengen sind nötig, bevor aus den günstigen Blockgrößen oder einem theoretischen Sortiervorteil eine allgemeine Einstellung abgeleitet wird. Zugleich sind Laden, Zuführung beziehungsweise Serialisierung und Worker-Rechenzeit getrennt zu messen. Die bisherige serielle Phase fasst mehrere Vorgänge zusammen; ohne ihre Aufteilung lässt sich weder die Amdahl-Grenze einer einzelnen Optimierung zuordnen noch sicher entscheiden, ob strömendes Laden, weniger Datenkopien oder eine andere Datenbankanbindung zuerst umgesetzt werden sollte.

Zweitens sollten die Modellaussagen ausdrücklich als falsifizierbare Messfragen behandelt werden. Eine Wiederholung mit mindestens fünf Läufen je Konfiguration auf der dokumentierten Vierkernmaschine soll zunächst die bisherige Stichprobe absichern; eine zusätzliche Reihe auf Rechnern mit mindestens acht Kernen prüft, wie sich die Skalierung dort verhält. Das Amdahl-Modell sagt voraus, dass bei gleichbleibenden Annahmen der Speedup innerhalb der aus f abgeleiteten Grenze bleibt und der serielle Anteil weitgehend unabhängig von p ist. Eine Wiederholung der Chunk-Reihe prüft, ob sich das flache Optimum im modellierten Bereich bestätigt. Kandidaten sind zusätzlich einmal in der bisherigen aufsteigenden und einmal in absteigender Reihenfolge ihres geschätzten Vergleichsaufwands zu verarbeiten: Die Theorie verbessert für die absteigende Reihenfolge die Schranke, doch ein praktischer Sortierungsvorteil ist in Abschnitt 9.10 ausdrücklich noch nicht gemessen. Alle Vergleiche müssen dieselben Queries, Kandidaten und Trefferbedingungen verwenden, damit Änderungen der Trefferlast nicht als Planungseffekt fehlinterpretiert werden.

Drittens folgt aus der Engpassanalyse eine klare Priorisierung. Beim Python-Backend kann die Parallelisierung der Vergleiche weiterhin einen wesentlichen Anteil der Zeit beeinflussen. Beim schnelleren C++-Backend ist dagegen der serielle Anteil größer; dort ist zuerst die Lade- und Zubringerphase zu optimieren. Diese Folgerung ist keine Aussage, dass eine Backend-Sprache grundsätzlich schlechter parallelisiere, sondern ergibt sich aus den gemessenen Phasenanteilen und dem dadurch kleineren Parallelüberschuss. Eine Implementierung sollte deshalb weder die Zahl der Worker noch eine Blockgröße isoliert optimieren, sondern Gesamtzeit, Phasenanteile und Effizienz gemeinsam berichten.

Eine vierte, nachrangige Forschungsrichtung ist Parallelität innerhalb eines einzelnen Vergleichs. Abschnitt 9.10.9 zeigt, dass die Schichten prinzipiell als unabhängige Teilaufgaben behandelt werden können; die anschließende Zusammenführung erfordert je nach kohärentem oder unabhängigem Modus unterschiedliche Operationen. Der mögliche Parallelitätsgrad ist jedoch durch die Zahl der Schichten begrenzt, und die aktuelle Python-

Implementierung berechnet Teile der Schichtrepräsentation bereits vor der Paarprüfung. Eine solche Änderung sollte deshalb erst dann prototypisiert werden, wenn Messungen belegen, dass einzelne Queries zu wenige Kandidaten für die vorhandene Parallelität über Kandidaten bieten. Für den Python-Kern ist außerdem die Global-Interpreter-Lock-Grenze zu berücksichtigen; Threads sind nur dann eine plausible Alternative, wenn die rechenintensive native Routine die Sperre freigibt oder der Kern entsprechend geändert wird.

Schließlich müssen Annahmen und Fehlerfälle als Teil der Implementierung überprüfbar bleiben. Gemeinsame Konformitätstests sollten A1 prüfen, indem sie für dieselben Kandidaten und Queries bei mehreren Worker-Zahlen, Blockgrößen und Fertigstellungsreihenfolgen identische Entscheidungen und Trefferlisten verlangen. Für A2 reichen reine Endzeiten nicht: Zu protokollierende Freigabe-, Start- und Endzeiten der Blöcke sowie Leerlaufintervalle würden zeigen, ob die tatsächliche Zuweisung dem nicht-leerlaufenden FIFO-Modell entspricht und ob die gemessene Makespan mit den hergeleiteten Schranken vereinbar ist. Abweichungen wären Anlass, entweder die Annahmen oder das Kostenmodell zu überarbeiten, nicht die Schranke ungeprüft auf den laufenden Prozess-Pool zu übertragen. Ausfälle eines Workers, Speichermangel und abgebrochene Aufgaben sind als Fehler auszuweisen und dürfen nicht wie leere oder erfolgreiche Trefferlisten erscheinen. Damit werden die Modellannahmen aus Abschnitt 9.10 mit den in Themenfeld G vorgeschlagenen Schnittstellen- und Backendtests verbunden. Erst nach dieser Absicherung sind automatische Anpassungen von Worker-Zahl oder Blockgröße für den produktiven Einsatz sinnvoll.

12.3.9 Optimierung der Implementierung des Vergleichs

Die Ursachenanalyse in Abschnitt 6.3 hat gezeigt, dass die Referenzimplementierung des Subgraph Algorithmus die Tabelle der längsten gemeinsamen Teilsequenz für jede Rotation vollständig aufbaut und in reinem Python arbeitet. Daraus ergeben sich drei Hebel, die die asymptotische Schranke $O(n^3)$ nicht verändern, die gemessene Zeit aber erheblich senken können. Erstens kann die Suche über die Rotationen abgebrochen werden, sobald eine Rotation die Bedingung erfüllt; im günstigen Fall genügt dann eine einzige Tabelle. Zweitens lässt sich die Tabelle zeilenweise abbrechen, wenn bereits feststeht, dass die geforderte Mindestlänge nicht mehr erreichbar ist. Drittens kann die innere Schleife mit `numpy` vektorisiert oder mit `numba` bzw. als C++-Erweiterung kompiliert werden. Die Wahl der Anbindung ist beim C++-Weg keine Nebensache: Abschnitt 12.3.24 begründet, weshalb eine Library der CLI vorzuziehen ist. Da der Vergleich nach der Abschätzung in Abschnitt 7 etwa 95 % der Laufzeit ausmacht, wirkt jede Beschleunigung des Einzelvergleichs fast vollständig auf die Gesamtzeit durch; zusammen mit den Prozessen multiplizieren sich die Gewinne. Diese Maßnahmen müssen allerdings durch die differenziellen Tests aus Abschnitt 12.3.1 abgesichert werden, damit die Optimierung das Ergebnis nicht verändert.

12.3.10 GPU-Beschleunigung der Signatur- und LCS-Berechnung

Die Signaturberechnung (Schritt 1 des Subgraph Algorithmus) ist eine Matrix-Vektor-Operation der Form $\sigma_j = \sum_i A_{ij} 2^i + j 2^n$, die sich als gewichtete Spaltensummen mit Bitmasken-Gewichten 2^i darstellen lässt — eine klassische BLAS-Level-2-Operation, die auf GPUs (CUDA, via CuPy oder PyTorch) hochparallel ausgeführt werden kann. Für

C Kandidaten mit $n \times n$-Matrizen ergibt sich ein Tensor der Form (C, n, n), dessen Signaturen in einem einzigen GPU-Kernel-Aufruf berechnet werden könnten, was Schritt 1 von $O(C \cdot n^2)$ sequentiellen auf $O(C \cdot n^2 / P_{\text{GPU}})$ effektiv parallele Operationen reduziert (P_{GPU} = Anzahl CUDA-Kerne). Die LCS-Berechnung (Schritt 3) ist aufgrund der sequentiellen Abhängigkeiten der DP-Tabelle ($dp[i][j]$ hängt von $dp[i-1][j-1]$ ab) schwieriger zu parallelisieren, jedoch existieren Wellenfront-Parallelisierungen (Anti-Diagonalen der DP-Tabelle sind unabhängig berechenbar), die auf GPUs in $O(n)$ Schritten mit $O(n)$ Prozessoren statt $O(n^2)$ sequentiellen Schritten realisierbar wären (Brent-Theorem).

12.3.11 Hybridstrategie: Adaptive Verfahrenswahl nach Kandidatengröße

Die in Abschnitt 6 beobachtete Umkehrung des Speedup-Verhältnisses bei $n_B \approx 14$ (Tabelle 6.1, Abbildung 6.2) motiviert eine *adaptive Hybridstrategie*: Da sowohl n_A (Query-Größe n_Q) als auch n_B (Kandidatengröße n_k) zum Zeitpunkt des Vergleichs bekannt sind (sie sind Teil des Tupels $(id_k, L_k, A_k, n_k, m_k) \in \mathcal{C}(Q, \mathcal{D})$, vgl. Abschnitt 3), könnte die Vergleichsfunktion `compare_graphs` zur Laufzeit zwischen dem Subgraph Algorithmus und VF2 umschalten:

$$\text{Verfahren}(n_Q, n_k) = \begin{cases} \text{Subgraph Algorithmus} & \text{falls } \max(n_Q, n_k) < \theta \\ \text{VF2} & \text{falls } \max(n_Q, n_k) \geq \theta \end{cases}$$

mit einem aus den Messdaten von Abschnitt 6 kalibrierten Schwellwert $\theta \approx 14$.

Satz 25 (Optimalität der Hybridstrategie auf Messbasis)**.** Sei $T_{\text{Sub}}(n)$ und $T_{\text{VF2}}(n)$ die (für eine feste Graphenfamilie $G(n,p)$ erwartete) Laufzeit beider Verfahren. Definiere $T_{\text{Hybrid}}(n) = \min(T_{\text{Sub}}(n), T_{\text{VF2}}(n))$. Dann gilt für jede Folge von Vergleichsanfragen $(n^{(1)}, \ldots, n^{(C)})$:

$$\sum_{i=1}^{C} T_{\text{Hybrid}}(n^{(i)}) \leq \min\left(\sum_{i=1}^{C} T_{\text{Sub}}(n^{(i)}),\ \sum_{i=1}^{C} T_{\text{VF2}}(n^{(i)}) \right).$$

Beweis. Da $T_{\text{Hybrid}}(n^{(i)}) \leq T_{\text{Sub}}(n^{(i)})$ und $T_{\text{Hybrid}}(n^{(i)}) \leq T_{\text{VF2}}(n^{(i)})$ für jedes i nach Definition des Minimums gilt, folgt durch Summation über $i = 1, \ldots, C$ sofort $\sum_i T_{\text{Hybrid}}(n^{(i)}) \leq \sum_i T_{\text{Sub}}(n^{(i)})$ und ebenso $\leq \sum_i T_{\text{VF2}}(n^{(i)})$, also auch $\leq$ dem Minimum beider Summen. □

Bemerkung 23. Satz 25 ist im strengen Sinne trivial (eine punktweise Minimum-Strategie ist niemals schlechter als jede ihrer Komponenten), seine praktische Bedeutung liegt jedoch darin, dass er die Hybridstrategie als *strikt dominant* gegenüber jeder festen Verfahrenswahl ausweist — *vorausgesetzt*, der Umschalt-Overhead (Bestimmung von $\max(n_Q, n_k)$ und Verzweigung) ist $O(1)$, was hier der Fall ist, da n_Q und n_k bereits in $\mathcal{D}_t$ materialisiert vorliegen (Abschnitt 3). Die Schranke $O(n^3)$ aus Satz 2 bleibt für die Hybridstrategie *nicht* formal erhalten, da T_{VF2} keine polynomielle Worst-Case-Schranke besitzt; für $n \geq \theta$ würde die Pipeline daher weiterhin den Subgraph Algorithmus als *Fallback mit garantierter Schranke* benötigen, etwa mit einem Zeitbudget τ: Falls VF2 nach τ nicht terminiert hat, wird auf den Subgraph Algorithmus zurückgegriffen (*Race-Verfahren*). Dies verbindet die empirische Average-Case-Effizienz von VF2 (Abschnitt 6) mit der formalen Worst-Case-Garantie des Subgraph Algorithmus.

Themenfeld D: Skalierung auf Systemebene

12.3.12 Asynchrone Suche mit Fortschrittsanzeige und Teilergebnissen

Eine Suche mit mehreren hunderttausend Kandidaten dauert Minuten (bis zu 403 Sekunden in Abschnitt 7) und blockiert den HTTP-Aufruf bis zum Ende. Für interaktive Nutzung eignet sich ein asynchrones Modell: Die API nimmt die Suche an, liefert sofort eine Kennung zurück und stellt den Fortschritt (verglichene Kandidaten von C) sowie die bisher gefundenen Treffer über Abfrage, Server-Sent Events oder WebSockets bereit. Der Prozess-Pool liefert Ergebnisse bisher in der Reihenfolge der Kandidaten; mit `as_completed` oder blockweiser Ausgabe könnten Treffer sofort sichtbar werden. Da die Kandidaten nach Knotenzahl sortiert sind, ließe sich zusätzlich ein „Anytime"-Verhalten erreichen: Der Nutzer sieht früh erste Treffer und kann die Suche abbrechen. Das senkt die wahrgenommene Antwortzeit unabhängig von der tatsächlichen Gesamtzeit und ist ohne Änderung des Algorithmus möglich.

12.3.13 Approximative Suche und geschichtete Stichproben

Die Beschränkung $K = 10.000$ (Abschnitt 4) gefährdet die Vollständigkeitsgarantie aus Satz 3, falls $|\mathcal{C}(Q, \mathcal{D})| > K$. Eine Abmilderung wäre eine *geschichtete Stichprobe*: Partitioniere $\mathcal{C}(Q, \mathcal{D})$ in Schichten nach n_k (also $\mathcal{C}_3, \ldots, \mathcal{C}_{20}$) und ziehe aus jeder nichtleeren Schicht $\mathcal{C}_i$ mindestens $\lceil K/18 \rceil$ Kandidaten, bevor das verbleibende Budget proportional zur Schichtgröße verteilt wird. Dies garantiert, dass Treffer mit seltenen Knotenzahlen n_k (z. B. $n_k = 20$, am dünnsten besetzte Schicht) nicht systematisch unterrepräsentiert werden — ein Problem, das bei einfachem `LIMIT K` nach `ORDER BY node_count ASC` auftritt (kleine n_k werden bevorzugt, große n_k ggf. nie erreicht).

Lemma 8 (Verzerrung durch `LIMIT`)**.** Sei $\mathcal{C}(Q, \mathcal{D})$ sortiert nach n_k aufsteigend und $K < |\mathcal{C}(Q, \mathcal{D})|$. Dann enthält das durch `LIMIT K` gelieferte Präfix $\mathcal{C}_K$ keinen Kandidaten mit $n_k > n^*$, wobei n^* das kleinste n mit $|\{k : n_k \leq n\}| \geq K$ ist.

Beweis. Da `ORDER BY node_count ASC` eine totale Ordnung auf $\mathcal{C}(Q, \mathcal{D})$ induziert und `LIMIT K` die ersten K Elemente dieser Ordnung zurückgibt, ist $\mathcal{C}_K = \{k \in \mathcal{C}(Q, \mathcal{D}) : \text{rank}(n_k) \leq K\}$. Per Definition von n^* als kleinstem Wert mit kumulativer Häufigkeit $\geq K$ folgt $n_k \leq n^*$ für alle $k \in \mathcal{C}_K$. □

Dieses Lemma motiviert formal die Notwendigkeit der oben skizzierten Schichtung für eine unverzerrte approximative Suche.

12.3.14 Caching häufiger Queries

Da biologische Analysen oft wiederholt nach denselben oder ähnlichen Motiven suchen (z. B. Standard-Pfadabschnitte der Glykolyse), bietet sich ein LRU-Cache auf Ebene des Signatur-Arrays $\Sigma(A_Q)$ an: Wird $\Sigma(A_Q)$ bereits für eine vorherige Anfrage berechnet, kann Schritt 1 des Subgraph Algorithmus übersprungen werden (Ersparnis $O(n_Q^2)$ pro wiederholter Anfrage). Bei einer angenommenen Cache-Trefferquote $\alpha \in [0, 1]$ reduziert sich die erwartete Laufzeit aus Satz 2 zu

$$\mathbb{E}[T_{\text{Suche}}] = (1 - \alpha) \cdot O(n_Q^2) + O(C \cdot n^3),$$

was insbesondere bei kleinem C (großes n_Q, vgl. Abbildung 5.2) relativ stärker zu Buche schlägt.

12.3.15 Inkrementelle Aktualisierung der Signatur-Indizes

Bei `INSERT` bzw. `DELETE` einzelner Netzwerke (wie in der Gen-DB-API über `POST/DELETE /api/networks`) werden Signaturen aktuell vollständig neu berechnet (Aufwand $O(n^2)$ pro Operation, vernachlässigbar). Bei einer zukünftigen Erweiterung um *Batch-Updates* (Änderung mehrerer Kanten gleichzeitig) wäre jedoch eine inkrementelle Aktualisierung der Signaturen wünschenswert: Da $\sigma_j = \sum_i A_{ij} 2^i + j 2^n$ linear in den Einträgen A_{ij} ist, lässt sich eine Änderung $A_{ij} \to A'_{ij}$ als

$$\sigma'_j = \sigma_j + (A'_{ij} - A_{ij}) \cdot 2^i$$

in $O(1)$ pro geänderter Kante nachführen, ohne die gesamte Spalte neu zu durchlaufen — eine direkte Konsequenz der Linearität der Signaturfunktion.

12.3.16 Betrieb, Reproduzierbarkeit und Beobachtbarkeit

Für den produktiven Betrieb sind mehrere organisatorische Erweiterungen sinnvoll. Die Dockerdateien des Projekts erlauben bereits einen reproduzierbaren Aufbau; sie sollten um einen automatisierten Testlauf und eine Messung der Suchzeit in einer Continuous-Integration-Pipeline ergänzt werden. Der Server sollte die Zeiten der einzelnen Phasen (Laden, Vergleichen, Ergebnisaufbau) und die Zahl der Kandidaten protokollieren oder als Metriken veröffentlichen. Das würde die in Abschnitt 7 fehlende Phasenzerlegung liefern und den seriellen Anteil direkt messbar machen, anstatt ihn aus dem Speedup abzuschätzen. Eine Begrenzung von Anfragegröße, Laufzeit und Zahl gleichzeitiger Suchen schützt den Dienst vor Überlastung.

Themenfeld E: Weiterentwicklung der Evaluation

12.3.17 Wiederholung und Erweiterung der Parallelisierungsmessung

Die Messreihe aus Abschnitt 7 ist als erste Einordnung gedacht; Abschnitt 9.10 ergänzt sie bereits um Modelle und die abgeschlossenen Multi-Omics-Worker- und Chunk-Messungen. Offen bleibt eine breitere empirische Prüfung: (i) Wiederholung mit mindestens 50 Anfragen, damit Streuungsmaße und Konfidenzintervalle berechnet werden können; (ii) Durchführung auf Rechnern mit vier, acht, sechzehn und mehr echten Prozessoren, um die Kurve $S(p)$ jenseits der bisher untersuchten Konfigurationen zu bestimmen; (iii) Trennung von Datenbankserver und Anwendung auf zwei Rechner, um den seriellen Ladeanteil einzugrenzen; (iv) Variation der Queries (Knotenzahlen oberhalb und unterhalb von 15, verschiedene Kantendichten und Kandidatenzahlen); (v) Messung mit und ohne Kandidatenlimit; (vi) mindestens fünf Wiederholungen der Chunk-Reihe, sowohl in der bestehenden Kandidatenreihenfolge als auch mit absteigender Reihenfolge nach geschätztem Vergleichsaufwand. Die Phasen Laden, Zuführung und Vergleich sind einzeln zu instrumentieren. Mit diesen Daten ließen sich Amdahl- und Gustafson-Modell sowie die Schranken für Blockbildung und Lastverteilung gegen weitere Messpunkte prüfen, statt ihre Gültigkeit aus wenigen Konfigurationen abzuleiten.

12.3.18 Benchmark-Suite und kontinuierliche Performanzmessung

Schließlich wäre die Entwicklung einer reproduzierbaren Benchmark-Suite sinnvoll, die die in Abschnitt 5 dargestellten Diagramme automatisch aus Live-Messungen (statt aus dem hier verwendeten analytischen Modell) generiert, etwa durch Instrumentierung der `search_subgraph`-Funktion mit `time.perf_counter()` und Protokollierung von $(N, n_Q, m_Q, C, T_{\text{gemessen}})$-Tupeln in einer separaten Metriktabelle. Eine solche Suite ermöglichte die empirische Validierung der in Satz 2 hergeleiteten asymptotischen Schranken und die Kalibrierung des Kandidatenlimits K in Abhängigkeit von einem Antwortzeit-Budget (z. B. 200 ms für interaktive Anwendungen). Die in Abschnitt 6 durchgeführten Experimente liefern hierfür einen konkreten Ausgangspunkt: `bench_compare.py` kann als Kern einer kontinuierlichen Integrationspipeline (CI) dienen, die bei jeder Änderung der Implementierung des Subgraph Algorithmus automatisch prüft, ob die gemessene Laufzeit weiterhin innerhalb der durch Satz 1 vorhergesagten Größenordnung liegt (Regressions-Schranke, z. B. $T_{\text{Sub}}(n) \leq c \cdot n^3$ für eine aus historischen Messungen kalibrierte Konstante c).

12.3.19 Erweiterung des Vergleichsbenchmarks

Die in Abschnitt 6 durchgeführten Experimente beschränken sich auf den Vergleich mit VF2 auf Erdős–Rényi-Zufallsgraphen $G(n, 0{,}3)$ mit $n \leq 18$. Eine umfassendere Evaluation sollte folgende Erweiterungen umfassen:

- **Weitere Referenzalgorithmen:** Integration von Ullmann [11] (z. B. über `networkx.algorithms.isomorphism.GraphMatcher` mit `ISMAGS` oder externe Implementierungen von RI [13]), um die in Tabelle 6.2 rein literaturbasierte Einordnung durch eigene Messungen auf identischer Hardware zu ersetzen.
- **Reale biologische Netzwerke:** Statt $G(n, p)$-Zufallsgraphen sollten Benchmark-Datensätze wie die KEGG-Pathway-Datenbank (metabolische Netzwerke) oder der DIP/STRING-Datensatz (Protein-Interaktionsnetzwerke) verwendet werden, da reale biologische Netzwerke charakteristische Eigenschaften (Skalenfreiheit, hohe Clusterkoeffizienten, Hub-Knoten) aufweisen, die sowohl das Pruning-Verhalten von VF2/RI als auch die Verteilung der Zeilenkomponenten $\rho(A)$ des Subgraph Algorithmus systematisch beeinflussen können.
- **Größere n:** Die hier durchgeführten Experimente decken $n \leq 18$ ab (entsprechend dem Wertebereich $\{3, \ldots, 20\}$ aus der Gen-DB-Generierung, Abschnitt 5). Für $n > 20$ (z. B. größere Stoffwechselnetzwerke mit 50–200 Knoten) wäre zu erwarten, dass die kubische Schranke des Subgraph Algorithmus zunehmend dominiert, während VF2/RI auf solchen Graphen — je nach Dichte und Symmetriegrad — entweder weiterhin sub-exponentiell bleiben (geringe Dichte, viele eindeutige Labels) oder in den exponentiellen Worst Case geraten (hohe Symmetrie, viele Automorphismen). Eine systematische Messreihe für $n \in \{20, 30, 50, 100\}$ wäre daher ein wichtiger nächster Schritt.
- **Statistische Signifikanz:** Die hier verwendeten 8 bzw. 5 Wiederholungen pro Datenpunkt (Tabelle 6.1, Abbildung 6.3) genügen für eine erste Tendenzaussage, jedoch nicht für eine belastbare statistische Aussage (z. B. Konfidenzintervalle, t-Test

auf Mittelwertunterschiede). Eine erweiterte Messreihe mit ≥ 100 Wiederholungen pro Konfiguration und expliziter Angabe von Konfidenzintervallen (95 %) wäre für eine Publikation in einer Fachzeitschrift erforderlich.

– **Implementierungssprache:** Die hier verwendete Referenzimplementierung des Subgraph Algorithmus ist reines Python mit `numpy`, während `networkx` ebenfalls in Python implementiert ist (mit C-Erweiterungen für einige Kernoperationen). Ein faircr Vergleich auf Implementierungsebene (z. B. beide Verfahren in C++ oder beide in einer JIT-kompilierten Umgebung wie `numba`) würde den Einfluss des Implementierungs-Overheads von der algorithmischen Komplexität trennen und die in Satz 1 bewiesene asymptotische Aussage von konstanten Faktoren isolieren. Für C++ ist dabei die Library-Anbindung (Abschnitt 12.3.24) die geeignete Messgrundlage, weil sie Prozessstart und JSON-Pfad aus dem Vergleich entfernt.

12.3.20 Formale Charakterisierung des Übergangsbereichs θ

Die empirisch beobachtete Umkehrung des Geschwindigkeitsvorteils bei $n_B \approx 14$ (Abschnitt 6) wirft die Frage auf, ob sich der Schwellwert θ aus Abschnitt 12.3.11 formal als Funktion der Graphdichte p charakterisieren lässt. Intuitiv gilt: Je größer p, desto eher findet VF2 *sehr früh* einen Match oder einen Konsistenzbruch (da fast jeder Knoten mit fast jedem anderen verbunden ist, sind die meisten Teilabbildungen ϕ bereits nach wenigen Schritten konsistent oder inkonsistent), während die Kosten des Subgraph Algorithmus $\Theta(n^3)$ *unabhängig* von p sind (Satz 1 macht keine Annahme über p). Dies legt die folgende Vermutung nahe:

Bemerkung 24 (Vermutung: Dichteabhängigkeit von θ)**.** Für $G(n,p)$ mit $p \to 1$ (vollständige Graphen) konvergiert die erwartete VF2-Laufzeit gegen $\Theta(n^2)$ (jede Teilabbildung ist sofort konsistent, das Pruning entscheidet nach $O(n)$ Rekursionsschritten mit $O(n)$ Aufwand pro Schritt für Konsistenzprüfung), während $T_{\text{Sub}}(n) = \Theta(n^3)$ unverändert bleibt. Für $p \to 0$ (sehr dünne Graphen) hingegen terminiert VF2 bei den meisten Knotenpaaren sofort durch Verletzung der Kantenbedingung, was ebenfalls zu $\Theta(n^2)$ oder besser führt. Der Schwellwert $\theta(p)$ wäre demnach für p nahe 0 und nahe 1 *kleiner* als für $p \approx 0{,}3$–$0{,}5$ (Bereich maximaler struktureller Unsicherheit, in dem VF2 die meisten Rekursionsschritte benötigt) — d. h. $\theta(p)$ wäre als Funktion von p vermutlich *unimodal* mit Maximum bei mittlerer Dichte. Ein formaler Beweis dieser Vermutung würde eine probabilistische Analyse des VF2-Suchbaums auf $G(n,p)$ erfordern, analog zu den in [10] angedeuteten, aber nicht vollständig ausgeführten Average-Case-Betrachtungen.

Eine systematische Messreihe über das zweidimensionale Gitter $(n,p) \in \{4,\ldots,20\} \times \{0{,}1, 0{,}2, \ldots, 0{,}9\}$ — eine direkte Erweiterung des in Abschnitt 6 durchgeführten Experiments — würde die in Bemerkung 24 formulierte Vermutung empirisch prüfbar machen und gleichzeitig eine Tabelle $\theta(p)$ liefern, mit der die Hybridstrategie aus Abschnitt 12.3.11 dichteadaptiv kalibriert werden könnte (d. h. θ würde nicht als globale Konstante, sondern als Funktion der für den jeweiligen Kandidaten k aus $m_k/(n_k(n_k-1))$ geschätzten Dichte $\hat{p}_k$ nachgeschlagen).

Themenfeld F: Anwendungsebene

12.3.21 Visuelle Exploration und Graph-Layout

Das aktuelle Frontend zeigt Adjazenzmatrizen als Zahlentabellen und Listen. Eine Erweiterung um eine graphische Darstellung (Force-Directed-Layout, z. B. mittels D3.js oder Cytoscape.js) würde die Interpretierbarkeit von Suchergebnissen erheblich verbessern, insbesondere für die in Abschnitt 5 diskutierten `subgraph`-Treffer, bei denen die *Einbettung* des Query-Graphen im größeren Kontextgraphen visuell hervorgehoben werden könnte (z. B. durch Hervorhebung der n_Q Knoten, auf die ϕ aus Abschnitt 12 (Label-Erweiterung) abbildet).

12.3.22 Validierung mit realen Anwendungsfällen

Der größte offene Punkt auf Anwendungsebene ist der Nachweis, dass die Suche für Biologinnen und Biologen nützliche Treffer liefert. Dazu sollten bekannte Motive (etwa Teilpfade der Glykolyse oder des p53-Netzwerks) gegen Datensätze aus öffentlichen Datenbanken gesucht und die Treffer von Fachleuten bewertet werden. Maßgeblich sind dabei die Präzision (wie viele Treffer sind fachlich sinnvoll) und die Vollständigkeit (werden bekannte Vorkommen gefunden). Die Ergebnisse ließen sich mit den Treffern etablierter Verfahren wie VF2 oder RI vergleichen und würden zeigen, an welchen Stellen die schwächere Relation $\preceq$ zu Abweichungen führt. Erst eine solche Validierung beantwortet, ob die zyklische Relation für die Praxis hinreichend ist oder ob die gefärbte Variante und eine Verfeinerung durch VF2 als Nachprüfung der Treffer nötig sind.

Themenfeld G: Kopplung und Schnittstellen

Dieses Themenfeld überträgt die Position aus Abschnitt 8.10 in konkrete Maßnahmen. Die C++-Library-Anbindung und eine erste gepaarte Messreihe sind umgesetzt. Offen bleiben die dauerhafte Absicherung der Backend-Konformität, die Wiederholung auf breiterer Datenbasis sowie die Verbesserung der Phaseninstrumentierung. Die CLI ist damit eine optionale Werkzeug- oder Isolationsschnittstelle, nicht mehr Voraussetzung für den produktiven Suchpfad.

12.3.23 Gemeinsame Schnittstellenspezifikation und Konformitätstests

Gegenwärtig ist die Gleichheit der beiden Backends eine Laufzeitprüfung des Experiments: Das Skript vergleicht die Treffer-IDs und die aktive Implementierung (Abschnitt 8.4). Besser ist es, die Zusicherung an die Schnittstelle zu binden. Die Spezifikation legt fest, welche Ergebniswerte der Vergleich liefert (A ist in B enthalten, B ist in A enthalten, keine Teilgraph-Beziehung, strukturell gleich mit der jeweiligen Wahl des beizubehaltenden Graphen), nach welcher Regel bei identischen Graphen anhand der Kantenzahl entschieden wird, wie Fehler gemeldet werden und welche Eingaben zulässig sind (quadratische 0/1-Matrizen).

Aus der Spezifikation folgt eine Suite von Konformitätstests, die gegen *jede* Implementierung läuft, also gegen den Python-Algorithmus, die Library und gegebenenfalls die CLI. Die Suite enthält feste Beispielpaare mit bekannten Ergebnissen, zufällige Paare mit festem Seed und die Eigenschaftstests aus Themenfeld A, etwa dass jedes Netzwerk

Teil von sich selbst ist und dass der Vergleich nicht von der zyklischen Anordnung der Zeilenkomponenten abhängt. Zwei Implementierungen gelten als austauschbar, wenn sie die Suite bestehen. Der Abgleich der Treffer-IDs im Experiment wird dadurch von einer Laufzeitkontrolle zu einer zusätzlichen Absicherung. Zugleich wandert die fachliche Regel für identische Graphen an die Stelle, an der sie hingehört, nämlich in die Schnittstelle des Algorithmus.

12.3.24 Ausbau der Library-Anbindung des C++-Kerns

Die Library-Anbindung ist umgesetzt: Die statische C++-Bibliothek wird mit einem Wrapper für den direkten Aufruf aus Python vorbereitet, und die Suchworker laden sie innerhalb ihres Prozesses. Der gemessene Such-Speedup von 6,36× gegenüber Python bestätigt, dass dieser Integrationsweg für den untersuchten Datensatz praktisch wirksam ist. Die folgenden Maßnahmen betreffen nun nicht mehr die Grundentscheidung, sondern Robustheit, Portabilität und weitere Optimierung:

- **Konformität automatisieren.** Der Experimentlauf vergleicht bereits Treffer-IDs und prüft, welches Backend aktiv ist. Diese Laufzeitkontrollen sollten durch gemeinsame Tests mit festen Fällen, Randbedingungen und Eigenschaftstests ergänzt werden, damit jede Änderung am C++-Wrapper oder Python-Backend vor dem Suchlauf gegen dieselbe Semantik geprüft wird.

- **Datenübergabe und Kopien messen.** Die aktuelle C-Schnittstelle übergibt einzelne Matrizen an den nativen Aufruf. Zu untersuchen ist, welcher Anteil der verbleibenden 66,7 % nichtvergleichender beziehungsweise serieller Suchzeit auf Arraykonvertierung und Übergabe entfällt. Erst danach ist zu entscheiden, ob Batch-Verarbeitung oder ein zusammenhängendes Array für Kandidaten messbare Vorteile bietet.

- **Bau und Auslieferung absichern.** Die Binärdatei ist plattformabhängig; Bau, Laden und Fehlerfälle sollten in der Continuous-Integration-Pipeline für unterstützte Betriebssysteme automatisiert werden. Fehlende oder inkompatible Bibliotheken müssen explizit gemeldet werden; ein stiller Wechsel auf Python würde Ergebnisse und Performance-Messungen verfälschen.

- **Schnittstelle stabil halten.** Ergebniswerte, ungültige Eingaben und Fehlerverhalten sollten unabhängig vom Backend dokumentiert werden. Die Python-Referenz bleibt als Rückfall und Vergleichsimplementierung verfügbar, muss aber dieselben Konformitätstests bestehen.

Offen bleibt der einmalige Aufwand für Bau und Test je Plattform sowie der wiederkehrende Wartungsaufwand bei Änderungen am C++-Kern. Die gemessenen Laufzeiten beantworten die Performance-Frage für die aktuelle Windows-Konfiguration, nicht aber die Portabilität. Automatisierte Builds und direkte Konformitätstests bilden deshalb die Voraussetzung dafür, den beobachteten Beschleunigungsgewinn zuverlässig in künftigen Releases zu erhalten.

Themenfeld H: Multi-Omics-Suche

12.3.25 Abgebrochene Datenbankgrößenreihe vervollständigen

Die Ende-zu-Ende-, Worker- und Chunk-Reihen aus Abschnitt 9.5 sind abgeschlossen und bereits gepaart ausgewertet. Offen blieb die Datenbankgrößenreihe: Die beiden kleinsten Größen (2.000 und 5.000 Zufallsnetzwerke) sind vollständig, bei 10.000 Netzwerken liegt nur ein Teil vor, und 20.000 wurde nicht erreicht. Eine Fortsetzung sollte entweder diese Reihe mit denselben Queries und Wiederholungen vervollständigen oder die fehlenden Stufen als eigene Messung neu durchführen. Dabei müssen Anzahl der Multi-Omics-Kandidaten, Ladezeit, Vergleichszeit und Trefferaufbau je Backend getrennt erhalten bleiben.

Für die bereits abgeschlossene Suche sind zusätzlich zur Gesamtzeit die Phasenanteile zu berichten: Mit C++ dominiert nun das Laden, während die Vergleichsphase nur noch 27,6 % der Antwortzeit beansprucht. Die Worker-Ergebnisse sollten im Kontext des Vierkernprozessors und die Chunk-Ergebnisse ausdrücklich als kleine, zweimal wiederholte Stichprobe interpretiert werden. Eine weitergehende statistische Absicherung erfordert mehr Queries, Wiederholungen und weitere Hardwarekonfigurationen; erst danach sind allgemeine Empfehlungen zu Worker-Zahl und Chunk-Größe gerechtfertigt.

12.3.26 Biologische Semantik und reale Multi-Omics-Daten validieren

Die synthetischen Matrizen sichern bekannte technische Sollfälle, bilden aber keine realen Erhebungsprozesse ab. Eine Folgestudie sollte öffentlich verfügbare und fachlich annotierte Multi-Omics-Datensätze auswählen und für jede Schicht dokumentieren, welche Entitäten und Interaktionen die Knoten und Kanten repräsentieren. Vor dem Vergleich sind Zuordnungen gemeinsamer Entitäten zwischen den Schichten festzulegen; falls die Knotenräume nicht identisch sind, muss das Datenmodell diese Zuordnungen explizit statt durch eine künstlich gemeinsame Knotennummerierung repräsentieren.

Gemeinsam mit Fachleuten ist zu prüfen, welche Suchfrage die Modi beantworten sollen. Der kohärente Modus verlangt dieselbe strukturelle Position in allen Schichten, während der unabhängige Modus verschiedene Positionen erlaubt. Diese algorithmische Definition sagt für sich genommen nichts über zeitliche Reihenfolge, Regulation oder Kausalität aus. Eine Evaluation sollte deshalb bekannte positive und negative Beispiele, Präzision und Vollständigkeit sowie eine qualitative Bewertung der gefundenen Muster umfassen. Sie muss außerdem festhalten, ob Knotenlabels, fehlende Messwerte, gerichtete Beziehungen und Schichtgewichte in die Relation eingehen sollen.

12.3.27 Skalierung und Modellgrenzen mehrschichtiger Netzwerke

Die Knotenzahl und Schichtzahl wurden im Experiment nur in begrenzten Bereichen variiert. Größere Werte können sowohl die Kosten des Vergleichs als auch die Speicher- und Übergabekosten verändern. Eine Folgeevaluation sollte deshalb Schichtzahlen oberhalb von acht und Netzwerke jenseits von 63 Knoten untersuchen, sobald die Implementierung und der Speicherbedarf dies erlauben. Dabei ist zu prüfen, ob die Laufzeit annähernd mit der Zahl der Schichten wächst oder ob die gemeinsame Suche und frühe Abbrüche ein anderes Profil erzeugen.

Die aktuelle Versuchskonstruktion verwendet pro Netzwerk eine gemeinsame Knotenzahl für alle Matrizen. Das ist eine klare Ausgangsannahme, aber keine allgemeine Eigenschaft von Omics-Daten. Erweiterungen auf unterschiedliche Knotenräume benötigen explizite Relationen zwischen Schichten und eine formale Definition, wann Teilstrukturen über solche Abbildungen hinweg als Treffer gelten. Vor einer Optimierung sind daher zunächst Relation und biologische Interpretation zu spezifizieren; erst anschließend lassen sich notwendige Datenbankfilter, Komplexitätsaussagen und Ground-Truth-Tests korrekt ableiten.

12.3.28 Batch- und persistente Prozessschnittstellen als Übergangslösung

Falls eine Library kurzfristig nicht umsetzbar ist, lässt sich die CLI-Anbindung entschärfen, ohne ihre Grenze zu ändern. Zwei Varianten stehen zur Wahl: ein Batch-Aufruf, bei dem ein Prozess viele Vergleiche bearbeitet und die Ergebnisse gesammelt zurückgibt, und ein persistenter Prozess, der Aufträge über `stdin` entgegennimmt und über `stdout` beantwortet. Bei B Vergleichen je Prozessaufruf teilt sich der Prozessstart auf alle Vergleiche auf. Eine Modellrechnung, die auf der gemessenen Mindestlatenz von 10,081 Millisekunden beruht und die Serialisierungskosten δ_{ser} je Paar getrennt führt, ergibt

$$\delta_{\text{eff}}(B) \approx \frac{10{,}081\,\text{ms}}{B} + \delta_{\text{ser}}.$$

Mit $B = 256$ läge der Prozessanteil rechnerisch bei rund 0,04 Millisekunden je Vergleich. δ_{ser} ist nicht gemessen; der Wert zeigt nur, dass sich der Prozessanteil durch Bündelung beseitigen lässt, während die JSON-Übersetzung bestehen bleibt. Die Übergangslösung löst damit das Kostenproblem der CLI, nicht aber das Beschreibungsproblem: Die fachlichen Regeln und die Fehlerabbildung bleiben im Übersetzungscode. Sie sollte deshalb nur gewählt werden, wenn die Library nach der Messung nicht tragfähig ist, und in diesem Fall ebenfalls gegen die Konformitätstests laufen.

12.3.29 Messprogramm für die Kopplungsfrage

Die abgeschlossene Messung vergleicht Python und Library bereits auf denselben sechs Queries und derselben Worker-Zahl. Eine Folgeuntersuchung sollte diese Messung mit mindestens mehreren Dutzend Queries und Wiederholungen pro Query erweitern, damit Streuungsmaße und Konfidenzintervalle berechnet werden können. CLI- und Batch-CLI-Varianten sind nur dann zusätzlich aufzunehmen, wenn sie als reale Alternativen für Isolation, Interoperabilität oder Fehlersuche bewertet werden; sie sind nicht nötig, um den inzwischen gemessenen Library-Vorteil festzustellen.

Die Folgeuntersuchung sollte die Phasen feiner aufteilen: Datenbankabfrage, Übergabe der Daten an den Rechenkern, Rechenzeit des Kerns und Ergebnisauswertung. Damit lässt sich der verbleibende Aufwand δ genauer bestimmen und prüfen, ob er sich durch eine gebündelte Übergabe senken lässt. Zu untersuchen sind außerdem der Backend-Speedup über unterschiedliche Knotenzahlen und Kantendichten, identische Treffer für einen größeren Query-Satz und die Skalierung mit mehr als zwei Workern. Mindestens zwei Betriebssysteme sollten einbezogen werden, um sowohl die Portabilität als auch den Einfluss des Betriebssystems auf den Wrapper zu bewerten.

12.3.30 Prüfung künftiger Fremdmodule

Mehrere Erweiterungen dieses Ausblicks führen weitere Bausteine in das System ein. Damit sie nicht jeweils neu begründet werden müssen, genügen fünf Prüffragen: Ist die Grenze schmal und benannt? Liegen fachliche Regeln, etwa die Ergebnisabbildung, auf einer Seite der Grenze? Ist der Schnittstellenaufwand δ im Kostenmodell ausgewiesen? Lässt sich der Baustein mit denselben Mitteln testen wie der Rest des Systems? Rechtfertigt der Zweck, etwa Isolation oder Skalierung über Rechner, einen Wechsel der Beschreibung? Für die Maßnahmen des Ausblicks ergibt sich folgende Einordnung:

- **Vektorisierung mit `numpy` und Kompilierung mit `numba`** (Abschnitt 12.3.9): bleiben in der Beschreibung des Aufrufers und erfüllen die Fragen ohne Weiteres.
- **C++-Erweiterung oder Library**: erfüllt die Fragen, sofern die fachliche Regel in die Schnittstelle wandert und die Konformitätstests bestehen.
- **GPU-Beschleunigung**: Die Übergabe als Tensor der Form (C, n, n) entspricht der blockweisen Schnittstelle und hält die Grenze schmal. Würde dagegen je Kandidat übertragen, überstiege der Transferaufwand voraussichtlich den Gewinn; die Frage nach δ ist deshalb vor der Umsetzung zu beantworten.
- **Verteilung mit Apache Spark**: führt eine weitere Beschreibung mit eigenem Datenformat ein. Der Zweck, Skalierung über Rechner, kann den Bruch rechtfertigen, sofern die Grenze grob ist, also einmal je Partition und nicht je Vergleich verläuft.
- **VF2 in der Hybridstrategie**: Über `networkx` bleibt der Baustein in der Beschreibung des Backends. Eine externe Implementierung wie RI [13] wäre je nach Anbindung ein Fremdmodul und würde dieselbe Prüfung durchlaufen.
- **Asynchrone Suche und Metriken**: betreffen Schichtgrenzen, die ohnehin über benannte Schnittstellen laufen.
- **Kodierung und Indexsuche der universellen Kodierung** (Themenfeld I): Die Kodierung liegt vollständig in Python, der Kern bleibt schmal und kennt nur Stapel; die fachliche Regel (Schema, Spaltenordnung, Koordinaten) liegt auf der Seite der Kodierung. Der Aufwand δ entsteht nur bei der Übergabe der Stapel an die native Bibliothek und ist dann je Anfrage und nicht je Vergleich zu veranlagen. Die Tests folgen aus den Sätzen und laufen mit denselben Mitteln wie der Rest des Systems.

Themenfeld I: Universelle Kodierung und Indexsuche

Abschnitt 10 hat den Kern, die Kodierung, den optimalen Vergleich und den invertierten Index formal begründet, Abschnitt 11 legt das Experiment fest, das ihre Laufzeitaussagen prüft. Dieses Themenfeld ist deshalb das umfangreichste Folgeprogramm der Arbeit: Es umfasst die Durchführung, Wiederholung und Auswertung der Messung, die Umsetzung weiterer Strukturarten, die Entwicklung konkreter Schemata, die fachliche Validierung der Suchsemantik und die Klärung der offenen theoretischen Fragen. Jede Maßnahme knüpft an einen bewiesenen Satz oder eine Hypothese des Experiments an; der Satz bestimmt zugleich, was der zugehörige Test prüfen muss.

12.3.31 Umsetzung der Kodierung in Python

Der erste Schritt ist eine Schnittstelle je Strukturart mit einem versionierten Schema $\Sigma = (\Lambda, T, (k_t), (W_t))$, einer Funktion `encode` für $\Phi_{\Sigma,\kappa}$ und einer Funktion `decode` für die Umkehrung nach Satz 19. Dazu gehört ein Entitätsregister, das κ als persistente Abbildung führt und die in Definition 24 geforderte Eigenschaft sichert: Neue Entitäten erhalten stets neue, größere Koordinaten, bestehende Koordinaten ändern sich nie. Das Register ist die einzige Stelle, an der bei globalen Koordinaten Zustand entsteht, und muss deshalb gegen gleichzeitige Zugriffe, Rücksicherung und Zusammenführen von Datenbanken abgesichert werden. Zu klären ist außerdem, wie kontinuierliche Messwerte quantisiert werden. Die Wahl der Stufenzahl W_t bestimmt die Schichtzahl L_Σ und damit Speicher und Vergleichsaufwand, und die Verlustfreiheit gilt nur bezüglich der quantisierten Struktur. Die Aufwandsaussage $O(|S| \log |S|)$ aus Satz 20 ist durch eine Lastmessung zu bestätigen, insbesondere für Strukturen mit vielen mehrstelligen Relationen, bei denen das Sortieren der Relationsknoten nach $(t, \kappa$-Tupel) überwiegt.

Die Rechnung am Beispiel zeigt die Größenordnung der Schichtzahl: Für ein Schema mit einem zweistelligen Typ ohne Gewichte und einer dreistelligen Reaktion ohne Gewichte und ohne Knotenmerkmale ist $L_\Sigma = 1+0+1+(3+1-1) = 5$, nämlich die Existenzschicht, eine Binärschicht und drei Inzidenzschichten. Jedes Knotenmerkmal und jede Gewichtsstufe fügt Schichten hinzu. Für reale Schemata sollte daher früh bestimmt werden, wie stark die Schichtzahl in den Faktor L der Schranken (Sätze 21 und 24) eingeht.

12.3.32 Indextabelle, Wörterbücher und Pflege des Index

Der Paarindex wird als Tabelle mit den Spalten Schicht, Komponentenpaar, Art (zyklisch oder linear), Netzwerk und Länge geführt, mit einem B-Baum über Schicht und Paar; die Komponenten werden über ein je Schicht injektives Wörterbuch ι_ℓ in Zahlen abgebildet. Für eine Anfrage werden die linearen Paare der Query nachgeschlagen, die Trefferlisten nach der Länge gefiltert und je aktiver Schicht geschnitten; für den kohärenten Modus verifiziert ein Vergleich nach Satz 21 die Obermenge (Satz 23). Folgende Fragen sind in der Umsetzung zu beantworten:

- **Größe des Index.** Je Struktur und Schicht entstehen höchstens $|M_k|$ zyklische und $|M_k| - 1$ lineare Paare, insgesamt also $O(\sum_k L\,|M_k|)$ Einträge (Satz 24). Für die Testdatenbank mit einer Million Netzwerken und im Mittel 11,5 Knoten ergibt das bei $L = 1$ rechnerisch höchstens rund $2{,}2 \cdot 10^7$ Einträge ($2 \cdot 11{,}5 - 1 = 22$ je Netzwerk); dies ist eine Obergrenze aus der Modellrechnung und keine Messung. Bei mehrschichtigen Strukturen wächst sie mit L. Zu messen sind Speicherbedarf, Aufbauzeit und Einfluss auf Einfüge- und Löschoperationen.

- **Pflege.** Das Einfügen oder Löschen einer Struktur kostet $O(L\,|M_k| + \nu(M_k))$. Zu prüfen ist, ob dies bei massenhaftem Einlesen in Stapelläufen und innerhalb von Transaktionen der Datenbank handhabbar bleibt, und wie die Wörterbücher bei neuen Komponenten wachsen.

- **Häufige Paare.** Die Laufzeit hängt von der Trefferlast H ab. Besonders häufige Paare, etwa solche aus leeren oder sehr verbreiteten Komponenten, erzeugen lange Listen. Naheliegende Maßnahmen, die noch nicht untersucht sind, sind das Schneiden in aufsteigender Listenlänge mit frühem Abbruch bei leerem Zwischenergebnis und das Ausnehmen von Paaren, deren Trefferliste einen festen Anteil der Datenbank

überschreitet. Letzteres verändert das Ergebnis nur dann nicht, wenn der Ausschluss vor dem Schnitt aus der Definition begründet ist; dies müsste bewiesen werden.

– **Auswahl der aktiven Schichten.** Die Menge I aktiver Schichten bestimmt die Anfrage (weniger Schichten liefern mehr Treffer, Lemma 6(d)). Die Oberfläche und die Schnittstelle der API müssen Nutzern erlauben, Schichten gezielt zu wählen, etwa die Existenzschicht nicht einzuschränken, und müssen die Folge für die Trefferzahl sichtbar machen.

12.3.33 Differenzielle Prüfung und Eigenschaftstests der Kodierung

Die Tests folgen aus den Sätzen und knüpfen an die differenzielle Prüfung der Multi-Omics-Suche an. Vorgesehen sind der Rundlauf decode∘encode = id auf zufälligen Strukturen jedes Schemas (Satz 19), die Gleichheit der Ergebnisse von Indexsuche und Einzelvergleich über alle Paare gespeicherter Strukturen und Queries in beiden Modi und beiden Richtungen (Sätze 23 und 21), die Teilmengenbeziehung $\mathrm{Match}_{\mathrm{coh}} \subseteq \mathrm{Match}_{\mathrm{ind}}$ und die Monotonie unter Projektion der Schichten (Lemma 6), die Nicht-Transitivität als Regressionsfall mit dem Gegenbeispiel des Lemmas und die Unzulässigkeit des Kantenzahl-Filters als Regressionsfall mit dem Gegenbeispiel aus Lemma 7. Dieser Test schützt davor, den Kantenfilter aus Abschnitt 4.2 versehentlich auf die neue Relation zu übertragen. Hinzu kommt der Abgleich der Python-Indexsuche mit der nativen Bibliothek auf allen Stapeln, deren Koordinaten in 64 Bit passen. Diese Tests gehören in die gemeinsame Konformitätssuite aus Themenfeld G, damit jede Implementierung des Kerns an derselben Spezifikation gemessen wird.

12.3.34 Auswertung, Wiederholung und Erweiterung der Messung

Das Experiment (Abschnitt 11) wurde auf der dort dokumentierten Hardware begonnen; die Metadaten markieren den Lauf jedoch als unvollständig. Als erster Schritt ist der Lauf regulär abzuschließen oder kontrolliert zu wiederholen. Danach sind alle vier Varianten und die vorgesehenen drei Wiederholungen in der Ende-zu-Ende-Auswertung zu berücksichtigen, bevor Laufzeit- und Skalierungsaussagen verallgemeinert werden. Anschließend sind folgende Erweiterungen sinnvoll:

– **Wiederholung auf weiterer Hardware und mit größerer Datenbank.** Die Messung verwendet 20.000 Netzwerke und eine Konfiguration. Wiederholungen auf Rechnern mit mehr Kernen, mit getrenntem Datenbankserver und mit Datenbanken der Größenordnung der Evaluation (10^6 Netzwerke) zeigen, ob Exponent und Speedup übertragbar sind. Zu trennen sind Messungen mit kalter und warmer Datenbank, weil der Index in PostgreSQL gelesen wird.

– **Variation der Häufigkeit.** Die Häufigkeitsannahme des Korollars 12 ist der kritische Punkt. Die Messung sollte h kontrolliert variieren, etwa durch Netzwerke mit Hubs, durch skalenfreie Verteilungen oder durch Kantenwahrscheinlichkeiten je Schicht, und den Übergang vom günstigen Fall zum Fall $H = \Theta(N\, m_Q |I|)$ abbilden.

– **Feinere Phasen der Indexsuche.** Zu trennen sind Wörterbuchzugriff, Lesen der Trefferlisten, Schnitt, Laden der Treffer und, im Modus coh, Verifikation. So lässt

sich prüfen, ob der Ladeanteil gegenüber der bisherigen C++-Suche sinkt (Hypothese H9), und der Beitrag der nativen Bibliothek zur Verifikation bestimmen.

– **Listen- und Schnittstrategien.** Das Schneiden in aufsteigender Listenlänge mit frühem Abbruch und das Ausnehmen sehr häufiger Paare sind nicht untersucht. Letzteres verändert das Ergebnis nur dann nicht, wenn der Ausschluss aus der Definition begründet ist; dies wäre zu beweisen.

– **Vergleich mit der klassischen Isomorphie.** Auf kleinen Strukturen ist die Treffermenge von $\sqsubseteq$ gegen eine exakte Referenz (VF2) in Präzision und Vollständigkeit zu messen (siehe Themenfeld I, Beziehung zur Subgraph-Isomorphie).

– **Weitere Schemata.** Das Experiment verwendet ein Matrixschema. Die Kernunabhängigkeit ist an mindestens einem weiteren Schema mit mehrstelligen Relationen zu messen, damit Kodierungsaufwand (Satz 20) und Schichtzahl L_Σ an einem nicht matrixförmigen Fall belegt sind.

Wie bei den bisherigen Reihen gelten die Einschränkungen synthetischer Daten, einer einzelnen Hardwarekonfiguration und kleiner Stichproben, bis sie durch Wiederholungen und reale Strukturen aufgehoben sind.

12.3.35 Schemata für weitere biologische Strukturarten

Der Nutzen des Kapitels liegt darin, dass eine neue Strukturart nur ein neues Schema erfordert. Das ist an konkreten Fällen zu zeigen: Genregulationsnetzwerke mit gerichteten, gewichteten Kanten, Stoffwechsel- und Reaktionsnetze mit dreistelligen Reaktionen aus Substrat, Enzym und Produkt, Protein-Protein-Interaktionsnetzwerke als symmetrische Relation, Hypergraphen für Komplexe, Pfade und Sequenzen als Nachfolgerrelation, Ontologie-Graphen sowie mehrschichtige Multi-Omics-Netzwerke als eine Relationsart je Schicht. Für jedes Schema ist festzuhalten, welche Entitäten und Merkmale vorkommen, wie L_Σ ausfällt, welche Gewichte quantisiert werden und wie die Spaltenordnung festgelegt wird. Für gerichtete Reaktionsnetze mit Enzymen ist zusätzlich zu klären, ob die Rollenreihenfolge in den Inzidenzschichten die gewünschte Suche abbildet, und ob die Existenzschicht in der Suche einzuschränken ist.

Die Verbindung zu den früheren Vorschlägen ist ausdrücklich herzustellen. Die beschriftete Relation und der Label-Index aus den Themenfeldern A und B entsprechen formal den Schichten Lab_a und ihrem Index. Anders als dort beschrieben, verlangt $\sqsubseteq$ jedoch keine Knotenabbildung, die Labels erhält, sondern gemeinsame Nachbarschaftspaare benachbarter Spalten. Es ist zu prüfen, ob die Schichten Lab_a die in Themenfeld B beschriebene notwendige Bedingung (Multimenge der Labels der Query im Multiset der Kandidaten) liefern; dies folgt aus den bisherigen Sätzen nicht und müsste aus der Definition der Relation bewiesen werden, bevor es als Vorauswahl verwendet wird. Dasselbe gilt für die Multi-Omics-Daten mit verschiedenen Knotenräumen in den Schichten (Themenfeld H): Globale Koordinaten identifizieren Entitäten über Schichten hinweg und ersetzen die dort geforderte explizite Zuordnung zumindest für die Entitäten, die im Register geführt werden; ob die resultierende Relation die fachlichen Fragen beantwortet, bleibt zu validieren.

12.3.36 Koordinatensystem, Schemaänderung und Versionierung

Das Kapitel unterscheidet lokale Koordinaten (Rang innerhalb der Struktur) und globale Koordinaten (Register der Datenbank). Lokale Koordinaten ergeben den Matrixstapel und die bisherige Multi-Omics-Relation und passen in die 64-Bit-Darstellung der nativen Bibliothek, solange $n \leq 63$ gilt. Globale Koordinaten machen die Gleichheit von Komponenten zur Gleichheit von Entitätsmengen und sind damit fachlich aussagekräftiger. Für die Praxis ist zu entscheiden, welche Koordinaten welches Schema verwendet, wie die Spaltenordnung bei globalen Koordinaten gewählt wird und ob mehrere Koordinatensysteme nebeneinander im Index geführt werden sollen. Zu untersuchen ist, wie stark die Treffermenge von der Wahl von κ abhängt: Da $\sqsubseteq$ benachbarte Spalten vergleicht und die Spalten nach κ geordnet sind, kann eine andere Vergabereihenfolge eine andere Treffermenge ergeben. Eine Ordnungsrobustheit, etwa durch Wahl kanonischer Ordnungen, wäre ein eigenes Teilziel.

Die Schemaänderung erfordert nach dem Kapitel eine Neukodierung in $O(|S| \log |S|)$ je Struktur oder ein neues Schema mit eigenem Index. Offen ist, wie Schemata versioniert, wie Neukodierungen in einer laufenden Datenbank ohne Ausfall durchgeführt und wie alte und neue Indizes bis zum Abschluss nebeneinander betrieben werden.

12.3.37 Fachliche Validierung der Suchsemantik

Bemerkung 22 hält fest, dass die Sätze Kodierung, Korrektheit und Aufwand sichern, nicht die fachliche Validität. Eine Validierungsstudie sollte für jedes Schema bekannte positive und negative Beispiele sammeln, mit Fachleuten bewerten, welche Treffer biologisch relevant sind, und Präzision und Vollständigkeit der Suchmodi bestimmen. Dabei sind zwei Eigenschaften der Relation zu berücksichtigen: Die Nicht-Transitivität verbietet es, Treffer als Halbordnung zu lesen, und die Schichtauswahl I bestimmt die Aussage. Der Modus ind erlaubt in jeder Schicht ein anderes Paar, der Modus coh verlangt dasselbe Paar in allen Schichten; welcher Modus für welche Frage gebraucht wird, ist mit Anwendern zu klären, wie es bereits für die Multi-Omics-Suche gefordert wurde (Themenfeld H).

12.3.38 Beziehung zur klassischen Subgraph-Isomorphie

Bemerkung 21 zeigt, dass eine in Polynomialzeit berechenbare Abbildung auf Stapel, die die Subgraph-Isomorphie exakt wiedergibt, nur bei P = NP existieren kann. Daraus ergeben sich zwei Folgefragen. Erstens: Unter welchen Zusatzbedingungen ist $\sqsubseteq$ eine *notwendige* Bedingung der Isomorphie und damit ein zulässiger Filter vor einer exakten Prüfung etwa mit VF2 (Abschnitt 6)? Nach Lemma 7 ist zumindest der Kantenzahl-Filter nicht zulässig; ob die Relation selbst ein zulässiger Filter ist, geht aus den Sätzen nicht hervor und wäre zu beweisen oder zu widerlegen. Zweitens: Wie nahe liegt die Treffermenge von $\sqsubseteq$ an der der Isomorphie auf realen und synthetischen Netzwerken? Hierfür ist ein Vergleich mit einer exakten Referenz zu messen (Präzision und Vollständigkeit auf kleinen Strukturen, auf denen VF2 praktikabel ist). Die Hybridstrategie aus Themenfeld C ließe sich damit zu einem dreistufigen Verfahren erweitern: Indexsuche als Vorauswahl, Verifikation durch den Kern und, wo gewünscht, exakte Prüfung durch ein Verfahren mit anderer Relation. Dies wäre ein weiterer Baustein im Sinne der Prüffragen aus Themenfeld G.

12.3.39 Offene theoretische Fragen

Aus dem Kapitel bleiben mehrere theoretische Fragen offen, die für die Einordnung und Weiterentwicklung wichtig sind:

- **Lücke für wachsendes L.** Zwischen der unteren Schranke $\Omega(|M| + |M'|)$ und der oberen Schranke $O(L(|M| + |M'|))$ liegt ein Faktor L, der für die kohärente und unabhängige Relation ungeklärt ist. Zu untersuchen ist, ob sich die Schranke für festes Verhältnis von Komponentengröße und L verschärfen lässt, etwa durch gemeinsames Internieren der Tupel oder durch bessere Hash-Konstruktionen, oder ob eine schärfere untere Schranke gilt.

- **Deterministische Schranke.** Die obere Schranke gilt in erwarteter Zeit durch Hashing; mit Sortieren ist sie deterministisch bis auf einen logarithmischen Faktor. Offen ist, ob eine deterministische Linearzeit im betrachteten Modell erreichbar ist.

- **Untere Schranke für die Indexsuche.** Bewiesen ist die untere Schranke für den Einzelvergleich und die Ausgabegröße als untere Schranke im ungünstigsten Fall. Eine feinere Aussage zur Abhängigkeit von H und zur Optimalität der Indexstruktur selbst steht aus.

- **Weitere Relationen auf Stapeln.** Die Relation vergleicht benachbarte Spalten. Zu untersuchen sind Erweiterungen auf längere gemeinsame Teilfolgen oder mehrere gleichzeitig zu erfüllende Paare, die sich der Isomorphie annähern, und die Frage, ob solche Erweiterungen noch eine Indexsuche mit ähnlichen Schranken erlauben.

- **Transitivität und Äquivalenzklassen.** Die Relation ist nicht transitiv. Es ist zu klären, ob sich durch Zusatzbedingungen, etwa einen Schnitt in $\sqsubseteq_{\mathrm{ind}}$ und der Umkehrung, eine Äquivalenzrelation gewinnen lässt, die für Gruppierung und Redundanzerkennung in der Datenbank geeignet ist.

12.3.40 Native Bibliothek und Anbindung an Experimente

Die C++-Bibliothek `csubgraph` bleibt nach dem Kapitel als optionaler Beschleuniger erhalten, wird aber für Korrektheit und Laufzeitaussagen nicht benötigt. Mit der Indexsuche verschiebt sich der Kostenschwerpunkt: Die bisherigen Messungen zeigen, dass nach der Beschleunigung der Vergleichsphase das Laden und die Übergabe dominieren; die Indexsuche reduziert genau diese Phasen, während die Verifikation im Modus coh weiterhin von der Bibliothek profitieren kann. Zu entscheiden ist, ob die Schnittstelle der Bibliothek auf Stapel beliebiger Koordinaten erweitert, mit einer Fallunterscheidung für 64-Bit-Koordinaten betrieben oder auf die Verifikation beschränkt wird. In allen Fällen gilt die Prüffrage nach der Grenze: Die Übergabe einer Menge von Kandidatenstapeln je Anfrage hält die Grenze schmal, die Übergabe je Vergleich nicht. Die Ergebnisse der bisherigen Experimente (Backend-Vergleich, Worker-Reihen, Chunk-Reihen) bleiben die Referenz; die Messung der Indexsuche muss auf denselben Datensätzen und mit denselben Kennzahlen wiederholbar sein, damit die Verbesserung gegenüber der Pipeline eindeutig zugeordnet werden kann.

12.4 Priorisierung der nächsten Entwicklungsschritte

Die beschriebenen Erweiterungen sollten in einer Reihenfolge umgesetzt werden, die zunächst die fachliche Korrektheit und Aussagekraft absichert, danach die Messbarkeit herstellt und erst anschließend die Laufzeit optimiert. Die Parallelisierung der Vergleiche, die zuvor als eine von mehreren möglichen Optimierungen aufgeführt war, ist inzwischen umgesetzt und gemessen; sie entfällt daher als eigener Entwicklungsschritt, nicht jedoch ihre Verfeinerung. Die Kopplungsfrage (Themenfeld G) steht früh in der Reihenfolge, weil sie bestimmt, auf welcher Anbindung spätere Messungen und Optimierungen aufbauen. Ebenso steht die Umsetzung der universellen Kodierung (Themenfeld I) früh, weil sie bereits bewiesene Aussagen in Tests überführt und die Voraussetzung für die Indexsuche ist; die Messung der Indexsuche folgt, sobald Kodierung und Index differenziell geprüft sind, und liegt vor den Optimierungen des Einzelvergleichs, weil sie klärt, wie viele Einzelvergleiche überhaupt noch anfallen.

Tabelle 12.1: Vorgeschlagene Reihenfolge der Weiterentwicklung.

Nr.	Maßnahme	Nutzen	Aufwand
1	Differenzielle Tests der Vorauswahl und Eigenschaftstests (Abschnitt 12.3, Feld A)	sichert Korrektheit und alle weiteren Optimierungen ab	gering
2	Gemeinsame Konformitätstests für Python und C++ (Feld G)	sichert identische Semantik bei jeder Änderung	gering bis mittel
3	Kodierung in Python: Schnittstelle `encode`/`decode`, Entitätsregister, Rundlauf- und Eigenschaftstests (Feld I)	überführt die bewiesene Verlustfreiheit in einen Test und ist Voraussetzung für die Indexsuche (Matrixschema umgesetzt; weitere Schemata offen)	gering bis mittel
4	Feinere Phasen- und Schnittstellenmessung (Feld D und G)	erklärt den Restanteil der C++-Suchzeit	gering bis mittel
5	Wiederholung der Backend- und Parallelisierungsmessung (Feld E und G)	belastbare Aussagen zu Speedup und $S(p)$	mittel
6	Bau und Laden der C++-Library für unterstützte Plattformen automatisieren (Feld G)	reproduzierbare Bereitstellung und Fehlererkennung	mittel
7	Batch- oder persistente CLI-Schnittstelle nur bei Bedarf (Feld G)	erhält Isolation oder Sprachneutralität bei amortisierten Startkosten	gering bis mittel
8	Suche in beide Richtungen (Feld A)	vollständige Suchsemantik	mittel
9	Indextabelle, Wörterbücher und Indexsuche mit differenzieller Prüfung gegen den Einzelvergleich (Feld I)	Suche ohne Einzelvergleiche, Aufwand von der Trefferlast statt von N abhängig (Prüfung im Experiment, Abschnitt 11)	mittel bis hoch

Tabelle 12.1: Vorgeschlagene Reihenfolge der Weiterentwicklung (Fortsetzung).

Nr.	Maßnahme	Nutzen	Aufwand
10	Vollständiger Messlauf des Experiments zur Indexsuche, Auswertung der Hypothesen H1 bis H9, Einträge in die Arbeit (Feld I)	bestätigt oder widerlegt die Laufzeitfolgerungen des Kapitels	mittel
11	Wiederholung der Indexmessung auf weiterer Hardware, mit größerer Datenbank und mit kontrollierter Häufigkeit h (Feld I)	Übertragbarkeit der Exponenten und Prüfung der Häufigkeitsannahme	mittel
12	Messung eines zweiten Schemas mit mehrstelligen Relationen und Vergleich mit exakter Referenz auf kleinen Strukturen (Feld I)	belegt Kernunabhängigkeit über den Matrixfall hinaus und bestimmt Präzision und Vollständigkeit	mittel bis hoch
13	Optimierung des Einzelvergleichs: Abbruch, Vektorisierung (Feld C)	wirkt fast vollständig auf die Gesamtzeit	mittel
14	Verfeinerte Parallelisierung: strömendes Laden, Query pro Task, Blockgröße (Feld C)	kleinerer serieller Anteil	mittel
15	Asynchrone Suche mit Teilergebnissen (Feld D)	bessere Bedienbarkeit bei langen Suchen	mittel
16	Beschriftete Relation, Label-Indizes, Label-Schichten der Kodierung (Felder A, B und I)	fachlich sinnvollere und kleinere Trefferlisten	mittel bis hoch
17	Mehrdimensionaler Index, Hash-Index (Feld B)	schärfere Vorauswahl	mittel
18	Hybridstrategie mit VF2 und Fallback (Feld C)	Geschwindigkeit bei garantierter Schranke	hoch
19	Abgebrochene Multi-Omics-Datenbankgrößenreihe vervollständigen (Feld H)	Skalierung über N belastbar beurteilen	mittel
20	Schemata für weitere Strukturarten, Koordinatenwahl, Schemaversionierung (Feld I)	zeigt die Kernunabhängigkeit an konkreten Fällen und klärt den Betrieb	mittel bis hoch
21	Multi-Omics-Relation mit realen, annotierten Daten validieren (Felder F und H)	biologische Aussagekraft und Nutzen klären	hoch
22	Fachliche Validierung der universellen Kodierung und Beziehung zur Subgraph-Isomorphie (Felder F und I)	klärt, welche Schemata und Suchmodi biologisch sinnvolle Treffer liefern	hoch
23	Validierung der allgemeinen Suche mit realen Netzwerken (Feld F)	Nachweis der Praxistauglichkeit	hoch
24	GPU-Beschleunigung, verteilte Ausführung, jeweils nach den Prüffragen aus Feld G (Feld C)	Skalierung für sehr große Datenbanken	hoch

Die Reihenfolge trennt vier Ziele: Vollständigkeit und semantische Korrektheit (Nummern 1, 3 und 8), reproduzierbare Austauschbarkeit der Backends (Nummern 2 und 6),

Umsetzung und Messung der universellen Kodierung (Nummern 3, 9 bis 12) sowie empirisch belastbare Leistungswerte und darauf aufbauende Optimierungen und Validierungen (Nummern 4, 5, 7 und 13 bis 24). Die Messung der Indexsuche (Nr. 10) folgt unmittelbar auf Kodierung und Index (Nummern 3 und 9), sobald die differenzielle Prüfung des Experiments keine Abweichung zeigt; sie entscheidet, wie viel von den Optimierungen des Einzelvergleichs und der Parallelisierung für die Gesamtzeit noch relevant ist. Die Wiederholungen (Nr. 11) und das zweite Schema (Nr. 12) folgen erst, wenn der erste Lauf Hypothesen bestätigt oder widerlegt hat, weil sie sonst auf einer ungeklärten Grundlage aufbauen. Die vervollständigte Datenbankgrößenreihe der Multi-Omics-Suche liefert die Vergleichsbasis für die Datenbankgrößenreihe des Experiments.

12.5 Abschließende Bemerkung

Diese Arbeit beschreibt Gen-DB als Full-Stack-System zur Suche in biologischen Netzwerken und verbindet Architektur, formale Analyse, eine Evaluation an einer Million synthetischer Netzwerke sowie Experimente zu VF2, Parallelisierung, Implementierungsbackends und geschichteten Multi-Omics-Netzwerken. Sie schließt mit einer universellen Kodierung biologischer Strukturen, die den Vergleichskern von der Art der Daten trennt und für die eine optimale Suche bewiesen wird, und mit einem Experiment, das diese Aussagen an einer Implementierung prüft. Ihr zentraler Gedanke ist die Trennung zweier Aufgaben: Eine datenbankseitige Vorauswahl verkleinert den Suchraum, und der Subgraph Algorithmus entscheidet anschließend über die verbliebenen Kandidaten mit einer bewiesenen kubischen Zeitschranke. Die Analyse zeigt, unter welchen Bedingungen diese Pipeline korrekt und vollständig ist; die Experimente zeigen, wie stark die tatsächliche Laufzeit von Kandidatenzahl, Implementierung und Ausführungsmodell abhängt.

Besonders deutlich wird das im erneuerten Backend-Experiment. Bei sechs identischen Suchanfragen und 931.899 Kandidaten sinkt die Laufzeit mit der C++-Library von 1.180,28 auf 185,62 Sekunden, ein Speedup von 6,36×. Auch der direkte Mikrobenchmark zeigt einen erheblich schnelleren C++-Aufruf. Gleichzeitig ist der Ende-zu-Ende-Gewinn deutlich kleiner als der Mikrobenchmark-Speedup: Nach Beschleunigung des Vergleichskerns machen Datenbankzugriff, Übergabe und Ergebnisaufbau einen großen Teil der verbleibenden Suchzeit aus. Die Messung führt damit nicht nur zu einem positiven Ergebnis für die Library, sondern auch zu einer konkreten nächsten Frage: Welche nichtvergleichenden Phasen lassen sich sicher messen und optimieren, ohne die Korrektheit der Suche zu beeinträchtigen?

Das Multi-Omics-Kapitel überträgt die Untersuchung auf mehrere gekoppelte Netzwerkschichten und unterscheidet eine gemeinsame, kohärente Trefferposition von unabhängigen Trefferpositionen je Schicht. Die abgeschlossene differenzielle Prüfung fand bei 5.000 Paaren und 10.000 Modusentscheidungen keine Abweichung zwischen Python und C++; im Mikrobenchmark lag der geometrische Mittelwert des Speedups bei 8,11×. In der vollständigen Suche mit 336 gepaarten Läufen je Backend sinkt die summierte Laufzeit von 1.401,16 auf 620,43 Sekunden, ein Speedup von 2,26. Alle 1.722 erwarteten eingebetteten Treffer wurden gefunden, und die Treffer-IDs stimmten in jedem Paar überein. Die Worker- und Chunk-Messungen zeigen begrenzte, backendabhängige Skalierung; die Datenbankgrößenreihe wurde vor Abschluss abgebrochen. Das sind belastbare technische Ergebnisse für synthetische Daten auf der dokumentierten Hardware, keine Validierung biologischer Muster oder Kausalbeziehungen.

Der Abschnitt zur universellen Kodierung ergänzt diese Ergebnisse um eine allgemeine

Aussage. Bisher wurde für jede neue Art biologischer Daten ein eigener Vergleichspfad formuliert und umgesetzt, zuerst für das Einzelnetzwerk, dann für das mehrschichtige Netzwerk. Das Ergebnis dieser Arbeit ist, dass dies nicht notwendig ist: Der Kern kennt nur Stapel von Mengenfolgen und die Relationen $\sqsubseteq_{\mathrm{ind}}$ und $\sqsubseteq_{\mathrm{coh}}$; jede endliche attributierte relationale Struktur lässt sich verlustfrei und in fast linearer Zeit in einen solchen Stapel kodieren; Vergleich und Indexsuche sind für beliebige Stapel korrekt, der Vergleich ist bis auf einen konstanten Faktor optimal, und die Indexsuche liefert das exakte Ergebnis ohne Einzelvergleiche mit einem Aufwand, der von der Trefferlast und nicht von der Datenbankgröße abhängt. Für weitere biologische Strukturen muss künftig nur noch ein Schema mit seiner Kodierung in Python ergänzt werden. Das ist dieselbe Trennung, die die Arbeit von Anfang an leitet, nun auf die Daten selbst angewandt: Die Vorauswahl ist vom Vergleich getrennt, die Sprachgrenze von der Rechnung, und jetzt die Struktur der Daten vom Kern.

Diese Aussagen der universellen Kodierung haben einen anderen Charakter als die gemessenen Ergebnisse der früheren Abschnitte. Sie sind bewiesen und in Python umgesetzt; ihre Laufzeitvorteile gegenüber der Pipeline sind Folgerungen aus Sätzen, für die Abschnitt 11 eine Prüfung festlegt. Das Experiment ist so angelegt, dass es die Sätze nicht nur bestätigen, sondern auch an ihre Grenze führen kann: Neun Hypothesen mit Widerlegungskriterien, eine konstruierte Ground Truth und eine Auswertung, die vor dem Lauf feststeht, verhindern, dass das Ergebnis nachträglich gedeutet wird. Die Unabhängigkeit von der Datenbankgröße gilt nur unter einer Häufigkeitsannahme; die Messung soll zeigen, für welche Anfragen und Daten sie trägt und für welche nicht, etwa bei kurzen Anfragen mit wenigen Schichten, bei denen die Trefferlast von der Größenordnung der Datenbank ist. Die Optimalität bezieht sich auf die definierte Relation und nicht auf die klassische Teilgraph-Isomorphie, die nach Bemerkung 21 ohne P = NP nicht exakt in dieser Form abbildbar ist; die Relation ist nicht transitiv, kennt keinen zulässigen Kantenzahl-Filter, und ihre biologische Validität ist offen. Der Ausblick (Themenfeld I) und die Priorisierung (Nummern 3 und 9 bis 12 sowie die fachliche Validierung) übersetzen dies in konkrete Schritte: Durchführung und Auswertung des Messlaufs, Wiederholung auf weiterer Hardware und mit kontrollierter Häufigkeit, ein zweites Schema, der Vergleich mit einer exakten Referenz und die fachliche Validierung.

Die Ergebnisse bleiben in ihrem Geltungsbereich begrenzt. Die Multi-Omics-Such-, Worker- und Chunk-Messungen umfassen synthetische Daten und eine Windows-11-Konfiguration mit vier physischen Kernen; Worker- und Chunk-Reihen beruhen auf sechs Queries und zwei Wiederholungen. Die DB-Skalierung ist unvollständig. Keine der Messungen belegt dieselbe Beschleunigung auf realen biologischen Netzwerken oder anderer Hardware; übereinstimmende Treffer in den geprüften Stichproben sind kein allgemeiner Korrektheitsbeweis. Offen bleiben außerdem die biologische Angemessenheit der Relationen, die Berücksichtigung von Knotenlabels, die Suchrichtung, mögliche Kandidatenlimits und eine breitere statistische Absicherung. Als nächste Schritte folgen die Vervollständigung der DB-Größenreihe, Wiederholungen auf weiterer Hardware, Konformitätstests, die Umsetzung der universellen Kodierung mit Indexsuche und biologische Validierung mit annotierten Multi-Omics-Daten.

Der zentrale Gedanke der Trennung gilt somit für den Suchablauf wie für die Sprachgrenze: Eine klar definierte Schnittstelle macht Verantwortung und Kosten nachvollziehbar. Für den untersuchten Suchpfad hat die Library diese Gestaltungsentscheidung nicht nur strukturell, sondern auch experimentell gestützt; die frühere CLI-Messung zeigt zugleich, wie sehr ein ungünstiger Aufrufpfad den Rechenkern verdecken kann. Die Arbeit liefert

damit keinen universellen Leistungsanspruch, sondern eine nachvollziehbare Grundlage für Gen-DB: einen formal analysierten Suchablauf, ein konkret gemessenes schnelleres Backend, eine bewiesene und umgesetzte universelle Kodierung mit optimaler Indexsuche, die neue Strukturarten ohne Änderung des Kerns zulässt, ein Experiment, das ihre Aussagen mit benannten Widerlegungskriterien prüfbar macht, und einen priorisierten Weg von der Prototyp-Implementierung zu einem validierten und belastbar betriebenen Suchsystem.

Literaturverzeichnis

[1] Stephan Epp. *Der Subgraph Algorithmus.*, 2026. Verfügbar unter: https://github.com/naphets86/subgraph.

[2] Amir Abboud, Arturs Backurs, Virginia Vassilevska Williams. *Tight Hardness Results for LCS and Other Sequence Similarity Measures*. In: Proceedings of FOCS 2015.

[3] Paul Erdős, Alfréd Rényi. *On Random Graphs I.* Publicationes Mathematicae (Debrecen), 6:290–297, 1959.

[4] The PostgreSQL Global Development Group. *PostgreSQL 15 Documentation*. https://www.postgresql.org/docs/15/.

[5] Sebastián Ramírez et al. *FastAPI Documentation*. https://fastapi.tiangolo.com/.

[6] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*, 4. Auflage. MIT Press, 2022.

[7] Richard P. Brent. *The Parallel Evaluation of General Arithmetic Expressions*. Journal of the ACM, 21(2):201–206, 1974.

[8] Antonin Guttman. *R-Trees: A Dynamic Index Structure for Spatial Searching*. In: Proceedings of ACM SIGMOD, 1984.

[9] Robert A. Wagner, Michael J. Fischer. *The String-to-String Correction Problem*. Journal of the ACM, 21(1):168–173, 1974.

[10] Luigi P. Cordella, Pasquale Foggia, Carlo Sansone, Mario Vento. *A (Sub)Graph Isomorphism Algorithm for Matching Large Graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 26(10):1367–1372, 2004.

[11] Julian R. Ullmann. *An Algorithm for Subgraph Isomorphism*. Journal of the ACM, 23(1):31–42, 1976.

[12] Aric A. Hagberg, Daniel A. Schult, Pieter J. Swart. *Exploring Network Structure, Dynamics, and Function using NetworkX*. In: Proceedings of the 7th Python in Science Conference (SciPy), 2008.

[13] Vincenzo Bonnici, Rosalba Giugno, Alfredo Pulvirenti, Dennis Shasha, Alfredo Ferro. *A Subgraph Isomorphism Algorithm and Its Application to Biochemical Data*. BMC Bioinformatics, 14(Suppl 7):S13, 2013.

[14] Huahai He, Ambuj K. Singh. *Graphs-at-a-time: Query Language and Access Methods for Graph Databases*. In: Proceedings of ACM SIGMOD, 2008.

[15] Gene M. Amdahl. *Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities*. In: Proceedings of the AFIPS Spring Joint Computer Conference, 1967.

[16] John L. Gustafson. *Reevaluating Amdahl's Law*. Communications of the ACM, 31(5):532–533, 1988.

[17] Stephen A. Cook. *The Complexity of Theorem-Proving Procedures*. In: Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC), S. 151–158, 1971.

[18] Richard M. Karp. *Reducibility Among Combinatorial Problems*. In: R. E. Miller, J. W. Thatcher (Hrsg.), Complexity of Computer Computations, Plenum Press, S. 85–103, 1972.

[19] Michael R. Garey, David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[20] Russell Impagliazzo, Ramamohan Paturi. *On the Complexity of k-SAT*. Journal of Computer and System Sciences, 62(2):367–375, 2001.

[21] Karl Bringmann, Marvin Künnemann. *Quadratic Conditional Lower Bounds for String Problems and Dynamic Time Warping*. In: Proceedings of FOCS 2015, S. 79–97.

[22] Daniel S. Hirschberg. *A Linear Space Algorithm for Computing Maximal Common Subsequences*. Communications of the ACM, 18(6):341–343, 1975.

[23] Brendan D. McKay, Adolfo Piperno. *Practical Graph Isomorphism, II*. Journal of Symbolic Computation, 60:94–112, 2014.

[24] László Babai. *Graph Isomorphism in Quasipolynomial Time*. In: Proceedings of the 48th Annual ACM Symposium on Theory of Computing (STOC), S. 684–697, 2016.

[25] Vincenzo Carletti, Pasquale Foggia, Alessia Saggese, Mario Vento. *Challenging the Time Complexity of Exact Subgraph Isomorphism for Huge and Dense Graphs with VF3*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 40(4):804–818, 2018.

[26] Jinsoo Lee, Wook-Shin Han, Romans Kasperovics, Jeong-Hoon Lee. *An In-depth Comparison of Subgraph Isomorphism Algorithms in Graph Databases*. Proceedings of the VLDB Endowment, 6(2):133–144, 2013.

[27] Xifeng Yan, Philip S. Yu, Jiawei Han. *Graph Indexing: A Frequent Structure-based Approach*. In: Proceedings of ACM SIGMOD, S. 335–346, 2004.

[28] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, Uri Alon. *Network Motifs: Simple Building Blocks of Complex Networks*. Science, 298(5594):824–827, 2002.

[29] Albert-László Barabási, Réka Albert. *Emergence of Scaling in Random Networks*. Science, 286(5439):509–512, 1999.

[30] Minoru Kanehisa, Susumu Goto. *KEGG: Kyoto Encyclopedia of Genes and Genomes*. Nucleic Acids Research, 28(1):27–30, 2000.

[31] Damian Szklarczyk et al. *STRING v11: Protein–Protein Association Networks with Increased Coverage, Supporting Functional Discovery in Genome-Wide Experimental Datasets*. Nucleic Acids Research, 47(D1):D607–D613, 2019.

[32] Charles R. Harris et al. *Array Programming with NumPy*. Nature, 585:357–362, 2020.

[33] Siu Kwan Lam, Antoine Pitrou, Stanley Seibert. *Numba: A LLVM-based Python JIT Compiler*. In: Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, 2015.

[34] Mikko Kivelä, Alex Arenas, Marc Barthelemy, James P. Gleeson, Yamir Moreno, Mason A. Porter. *Multilayer Networks*. Journal of Complex Networks, 2(3):203–271, 2014.

[35] Manlio De Domenico, Albert Solé-Ribalta, Emanuele Cozzo, Mikko Kivelä, Yamir Moreno, Mason A. Porter, Sergio Gómez, Alex Arenas. *Mathematical Formulation of Multilayer Networks*. Physical Review X, 3:041022, 2013.

[36] Yehudit Hasin, Marcus Seldin, Aldons Lusis. *Multi-omics Approaches to Disease*. Genome Biology, 18:83, 2017.

[37] Bo Wang, Aziz M. Mezlini, Feyyaz Demir, Marc Fiume, Zhuowen Tu, Michael Brudno, Benjamin Haibe-Kains, Anna Goldenberg. *Similarity Network Fusion for Aggregating Data Types on a Genomic Scale*. Nature Methods, 11(3):333–337, 2014.

[38] Ronglai Shen, Adam B. Olshen, Marc Ladanyi. *Integrative Clustering of Multiple Genomic Data Types Using a Joint Latent Variable Model with Application to Breast and Lung Cancer Subtype Analysis*. Bioinformatics, 25(22):2906–2912, 2009.

[39] Chen Meng, Oana A. Zeleznik, Gerhard G. Thallinger, Bernhard Kuster, Amin M. Gholami, Aedín C. Culhane. *Dimension Reduction Techniques for the Integrative Analysis of Multi-omics Data*. Briefings in Bioinformatics, 17(4):628–641, 2016.

[40] Marylyn D. Ritchie, Emily R. Holzinger, Ruowang Li, Sarah A. Pendergrass, Dokyoon Kim. *Methods of Integrating Data to Uncover Genotype–Phenotype Interactions*. Nature Reviews Genetics, 16:85–97, 2015.

[41] Yifeng Li, Fang-Xiang Wu, Alioune Ngom. *A Review on Machine Learning Principles for Multi-view Biological Data Integration*. Briefings in Bioinformatics, 19(2):325–340, 2018.

[42] Shihua Zhang, Chun-Chi Liu, Wenyuan Li, Hui Shen, Peter W. Laird, Xianghong Jasmine Zhou. *Discovery of Multi-dimensional Modules by Integrative Analysis of Cancer Genomic Data*. Nucleic Acids Research, 40(19):9379–9391, 2012.

[43] Shihua Zhang, Qingjiao Li, Juan Liu, Xianghong Jasmine Zhou. *A Novel Computational Framework for Simultaneous Integration of Multiple Types of Genomic Data to Identify microRNA-Gene Regulatory Modules*. Bioinformatics, 27(13):i401–i409, 2011.

[44] Bin Zhang, Steve Horvath. *A General Framework for Weighted Gene Co-Expression Network Analysis*. Statistical Applications in Genetics and Molecular Biology, 4(1):Article 17, 2005.

[45] Peter Langfelder, Steve Horvath. *WGCNA: an R Package for Weighted Correlation Network Analysis*. BMC Bioinformatics, 9:559, 2008.

[46] Jan S. Hawe, Fabian J. Theis, Matthias Heinig. *Inferring Interaction Networks from Multi-omics Data*. Frontiers in Genetics, 10:535, 2019.

[47] Xifeng Yan, Philip S. Yu, Jiawei Han. *Substructure Similarity Search in Graph Databases*. In: Proceedings of ACM SIGMOD, S. 766–777, 2005.

[48] Haichuan Shang, Ying Zhang, Xuemin Lin, Jeffrey Xu Yu. *Taming Verification Hardness: An Efficient Algorithm for Testing Subgraph Isomorphism*. Proceedings of the VLDB Endowment, 1(1):364–375, 2008.

[49] Zhao Sun, Hongzhi Wang, Haixun Wang, Bin Shao, Jianzhong Li. *Efficient Subgraph Matching on Billion Node Graphs*. Proceedings of the VLDB Endowment, 5(9):788–799, 2012.

[50] Fei Bi, Lijun Chang, Xuemin Lin, Lu Qin, Wenjie Zhang. *Efficient Subgraph Matching by Postponing Cartesian Products*. In: Proceedings of ACM SIGMOD, S. 1199–1214, 2016.

[51] Shixuan Sun, Qiong Luo. *In-Memory Subgraph Matching: An In-depth Study*. In: Proceedings of ACM SIGMOD, S. 1083–1098, 2020.

[52] Lei Zou, Jinghui Mo, Lei Chen, M. Tamer Özsu, Dongyan Zhao. *gStore: Answering SPARQL Queries via Subgraph Matching*. Proceedings of the VLDB Endowment, 4(8):482–493, 2011.

[53] Shijie Zhang, Shirong Li, Jiong Yang. *GADDI: Distance Index Based Subgraph Matching in Biological Networks*. In: Proceedings of EDBT, S. 192–203, 2009.

[54] Shijie Zhang, Jiong Yang, Wei Jin. *SAPPER: Subgraph Indexing and Approximate Matching in Large Graphs*. Proceedings of the VLDB Endowment, 3(1):1185–1194, 2010.

[55] Wenfei Fan, Jianzhong Li, Shuai Ma, Nan Tang, Yinghui Wu, Yunpeng Wu. *Graph Pattern Matching: From Intractable to Polynomial Time*. Proceedings of the VLDB Endowment, 3(1):264–275, 2010.

[56] Yuanyuan Tian, Jignesh M. Patel. *TALE: A Tool for Approximate Large Graph Matching*. In: Proceedings of IEEE ICDE, S. 963–972, 2008.

[57] Zhiping Zeng, Anthony K. H. Tung, Jianyong Wang, Jianhua Feng, Lizhu Zhou. *Comparing Stars: On Approximating Graph Edit Distance*. Proceedings of the VLDB Endowment, 2(1):25–36, 2009.

[58] Xinbo Gao, Bing Xiao, Dacheng Tao, Xuelong Li. *A Survey of Graph Edit Distance*. Pattern Analysis and Applications, 13(1):113–129, 2010.

[59] Donatello Conte, Pasquale Foggia, Carlo Sansone, Mario Vento. *Thirty Years of Graph Matching in Pattern Recognition*. International Journal of Pattern Recognition and Artificial Intelligence, 18(3):265–298, 2004.

[60] Longbin Lai, Lu Qin, Xuemin Lin, Lijun Chang. *Scalable Subgraph Enumeration in MapReduce*. Proceedings of the VLDB Endowment, 8(10):974–985, 2015.

[61] Xiaowei Zhu, Wenguang Chen, Weimin Zheng, Xiaosong Ma. *Gemini: A Computation-Centric Distributed Graph Processing System*. In: Proceedings of USENIX OSDI, S. 301–316, 2016.

[62] Gene Myers. *A Fast Bit-Vector Algorithm for Approximate String Matching Based on Dynamic Programming*. Journal of the ACM, 46(5):395–415, 1999.

[63] Gonzalo Navarro. *A Guided Tour to Approximate String Matching*. ACM Computing Surveys, 33(1):31–88, 2001.

[64] Dan Gusfield. *Algorithms on Strings, Trees, and Sequences*. Cambridge University Press, 1997.

[65] Richard M. Karp, Michael O. Rabin. *Efficient Randomized Pattern-Matching Algorithms*. IBM Journal of Research and Development, 31(2):249–260, 1987.

[66] Donald E. Knuth, James H. Morris, Vaughan R. Pratt. *Fast Pattern Matching in Strings*. SIAM Journal on Computing, 6(2):323–350, 1977.

[67] David Eppstein. *Subgraph Isomorphism in Planar Graphs and Related Problems*. Journal of Graph Algorithms and Applications, 3(3):1–27, 1999. DOI: https://doi.org/10.7155/jgaa.00014.

[68] Christine Solnon. *AllDifferent-based filtering for subgraph isomorphism*. Artificial Intelligence, 174(12–13):850–864, 2010. DOI: https://doi.org/10.1016/j.artint.2010.05.002.

[69] Alpár Jüttner, Péter Madarasi. *VF2++—An improved subgraph isomorphism algorithm*. Discrete Applied Mathematics, 242:69–81, 2018. DOI: https://doi.org/10.1016/j.dam.2018.02.018.

[70] Ciaran McCreesh, Patrick Prosser, James Trimble. *The Glasgow Subgraph Solver: Using Constraint Programming to Tackle Hard Subgraph Isomorphism Problem Variants*. In: Principles and Practice of Constraint Programming – CP 2020, S. 316–324, 2020. DOI: https://doi.org/10.1007/978-3-030-51372-6_19.

[71] Ronald L. Graham. *Bounds on Multiprocessing Timing Anomalies*. SIAM Journal on Applied Mathematics, 17(2):416–429, 1969. DOI: https://doi.org/10.1137/0117039.

[72] Robert D. Blumofe, Charles E. Leiserson. *Scheduling Multithreaded Computations by Work Stealing*. Journal of the ACM, 46(5):720–748, 1999. DOI: https://doi.org/10.1145/324133.324234.

[73] Leonardo Dagum, Ramesh G. Menon. *OpenMP: An Industry-Standard API for Shared-Memory Programming*. IEEE Computational Science and Engineering, 5(1):46–55, 1998. DOI: https://doi.org/10.1109/99.660313.

[74] OpenMP Architecture Review Board. *OpenMP API Specification*, Version 5.2, 2021. https://www.openmp.org/spec-html/5.2/openmp.html (abgerufen am 10. Oktober 2026).

[75] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard*, Version 4.1, 2023. https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf (abgerufen am 10. Oktober 2026).

[76] Peter S. Pacheco. *An Introduction to Parallel Programming*. Morgan Kaufmann, 2011.

[77] Thomas Rauber, Gudula Rünger. *Parallel Programming for Multicore and Cluster Systems*, 2. Auflage. Springer, 2013. DOI: https://doi.org/10.1007/978-3-642-37801-0.

[78] Michael McCool, James Reinders, Arch Robison. *Structured Parallel Programming: Patterns for Efficient Computation*. Morgan Kaufmann, 2012.

[79] Georg Hager, Gerhard Wellein. *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press, 2010.

[80] David H. Bailey et al. *The NAS Parallel Benchmarks*. The International Journal of Supercomputing Applications, 5(3):63–73, 1991. DOI: https://doi.org/10.1177/109434209100500306.

[81] Lutz Prechelt. *An Empirical Comparison of Seven Programming Languages*. Computer, 33(10):23–29, 2000. DOI: https://doi.org/10.1109/2.876288.

[82] Stefan Behnel, Robert Bradshaw, Craig Citro, Lisandro Dalcin, Dag Sverre Seljebotn, Kurt Smith. *Cython: The Best of Both Worlds*. Computing in Science & Engineering, 13(2):31–39, 2011. DOI: https://doi.org/10.1109/MCSE.2010.118.

[83] Python Software Foundation. *Extending and Embedding the Python Interpreter*, Python 3.14 Documentation. https://docs.python.org/3.14/extending/index.html (abgerufen am 10. Oktober 2026).

[84] pybind11 contributors. *pybind11 Documentation: Seamless Interoperability between C++ and Python*, Version 3. https://pybind11.readthedocs.io/en/stable/ (abgerufen am 10. Oktober 2026).

[85] Python Software Foundation. *ctypes – A Foreign Function Library for Python*, Python 3.14 Library Reference. https://docs.python.org/3.14/library/ctypes.html (abgerufen am 10. Oktober 2026).

[86] Python Software Foundation. *Thread States and the Global Interpreter Lock*, Python 3.14 C API Reference. https://docs.python.org/3.14/c-api/threads.html (abgerufen am 10. Oktober 2026).

[87] Python Software Foundation. *multiprocessing – Process-Based Parallelism*, Python 3.14 Library Reference. https://docs.python.org/3.14/library/multiprocessing.html (abgerufen am 10. Oktober 2026).

[88] Python Software Foundation. *concurrent.futures – Launching Parallel Tasks*, Python 3.14 Library Reference. https://docs.python.org/3.14/library/concurrent.futures.html (abgerufen am 10. Oktober 2026).

[89] Carl Friedrich Bolz, Antonio Cuni, Maciej Fijalkowski, Armin Rigo. *Tracing the Meta-Level: PyPy's Tracing JIT Compiler*. In: Proceedings of the 4th Workshop on the Implementation, Compilation, Optimization of Object-Oriented Languages and Programming Systems, S. 18–25, 2009. DOI: https://doi.org/10.1145/1565824.1565827.

[90] Rui Pereira, Marco Couto, Francisco Ribeiro, Rui Rua, Jácome Cunha, João Paulo Fernandes, João Saraiva. *Energy Efficiency across Programming Languages: How Do Energy, Time, and Memory Relate?* In: Proceedings of the ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, S. 256–267, 2017. DOI: https://doi.org/10.1145/3136014.3136031.

[91] Andy Georges, Dries Buytaert, Lieven Eeckhout. *Statistically Rigorous Java Performance Evaluation*. In: Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems and Applications, S. 57–76, 2007. DOI: https://doi.org/10.1145/1297027.1297033.

[92] Tomáš Kalibera, Richard Jones. *Rigorous Benchmarking in Reasonable Time*. In: Proceedings of the 2013 International Symposium on Memory Management, S. 63–74, 2013. DOI: https://doi.org/10.1145/2464157.2464160.

[93] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, Peter F. Sweeney. *Producing Wrong Data Without Doing Anything Obviously Wrong!* In: Proceedings of the 14th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, S. 265–276, 2009. DOI: https://doi.org/10.1145/1508244.1508275.

[94] Philip J. Fleming, John J. Wallace. *How Not to Lie with Statistics: The Correct Way to Summarize Benchmark Results*. Communications of the ACM, 29(3):218–221, 1986. DOI: https://doi.org/10.1145/5666.5673.

[95] SPEC Open Systems Group. *SPEC CPU 2017 Run and Reporting Rules*. https://www.spec.org/cpu2017/Docs/runrules.html (abgerufen am 10. Oktober 2026).

[96] Raj Jain. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley, 1991.

[97] David J. Lilja. *Measuring Computer Performance: A Practitioner's Guide*. Cambridge University Press, 2000.

[98] Python Software Foundation. *timeit – Measure Execution Time of Small Code Snippets*, Python 3.14 Library Reference. https://docs.python.org/3.14/library/timeit.html (abgerufen am 10. Oktober 2026).

[99] Python Software Foundation. *time – Time Access and Conversions*, Python 3.14 Library Reference. https://docs.python.org/3.14/library/time.html (abgerufen am 10. Oktober 2026).